Distributed systems, Cloud and AI

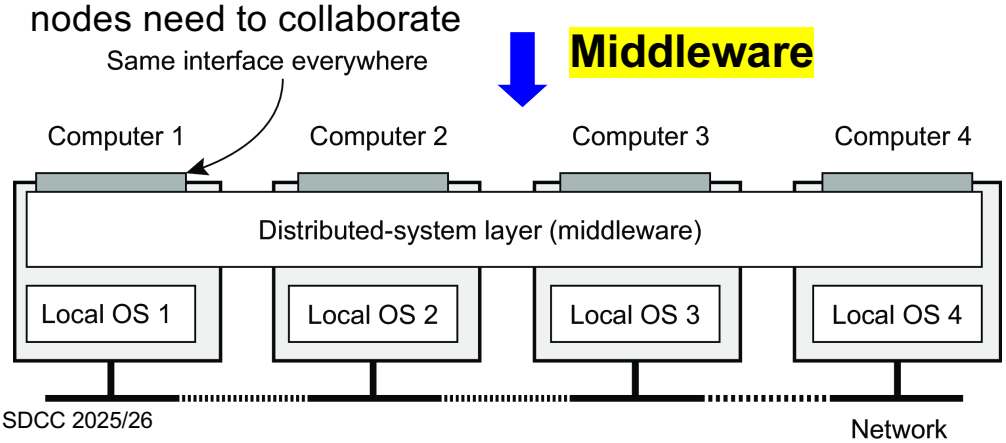
- AI has recently become practical as result of:
  - Distributed computing
  - Affordable cloud computing and storage costs
  - Examples: federated learning, distributed training of foundation models (huge computational and data storage needs)
- Distribute = to divide and dispense in portions
- A foremost strategy used in distributed computing you already know
  - Divide et impera: break larger (computational) problems down into numbers of smaller, interrelated, “manageable” pieces

Distributed system

- Multiple definitions of **distributed system (DS)**

[van Steen & Tanenbaum] A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system

  - Consists of autonomous computing elements (i.e., **nodes**), can be hardware devices (computer, phone, car, robot, …) or software processes
  - Users or applications perceive it as a **single system** (how?): nodes need to collaborate



Distributed system

[Lamport] A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable

- Emphasis on fault tolerance

- Who is Leslie Lamport?
  - Recipient of 2013 Turing award https://www.youtube.com/watch?v=pgWTmOyUjtM
  - His research contributions have laid the foundations of theory and practice of DS
    - Fundamental concepts such as causality, logical clocks and Byzantine failures
    - Algorithms to solve many fundamental problems in DS
  - Inventor of LaTeX

Why make a system distributed?

- Share resources
  - Resource = computing node, storage, data, service, …
- Lower costs
- Improve performance
  - e.g., get data from a nearby node rather than one halfway round the world
- Improve availability and reliability
  - even if one node fails, the system as a whole keeps functioning
- Improve security
- Solve bigger problems
  - e.g., huge amounts of data, can't fit on one machine
- Support Quality of Service (QoS) and Quality of Experience (QoE)

Why to study distributed systems

- Distributed systems are more complex than centralized ones
  - e.g., no global clock, group membership, …
- Building them is harder… and building them correct is even much harder

  Distributed systems need radically different software than centralized systems do (Tanenbaum)
- Managing, and, above all, testing them is difficult

Some distinguishing features of DS

- Concurrency
  - Many things happen “at the same time”
  - Centralized system: design choice
  - Distributed system: fact of life to be dealt with
- No global clock
  - Centralized system: use computer’s physical clock for synchronization
  - Distributed system: many physical clocks and not necessarily synchronized among them
- Independent and partial failures
  - Centralized system: fails completely
  - Distributed system: fails partially (i.e., only a part), often due to communication; hard (and in general impossible) to hide partial failures and their recovery

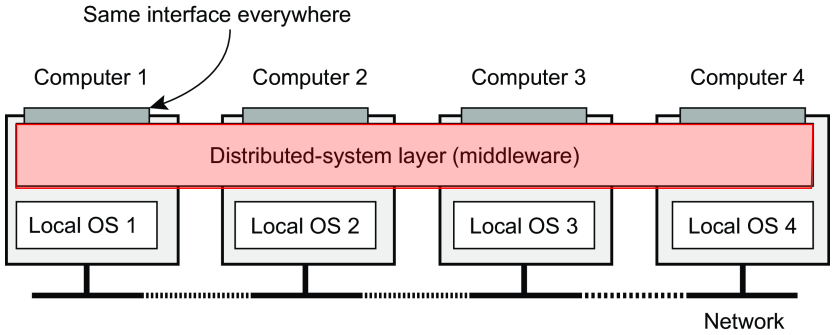
Challenges and design goals

- Challenges and goals associated with designing distributed systems
  1. **Heterogeneity**
  2. **Distribution transparency**
  3. **Openness**
  4. **Scalability**
  5. **Dependability**
  6. **Security**

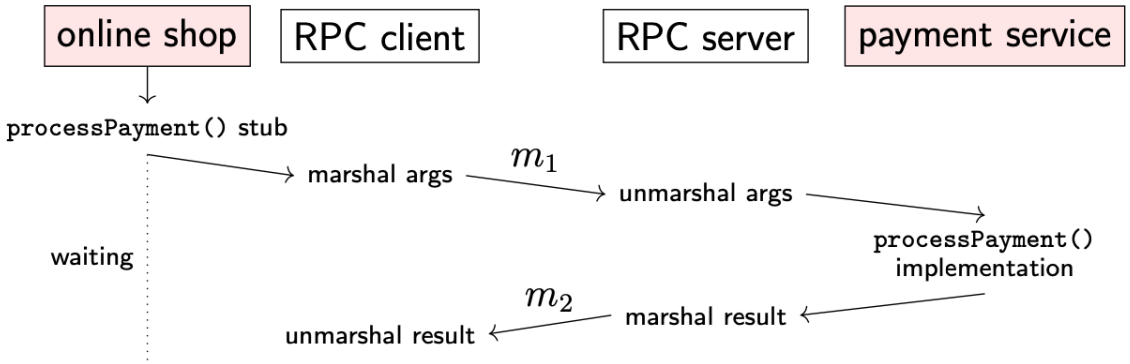
while improving performance and sustainability (energy efficiency, carbon-awareness), reducing monetary cost, etc.

Challenge 1: Heterogeneity

- Many sources of heterogeneity: network, hardware, operating system (OS), programming language, implementations by different developers
- How to address? **Middleware**: the "OS of a DS"
  - Sw layer placed on top of OS that provides a programming abstraction as well as masks heterogeneity
  - Contains commonly used components and functionalities (e.g., communication) thus avoiding developers to implement them again and from scratch



RPC: Behind the curtains



$m_1 =$

```
{
  "request": "processPayment",
  "card": {
    "number": "1234567887654321",
    "expiryDate": "10/2024",
    "CVC": "123"
  },
  "amount": 3.99,
  "currency": "GBP"
}
```

$m_2 =$

```
{
  "result": "success",
  "id": "XP61hHw2Rvo"
}
```


Challenge 2: Distribution transparency

- **Distribution transparency**: single coherent system where distribution of its *objects* (*resources* and *processes*) is *transparent* (i.e., invisible) to users and applications
- **Types** of distribution transparency (*ISO 10746*, Reference Model of Open Distributed Processing)

Access transparency

- Hide differences in data representation and how an object is accessed
  - e.g., use same mechanism for local or remote call

Location transparency

- Hide where an object is located
  - e.g., URL hides IP address

Relocation transparency

- Hide that an object may be moved to another location *while in use*

Challenge 2: Distribution transparency

Migration transparency

- Hide that an object may move to another location
  - e.g., communication between mobile phones

Replication transparency

- Hide that an object may be replicated
  - How? Same name for all replicas
  - Example: type command `dig www.youtube.com`
  - Require also location transparency

Concurrency transparency

- Hide that an object may be shared by several users
  - E.g.: concurrent accesses to DB table
  - Issue: leave shared object in a consistent state, e.g., by *locking* mechanisms

Failure transparency

- Hide failure and recovery of an object (Lamport's DS definition)

Degree of distribution transparency

- Aiming at *full* distribution transparency may be too much
  - We cannot always hide communication latency: sending a message from Roma to New York requires ~23 ms
  - We cannot completely hide failures in a large-scale DS
    - Cannot distinguish a slow server from a failing one
    - Cannot be sure that a server actually performed an operation before crashing
  - Price for achieving full transparency may be too high in term of performance
    - e.g., keeping data replicas *exactly* up-to-date takes time
    - e.g., *immediately flushing* write operations to disk for fault tolerance affects performance

  Trade-off between consistency and performance

Challenge 3: Openness

- Open DS: offers components that can easily be used by or integrated into other systems; consists of components that originate from elsewhere
- System should conform to well-defined interfaces
  - Defined through IDL (**Interface Definition Language**)
    - IDL is used to define data structures, interfaces (APIs), serialization formats in a language-agnostic way: complete and neutral
    - IDL examples: Protobuf, XDR, Thrift
- Benefits of openness
  - Interoperability
  - Portability of applications
  - Extensibility
- Examples: Jakarta EE, .Net, Web Services

Practice shows that many distributed systems are not as open as we'd like (van Steen & Tanenbaum)

Separation of policies and mechanisms

- To implement open and flexible DS, we need to organize the DS as a collection of relatively small and easily replaceable or adaptable component rather than as a monolithic system
- How? Separate policies from mechanisms
  - Policies refer to the principles guiding the decisions, while mechanisms represent the means to implement policies
- E.g., caching in a web browser:
  - Mechanism: store data and allow (dynamic) setting of caching policies
  - Caching policies:
    - Where to cache data?
    - How to free space when cache fills up?
    - When to refresh cached data?
    - Private or shared cache?

Separation of policies and mechanisms

- The other side of the coin
  - Strict separation can be counterproductive: the stricter the separation between policies and mechanisms, the more we need to ensure proper mechanisms, potentially leading to many configuration parameters and complex management
- Need to find a balance
- Possible solution: *self-configurable systems*

> *Finding the right balance in separating policies from mechanisms is one of the reasons why designing a distributed system is sometimes more an art than a science* (van Steen & Tanenbaum)

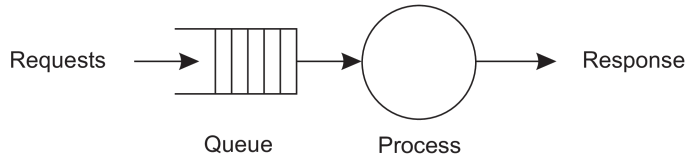
Challenge 4: Scalability

- **Scalability** is the property of a (distributed) *system* to keep an adequate level of performance notwithstanding a growing amount of:
  - Number of users and resources (**size** scalability)
  - Maximum distance between nodes (**geographical** scalability)
  - Number of administrative domains (**administrative** scalability)
- Most systems account only, to a certain extent, for size scalability

> *Many developers of modern distributed systems easily use the adjective scalable without making clear why their system actually scales.* (van Steen)

Size scalability

- Why a centralized system often lacks scalability
  - Computational capacity, limited by CPUs
  - Storage capacity, including transfer rate between CPUs and disks
  - Network between users and centralized system
- Formal analysis (see PMCS course)

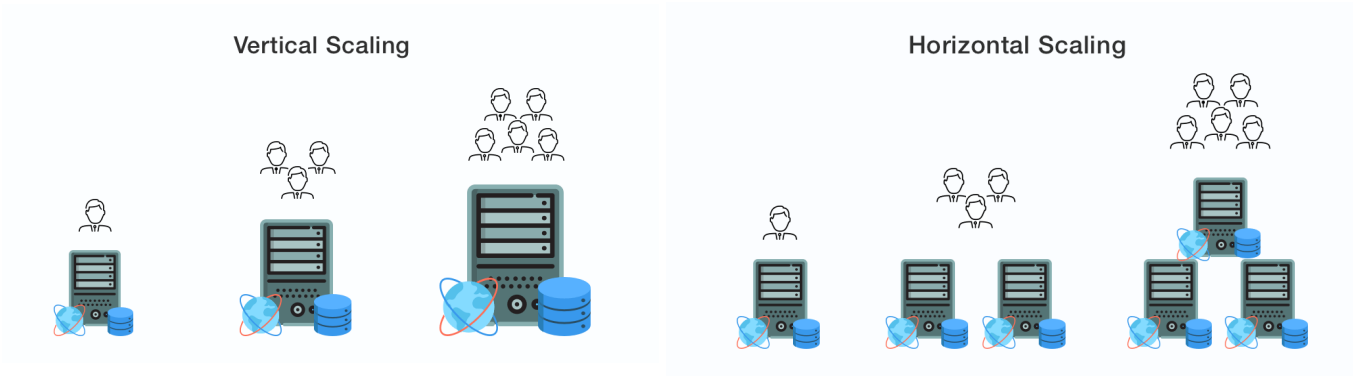


$$\frac{R}{S} = \frac{1}{1-U}$$

  - Service time S
  - Utilization U: fraction of time the service is busy
  - Response time R: total time take to process a request its arrival
  - If U is small, response-to-service time is close to 1: request is immediately processed
  - If U goes up to 1, system comes to a grinding halt. Solution: decrease S. How?

Size scalability

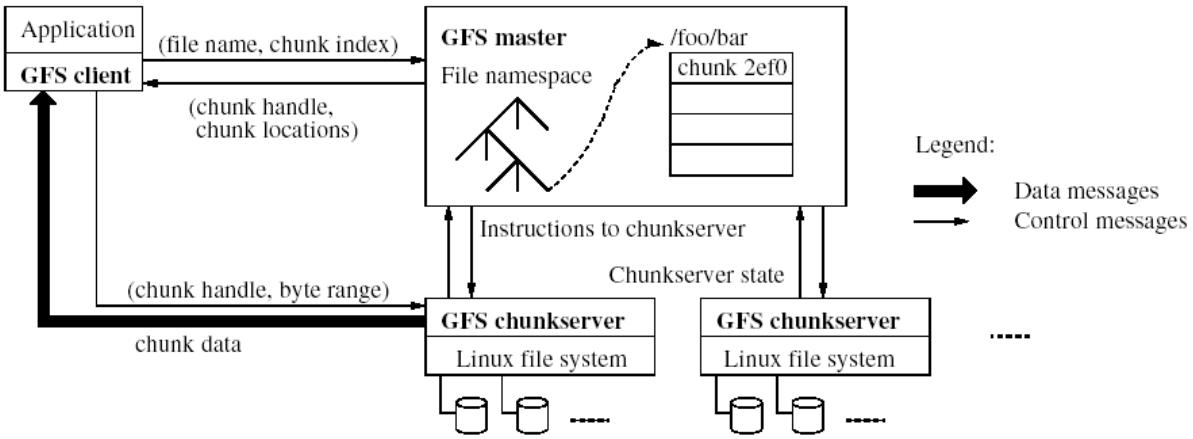
- Two directions for size scalability
  - **Vertical (scale-up)**: more powerful resources
  - **Horizontal (scale-out)**: more resources with same capacity



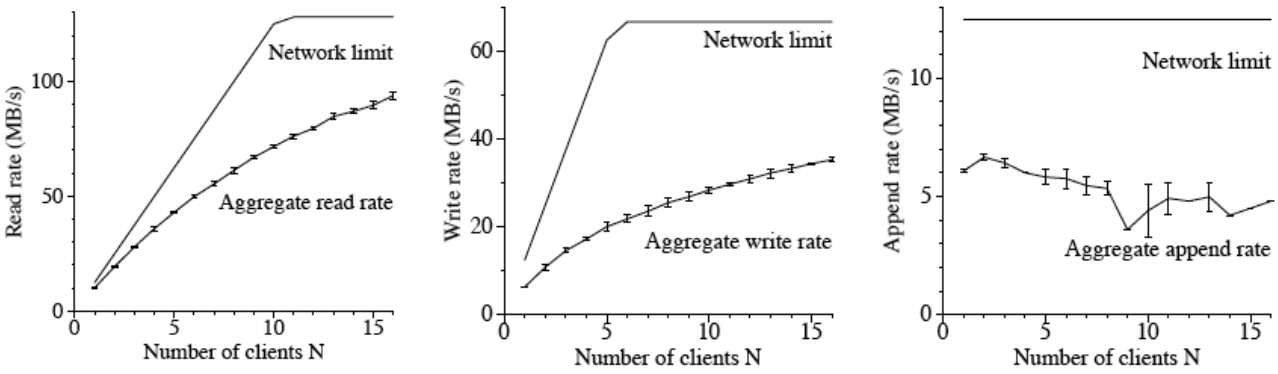
- Which resources? Physical, virtual (virtual machines, software containers, etc.)

Size scalability: GFS example

- Google File System (GFS)
  - Distributed file system based on divide et impera principle and horizontal scalability
  - Read, write, append operations
  - File divided into fixed blocks (chunks), which are distributed and replicated on multiple servers
  - Single master, multiple workers (chunkservers)



Size scalability: GFS example



(a) Reads (b) Writes (c) Record appends

- See experimental results
  - Scale parameter: number of clients
  - Scalability metric: aggregated read/write/append throughput (random file access)
  - Scalability criterion: the closer to network limit, the better
- Scalability is challenging!

Scaling techniques

1. Hide communication latency
   - Use asynchronous communication
   - Have separate handler for incoming response
   - Problem: not every application fits this model

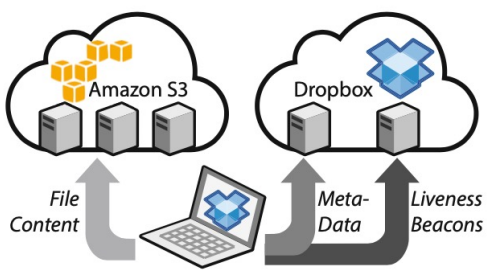
2. Move computation to clients

3. Distribute data and computation across multiple resources
   - Divide et impera: partition data and computation into smaller parts and distribute them across multiple DS resources
   - E.g., decentralized naming service (DNS), data-intensive distributed computation (Hadoop MapReduce and Spark)

Scaling techniques

4. Replicate DS resources and data
   - Make copies of data available at different DS nodes
   - Distribute processing on multiple DS nodes
   - Examples:
     - Distributed file systems and databases
     - Replicated Web servers
     - Web caches (in browsers and proxies)

- Example: in a cloud storage service (e.g., Dropbox, OneDrive, GDrive) data are locally cached on your device and replicated across multiple cloud servers



The problem with replication

- Applying replication is easy, but
- Having multiple copies leads to inconsistency: modifying one copy makes that copy different from the rest
- Trade-off: depending on application type, a certain degree of inconsistency can be tolerated
  - Blog, shared file, shopping cart, on-line auction, air traffic control
- We will study different consistency models to choose from

Challenge 5: Dependability

- **Dependability** refers to the degree that a computer system can be relied upon to operate as expected
  - partial failures make it intricate for distributed systems
- Requirements related to dependability
  - **Availability**: readiness for usage
  - **Reliability**: continuity of service delivery
  - **Safety**: very low probability of catastrophic consequences
  - **Maintainability**: how easily a failed system can be repaired

Dependability: availability

Can I use the system now?

- System is ready to use (i.e., operational) immediately
- Availability $A(t)$ of component C: probability that C is functioning correctly at time t

- Availability = uptime / (uptime + downtime)

- Normally expressed as number of 9's https://uptime.is/
  - A = 99%: two nines

    1-A=1% average days per year

    downtime per year = 0.01*365.2425 d = 3d 15h 39m 29.5s
  - A = 99.99%: four nines

    downtime per year = 0.0001*365.2425 d = 52 m 35.7 s

Dependability: availability

- Metrics

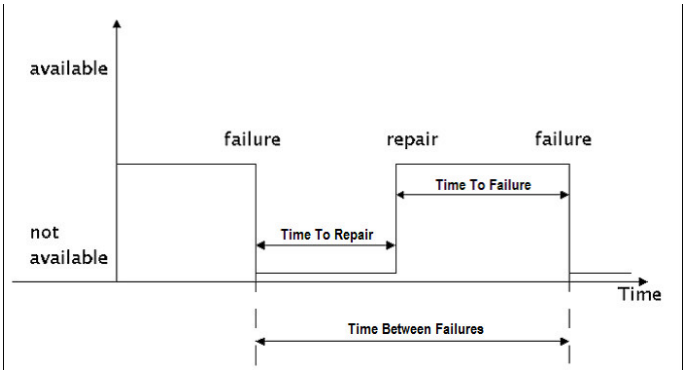
A = MTTF/(MTTF + MTTR)

Mean Time To Failure (MTTF): average time until a component fails

Mean Time To Repair (MTTR): average time needed to repair a component

Mean Time Between Failures (MTBF): MTTF + MTTR

MTBF = total operating time / number of failures



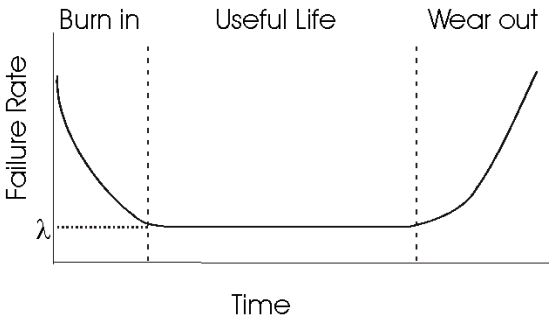
Dependability: reliability

Will the system be up as long as I need it?

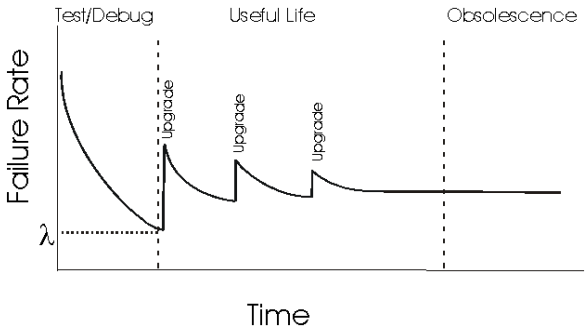
- System will run continuously without failure
- Reliability $R(t)$ of component C: conditional probability that C has been functioning correctly during $[0,t)$ given C was functioning correctly at the time $T = 0$

- Metrics: MTTF (and failure rate)

Bathtub curve for hardware



Revised bathtub curve for software



Availability vs reliability

- Availability ≠ reliability (when the system is repairable)

- Example A: system that goes down for 1 ms every hour
  - Highly available: > 99,9999% (= 1 - 1/(3600*1000))
  - Unreliable, because MTBF = 1 hour and there are 24*365=8780 failures per year

- Example B: system that never crashes but is shutdown for 2 weeks every year
  - Highly reliable, because MTBF = 1 year and there is only 1 failure per year
  - But only 96% available (= 1 - 14/365)

Failure, error and fault

- **Failure**: the system (or a component) is not living up to its specifications
  - Example: software crashes or produces wrong result
  - Can be cascading
- **Error**: wrong internal state, may lead to failure
  - Example: memory stores wrong value
- **Fault**: cause of an error
  - Example: memory is damaged
  - Can be transient, intermittent, permanent
- Relationship: fault $\rightarrow$ error $\rightarrow$ failure

Dependability: tools

- **Fault prevention**
  - Prevent fault occurrence
- **Fault tolerance**
  - Build component and make it mask fault occurrence
- **Fault removal**
  - Reduce presence, number, or seriousness of faults
- **Fault forecasting**
  - Estimate current presence, future incidence, and consequences of faults

Challenge 6: Security

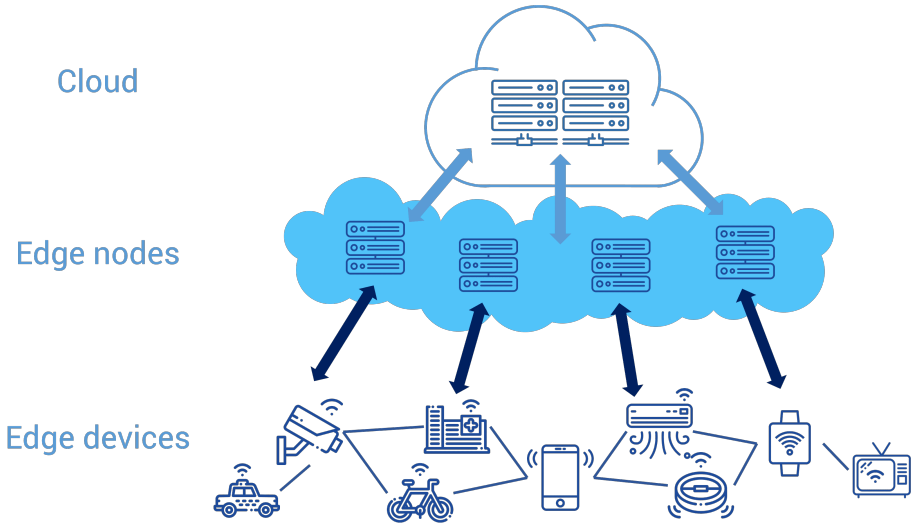
- A distributed system that is not secure, is not dependable
- What we need
  - **Confidentiality**: information is disclosed only to authorized parties
  - **Integrity**: ensure that alterations to system assets can be made only in an authorized way
- Authorization, authentication, trust
  - Authentication: verifying correctness of claimed identity
  - Authorization: does an identified entity has proper access rights?
  - Trust: one entity can be assured that another will perform particular actions according to a specific expectation

Security mechanisms

- Keeping it simple: it is all about encrypting and decrypting data using security keys
- Symmetric vs asymmetric cryptosystem
  - Symmetric: same encryption and decryption key, both keys should be kept secret
  - Asymmetric: public key and private key
- Secure hashing
  - In practice, we use secure hash functions: $H(data)$ returns a fixed-length string
    - Any change from *data* to *data*$*$ will lead to a completely different string $H(data*)$
    - Given a hash value h, it is computationally infeasible to find a *data* with $h = H(data)$

Edge computing

- Brings computation and storage at the network edges, in proximity of data producers (e.g., IoT devices) and consumers (e.g., users)



Distributed pervasive systems

- Distributed systems where computing is embedded across many devices, which are
  - often small, mobile, embedded, battery-powered
  - often spread out geographically
  - coordinated to work together seamlessly
    - The system naturally blends into the user's environment
- Sub-categories:
  - Ubiquitous computing systems: pervasive and continuously present (continuous interaction with users), e.g., smart home system
  - Mobile computing systems: pervasive with mobile devices, e.g., fleet of drones for emergency management
  - Sensor networks: pervasive, that work together to sense and monitor data about environmental or physical conditions, e.g., air quality

Pitfalls in realizing distributed systems

- Many distributed systems are needlessly complex because of errors in design and implementation that were patched later
- Common wrong assumptions by architects and designers of distributed systems ("The Eight Fallacies of Distributed Computing", Peter Deutsch, 1991-92):
  1. The network is reliable
     - "You have to design distributed systems with the expectation of failure" (Ken Arnold)
  2. Latency is zero
     - Latency is more problematic than bandwidth
     - "At roughly 300,000 km/s, it will always take at least 30 ms to send a ping from Europe to the US and back, even if the processing would be done in real time." (Ingo Rammer)
  3. Bandwidth is infinite

Pitfalls in realizing distributed systems

4. The network is secure
5. The topology does not change
   - That's right, it doesn't - as long as it stays in the test lab!
6. There is one administrator
7. Transport cost is zero
   - Going from the application level to the transport level is not free
   - Costs for setting and running the network are not free
8. The network is homogeneous

Technology is not the solution to everything!

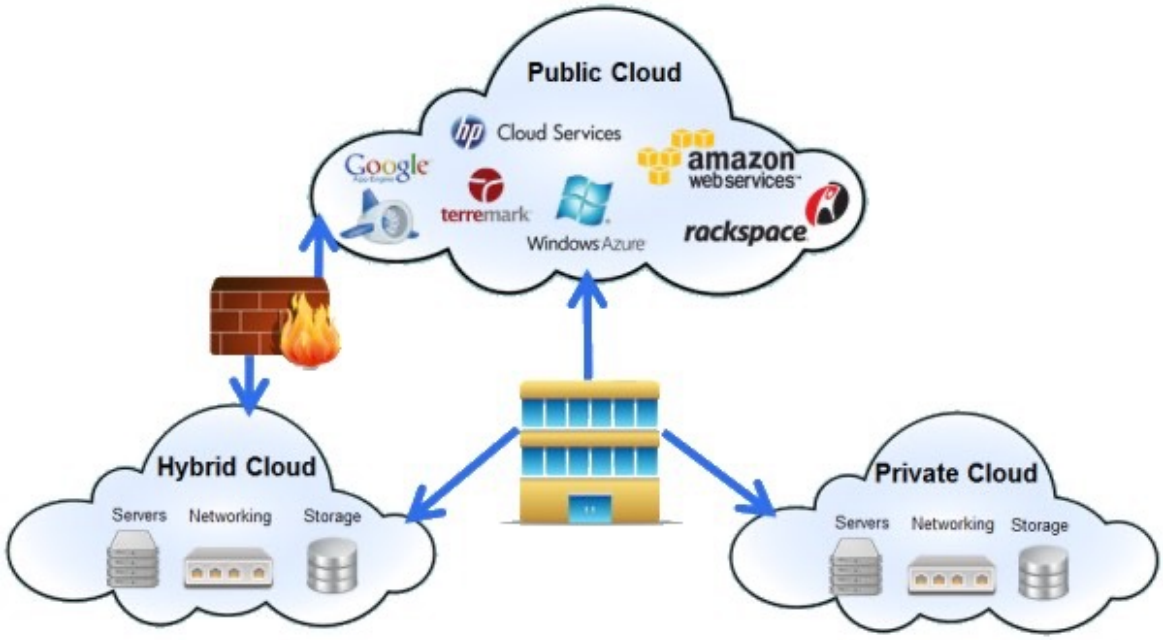
… sharing some essential characteristics

- **On-demand self-service**
  - Cloud resources can be provisioned on-demand by users, without requiring interactions with cloud provider
- Broad **network access**
  - Cloud resources accessed over Internet using standard access mechanisms that provide platform-independent access
  - Published service interface/API
- **Elasticity**
  - Ability for customers to quickly request, receive, and later release as many resources as needed
  - Cloud resources can be provisioned rapidly and elastically: they can be rapidly scaled out/in based on demand

… that share some essential characteristics

- Resource **pooling**
  - Cloud resources are pooled to serve multiple users using multi-tenancy
  - Multi-tenancy: multiple users served by the same physical hardware
- Resource **virtualization**
  - Resources: storage, processing, memory, network bandwidth, and even data centers
- **Pay-per-use** pricing model
  - No large up-front acquisition cost
- **Measured** service
  - Usage of cloud resources is measured and user is charged based on some specific metric

Cloud deployment models



Deployment models: public

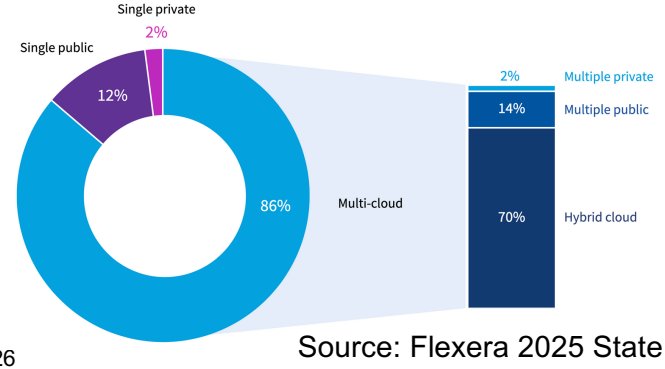
- Public cloud
  - Cloud infrastructure: provisioned for open use by the general public (*multi-tenant*)
  - Owned, managed, and operated by a business, academic, or government organization, or some combination of them
  - Exists on premises of cloud provider
  - Services can be free or fee-based

Deployment models: private

- Private/enterprise cloud
  - Cloud infrastructure: provisioned for exclusive use by a single organization (*single-tenant*) comprising multiple consumers (e.g., business units)
  - Owned, managed, and operated by the organization, a third party, or some combination of them
  - Exists on- or off-premises of cloud provider
  - ✓ Stronger security, customization
  - ✗ Lower economic advantages, scalability more laborious

Deployment models: hybrid

- Hybrid cloud
  - Mixed use of private and public clouds
  - Cloud infrastructure: composition of two or more distinct cloud infrastructures (private or public) that
    - remain unique entities
    - but are bound together by standardized or proprietary technology that enables data and application portability
  - The most popular deployment model



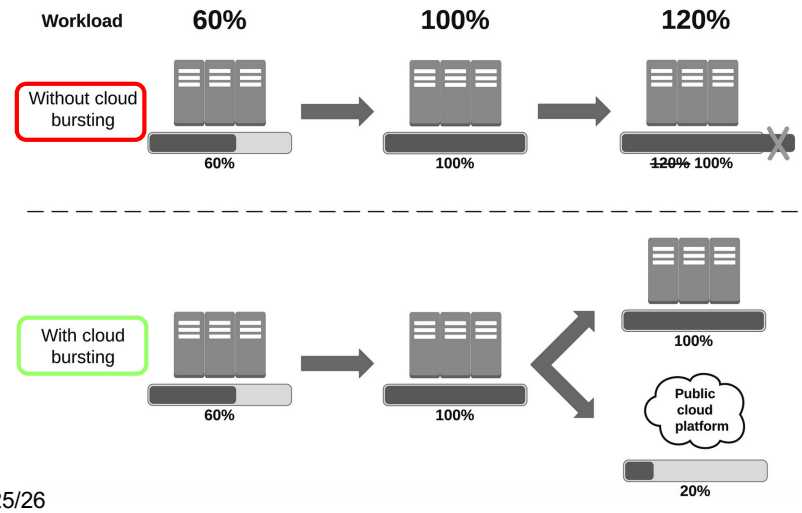
Source: Flexera 2025 State of Cloud Report

Deployment models: hybrid

- Pros
  - ✓ Balance resources and costs
  - ✓ Increase flexibility
    - Differentiate privacy, e.g., sensitive data is kept in private cloud
    - Differentiate services: apps are siloed on different clouds
    - Improve availability: public cloud for disaster recovery in case of unexpected outage
    - Improve geographic distribution and compliance: public cloud resources in different geographic regions
    - Improve performance: by means of cloud bursting
  - ✓ Reduce vendor lock-in risks
    - Vendor lock-in happens when someone is forced to continue using a service regardless of quality, because switching away from that service is not practical

Hybrid cloud: cloud bursting

- Use hybrid cloud dynamically to manage workload spikes when private cloud capacity is insufficient
  - Workload runs in a private cloud which reaches its peak capacity
  - Use public cloud to handle workload spikes without maintaining excess capacity on-premise



Service models: IaaS

- Infrastructure as a Service (IaaS)
  - Compute, storage and network resources as services
  - Customer capability
    - To provision fundamental computing resources (processing, storage, networks) where consumers deploy and run arbitrary software (including OS and apps)
  - Customer control or management
    - No control over underlying cloud infrastructure
    - Control over OS, storage, deployed apps
    - Limited control of selected network components (e.g., firewalls)

IaaS: examples

- Amazon Web Services (AWS): e.g., EC2, S3
- Google Cloud: e.g., Compute Engine, Cloud Storage
- Alibaba Cloud: e.g., Elastic Compute Service
- Aruba Cloud: e.g., Virtual Data Center
- CloudSigma
- DigitalOcean
- IBMCloud : e.g., Cloud Virtual Servers, Cloud Object Storage
- Microsoft Azure: e.g., Virtual Machines, Storage

IaaS: AWS EC2

- Let us launch a virtual machine on EC2 from AWS Management Console
- Few steps in 2-3 minutes:
  1. Sign in to AWS Management Console https://signin.aws.amazon.com/
  2. Open Amazon EC2 console by choosing EC2 under Compute
  3. Choose AWS Region for instance deployment
  4. Choose Launch Instance
  5. Choose software configuration (operating system and other sw) from available Amazon Machine Images (AMIs)
     - AMI provided by AWS, user community, or AWS Marketplace
  6. Choose resource configuration
     - T-shirt sizes: various combinations of CPU, GPU, memory, storage, and networking capacity
  7. If not yet configured, select security group to open a specific network port (e.g., SSH) to access the instance
  8. Select an existing key pair or create a new one to securely connect to the instance after it launches

Service models: PaaS

- Platform as a Service (PaaS)
  - Allows customers to develop, build, run and scale applications, without the complexity of building and maintaining the underlying infrastructure
  - Customer capability
    - To develop, deploy and test onto the cloud (consumer-created or consumer-acquired) apps realized using programming languages, application frameworks, development tools supported by PaaS provider
  - Customer control or management
    - No control over underlying cloud infrastructure (network, servers, OS, storage)
    - Control over deployed apps and possibly application hosting environment configurations

PaaS: examples

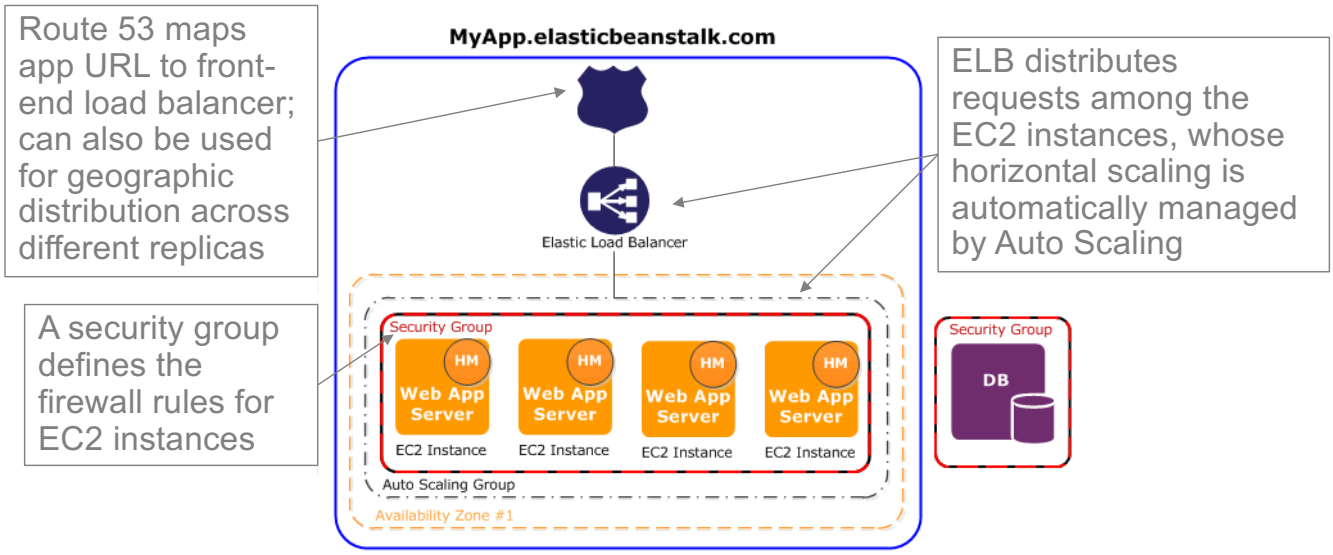
- Platforms for Web apps
  - AWS Elastic Beanstalk, Google App Engine, Azure App Service
- Platforms for building and deploying applications in containers and Kubernetes
  - AWS Elastic Container Service, Google Kubernetes Engine, Azure Kubernetes Service
- Platforms for entire application lifecycle
  - Include tools for CI/CD, container orchestration, and integration with various services
  - Heroku, Red Hat OpenShift, VMware Tanzu
- Salesforce Platform
  - Allows developers to build, deploy, and manage custom applications within Salesforce ecosystem
- Serverless computing/Function as a Service (FaaS)
  - AWS Lambda, Google Cloud Functions, Alibaba Function Compute

PaaS: AWS Elastic Beanstalk

- Supports applications developed in Go, Java, .NET, Node.js, PHP, Python, and Ruby
- Allows user to deploy and manage applications without having to learn about the infrastructure
- Reduces management complexity
  - User uploads the application source bundle (e.g., .war file) to Elastic Beanstalk and provides some information about the application
  - Elastic Beanstalk automatically launches an environment and creates and configures the AWS resources needed to run the code, handling the details of capacity provisioning, load balancing, scaling, and application health monitoring

PaaS: AWS Elastic Beanstalk

- Example of Elastic Beanstalk architecture
  - AWS resources (ELB, Auto Scaling group, EC2 instances) are automatically provisioned by Elastic Beanstalk



https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts-webserver.html

IaaS vs. PaaS

- Using IaaS
  - "Pure" virtualized resources: CPU, memory, storage, load balancer, ...
  - OS included
  - "T-shirt" size
- Using PaaS
  - Virtualized resources plus application framework
  - Additional services, e.g., horizontal scaling without having to configure it
  - ✗ Constraints on application and data architecture
  - ✗ Increased risk of vendor lock-in

IaaS and PaaS providers

- Main providers: AWS, Microsoft Azure, Google Cloud, IBM Cloud, Oracle, Alibaba
- Gartner's magic quadrant for strategic cloud platform services (2024)

Figure 1: Magic Quadrant for Strategic Cloud Platform Services

Gartner

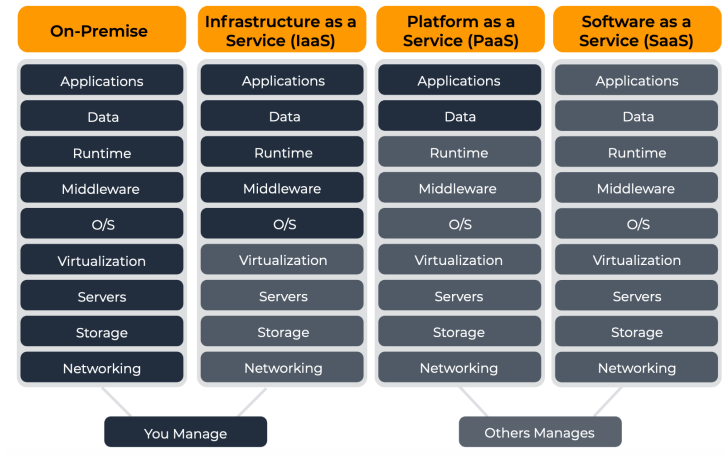
Service models: SaaS

- Software as a Service (SaaS)
  - Applications made available to customers over Internet, still the largest market
  - Customer capability
    - To use SaaS provider's applications running on a cloud infrastructure
    - Applications accessible from various client devices through Web or provider APIs
  - Customer control or management
    - No control over underlying cloud infrastructure
      - Network, servers, operating systems, storage, or even individual application capabilities
    - Possible exception: limited user-specific application configuration settings

SaaS: examples

- Communication & collaboration: Adobe Connect, Cisco WebEx, Google Mail, Zoom
- Personal productivity: Google Calendar, Google Drive, Microsoft 365
- File storage and sharing: Dropbox, OneDrive, iCloud, SugarSync
- CRM (Customer Relationship Management): salesforce.com
- Enterprise-level (accounting, ERP, HR, marketing, sales): NetSuite, SAP Business ByDesign, Workday, Zoho

Service models: wrapping up



- Additional service models, e.g.:
  - Database as a Service (DBaaS)
  - Function as a Service (FaaS)
  - Monitoring as a Service (MaaS)
  - … Anything as a Service (XaaS)

Cloud pricing models

- Cloud providers offer a number of pricing models

- Pay-per-use / on-demand
  - Users are charged based on cloud resources usage
  - Pricing granularity depends on service, e.g., by hour for AWS EC2, by ms for AWS Lambda
  - ✓ Maximum flexibility: no need to plan in advance

- Fixed / reserved
  - Users are charged a fixed amount per month for cloud resources
  - ✓ Significant discount compared to on-demand pricing
  - ✗ Upfront payment

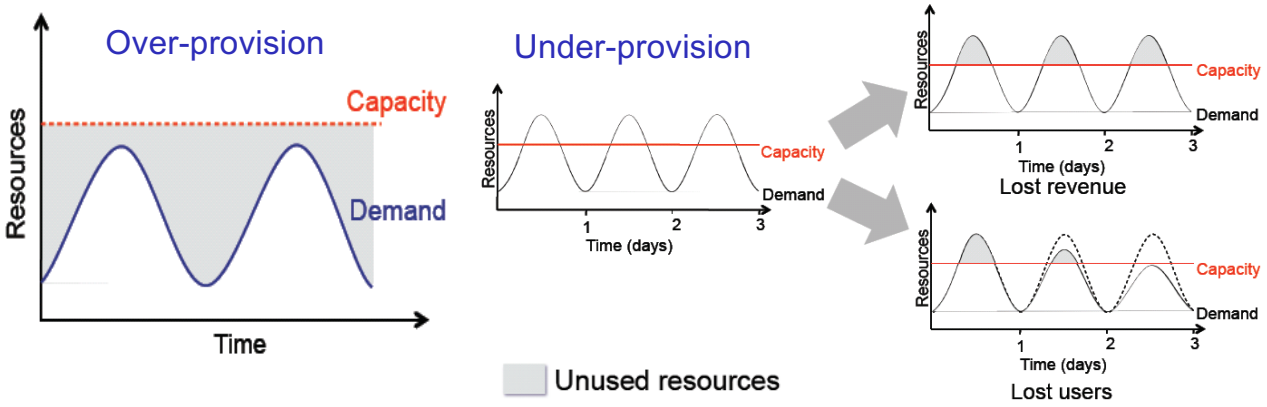
Cloud pricing models

- Dedicated
  - Physical resources (e.g., physical server) rather than virtual ones fully dedicated to user
  - ✓ Dedicated hardware, maximum isolation
  - ✗ Higher cost

- Spot
  - Variable pricing for cloud resources driven by market demand (supply and demand)
  - ✓ Substantial savings
    - E.g., AWS EC2 spot instances can be acquired at up a 90% discount compared to on-demand pricing https://aws.amazon.com/ec2/spot/
  - ✗ Interruptible by cloud provider
    - 👉 use for fault-tolerant workloads

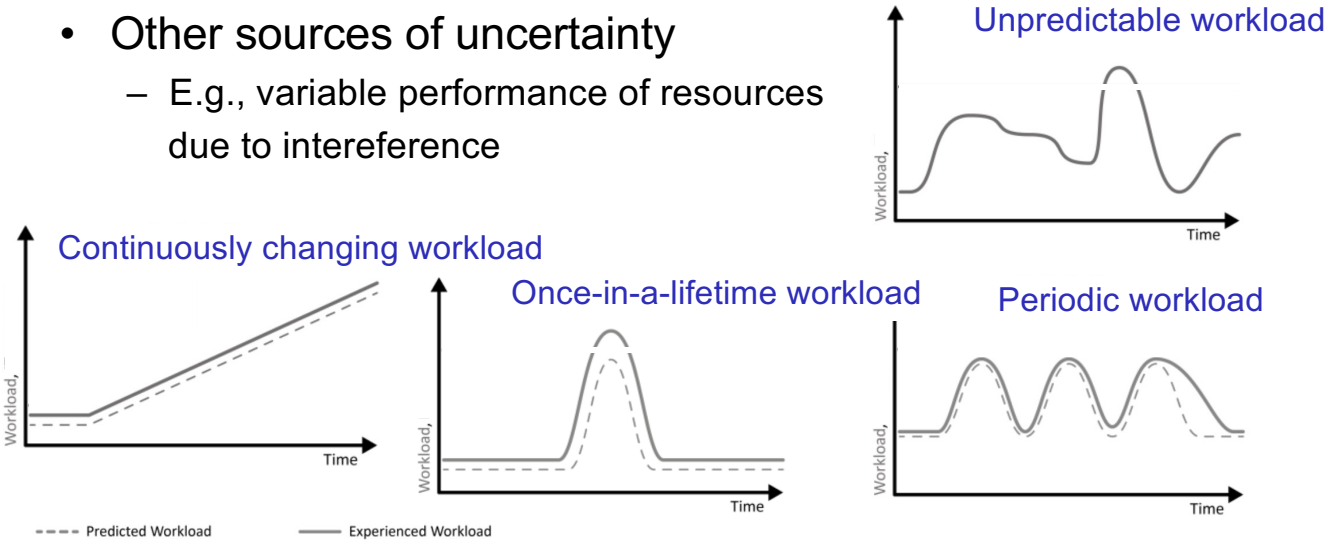
Capacity planning and resource provisioning

- Capacity planning: determines right sizing (of each tier) of application deployment in terms of:
  - Number of IT resources and their size
  - IT resources: computing, storage, memory and network
- Static provisioning: capacity is pre-provisioned
  - ✗ Excess capacity (over-provision based on peak load) or capacity lack (under-provision)



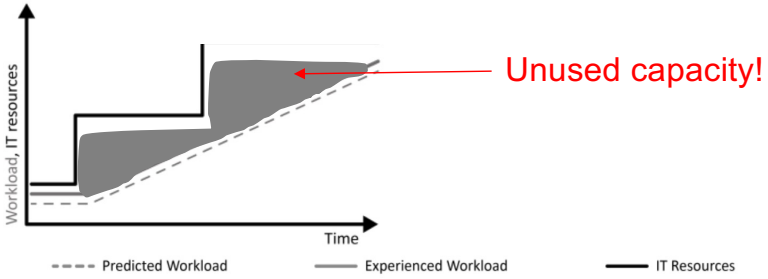
To make things more complicated: uncertainty

- Fluctuating workloads with different patterns
- Other sources of uncertainty
  - E.g., variable performance of resources due to intereference



A solution…

- Forecast demand
- Next, manually provision resources dynamically by scaling in/out or scaling up/down IT resources as needed

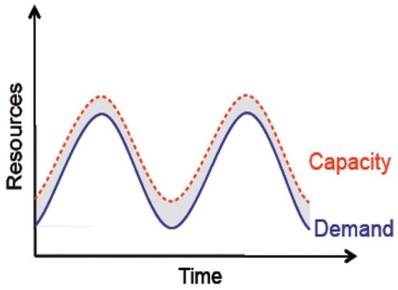


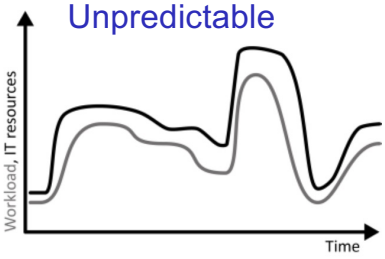
- Manual scaling: unpractical in cloud environments
  - Takes time, prone to human errors, inefficient, …

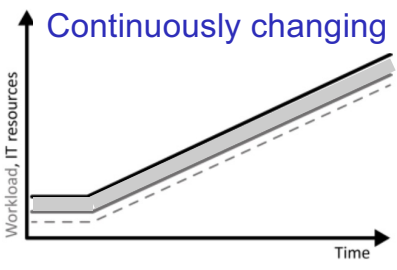
👉 Automated scaling is better!

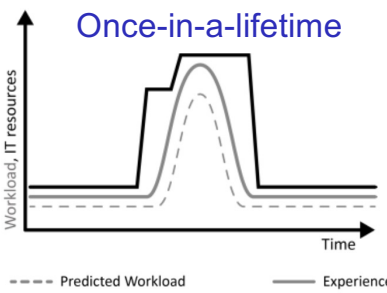
The solution: elastic scaling (aka auto-scaling)

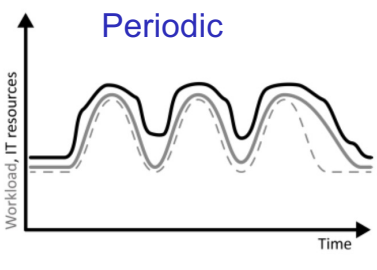
- Let's automatically align IT resources to changing workloads





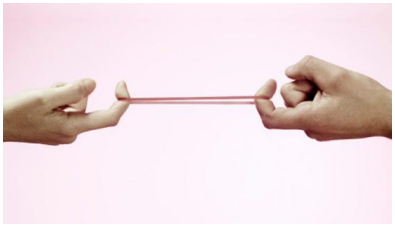






Elasticity

- (Physics) Property of returning to an initial form or state following deformation
  - Stretch when a force stresses it
  - Shrink when the stress is removed

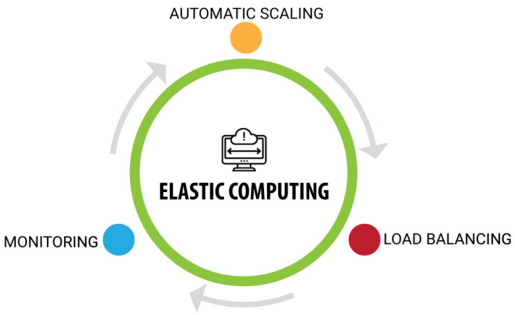


- (Computer science) Degree to which a computing system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible

Herbst et al., Elasticity in Cloud Computing: What It Is, and What It Is Not, ICAC 2013

Elasticity and Cloud computing

- Elasticity: a key feature of Cloud computing
- How do we achieve elastic computing?
- We need a self-adaptive software system

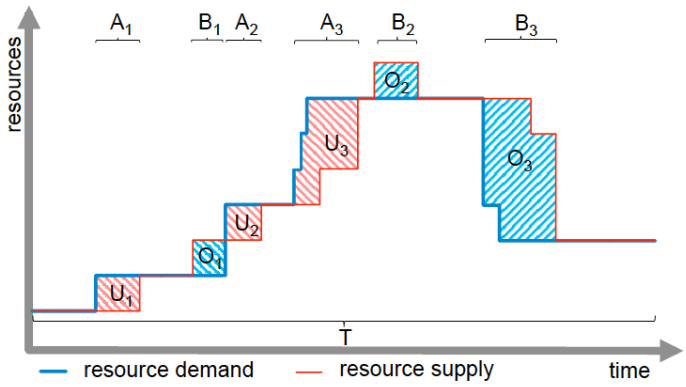


*In any system of sufficiently large scale, **automation** is not only necessary, but it is cheap.*

Ousterhout, Is scale your enemy, or is scale your friend?: technical perspective, Comm. ACM, 2011
https://dl.acm.org/doi/pdf/10.1145/1965724.1965748

Elasticity: how to measure

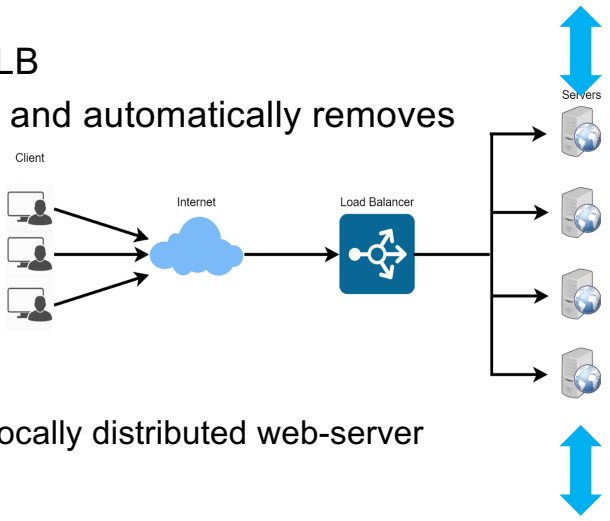
- Consider over-provision and under-provision



- Elasticity metrics:
  - Accuracy: sum of areas of over-provision (*O*) and under-provision (*U*) over the measurement period *T*
  - Timing: total amount of time spent in over-provision (*B*) and under-provision (*A*) over the measurement period *T*

Elasticity: architectural point of view

- Distribute requests among multiple server replicas using a *server-side* **load balancer** (LB)
  - A single external IP address (VIP) assigned to LB
  - Network (layer 4) or application (layer 7) LB
  - One-way LB (responses from replicas go directly to clients) or two-way LB (responses go through LB)
  - Hardware or software LB
  - Centralized or distributed LB
  - Health checks: LB detects and automatically removes unhealthy replicas



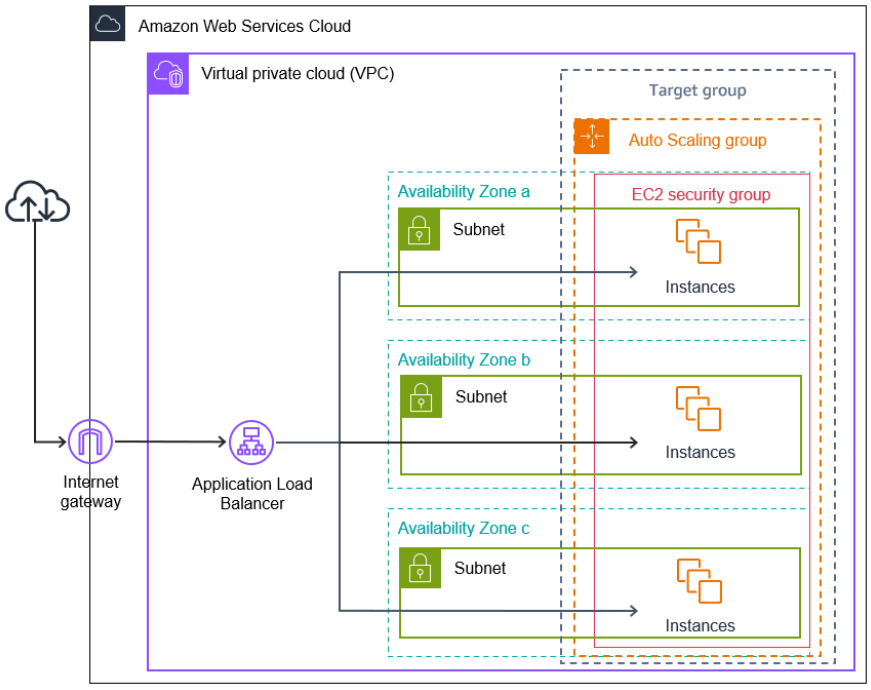
Cardellini et al., The state of the art in locally distributed web-server systems, ACM Comput. Surv., 2002

Elasticity: architectural point of view

- **Load balancing goals**
  - Maximize resource utilization
  - Minimize response time
  - Maximize throughput
- **Load balancing policies**
  - Stateless: Random, Round Robin
  - Server-aware: Least Loaded, Weighted Round Robin (assign weights proportional to server capacity and distribute requests in a circular way according to weights), Power of Two Choices (select randomly two servers and choose the least loaded of the two), …
  - Client-aware (depends on layer): Hashing, Session Stickiness, …
  - Client & server-aware: combination of client and server info
- Some products: Envoy https://www.envoyproxy.io, nginx https://nginx.org/en/docs/http/load_balancing.html

Elasticity: architectural point of view

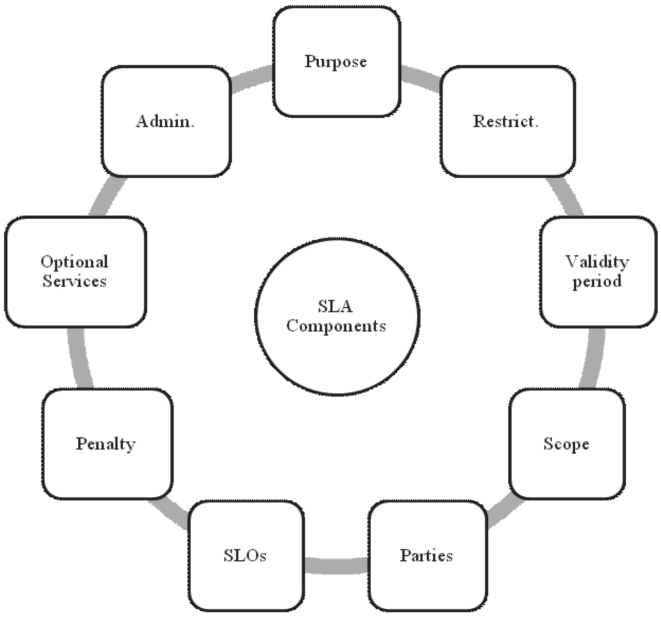
- Example of load balancing and auto-scaling using AWS



https://docs.aws.amazon.com/autoscaling/ec2/userguide/tutorial-ec2-auto-scaling-load-balancer.html

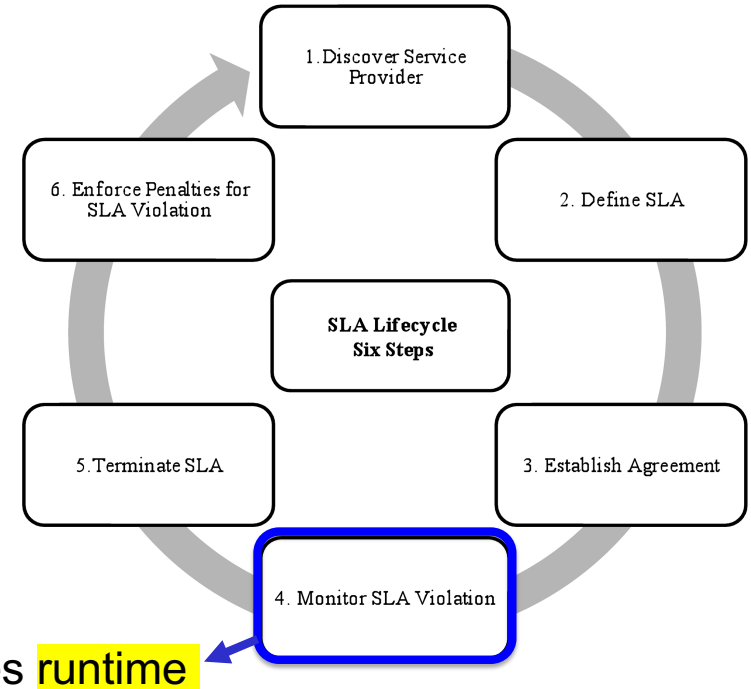
Service Level Agreement (SLA)

- Formal agreement (contract) between a provider and a consumer of a service
- Composed of one or more Service Level Objectives (SLOs):
  - SLO: condition on a measure of a specific metric (e.g., mean response time <= 1 s)
- Includes penalty and/or compensation in case of SLA violation



Source: Sun Microsystems Internet Data Center Group

SLA life cycle



Requires runtime monitoring of service

SLI and SLO

- Service Level Indicator (SLI)
  - A measureable metric
  - Examples: availability, response time, throughput
  - Availability is the most common SLI for cloud services

- Service Level Objective (SLO)
  - A predicate over a set of SLIs
  - Example: a fixed threshold on a single SLI
  - Example SLO:

monthly uptime percentage for a VM is at least 99.99%

SLI — SLO

Example: Amazon Compute SLA

- **Service Commitment** https://aws.amazon.com/compute/sla/?nc1=h_ls

  For each individual Amazon EC2 instance (“Single EC2 Instance”), AWS will use commercially reasonable efforts to make the Single EC2 Instance available with an Instance-Level Uptime Percentage of at least 99.5%, in each case during any monthly billing cycle (the “Instance-Level SLA”). In the event any Single EC2 Instance does not meet the Instance-Level SLA, you will be eligible to receive a Service Credit

Instance-Level Uptime Percentage	Service Credit Percentage
Less than 99.5% but equal to or greater than 99.0%	10%
Less than 99.0% but equal to or greater than 95.0%	30%
Less than 95.0%	100%

  Monthly Uptime Percentage 99.5% = 3h 39m 8s of downtime per month

  Monthly Uptime Percentage 99% = 7h 18m 17s of downtime per month

- **Service Credits**

  Service Credits are calculated as a percentage of the monthly bill… We will apply any Service Credits only against future payments… To receive a Service Credit, you must submit a claim…

Example: Amazon S3 SLA

https://aws.amazon.com/s3/sla/?nc1=h_ls

- **Service Commitment**

  AWS will use commercially reasonable efforts to make Amazon S3 available with a Monthly Uptime Percentage during any monthly billing cycle. In the event Amazon S3 does not meet the Service Commitment, you will be eligible to receive a Service Credit as described below.

  Monthly Uptime Percentage 99.9% = 43 min 49 sec of downtime per month

- **Definitions**
  - "Error Rate" means, for each request type to an Amazon S3 storage class: (i) the total number of internal server errors returned by Amazon S3 for such request type to the applicable Amazon S3 storage class as error status "InternalError" or "ServiceUnavailable" divided by (ii) the total number of requests for such request type during the applicable 5-minute interval of the monthly billing cycle. We will calculate the Error Rate for each AWS account as a percentage for each 5-minute interval in the monthly billing cycle. The calculation of the number of internal server errors will not include errors that arise directly or indirectly as a result of any of the Amazon S3 SLA Exclusions.
  - "Monthly Uptime Percentage" is calculated by subtracting from 100% the average of the Error Rates from each 5-minute period in the monthly billing cycle. If you did not make any requests in a given 5-minute interval, that interval is assumed to have a 0% Error Rate.

Issues with cloud SLAs

- Another example: https://cloud.google.com/compute/sla
- Issues related to cloud SLAs:
  - SLA jargon: confusing terminology
  - Availability
    - Averaging over a period can obscure the difference between many short outages vs. a few long ones
  - Lack of SLIs for response time
  - Service credits: only applicable to future payments
  - SLA violation detection: burden of detecting violations often on cloud customer
  - Time period for reporting SLA violations
  - Non-negotiable (or customizable) SLA for most users
  - Difficulty in comparing SLAs across different cloud providers

Cloud monitoring

- Goal: continuously track and assess health of systems and services deployed within cloud
  - Ensure that any performance issues, outages, or inefficiencies are quickly detected and addressed to maintain optimal system performance and availability
- Requires tools to collect and analyze data on system-oriented and application-oriented metrics from cloud resources and services
- System-oriented metrics

Type	Metrics
CPU	CPU utilization
Disk	Disk utilization, throughput (MB/sec for read/write, operations/sec)
Memory	Memory-Used, Memory-Free, Page-Cache
Interface	Throughput (Mbps incoming/outgoing)

- Application-oriented metrics: response times, error rates, transaction volumes, user interactions, …

Cloud monitoring: services and tools

- Examples of monitoring services offered by cloud providers
  - Amazon CloudWatch https://aws.amazon.com/cloudwatch
    - Provides monitoring for AWS cloud resources and applications, including system-level and applications metrics
    - Supports alarms, automated responses, and detailed logging for troubleshooting
  - Google Cloud Operations Suite https://cloud.google.com/monitoring
- Examples of third-party tools
  - Prometheus https://prometheus.io
    - Open-source monitoring tool, collects and stores metrics in a time series DB, which can then be queried for real-time analysis
  - Datadog https://www.datadoghq.com
  - Dynatrace https://www.dynatrace.com

Outside a cloud data center

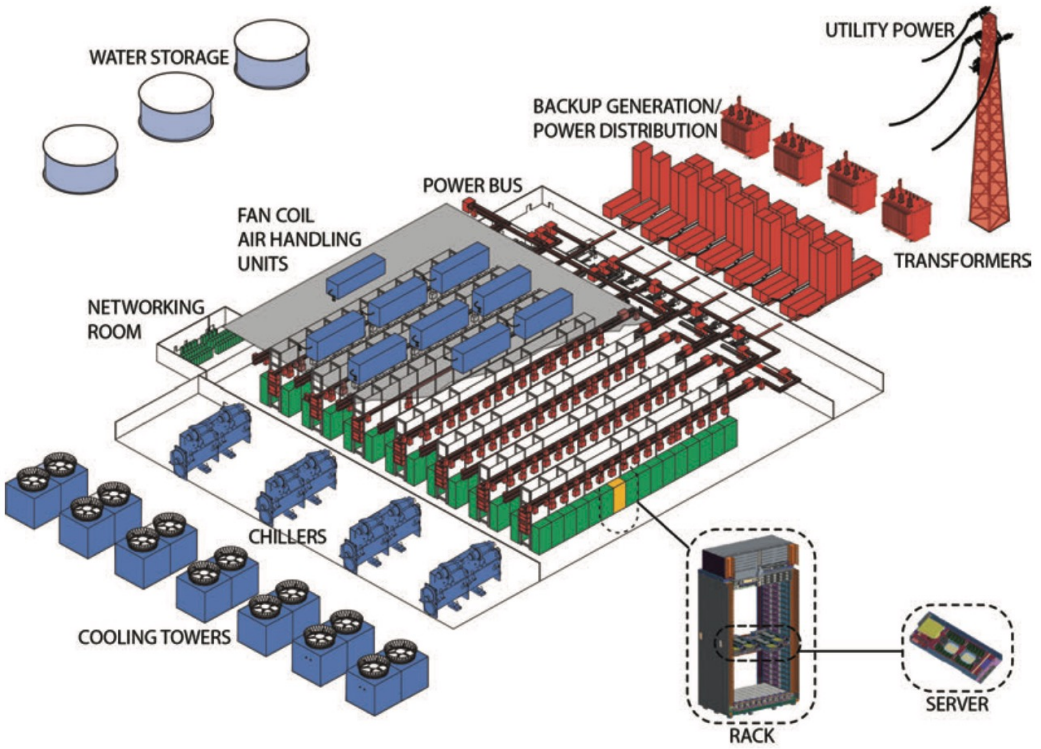
Cooling towers

- How large? ~100000 m^2
- How many servers? ~10^6
- Power-efficient?

Power Usage Effectiveness (**PUE**) = Facility power / IT equipment power

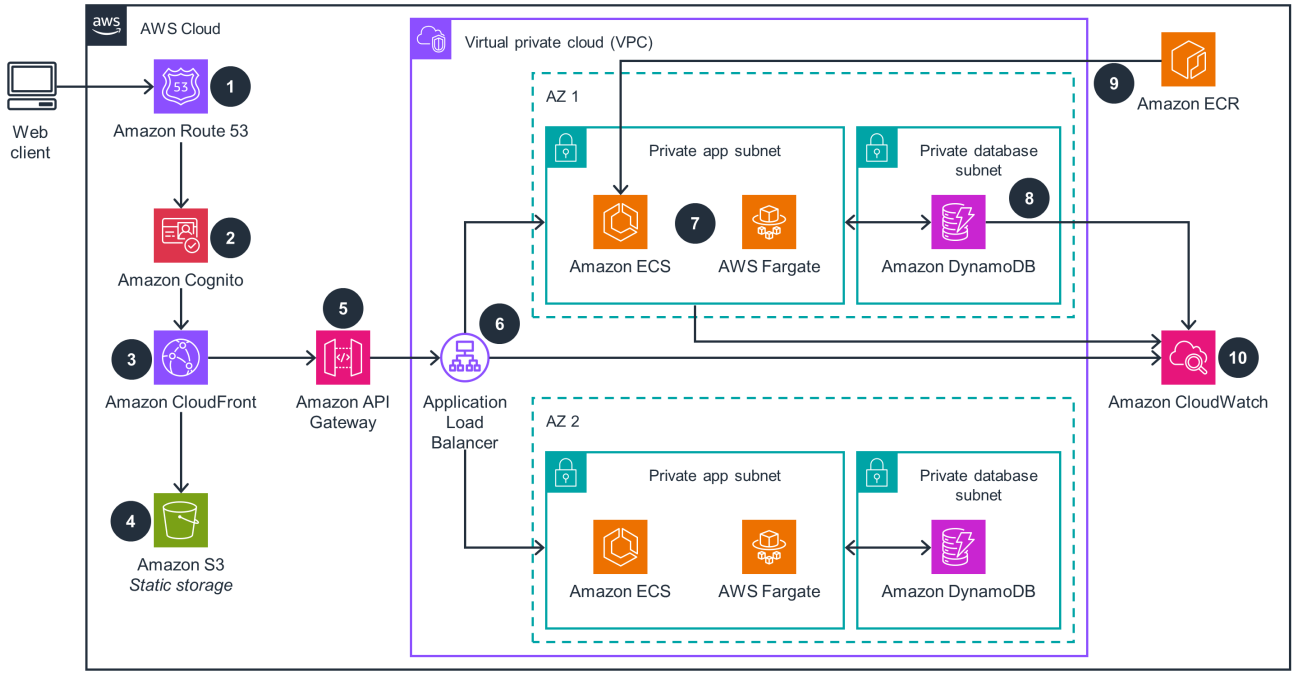
  - E.g., average PUE for Google data centers is 1.1

Main components of a cloud data center



Example of web application in cloud

- A scalable, secure 3-tier web application using containerization on AWS



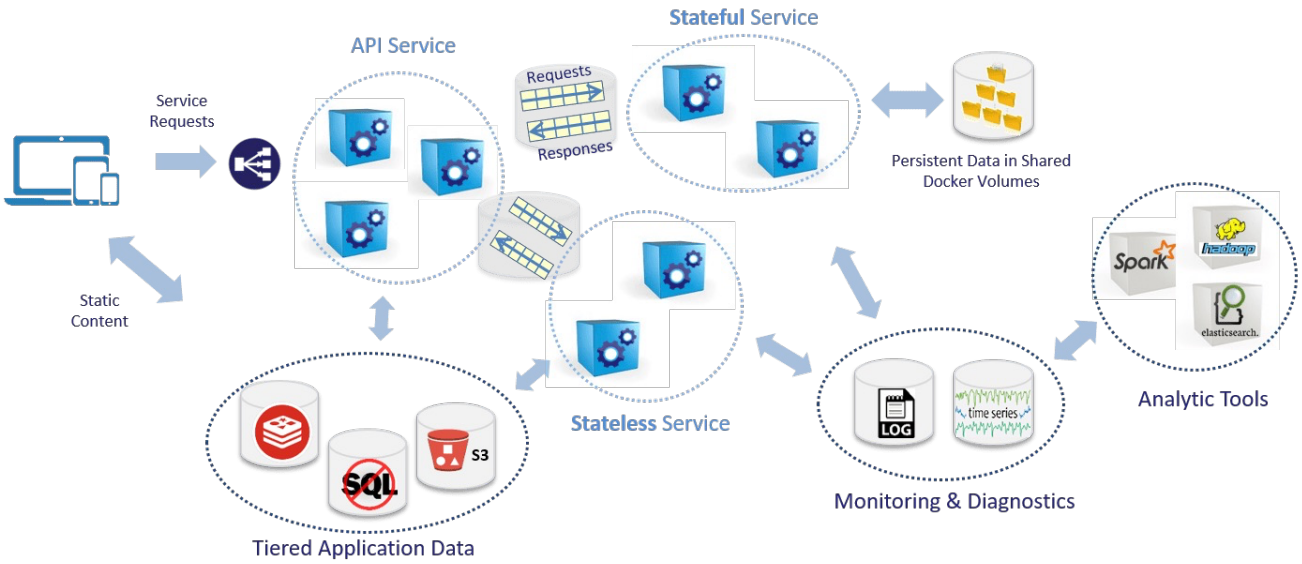
https://aws.amazon.com/it/solutions/guidance/building-a-containerized-and-scalable-web-application-on-aws

Cloud-native applications

- Cloud native: sw approach of building, deploying, and managing modern applications in cloud environments
- Cloud-native applications are a collection of small, independent, and loosely coupled services
- Goals: agile, highly scalable, flexible, and resilient applications
- We will study microservices and serverless applications

Cloud-native applications: high-level example

Stateful service: retain data or context between sessions or interactions



Cloud trends

- AI and ML Cloud services, Container-as-a-Service, serverless are rising
- Public cloud adoption still growing
- Multi-cloud is popular: concurrent usage of multiple clouds
  - Hybrid, multiple private, multiple public

Multi-cloud: challenges

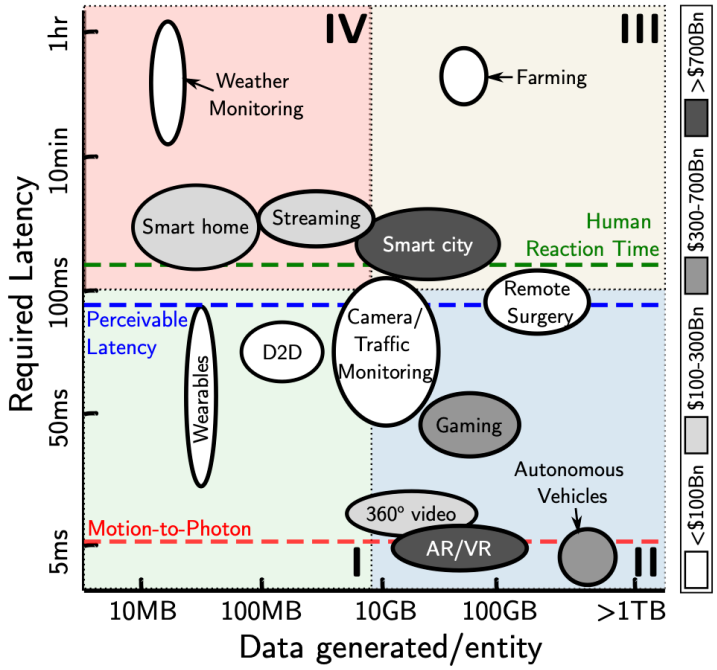
- Multi-cloud management is more complex
  - Managing resources spread across different clouds
  - Automating workload placement, configuration and maintenance (including identity management and data protection/encryption)
  - Monitoring performance, resource usage and costs of multiple infrastructures and applications
  - Assessing changes in service portfolio and pricing of cloud providers (FinOps)
  - Predicting performance, resource usage and costs
- How to deploy multi-cloud applications?
  - Commercial purpose-built products
  - **Infrastructure as Code (IaC)**: tools like Terraform, Ansible, Chef, or Puppet allow you to define cloud infrastructure in code and deploy it consistently across multiple cloud providers

Cloud trends: applications

- Convergence of different technologies, evolving landscape
- Current applications
  - IoT, Big Data, AI and ML
- Emerging applications
  - Generative AI
  - Metaverse: collective virtual 3D shared space, created by the convergence of virtually enhanced physical and digital reality
  - Web3: stack of technologies built on blockchain protocols that support the development of decentralized web applications and empower users to control their own identity, content, and data
  - Quantum computing: tasks like cryptography, optimization, and simulation of complex molecules

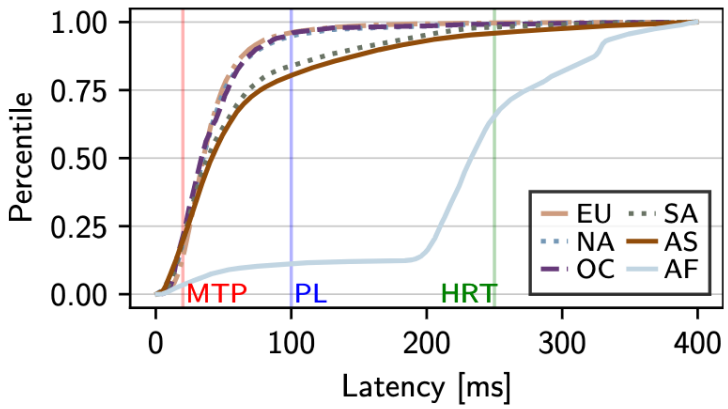
Does cloud suit every application?

- Different classes of applications, different performance requirements



Beyond cloud

- Challenge with latency-sensitive and location-aware applications
  - Apps like real-time manufacturing, self-driving vehicles, drones, cognitive assistants generate massive data volumes
  - Relying on cloud can be impractical due to high network latency
  - See round-trip times (RTTs) to nearest cloud data center



Motion-to-Photon (MTP)
Human Perceivable Latency (PL)
Human Reaction Time (HRT)

Source: Cloudy with a chance of short RTTs: analyzing cloud connectivity in the internet, https://tinyurl.com/3wztmmz6

Computing continuum: challenges

- Infrastructure and application management: interoperability and seamless integration
- Resource heterogeneity (capacity, energy, ...)
- Performance guarantee
- Security and privacy

The Computing Continuum

IoT/Edge — Fog — HPC/Cloud/Instrument

Size	Nano	Micro	Milli	Server	Fog	Campus	Facility
Example	Adafruit Trinket	Particle.io Boron	Array of Things	Linux Box	Co-located Blades	1000-node cluster	Datacenter
Memory	0.5K	256K	8GB	32GB	256G	32TB	16PB
Network	BLE	WiFi/LTE	WiFi/LTE	1 GigE	10GigE	40GigE	N*100GigE
Cost	$5	$30	$600	$3K	$50K	$2M	$1000M

Count = 10^9 Size = 10^1 … Count = 10^1 Size = 10^9

Beckman et al., Harnessing the Computing Continuum for Programming Our World, 2020 https://www.netlib.org/utk/people/JackDongarra/PAPERS/icl-utk-1358-2020.pdf

- Methodologies, programming models, tools and platforms for computing continuum are still in their infancy

Summing up: cloud benefits

IT benefits

- Scalability, elasticity, flexibility
- Anytime and anywhere accessibility
- Improved security
  - E.g., 3.47 Tbps DDoS attack from 10,000 servers successfully stopped in 2022 https://tinyurl.com/bdec45dy
- Organizational and operation agility
- Simplified management
  - E.g., software updates and versioning

Business benefits

- No advanced payment
  - Costs scale with use
  - CapEx to OpEx
- Rapid business innovation
- Increase in productivity

Issues for cloud customers

- Privacy, security and legal issues
  - Control over data: location of data centers, data access
    - Data sovereignty: data are subject to the laws and governance structures of the nation where they are collected
  - Data encryption, also in transit
  - Data integrity, tracing and recovery
  - GDPR compliance (accountability, privacy by design, privacy by default)

- We had “stormy” clouds
  - “You have zero privacy anyway. Get over it.” (Scott McNealy, co-founder of SUN, 1999)
  - “If you have something that you don’t want anyone to know, maybe you shouldn’t be doing it in the first place.... The reality is that search engines do retain information... It could become available later...” (Eric Schmidt, Google CEO, 2009)

Issues for cloud customers

- Now?
  - In Europe: CISPE Code of Conduct
    https://www.codeofconduct.cloud/
  - Be aware of privacy policies
    - "We use the information we collect in existing services to help us develop new ones. For example, understanding how people organized their photos in Picasa, Google’s first photos app, helped us design and launch Google Photos." (Google, 2024)

    https://policies.google.com/privacy
    - "Microsoft uses the data we collect to provide you with rich, interactive experiences. (…) We also use the data to operate our business, which includes analyzing our performance, meeting our legal obligations, developing our workforce, and doing research." (Microsoft, 2024)

    https://privacy.microsoft.com/en-gb/privacystatement

Issues for cloud customers

- Network latency
  - ms latency from users and devices to cloud data centers
  - How to address? Computing continuum

- Cloud portability
  - Customer's ability to move and adapt their applications and data between their own systems and cloud services, or across different cloud providers and deployment models
  - Risks of vendor lock-in
    - E.g., specific features offered by PaaS provider make it difficult to switch to another provider
    - E.g., difficulty in extracting data and migrating it to different provider
  - Towards portability
    - Containers: enable applications to run consistently across different cloud environments
    - Cloud automation tools (IaC tools)

Issues for cloud customers

- Cloud interoperability
  - Ability of public cloud services, private cloud services, and cloud service customer's system to understand and work with each other (compatibility of interfaces, configuration, authentication and authorization, methods data formats, ...)
  - Standards for cloud portability and interoperability
    - ISO/IEC 19941:2017, IEEE 2302-2021 Standard for Intercloud Interoperability and Federation
  - European initiative GAIA-X https://gaia-x.eu

- SLA negotiation and management
  - Lack of SLIs based on end-user experience
  - SLA monitoring often on customers' shoulders

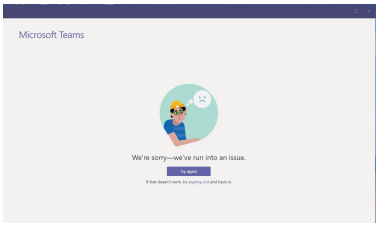
Issues for cloud customers

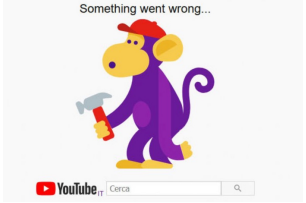
- The two faces of scalability and elasticity
  - Can your application scale?
    - Design application to leverage cloud capabilities
  - Does cloud provider support real elasticity?
    - Automatically provision and de-provision resources on demand, without performance degradation
    - Cloud providers impose limits on resources (e.g., number of VMs, storage size)
    - Stateful applications can face challenges with elasticity because maintaining session persistence and data consistency can be complex

Jiang et al., The Limit of Horizontal Scaling in Public Clouds, ACM TOMPSEC, 2020
https://media.qyjohn.net/blog/2020_Horizontal_Scaling.pdf

Issues for cloud customers

- Cloud outages: examples
  - 2020: AWS Kinesis went down, impacting other AWS services that use it (cascading failure). Cause: one connection per OS thread design led to resource exhaustion https://aws.amazon.com/message/11201
  - 2023: Google Cloud services in Europe hit by outage when Paris' data center caught fire https://thenewstack.io/google-cloud-services-hit-by-outage-in-paris
  - 2024: Microsoft cloud services, including Azure and Teams, experienced a global disruption due to a defective update from CrowdStrike, a cybersecurity company https://tinyurl.com/4tt3ex6p
  - 2025: Google Cloud outage affected Gmail, Drive, Meet, and third-party platforms like Spotify and Discord. Cause: technical failure in Google Cloud's API management system, leading to invalid quota updates https://tinyurl.com/4v7musa8
  - More examples in 2025 https://tinyurl.com/3pveavn6





Issues for cloud providers

- Increasing energy consumption
  - Energy consumption of data centers: 200 TWh in 2016, 460 TWh in 2022, 1000 TWh in 2026
- Uncertainty in service demands
  - Example of provider-side solution: spot instance market to sell unused Amazon EC2 capacity
- SLA management
  - Cascading effect
- Cloud interoperability
  - Interoperable tools with standardized interfaces
- Lack of skilled expertise

Summing up

- Cloud computing is:
  - here to stay and one of the most hyped subjects in IT
  - increasingly becoming an integral concept in IT
  - major trend in the way digital services are designed, implemented and delivered
  - key factor for the digitalization of the whole society
- However, cloud and its evolutions are complex systems
  - Heterogeneous resources, many technologies, plethora of applications with heterogeneous workloads and complex interactions
  - All difficult aspects of large-scale distributed systems
  - Recall: "old" problems become new problems on a totally different scale and open to new solutions

Software vs. system architecture of DS

All'interno dei sistemi distribuiti si distingue l'architettura del software da quella del sistema.

- **Software architecture**: logical organization and interaction of software components in a DS

- **System architecture**: final instantiation (including deployment) of a software architecture
  - Software components need to be placed on system resources
  - E.g., placing a containerized microservice on a machine

Possiamo dire che una volta che ho organizzato i miei container, quando procedo con il deployment sto effettivamente modificando l'architettura del sistema.

- Let's first focus on software architectures for DS

Distributed systems are often complex pieces of software, of which the components are by definition dispersed across multiple machines. To master their complexity, it is crucial that these systems be properly organized. There are different ways on how to view the organization of a distributed system, but an obvious one is to make a distinction between, on the one hand, the logical organization of the collection of software components, and on the other hand the actual physical realization. (libro DS)

Per stile architetturale, si intende come i componenti interagiscono fra loro, come scambiano i dati e come tutti assieme vengono uniti in un unico sistema.

Architectural styles for DS

- Architectural style: set of design decisions concerning sw architecture
  - Defined primarily in terms of components and connectors
- Component
  - Modular unit with well-defined interfaces
  - Replaceable within its environment
- Connector
  - Mechanism enabling interaction among components, handling communication, coordination or cooperation
  - Example: remote procedure call, messaging system
- Also includes:
  - How components are connected
  - What data is exchanged
  - How components and connectors are configured together

Un component deve essere sostituibile senza che il sistema venga spento. Solamente quando un componente necessita di riavvio affinché l'aggiornamento avvenga si possono prendere misure speciali. Tali misure di solito includono che un componente replica che è in standby prenda il suo posto durante la fase di riavvio. (Come visto nel corso di Legato)

appunto RPC come visto alla triennale

Main architectural styles for DS

- Layered style

pensa ad un design che sfrutta chain of responsability quindi il layer N fa una richiesta al layer N-1 e cosi via fino a quando non si risponde

- Service-oriented style
  - Object-oriented
  - Microservices
  - RESTful

- Publish-subscribe style

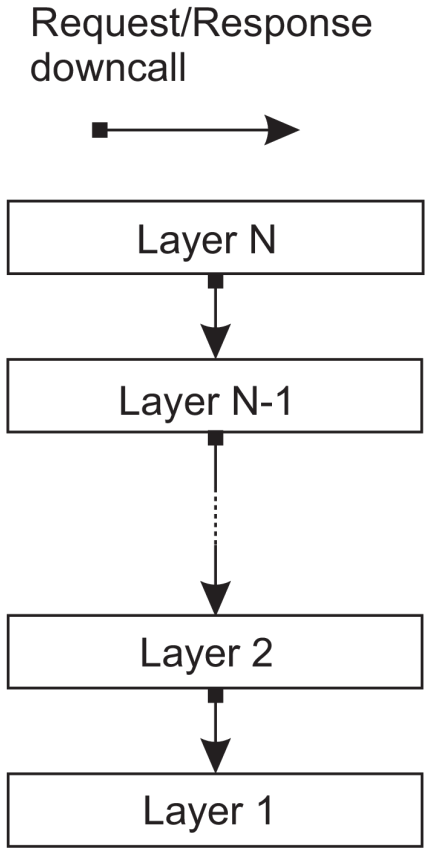
Layered style

- Components are organized in *layers*
- Component at layer *i* invokes component at layer *j (with j<i)*
- Components communicate by message exchange
  - Request/response downcall

✓ Separation of concerns among components

  - E.g., web app based on MVC design

appunto un esempio è il pattern Model View Controller



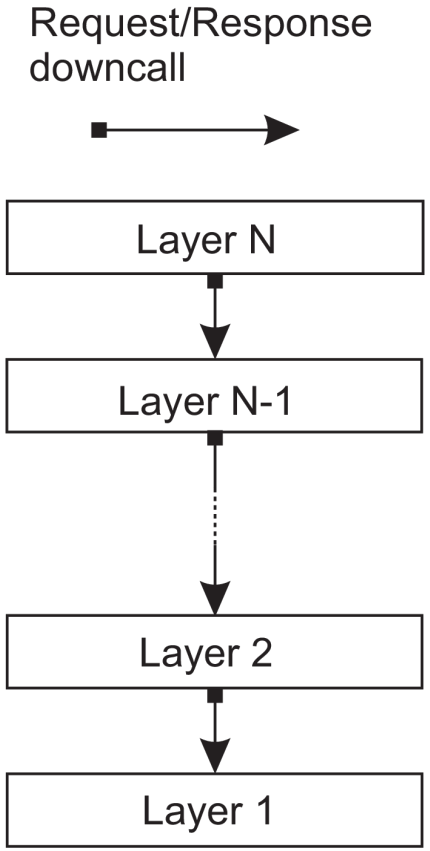
da notare la forte dipendenza tra i layer

Layered style

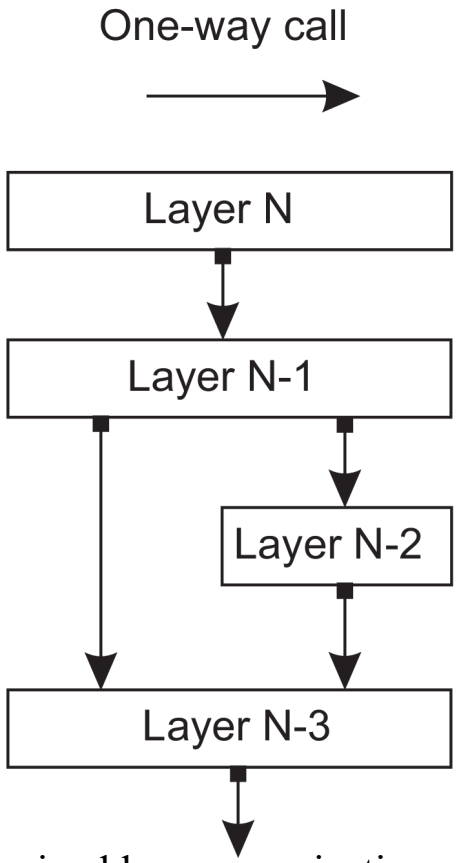
Layered style attentamente studiato ed utilizzato per lo stack di comunicazione via rete, vedasi TCP

- Different layered organizations

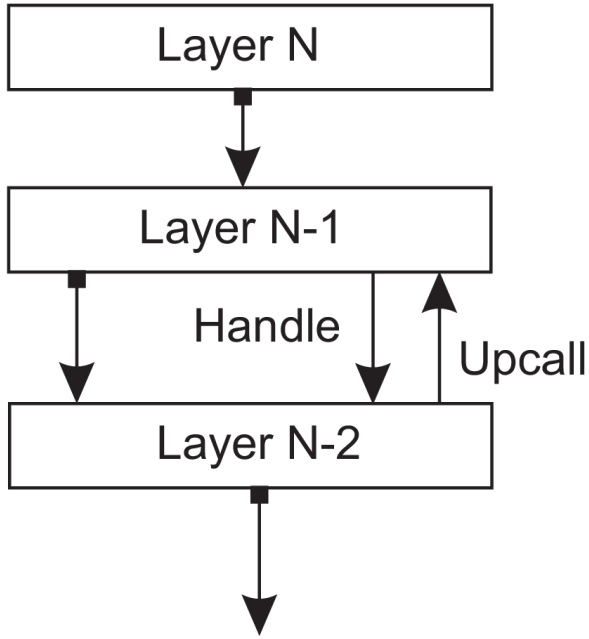
Si è arrivati a distinguere queste tre diverse organizzazioni



pure layer organization
usata di solito nelle
network communication



mixed layer organization.
Sia N-1 che N-2, utilizzano N-3

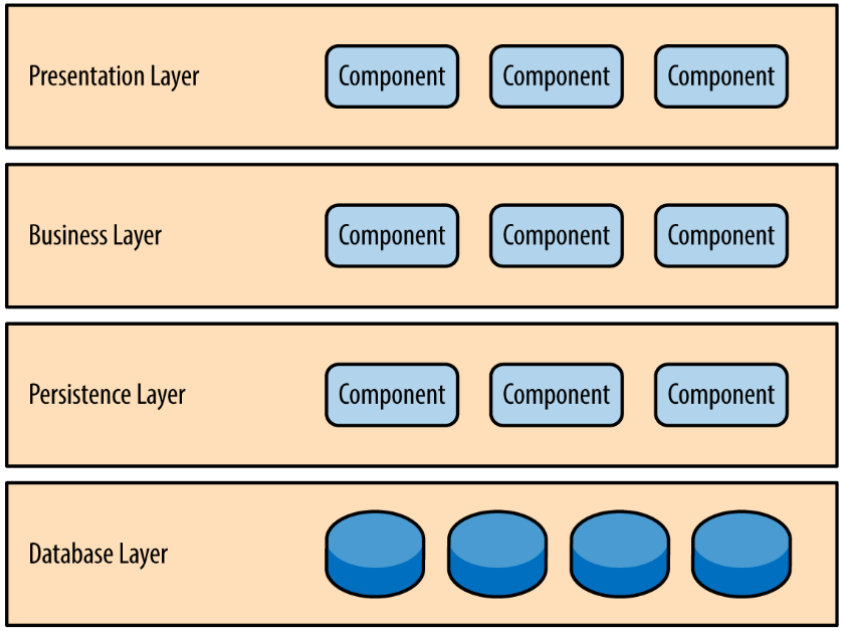


organization with upcalls.
Usata nei sistemi operativi quando si segnala un evento al cui termine si svolge un'operazione definita dal utente. (esempio apri file scaricato?)

Application layering

- Traditional layered architecture: presentation, business, persistence, database
  - In some cases, business and persistence layers are combined into a single business layer
  - Found in many distributed information systems, using traditional DB technology and accompanying applications

Di solito le applicazioni seguono una architettura divisa in 3 livelli: data level, processing level e application-interface level. Dove in linea con questi livelli un parte dell'applicazione gestisce le interazioni con l'esterno (user o altre app), una parte opera con un db o file system ed una parte contiene la logica di business che rappresenta il core dell'applicazione. Nella slide appunto si ha un presentation layer, un business layer ed un database layer.



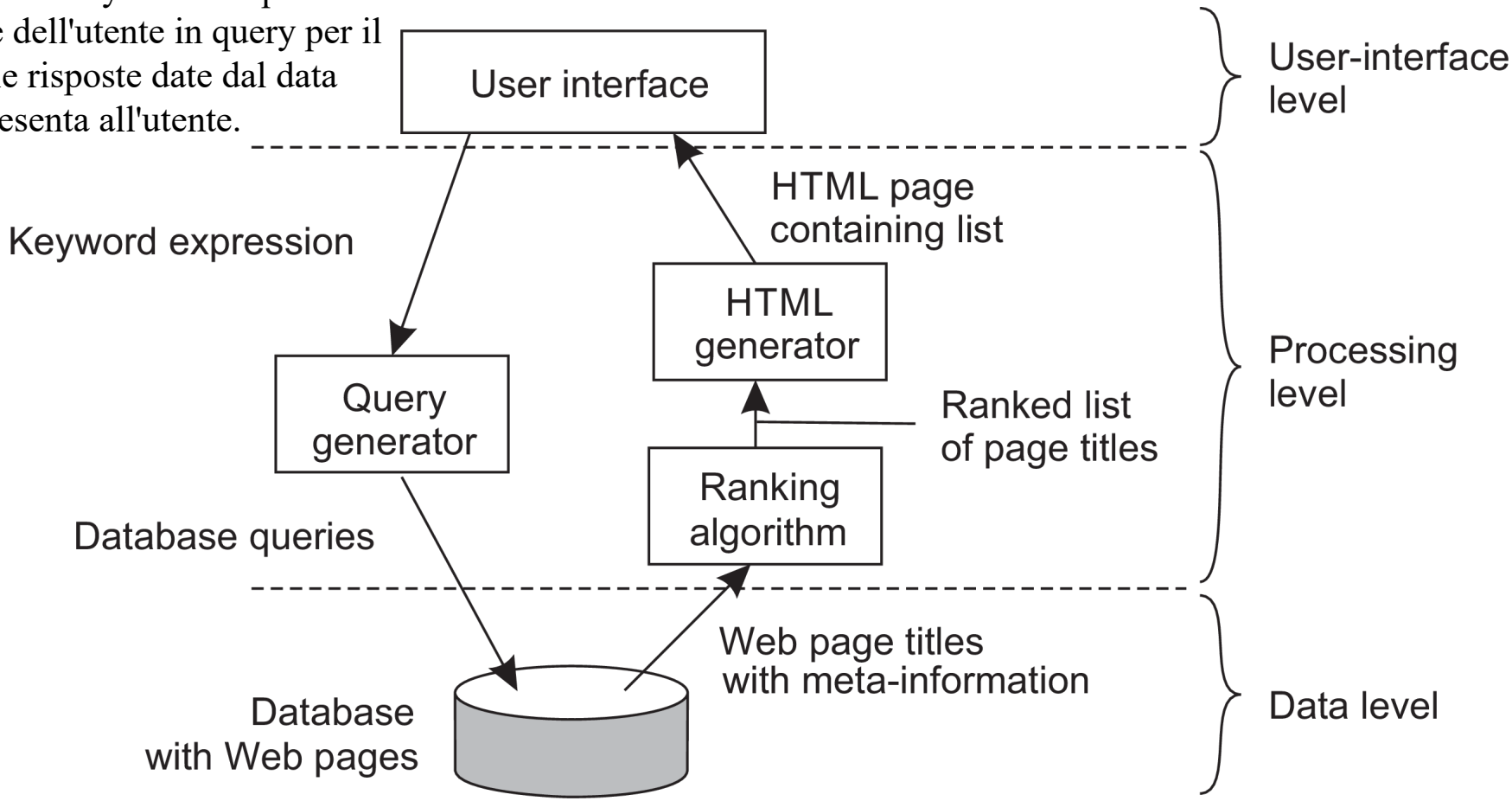
Decenni fa ormai fine anni '90, questo genere di architettura su 3 livelli, veniva ampiamente utilizzata, anche per la mancanza di VMs. Quindi su un server girava il presentation layer, su un server il business layer e su un terzo server il database. Oggigiorno questo tipo di applicazioni girano tutte sullo stesso server grazie alle VMs. Quindi una singola VM per ogni layer, ma sullo stesso server.

Application layering: example

- A simple Web search engine

Esempio di applicazione organizzata su 3 livelli: l'interfaccia è il presentation layer mentre il business layer si occupa di trasformare la descrizione dell'utente in query per il db, processa le risposte date dal data layer e le ripresenta all'utente.

Da notare l'alta dipendenza tra i livelli. Cosa succederebbe se magari all'interno del processing level una libreria utilizzata per fare tradurre la query in SQL per il database fosse rimossa? Che l'intera applicazione andrebbe buttata o comunque riscritta



Service-oriented style

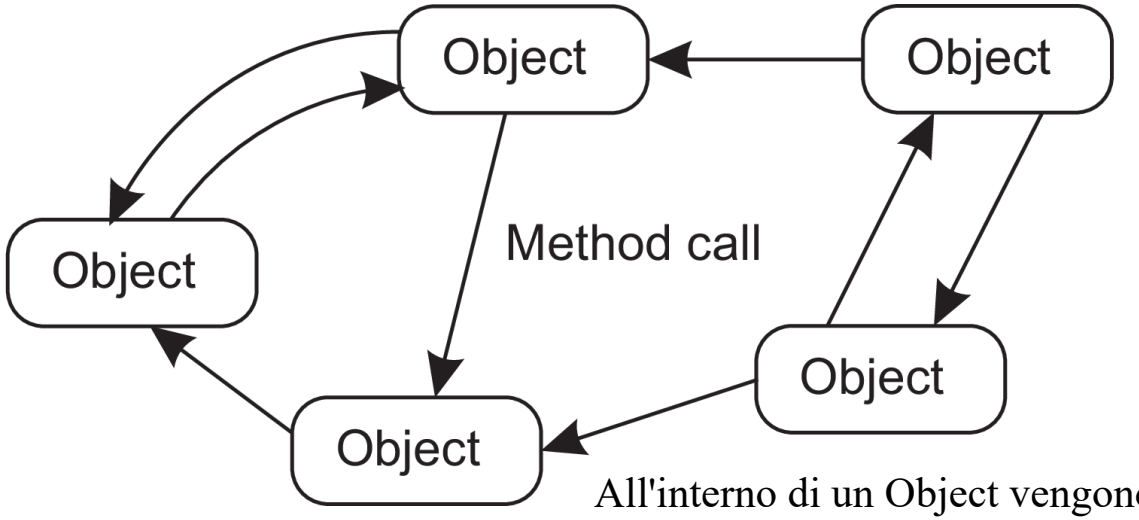
"Nasce" come soluzione al problema dell'alta dipendenza della layered organization

- A collection of separate, independent entities
- Each entity encapsulates a service Servizio che esegue come un processo a se
- Entity = service, object, or microservice
- Includes
  - Object-based architectural style
  - Microservices architectural style
  - RESTful architectural style

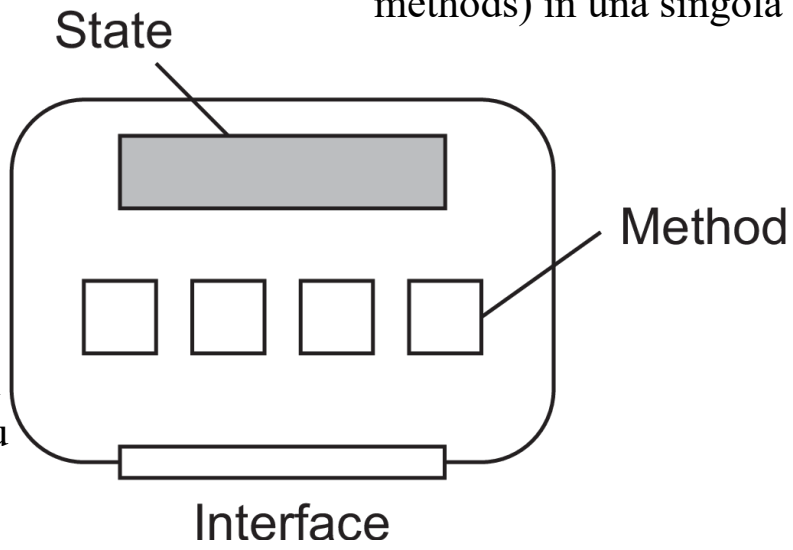
Ovviamente non è detto che la dipendenza si abbassi con questo design

Object-based style

- Component = object: encapsulates data and offers methods on data
  - Encapsulation and information hiding reduce management complexity
  - Reusability among different apps
  - Wrapping of legacy components
- Communication between components through RPC

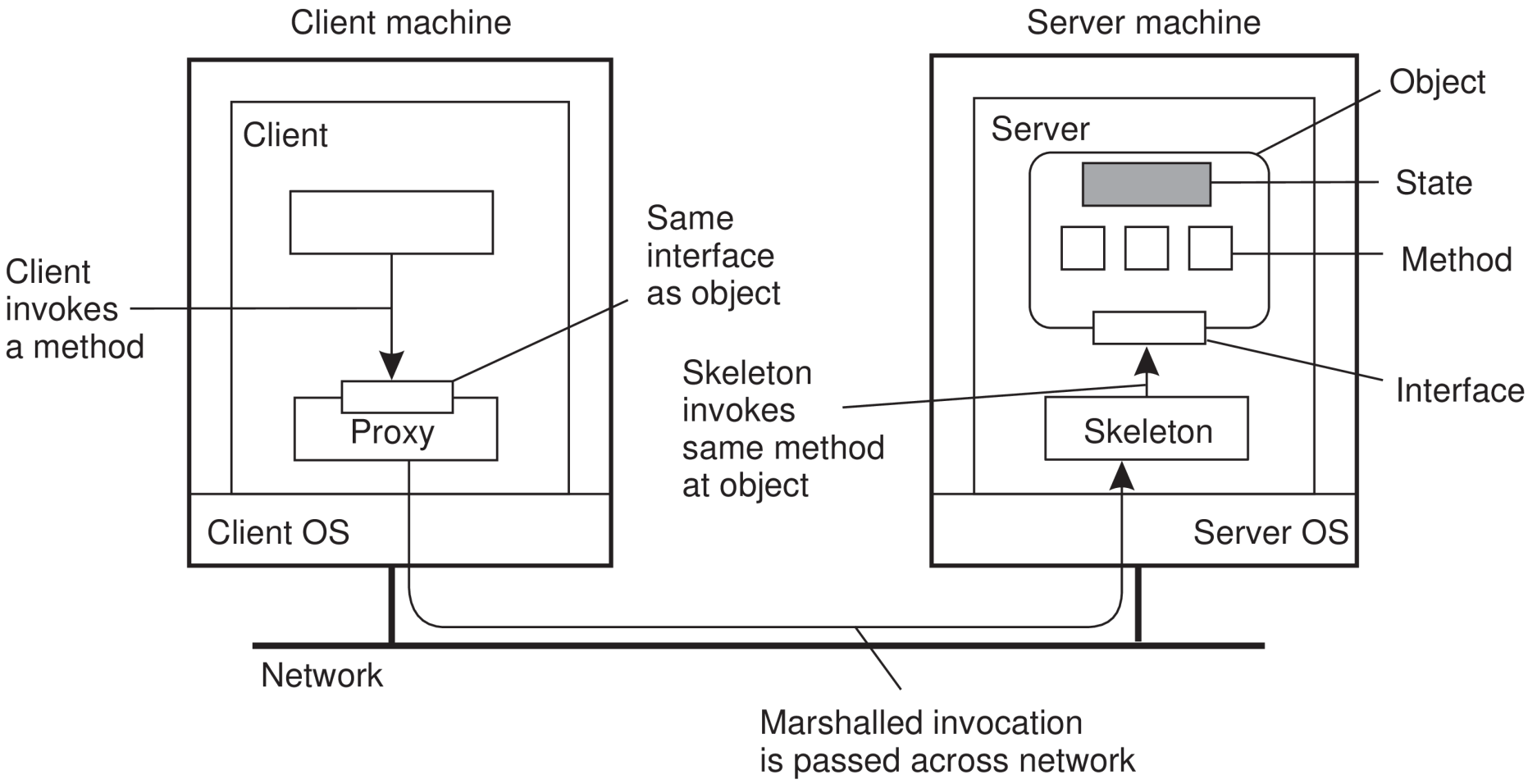


All'interno di un Object vengono incapsulati i dati (chiamati state) e le operazioni su di essi (chiamate methods) in una singola entity.



L'interfaccia di un object raccoglie i suoi dettagli implementativi e se viene definita a modo allora un object può essere sostituito da un altro che ha la stessa interfaccia, rendendo così gli object indipendenti dall'ambiente in cui lavorano. Questo permette di avere l'interfaccia su una macchina e l'object situato su un'altra. Vedi slide successiva.

Quando un client si lega ad un oggetto distribuito, un implementazione dell'interfaccia dell'oggetto detta proxy, viene caricata nello spazio di indirizzi del client. Il proxy è analogo al client stub in un sistema RPC. Si occupa di aggregare le invocazioni dei metodi in messaggi e di estrarre dai messaggi di risposta i risultati all'invocazione del metodo da parte del client. Il vero object si trova sul server che offre la stessa interfaccia che viene offerta nel client. Le richieste di invocazione sono prima passate al server stub, che le estrae per effettuare i metodi sull'interfaccia dell'oggetto nel server. Il server stub è poi responsabile di aggregare i risultati in un messaggio e mandarli come risposta al proxy del client. Lo stub del server è anche detto scheletro dato che fornisce i mezzi essenziali al middleware del server per accedere agli oggetti definiti dallo user. In pratica, spesso contiene codice incompleto sotto forma di classe in un linguaggio specifico che necessita ulteriore specializzazione da parte dello sviluppatore.



Microservices style

La separazione degli object come servizi a se stanti porta sulla strada per l'architettura basata su microservizi

- A “new” emerging architectural style for distributed apps that structures an application as a collection of loosely coupled services
- Address how to build, manage, and evolve architectures out of small, self-contained and independently-scalable services that communicate over well-defined APIs

Da notare che non c'è però uno standard per quanto riguarda la dimensione. Piccolo potrebbe pure essere 1GB

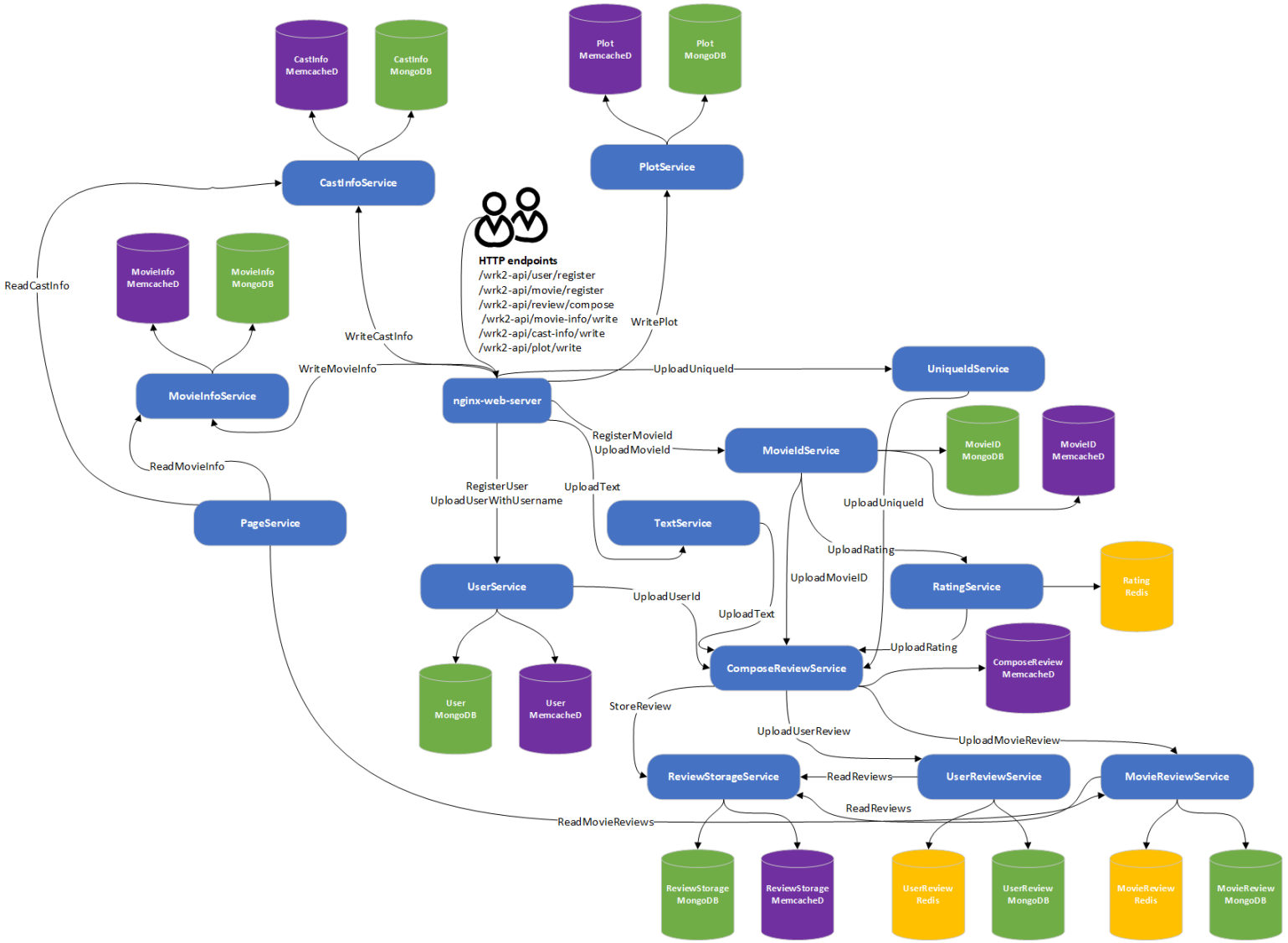
La modularità è la chiave

  - *Modularization*: decompose app into a set of independently deployable services, that are loosely coupled and cooperating and can be rapidly deployed and scaled
- See upcoming lessons

Ogni microprocesso, gira come un processo di rete indipendente.

Microservices style: example

- A social-media microservice architecture



https://github.com/delimitrou/DeathStarBench

RESTful style

Come alternativa ai microservice, che si sono utili ma richiedono anche un grosso sforzo di integrazione e orchestrazione.

- DS as a collection of resources, individually managed by components
- **Representational State Transfer (REST)**: proposed by Roy Fielding (co-author of HTTP/1.1)
  - Resources may be added, removed, retrieved, and modified by (remote) applications (HTTP methods)
  - Resources are identified through a single naming scheme (Uniform Resource Identifier, URI)

```
URI = scheme:[//authority]path[?query][#fragment]
authority = [userinfo@]host[:port]
```

- Components expose a uniform interface ovvero i metodi REST
- Messages sent to/from component are self-described
- Interactions are stateless chiusa la connessione NON si memorizza nulla
  - State must be transferred from clients to servers

REST operations

- Basic operations
  - Use HTTP methods: GET, PUT, POST and DELETE

Operation	Description
PUT	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource
POST	Modify a resource by transferring a new state

Example: S3 REST API

- S3: cloud storage service by AWS, organized as a key-based object store
- Objects (files) are stored into buckets (directories)
  - Flat structure: no directory hierarchy
    - Logical hierarchy simulated by using object names with directory structure: `photos/puppy.jpg`
  - Object *`objectname`* stored in bucket *`bucketname`* is uniquely referred to by its URI:
    `https://`*`bucketname`*`.s3.`*`Region`*`.amazonaws.com/`*`objectname`*

    e.g., `https://example-bucket.s3.us-west-2.amazonaws.com/photos/puppy.jpg`
- Operations carried out through HTTP requests:
  - Create bucket/object: `PUT`, along with its URI
  - List objects in bucket: `GET` on bucket
  - Read object: `GET` on full URI

Example: S3 REST API

- Retrieve object from bucket (`GetObject`)

```
GET /photos/puppy.jpg HTTP/1.1
Host: example-bucket.s3.us-west-2.amazonaws.com
Date: date
Authorization: authorization string
```

- Add object to bucket (`PutObject`)

```
PUT /photos/puppy.jpg HTTP/1.1
Host: example-bucket.s3.us-west-2.amazonaws.com
Date: date
Authorization: authorization string
```

See Amazon S3 REST API
https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html

Notes:

- HTTP Authorization header to authenticate S3 request
- You need permission for operations (e.g., WRITE permission on bucket): use IAM to create roles and manage permissions

Decoupling

- Strong dependencies between components introduce limitations Le dipendenze sono buone solo quando servono
- Solution: let components indirectly communicate through some intermediary nei sistemi distribuiti si mette un middleware come intermediario
  - Clean separation between computation and coordination

"All problems in computer science can be solved by another level of indirection"

(David Wheeler, Titan project)

- **Decoupling**: enabling factor to
  - Achieve greater flexibility
  - Define architectural styles that allow to better exploit distribution, scalability, and elasticity

Decoupling properties

- ***Space*** (or referential) decoupling
  - Anonymous components: do not need to know each other in order to communicate and cooperate

I componenti non devono conoscersi a vicenda per comunicare e cooperare. Componenti che possono essere sostituiti, aggiornati, replicati e migrati facilmente

- ***Time*** (or temporal) decoupling
  - Interacting components do not need to be present at the same time when communication occurs

Permette di avere asincronia

- ***Synchronization*** decoupling : no blocking anche qui asincronia
  - Interacting components do not need to wait each other and are not reciprocally blocked

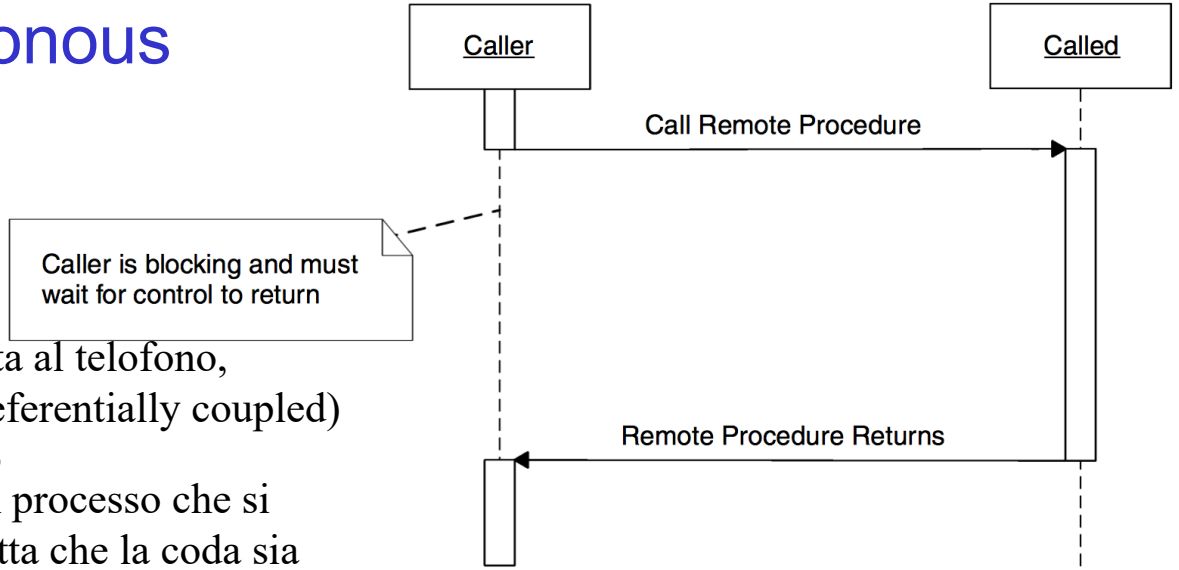
Grazie al decoupling un sistema distribuito può gestire in maniera flessibile i cambiamenti e fornire servizi elastici e affidabili. Tuttavia disaccopiare aumentare l'overhead in termini di performance.

Synchronous vs. asynchronous interaction

Synchronous

Esempi di processi coupled

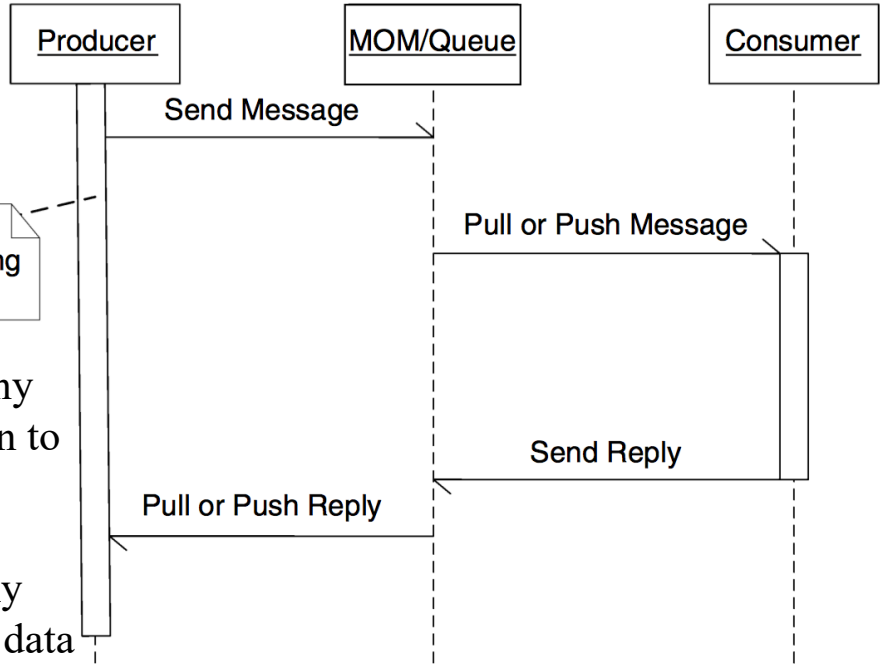
	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space



per direct si intende magari una chiamata al telofono, dovo due persone devono conoscersi (referentially coupled) e stare al telefono nello stesso momento

Event based è quando ad esempio si ha un processo che si sottoscrive alla coda per stampare ed aspetta che la coda sia libera (evento) ma deve rimanere attivo ad aspettare altrimenti non potrà stampare se inattivo. Shared data space:

Asynchronous

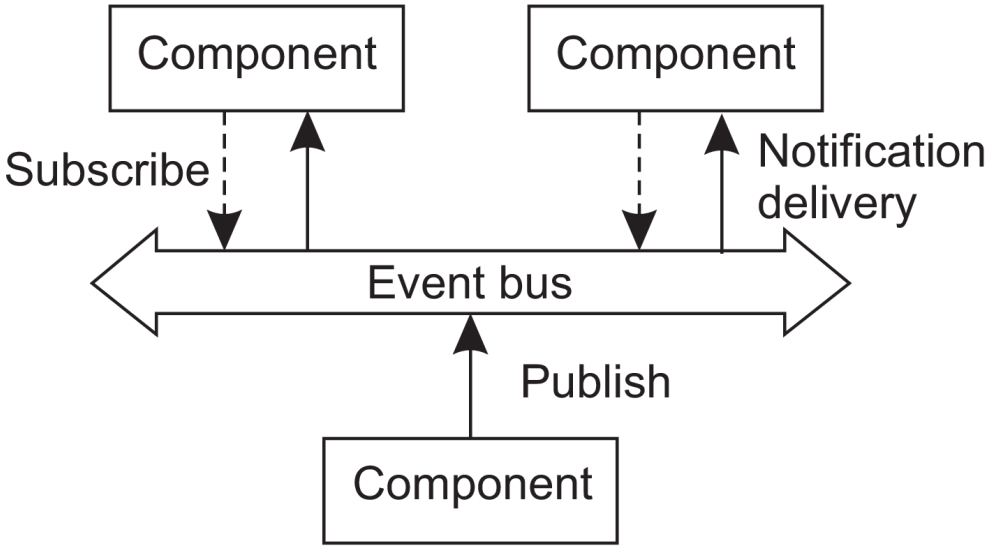


The key idea is that processes communicate entirely through tuples, which are structured data records consisting of several fields, very similar to a row in a db table. Processes can put any type of tuple into the shared data space. To retrieve a tuple, a process provides a search pattern that is matched against the tuples. Any tuple that matches is returned. Shared data spaces are thus seen to implement an associative search mechanism for tuples. When a process wants to extract a tuple from the data space, it specifies the values of the fields it is interested in. Any tuple that matches that specification is then removed from the data space and passed to the process.

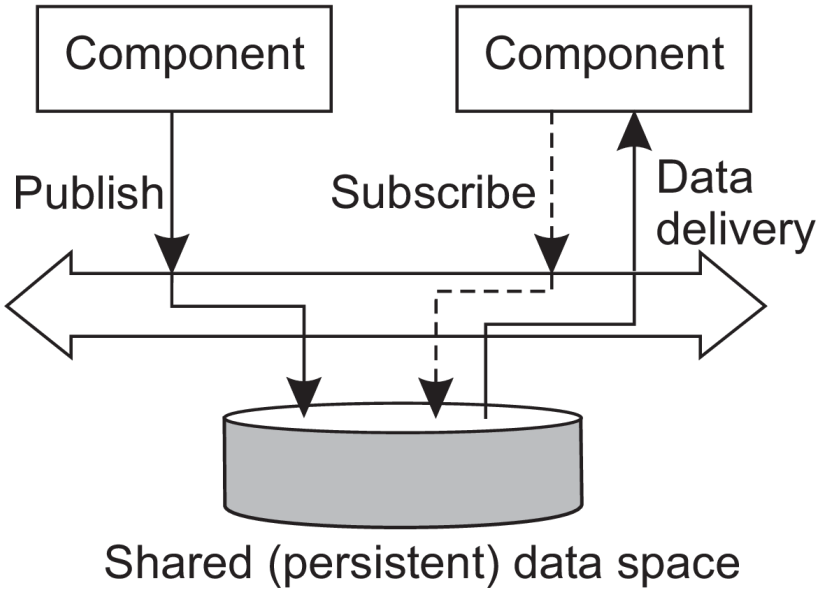
Architectural style evolution

- Introducing decoupling, alternative architectural styles where components communicate indirectly

Event-driven



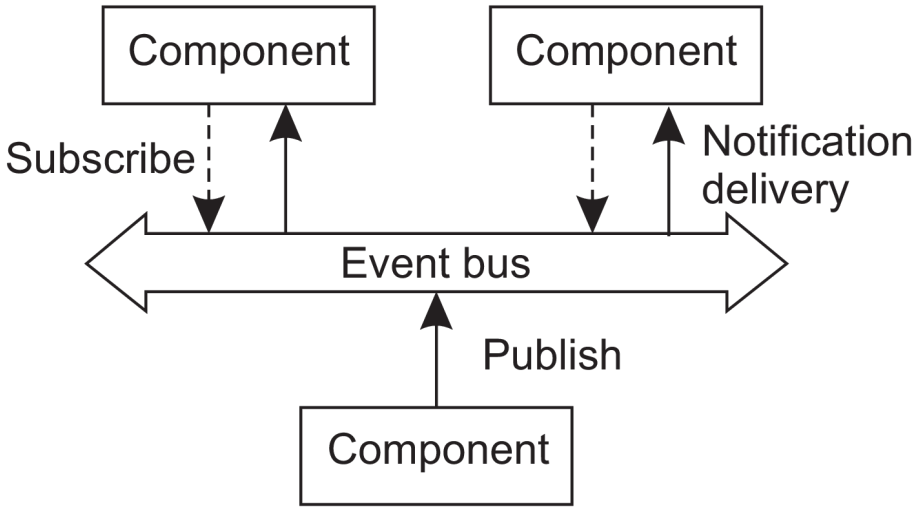
Data-oriented



Publish-subscribe

Event-driven style

- Components communicate through an event bus
  - Event: significant change in state (e.g., change in temperature, door opening)
- Components
  - Publish events
  - Subscribe to events they are interested in being notified
  - Receive notifications about events
- Communication
  - Anonymous
  - Based on message exchange
  - Asynchronous
  - Multicast



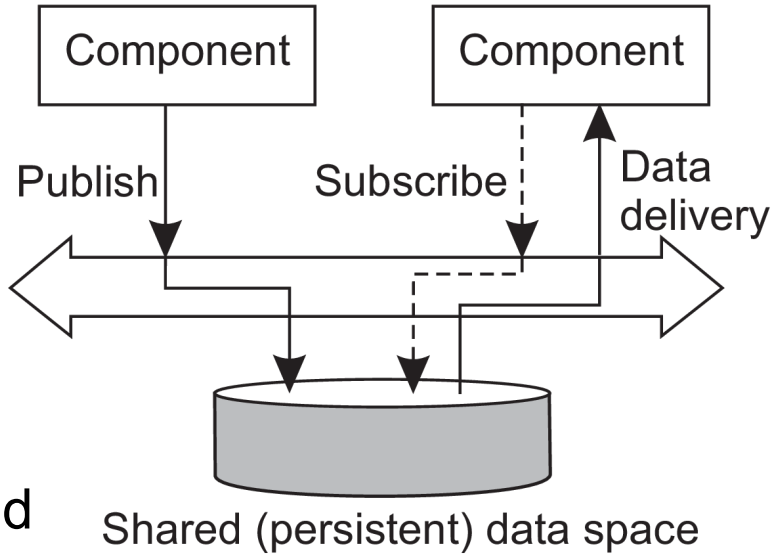
- Example: Java Swing

Which decoupling?

Spaziale sicuramente, dato che non devono conoscersi per collaborare, di sincronizzazione pure dato che una volta pubblicato l'evento non aspetta nessuno. Temporale dipende se il bus è in-memory o persistente

Data-oriented style

- Communication among components happens through shared data space: passive, sometimes (pro)active
  - Data added to or removed from shared space
- Shared data space API
  - `write`, `take`, `read` and variants (`takeIfExists`, `readIfExists`)
  - If active space: `notify` or `push` (avoid polling)
  - Concurrency control
- Examples of shared data spaces
  - GigaSpaces https://tinyurl.com/bdtc5yk6, TIBCO ActiveSpaces https://tinyurl.com/bkkxm768



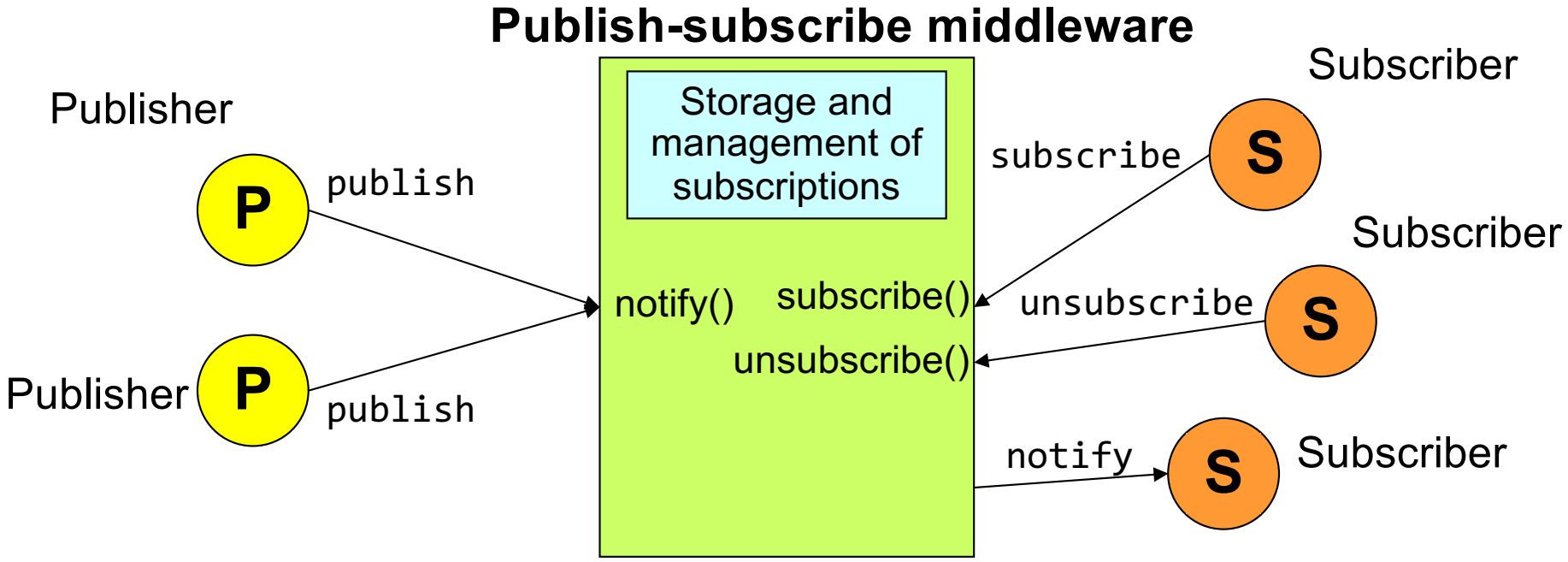
Which decoupling?

Tutte e tre. Anche se LO SPAZIO è concorrente i componenti non lo sono e comunque è gestita

How to implement shared space?

Publish-subscribe style

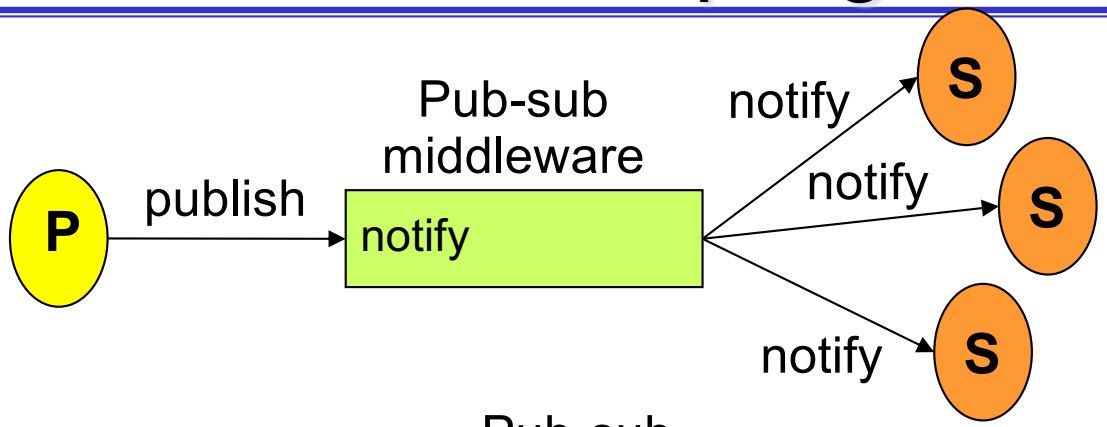
- Publishers (aka producers) generate events (`publish`) and are not interested in their delivery to subscribers (aka consumers)
- Consumers register as interested to events (`subscribe`) and are notified (`notify`) of their occurrence
- Full decoupling among components tutte e 3: spaziale, temporale e sincronizzazione



Publish-subscribe and decoupling

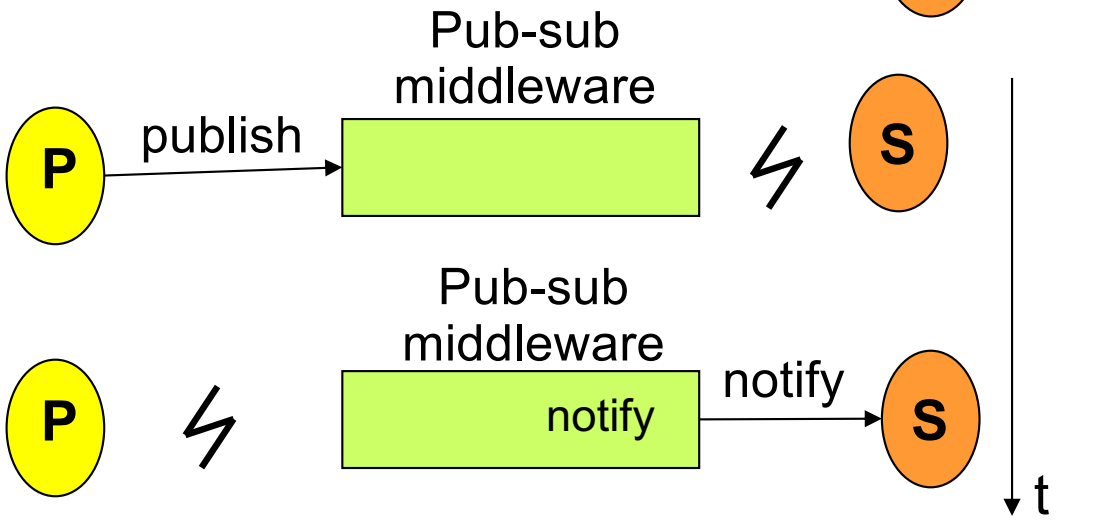
Space decoupling

un publisher pubblica e non sa chi viene notificato e nemmeno dove si trova



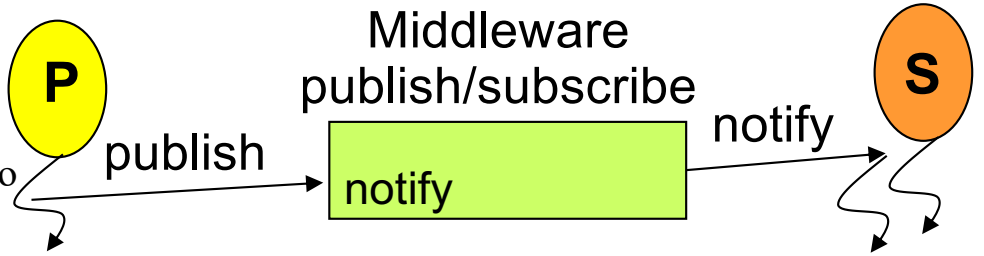
Time decoupling

appunto una volta messo il messaggio nel middleware, resta li e può essere letto quando si vuole da un publisher.



Synchronization decoupling

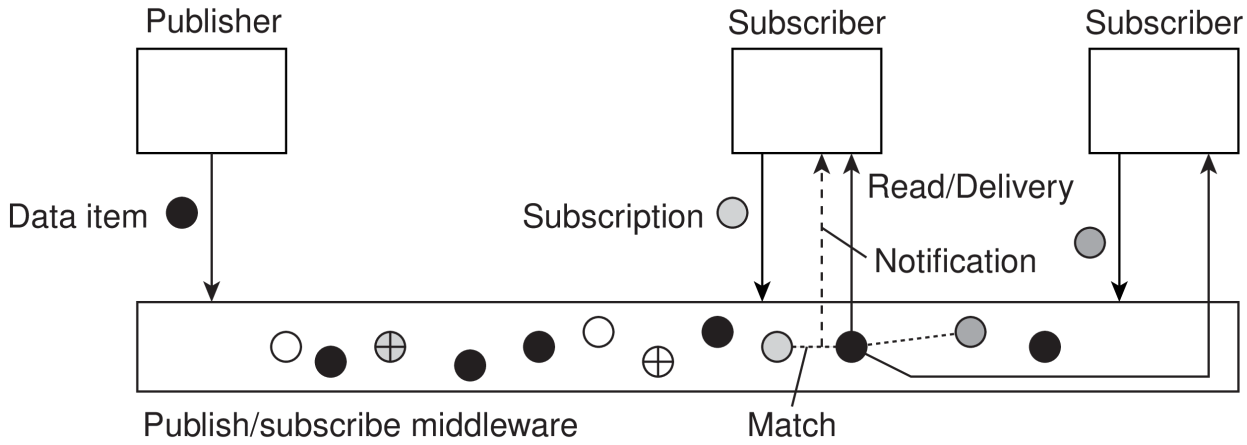
nessuno aspetta nessuno e non c'è un blocco di attesa per risposta o per la notifica. il subscriber può fare altro



Eugster et al., The many faces of publish/subscribe, *ACM Comput. Surv.*, 2003

Publish-subscribe: subscription models

- Issue: how to match events
  - Assume events are described by (attribute,value) pairs



Il modello può organizzarsi secondo la metodologia, push dove si notifica e si mandano i dati oppure pull dove si notifica solo l'evento.

- Topic-based subscription matching di un valore. molto semplice ma limitato
  - Specify an "attribute = value" series (e.g., type=news)
  - ✗ Limited expressiveness, only exact match
- Content-based subscription
  - Specify a series of filter (e.g., "attribute ∈ range")
  - Subscribers define complex conditions (e.g., price > 100 AND category = electronics)
  - ✗ Scalability challenges, why?

bisogna verificare N condizioni, quindi magari ho O(N) come costo e non O(1). e ciò avviene anche per N subscriber. alla peggio così valuto per N subscriber N condizioni logiche ovvero O(N^2)

Choosing an architectural style

Come spesso accade non esiste una soluzione unica

- No single solution: can tackle same problem with different architectural styles

- Choice depends often on extra-functional requirements:
  - Costs (resource usage, development effort needed)
  - Scalability and elasticity (effects of scaling and amount of available resources)
  - Performance (e.g., response time, latency)
  - Reliability and fault tolerance
  - Maintainability (extending system with new components)
  - Usability (ease of configuration and usage)
  - Reusability

System architecture of DS

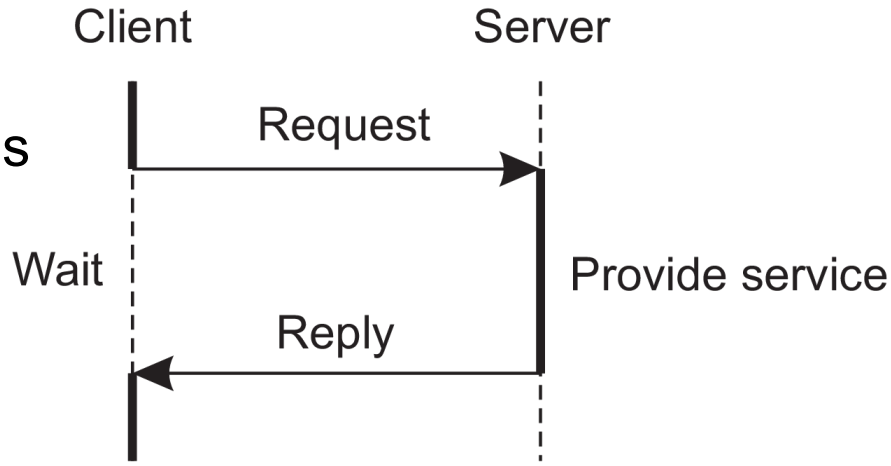
- Runtime instantiation of DS software architecture
  - Which components?
  - How do they interact with each other?
  - Where to deploy them?

- Types of system architectures
  - Centralized architectures
  - Decentralized architectures
  - Hybrid architectures

Trattiamo le diverse architetture nel dettaglio

Centralized system architectures

- Basic client-server model
  - Two groups: servers offer services and clients use services
  - Clients and servers can be on different machines
  - E.g., Web clients and servers



- Request/reply model
- Communication
  - based on message exchange
  - often *synchronous* and *blocking*
- Strong coupling: e.g., coexistence of interacting entities

Multi-tiered client-server architectures

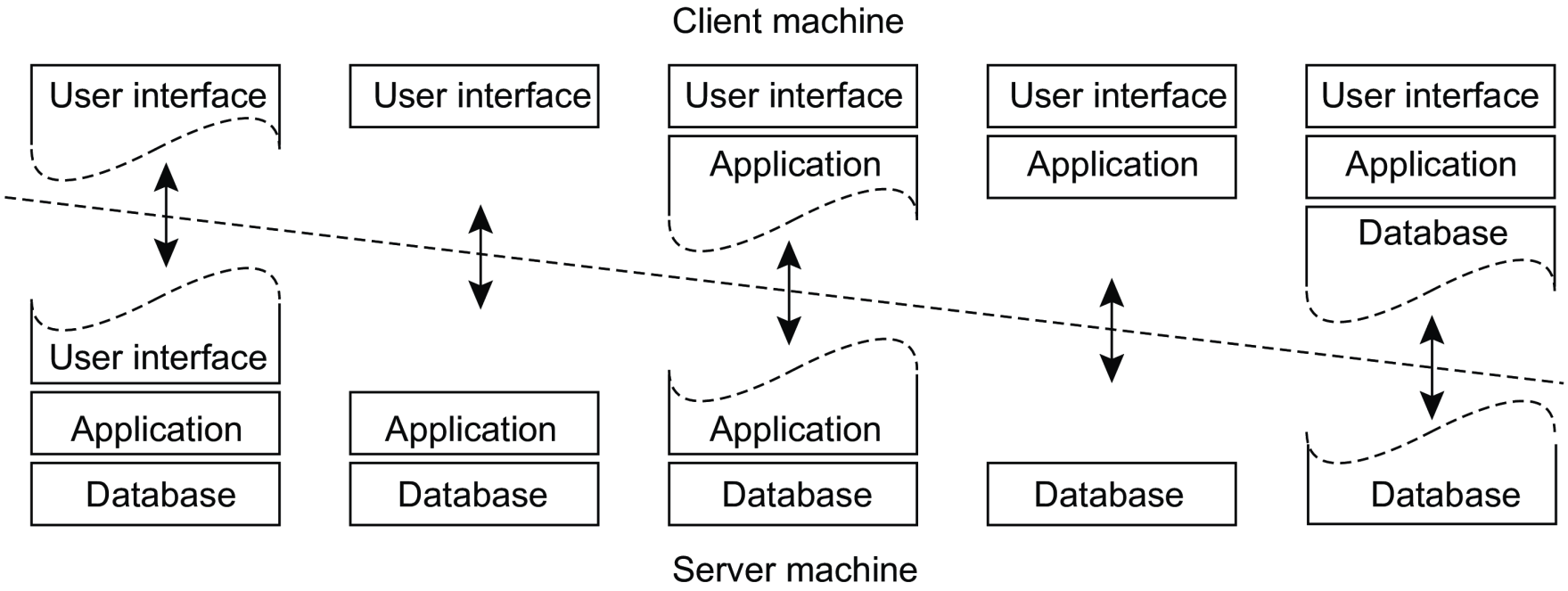
Un approccio per organizzare client e server è distribuire i tre layer (presentazione, business e data) su più macchine.

- How to map logical levels (***layers***) into physical levels (***tiers***)?
  - Two-tiered architectures è il primo approccio che fa una distinzione solo su due tipi
  - Three-tiered architectures

- Different configurations, depending on distribution of:
  1. presentation layer
  2. logic (aka application, business, processing) layer
  3. data layer

Multi-tiered client-server architectures

- Example: two-tiered configurations



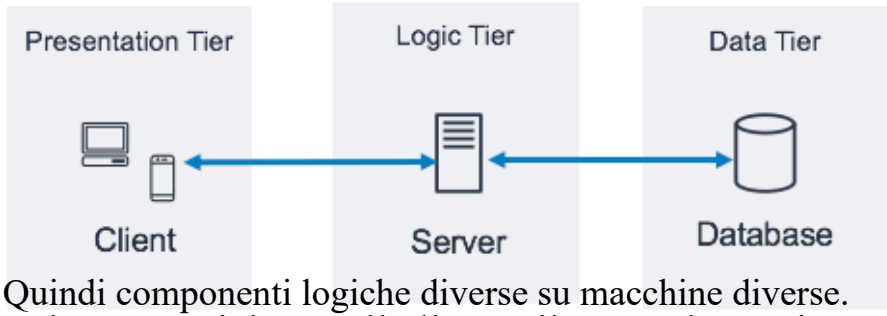
- More than three tiers?

Quanta più logica si vuole dare al client, ci si sposta verso destra, con il caso mediano, in cui il client ha un front end capace di operare in locale su dati cached mentre per operazioni più complesse fa richieste al server.

From multi-tiered architectures to…

Dai multi-tiered si passa ai symmetrically distributed systems architectures, dove si organizzano le applicazioni client-server in un'architettura multi tier.

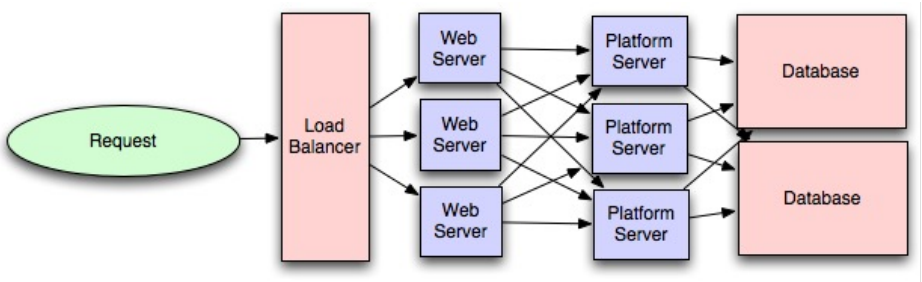
- Vertical distribution
  - Divide distributed applications into 3 logical layers and run each layer on a different tier



Quindi componenti logiche diverse su macchine diverse.

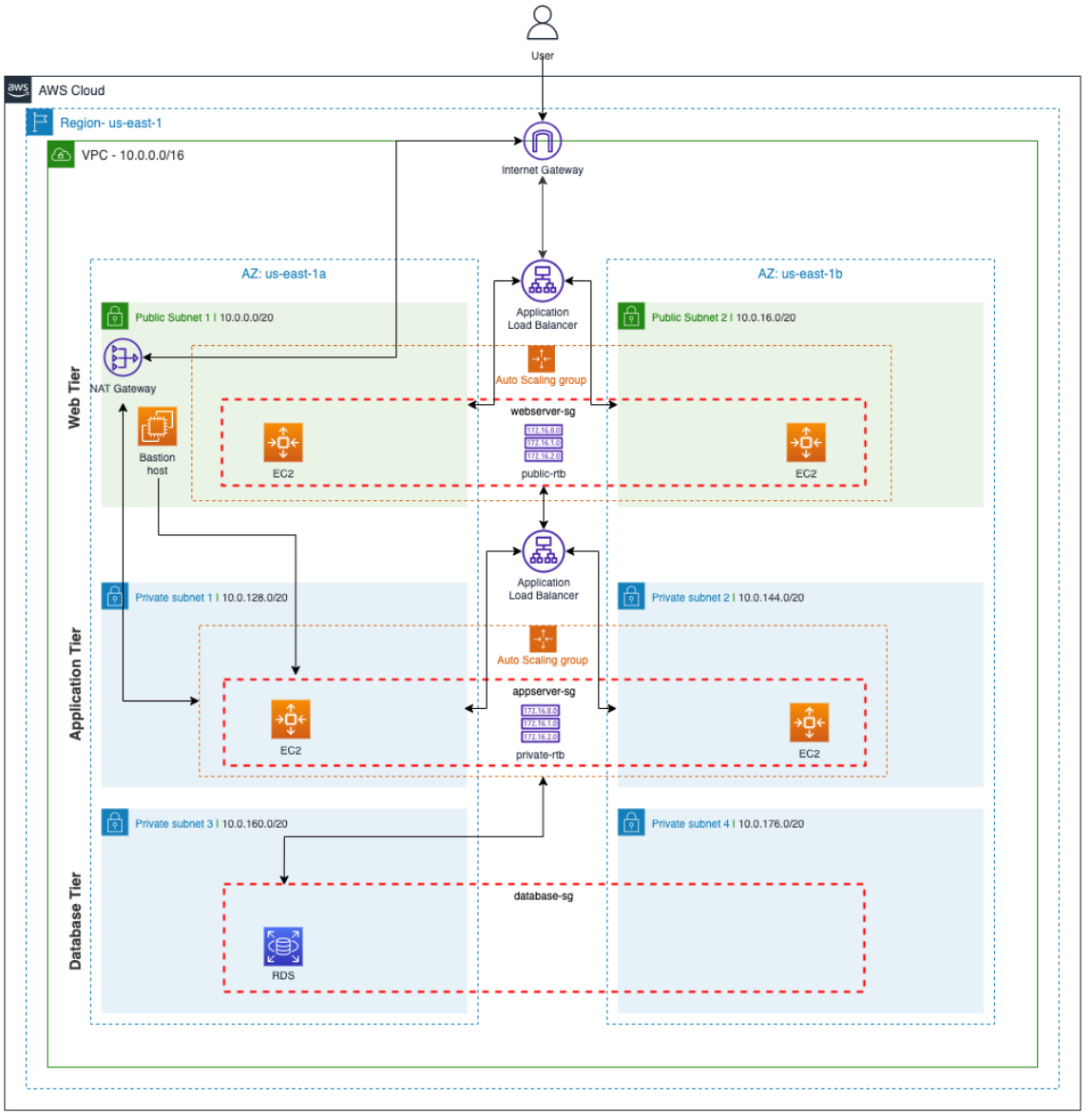
Nelle architetture moderne la distribuzione dei client e dei server conta. Si va a partizionare il client o il server in parti logicamente uguali ma ognuna con la propria porzione del set di dati completo con cui opera autonomamente, bilanciando il carico

- Horizontal distribution
  - Distribute each layer on multiple servers
  - Balance load among multiple servers through a load balancer
  - E.g.,: distributed Web cluster



Example: Web application in AWS

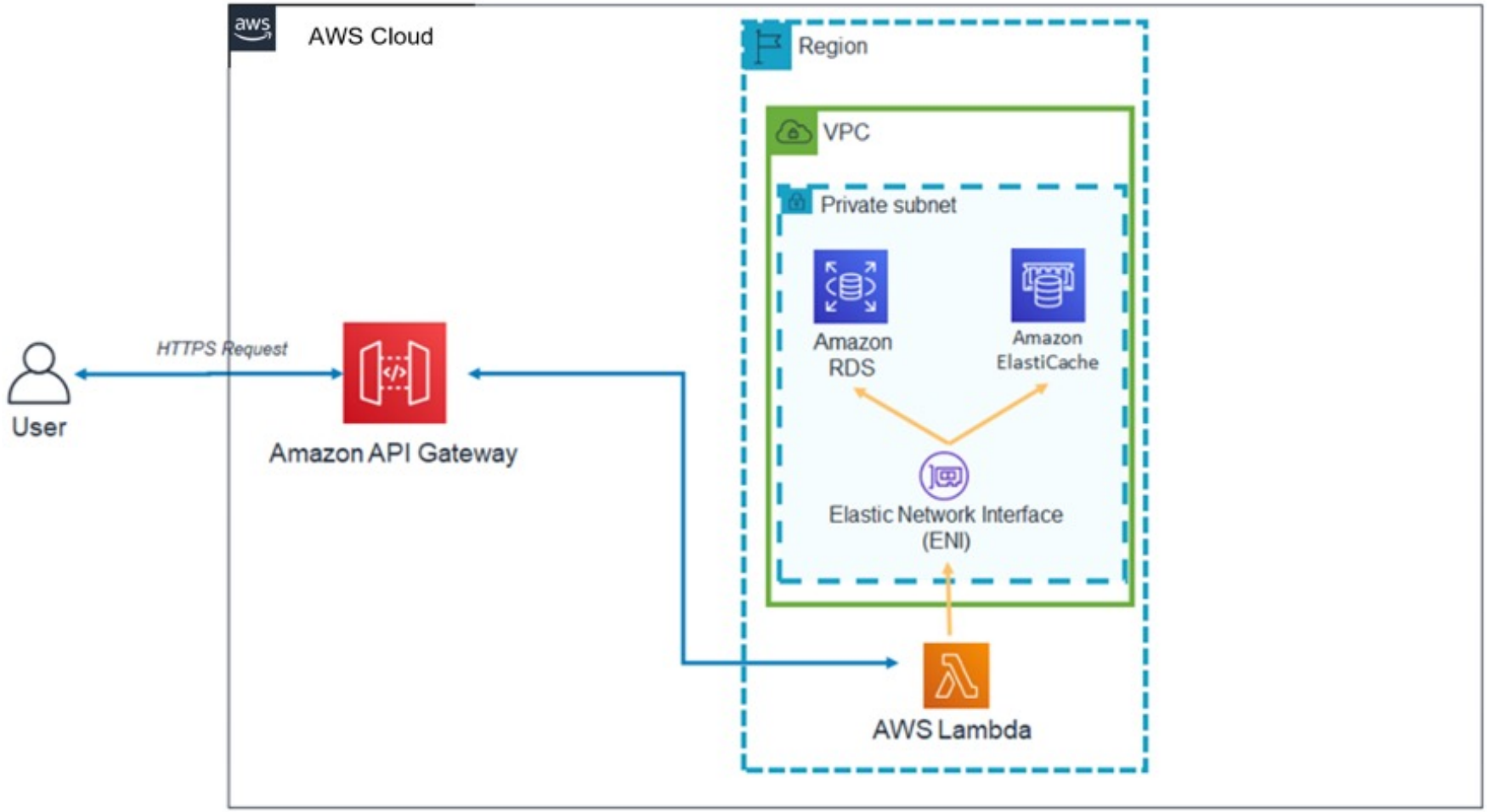
- Web application with horizontal distribution in AWS
  - Elasticity: tiers can scale out/in
  - High availability: replication in different availability zones
  - Security: tiers communicate with private IP



https://medium.com/@aaloktrivedi/building-a-3-tier-web-application-architecture-with-aws-eb5981613e30

Example: 3-tier serverless architecture

- Horizontal distribution is not visible: AWS Lambda has elastic scalability already built in



https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/three-tier-architecture-overview.html

Decentralized system architectures

Un tipo di sistema distribuito orizzontalmente sono i

- **Peer-to-peer** (P2P) systems

- P2P: decentralized systems and applications using distributed resources to perform key functions
  - "P2P is a class of applications that takes advantage of resources available at the edges of the Internet" (Shirny, 2000)
- Leverage shared resources: files, storage, compute power, bandwidth
  - Peers both give and receive resources

Da un punto di vista di alto livello, i processi che costituiscono un sistema peer-to-peer sono tutti eguali. Ciò significa che le funzioni che necessitano di essere svolte sono rappresentate da ogni processo che costituisce il sistema distribuito. Come conseguenza, gran parte dell'interazione tra i processi è simmetrica: ogni processo agirà come client e server allo stesso tempo, o come riferisce in gergo tecnico, agirà da servant

P2P systems: key characteristics

Dunque riassumendo:

- Peers are roughly symmetric in roles, privileges and responsibilities
  - Peers act as both clients and servers (*servents*)
  - Autonomous nodes located at network edge
  - Super-peer may exist with enhanced roles
- No centralized control
- Highly distributed
  - Can scale to hundreds of thousands of nodes
- Highly dynamic
  - Nodes can join or leave at any time (*churn*)
- Redundancy of information
  - To ensure resilience and availability

P2P: use cases

Rivedi anche appunti reti di calcolatori

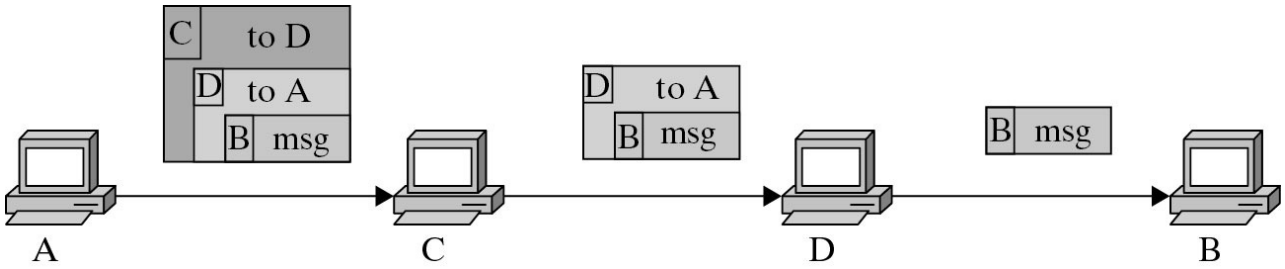
- Content sharing and storage
  - Content: file, video, etc.
  - File sharing: Kademlia, BitTorrent, WebTorrent, etc.
  - Video sharing: PeerTube, etc.
  - File storage: Freenet, IPFS, etc.
- Computing resource sharing
  - BOINC (sw platform for computing using volunteered resources), Folding@home (protein folding)
- Communication
  - e.g., jami, Matrix (+ Element), XMPP, etc.
- Blockchain and Web3
  - Bitcoin, Ethereum, Optimism, etc.: decentralized consensus and data sharing
  - libp2p networking library https://libp2p.io/

P2P: challenges

- Heterogeneity in peer resources
  - Peers differ in hardware, software, and network capabilities
- Scalability
  - Scale affects routing efficiency and bandwidth usage
- Location and interoperability
  - Data locality, network proximity, and cross-platform compatibility
- Fault tolerance and churn resilience
  - Peers join, leave, or fail randomly
  - Require robust failure recovery and data redundancy
- Performance
  - Routing efficiency, load balancing, self-organization
- Free-riding avoidance
  - Free-riders consume resources but don’t contribute
  - Requires incentive mechanisms to promote fairness

P2P: challenges

- Anonymity and privacy
  - Ensuring private communication
  - *Onion routing* (e.g., Tor) for anonymous message passing



- Trust and reputation management
  - Peers often don’t know each other
  - Requires mechanisms to evaluate trustworthiness and prevent abuse
- Network threats and attacks
  - Vulnerable to sybil attacks, DDoS, malicious nodes
  - Require defense mechanisms like verification, redundancy, or rate-limiting

Main tasks of a P2P node

Vediamo il caso in cui un sistema P2P venga utilizzato per file sharing.

- Let's consider file sharing Dividiamo il processo in 3 fasi

1. Bootstrap
   - How a new peer discovers others in the network
   - Solutions: static configuration, pre-existing caches, well-known nodes
2. Resource lookup
   - How to locate resources
   - Our focus
3. Resource retrieval
   - How to download or access the located resource

P2P as overlay networks

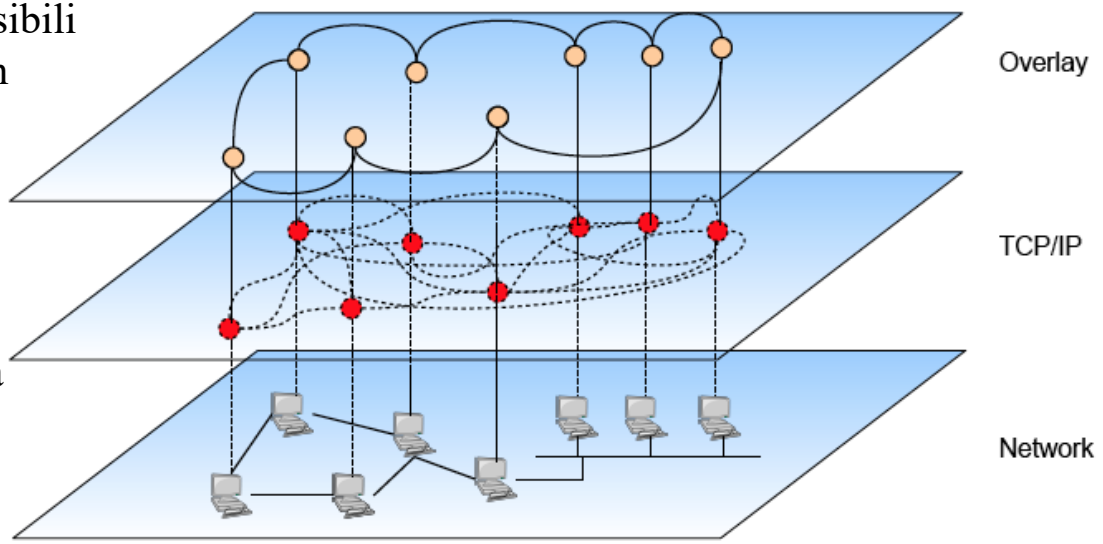
- P2P networks are often referred to as overlay networks
- **Overlay network**: logical network connecting peers built on top of physical IP network
  - Peers are connected by logical links, which may not match physical connections
  - Provides a resource location service by means of **application-level routing**

Un overlay network è una rete in cui i nodi sono formati dai processi e le connessioni (link) rappresentano i possibili canali di comunicazione. In genere appunto TCP/IP. Un overlay network può essere strutturato o meno.

Application-level abstraction

Da notare che un nodo potrebbe non comunicare in maniera diretta con un altro nodo e quindi debba inoltrare i messaggi attraverso i canali disponibili.

Da notare come nell'immagine il grafo connesso, sia ridotto al minimo possibile!



Overlay routing

- Basic idea:
  - Find a path to reach a resource in P2P system
- Unlike traditional routing
  - The target is a resource (file, CPU, storage space), not a node address
- Our focus: efficient routing to locate resource
- Once located, resource retrieval is easy
  - Usually a direct peer-to-peer interaction (e.g., via HTTP)

Overlay network operations

- Beyond routing, overlays enable:
  - Inserting and deleting resources
  - Adding and removing nodes
  - Identifying resources and nodes
- Resource identification
  - Use a Globally Unique IDentifier (GUID)
  - Generated by applying a secure hash function to resource state/data
- Peer identification
  - Also generated using a secure hash function

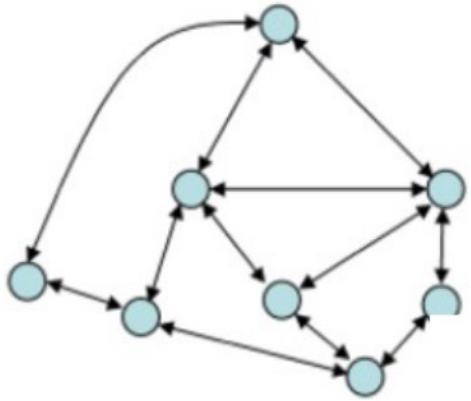
P2P overlay classification

- How to manage resources and nodes?
  - Depends on overlay network's type

Come detto precedentemente

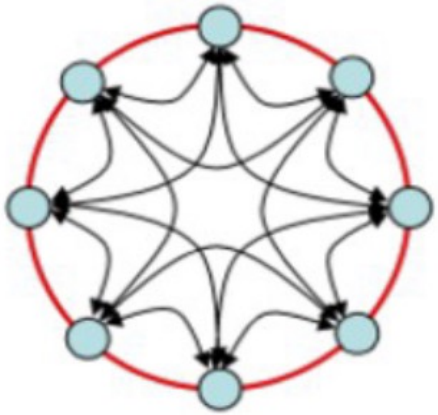
Unstructured overlay networks

molto dinamici ma complessi per individuare le risorse



Structured overlay networks

più rigidi ma con tecniche per individuare le risorse efficaci



Unstructured overlay network

- Overlay network
  - Built on random graphs without structure *by design*
  - Peers connect arbitrarily using simple and local rules
  - New nodes join by contacting a set of neighbors
  - No control over resource placement on nodes
  - Designed to handle highly dynamic
- Goal: handle highly dynamic networks
- Examples: Gnutella, Bitcoin
- Pros and cons:
  - ✓ Ease of maintenance: insertion and deletion of nodes and resources are easily managed
  - ✓ Highly resilient
  - ✗ Lookup is inefficient: resource location is complicated by the lack of structure

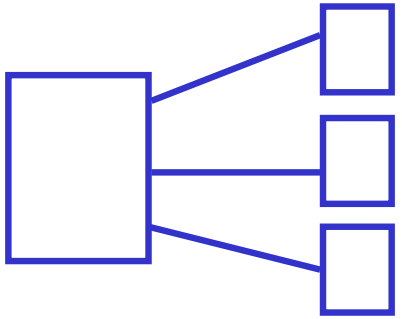
Ogni nodo in questo caso mantiene una lista di vicini, che viene aggiornata spesso.

In un random graph: un arco tra u e v esiste con probabilità P[<u,v>]

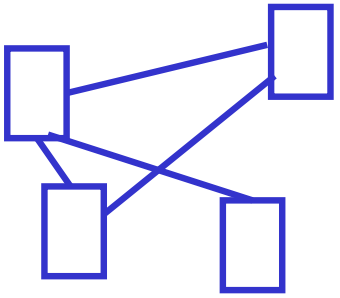
in realtà contatta un nodo noto e ottiene una lista di nodi

Unstructured overlay network: routing

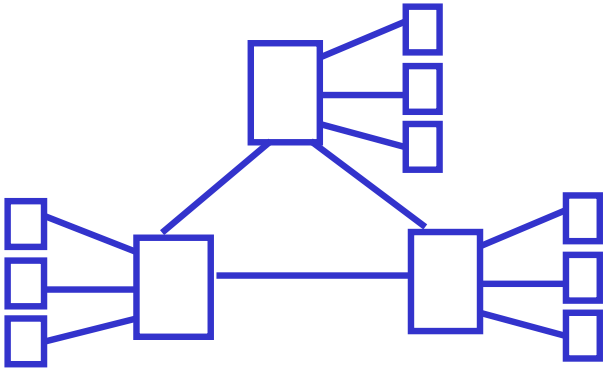
- Let's classify unstructured overlays by distribution of peer-resource index (*directory*)
- Centralized unstructured overlay: central directory (e.g., Napster)



- Decentralized unstructured overlay: distributed directory (e.g., Gnutella)

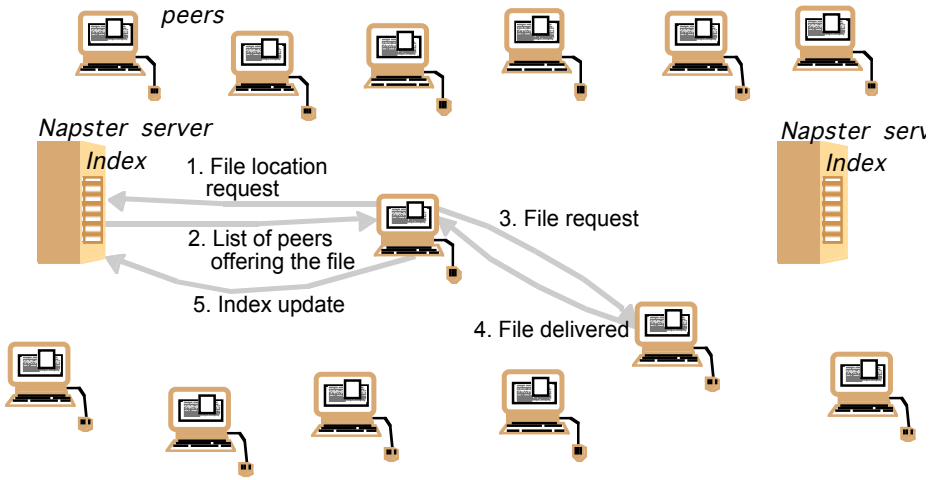


- Hybrid unstructured overlay: semi-centralized directory
  - Routing limited to *super-peers*



Centralized unstructured overlay

- One or few nodes (directory server) maintain a global index of resources
- Peers query directory to locate resources



✓ Simple

✗ Performance and scalability bottleneck

✗ Single point of failure (SPOF) for technical and legal reasons vedasi la fine di Napster

✗ Expensive to manage due to centralized control

✗ Single directory server: performance bottleneck and limited scalability

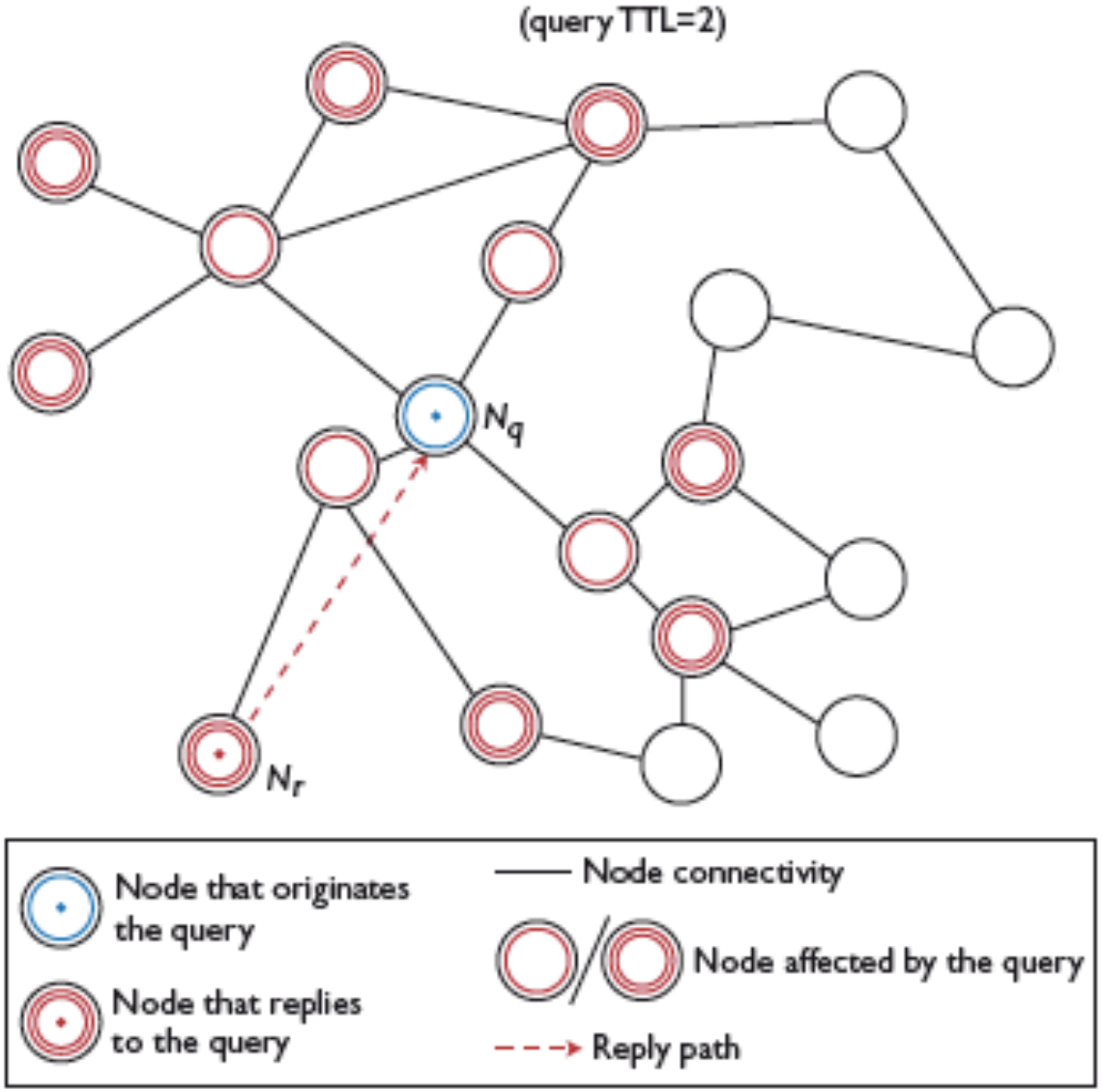
Decentralized unstructured overlay

- Fully decentralized resource lookup
- How to find resources without a directory?
  - **Query flooding**: broadcast queries to neighbors until resource is found or TTL expires
  - **Random walk**: send queries along random paths to reduce message overhead
  - **Gossiping** (covered in upcoming lesson): probabilistic message spreading among peers

Query flooding

- Originator sends query to its neighbors che la ignorano se hanno l'hanno già ricevuta.
- Each peer
  - Responds if it has the resource la risposta è diretta oppure ripercorre il path fino ad arrivare al nodo origine della richiesta.
  - Otherwise, forwards query to its neighbors (except sender)
- Optimization #1: limit search range
  - Use Time-to-Live (TTL), decreasing it by 1 at each hop
  - Stop forwarding when TTL=0 Molto importante scegliere un TTL adeguato
- Optimization #2: avoid cycles
  - Assign unique query ID so to not process same query multiple times
- Lookup cost: O(*N*), where *N* = number of nodes in P2P network abbastanza immediato come risultato

Query flooding: example



Query flooding

- Response routing options
- Direct routing
  - Resource owner sends response directly to query originator
- Backward routing
  - Response follows the same path back along which the query traveled
  - Uses query ID to retrace the path
  - Which pros?

vantaggi: non devo ricalcolare il percorso per l'instradamento della risorsa (ricordiamo che non per forza c'è un canale diretto), rischio ridotto di cicli e permette di avere un istradamento bilanciato.

Query flooding: cons

andiamo a inondare la rete di messaggi

- Communication overhead
  - Large number of messages
  - Even unsuccessful queries consume bandwidth

come si vedrà il gossiping con lo stesso numero di round, o intervallo di tempo raggiunge leggermente meno nodi ma con un costo (n° messaggi) minore

- High lookup cost: O(N)
  - How to choose TTL?
- Security risks
  - Vulnerable to DoS attacks
  - Risk of black-hole nodes under network congestion ?
- False negatives
  - No guarantee that (all) nodes with resource are reached IMPORTANTE
- Topology mismatch
  - Overlay “neighbors” may be physically far apart
  - Leads to inefficient routing and increased latency

LATENZA ELEVATA

Random walk

- The originator sends the query to one random neighbor
  - Each peer forwards the query to one of its neighbors, randomly chosen
  - Process continues until the resource is found
- Random walk vs. flooding
  - ✓ Lower message overhead
  - ✗ Longer lookup time

meno messaggi e quindi meno congestione, ma tempi di risposta MOLTO più grandi

- Optimization: *k*-random walks simultanei si intende
  - Originator starts k independent random walks
  - Sends the query to k randomly chosen neighbors
  - Each copy follows its own path
  - Balances lower traffic with faster discovery

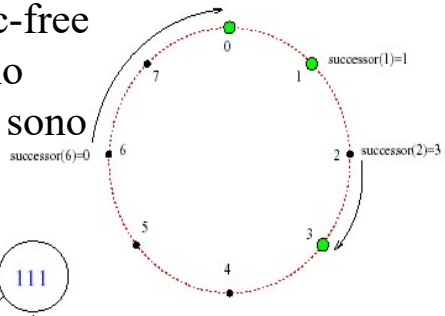
vedi libro per piccolo confronto tra flooding e random walk

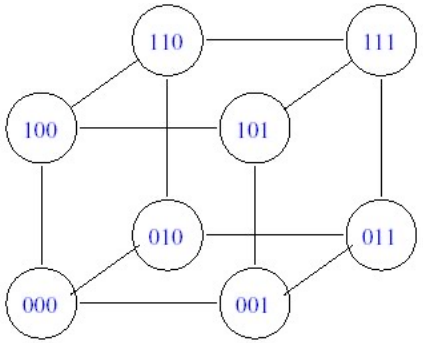
Ulteriori ottimizzazioni sono anche qui un TTL e un meccanismo per chiedere all'originator se serve ancora inoltrare la richiesta della risorsa o no

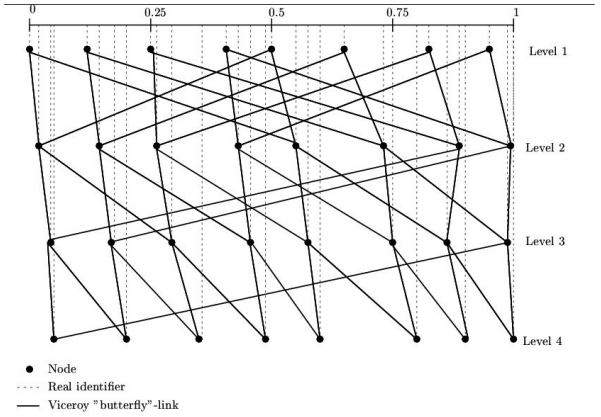
Structured overlay networks

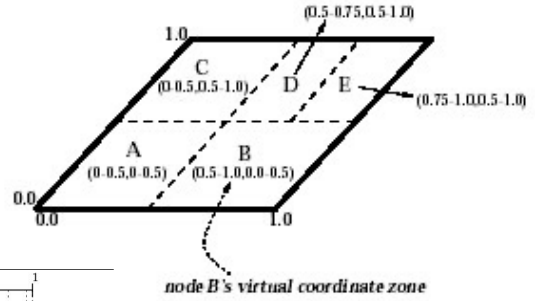
- Lookup queries are forwarded using a well-defined structure
- Each peer maintains information about a specific set of other peers
- Overlay network is structured
  - Constraints on how peers and resources are organized
  - Overlay topology: ring, tree, hypercube, grid, ...

I structured overlay network si basano su i cosidetti semantic-free index. In questi indici i dati sono associati a chiavi UNICHE che sono generate tramite funzioni hash.









Structured overlay networks

- Goals:
  - Improve scalability by lowering lookup cost
  - Reduce communication overhead compared to unstructured overlays
  - Enable efficient key-based resource lookup
    - Overlay structure keeps lookup cost limited
  - Bounded lookup complexity
  - Complexity guarantees also for peer join/leave

- Trade-offs/cons:

in sostanza il trade-off è mantenere consistente la rete strutturata che ha appunto costi in gestire nuovi nodi e quando un nodo lascia la rete, perché bisogna ridistribuire le chiavi in quella parte della rete tra i nodi che sono attivi

  - Peer join/leave is more expensive
  - Overlay structure must be maintained

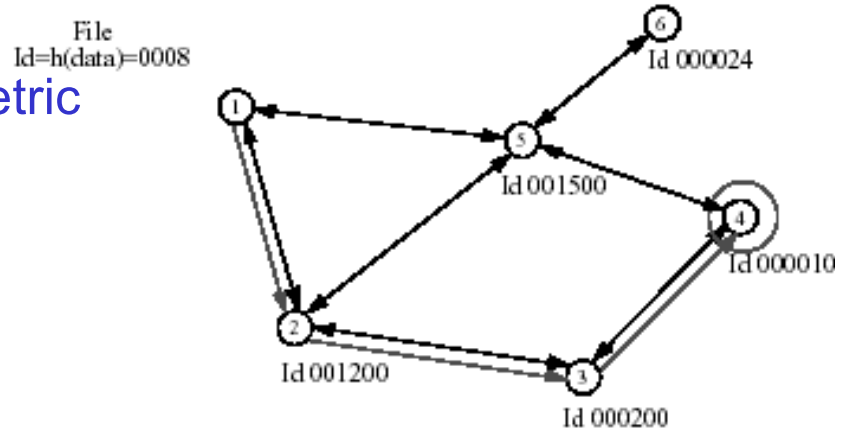
Routing in structured overlays

- Basic concepts
  - Each peer is responsible for a subset of resources and knows a subset of peers according to the overlay structure
  - Each resource is assigned a GUID
  - Each peer is assigned a GUID
  - GUIDs are computed via a secure hash function
  - Same large identifier space used for peers and resources GUIDs
  - Lookup query is routed to the peer whose GUID is closest to resource GUID
    - Closest: defined using a distance metric
- Routing is based on **Distributed Hash Table (DHT)**
  - A distributed key-value store spread across all peers

partizioniamo i peer e le risorse per affidarle in maniera bilanciata

ricordiamo Global Unique IDentifier

come per gli unstructured



Distributed Hash Table

- Distributed abstraction of conventional hash table (HT) that maps keys to values
- Conventional HT
  - Stores (*key, value*) pairs in a table of size *M* se ho poche chiavi, ho grosso spreco di memoria
  - Uses a hash function to map a key to an index *0 … M*-1
  - Lookup time: O(1)
  - Must handle collisions because multiple keys may hash to same value che si risolvono usando le liste di collisione.
- DHT
  - Conceptually similar to conventional HT: map resource keythrough hashing to determine the responsible peer (bucket)
  - DHT buckets are spread across peers ovvero un nodo ha responsabilità su un subset di chiavi
- Key challenge: how to efficiently map a resource key to the peer responsible for that key's bucket?

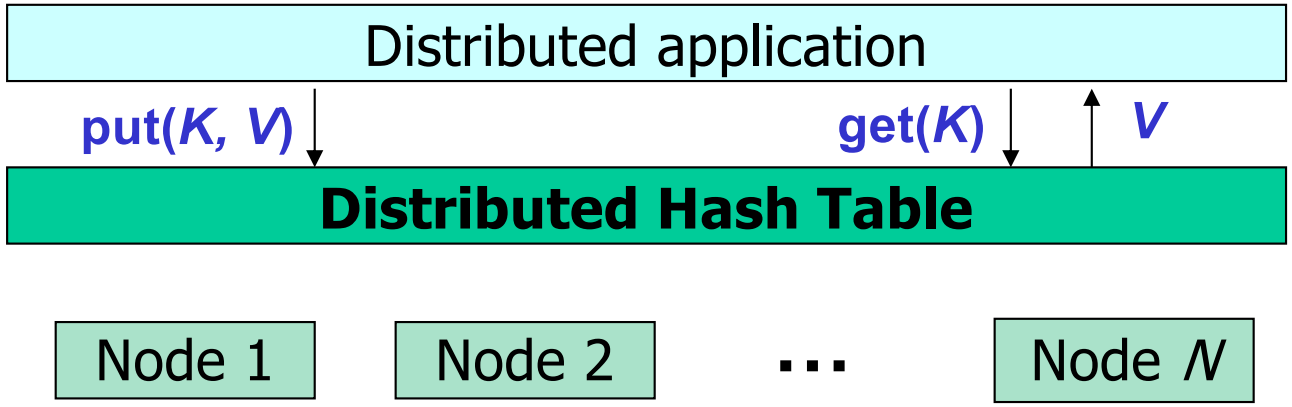
DHT: API

- DHT stores key-value pairs (key *K*, value *V*)
  - *K* = resource GUID, *V* = resource
- Common DHT API operations

  ***V* = get(*K*)**: retrieve *V* for *K* from the responsible node

  put(*K*, *V*): store *V* at the node responsible for *K*

  remove(*K*): delete the reference to *K* and its associated *V*



Why might DHT design be hard?

- Decentralized
  - No central authority
- Scalable
  - Low network traffic overhead
- Efficient nel senso che è difficile mantenere un efficienza alta con una latenza bassa.
  - Find items quickly (latency)
- Dynamic l'aspetto cruciale
  - Nodes fail or leave, new nodes join

Designing a DHT

- Mapping resources and nodes in DHTs
  - Both resources and nodes are mapped to the same identifier space using a hash function (e.g., SHA-1)
  - GUIDs are composed of *m* bits (usually *m* = 128 or 160)
  - Hash function is applied to metadata/data of resources (name, timestamp, content, ...) and of nodes (IP address, port, …)
- Resources are partitioned among nodes
  - Each node is assigned a contiguous range of keys and stores resources whose keys fall in its range
- Routing in DHT
  - Given *K*, map it to the node with GUID "closest" to *K*
- Resource replication can be used to improve availability and provide fault tolerance
  - E.g., neighbor replication: replicate resource on nodes that are neighbors in the ID space

ricorda K=GUID risorsa
se K=1034, e ho un nodo con GUID=1024, lui prende la key

Issues related to DHTs

- Avoid hotspots by evenly distributing key responsibility among peers per evitare colli di bottiglia e congestione in pochi nodi sempre uguali anche perché magari non sono nodi potenti a sufficienza
- Avoid remapping all keys if DHT size changes (i.e., when peers join or leave) si va solo a considerare la porzione di spazio dove avviene la join o la leave
  - **Consistent hashing** addresses these issues
- DHTs only directly support exact-match search IMPORTANTE
  - Since each resource is identified only by its key, to lookup a resource we need to know its key
  - Exact-match queries (e.g., based on resource name) are easy to perform
  - More complex queries (e.g., wildcard, range queries) are difficult and expensive to support
  - We consider only exact-match queries!

P2P systems based on DHTs

- Characterized by high scalability with respect to system size (i.e., *N*)
- Many proposals for DHT-based P2P systems, differ in:
  1. Definition of identifier space (and thus network topology)
  2. Selection of peers to communicate with (distance metric)
- Examples of DHT protocols:
  - **Chord** (MIT)
  - Pastry (Rice Univ., Microsoft)
  - Tapestry (Berkeley Univ.)
  - CAN (Berkeley Univ.)
  - **Kademlia** (NY Univ.)

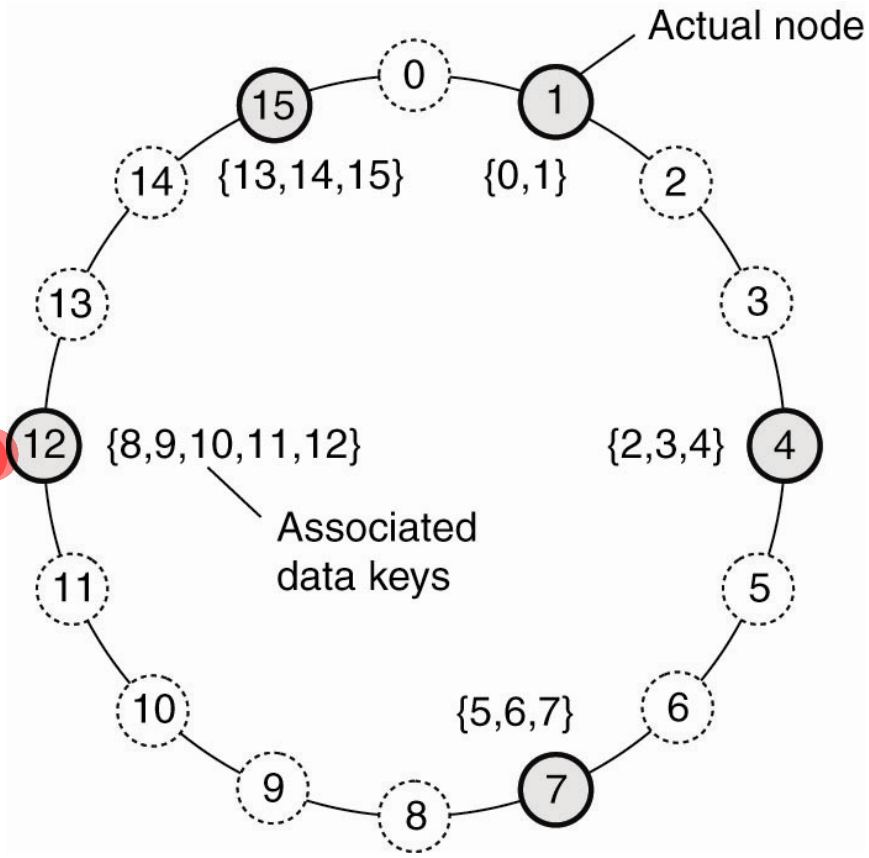
Why Chord?

- Elegant resource lookup algorithm for DHT
- Efficient: O(log *N*) message per lookup di meglio che vuoi?
- Scalable: O(log *N*) state per node
- Robust: survives massive failures
- Simple to analyze

Stoica et al., Chord: A scalable peer-to-peer lookup service for internet applications, ACM SIGCOMM 2001

Chord https://github.com/sit/dht/wiki

- Nodes (peers) and resources are mapped onto a **ring** using **consistent hashing**
- Each node is responsible for all keys between itself and its predecessor in counter-clockwise direction
  - Resource with key *K* is managed by the node whose identifier is the smallest id ≥ *K*
  - This node is called *succ*(*K*), ***successor*** of key *K*
    - E.g., *succ*(1)=1, *succ*(10)=12



Guarda anche appunti reti di calcolatori

- Distance between IDs is measured as linear difference on the ring: if x<=y, d(x, y) = y-x; else d(x,y) = 2^m -(x-y)

Consistent hashing

- Special hashing technique mapping both resources and nodes on the same identifier space (ring)
- Uses standard hash functions (e.g., SHA-1, MD5)
- Each node manages a contiguous interval of hash keys, not scattered keys
- Originally devised by Karger et al. (MIT, 1997) for Web distributed caching

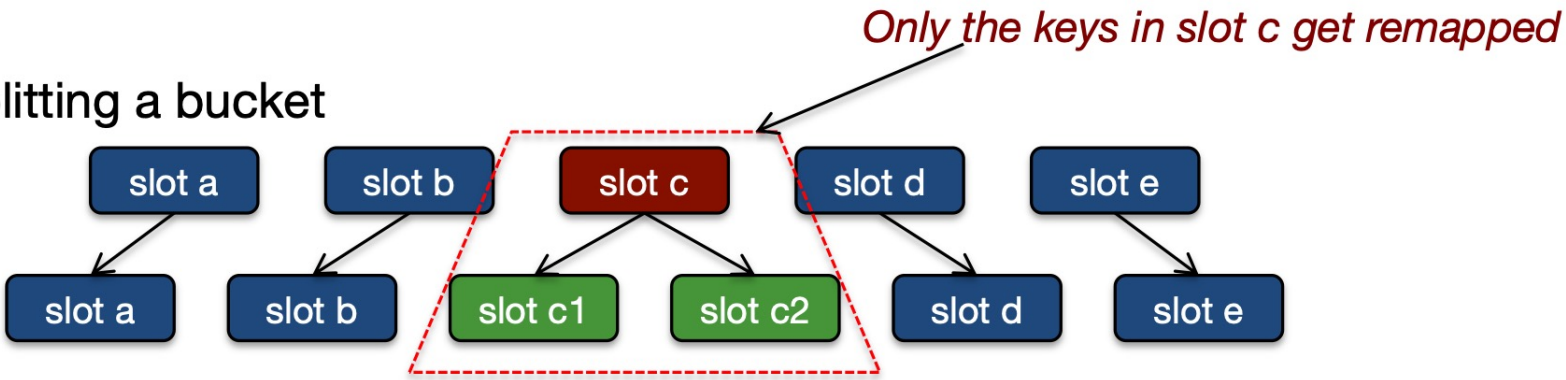
  Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web, STOC 1997
- Behind **Akamai** https://www.akamai.com/
- Widely adopted in real systems such as **Amazon Dynamo**, **Cassandra**, **Discord**, **Memcached**
- Details and Java implementation

  https://tom-e-white.com/2007/11/consistent-hashing.html

Chord: consistent hashing

- Consistent hashing is key to Chord robustness and performance
  1. When DHT resizes (nodes join or leave): minimal key remapping as most keys remain assigned to the same node
  – Practical impact: peers can join and leave with minimal disruption

Example: splitting a bucket



  2. All nodes are assigned roughly the same number of keys: load balancing among peers

Chord: towards routing

- The simplest approach: traverse the ring one node at a time
- Can we do better than O(*N*) lookup? ovvero flooding

- Another simple approach for great performance
  - Every node knows all others
  - When a peer receives a query, it searches its table of nodes to find the node that owns that key
  - ✓ Lookup is O(1) dht classica
  - ✗ Costly updates on join/leave
  - ✗ Not scalable for many peers (large tables)

- Chord uses a compromise to avoid large tables at each node: **finger table**
  - A partial list of nodes, progressively more distant

Chord: finger table

- Finger table (FT): routing table of each node
  - FT has *m* rows, with *m* = # GUID bits
  - If FT_p is the FT of node *p*, then $FT_p[i] = succ(p + 2^{i-1}) \bmod 2^m$, $1 \le i \le m$
    - $succ(p+1)$, $succ(p+2)$, $succ(p+4)$, $succ(p+8)$, $succ(p+16)$, …

quindi ad esempio se p=8, si avrà la tabella con 8+1,8+2,8+4 ecc. Attenzione: se ad esempio il nodo con ID 12 non esiste ma esiste il nodo 14 allora 8+4 darà come responsabile il nodo 14

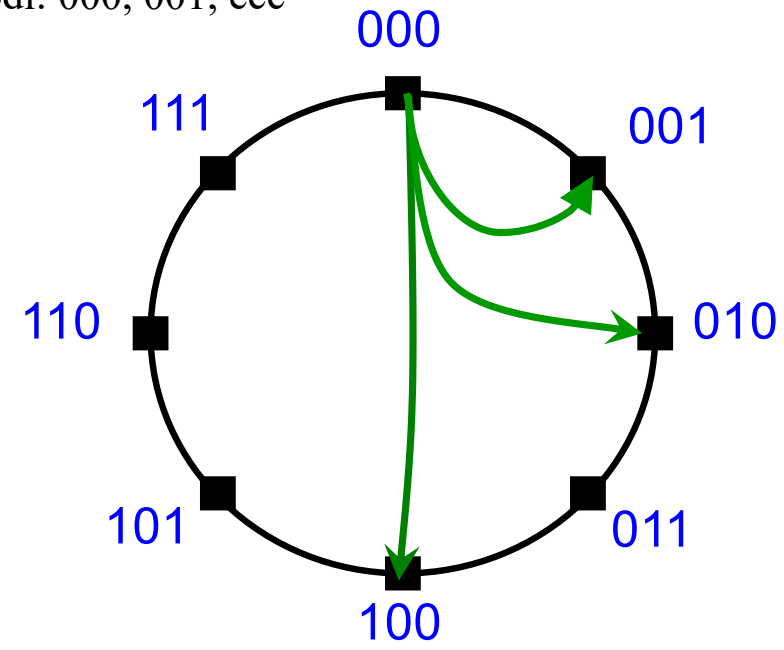
- Example with *m*=3 quindi 3 bit e tutte le sue conseguenti permutazioni come ID dei nodi: 000, 001, ecc
  - Finger table of node 0?

$FT_0[1]=0+2^0=1$

$FT_0[2]=0+2^1=2$

$FT_0[3]=0+2^2=4$

quindi ricorda m è sia il numero delle righe sia il numero di bit che si utilizzano per enumerare i GUID



Chord: FT's characteristics

- Each node stores information about only a small number of other nodes
  - Only *m* rows
- Each node knows more about nodes closely following it than about nodes farther away
- However, a node's FT generally does not contain enough information to directly determine the successor of any key
  - We need a routing algorithm to map each key *K* into *succ*(*K*)

Chord: routing algorithm

- How to map key K into $succ(K)$ starting from node p
  - If K belongs to the ring portion managed by p, lookup ends
  - If $p < K \leq FT_p[1]$, p forwards the request to its successor
  - Otherwise, p forwards the request to node q with index j in FT_p by considering the clockwise ordering

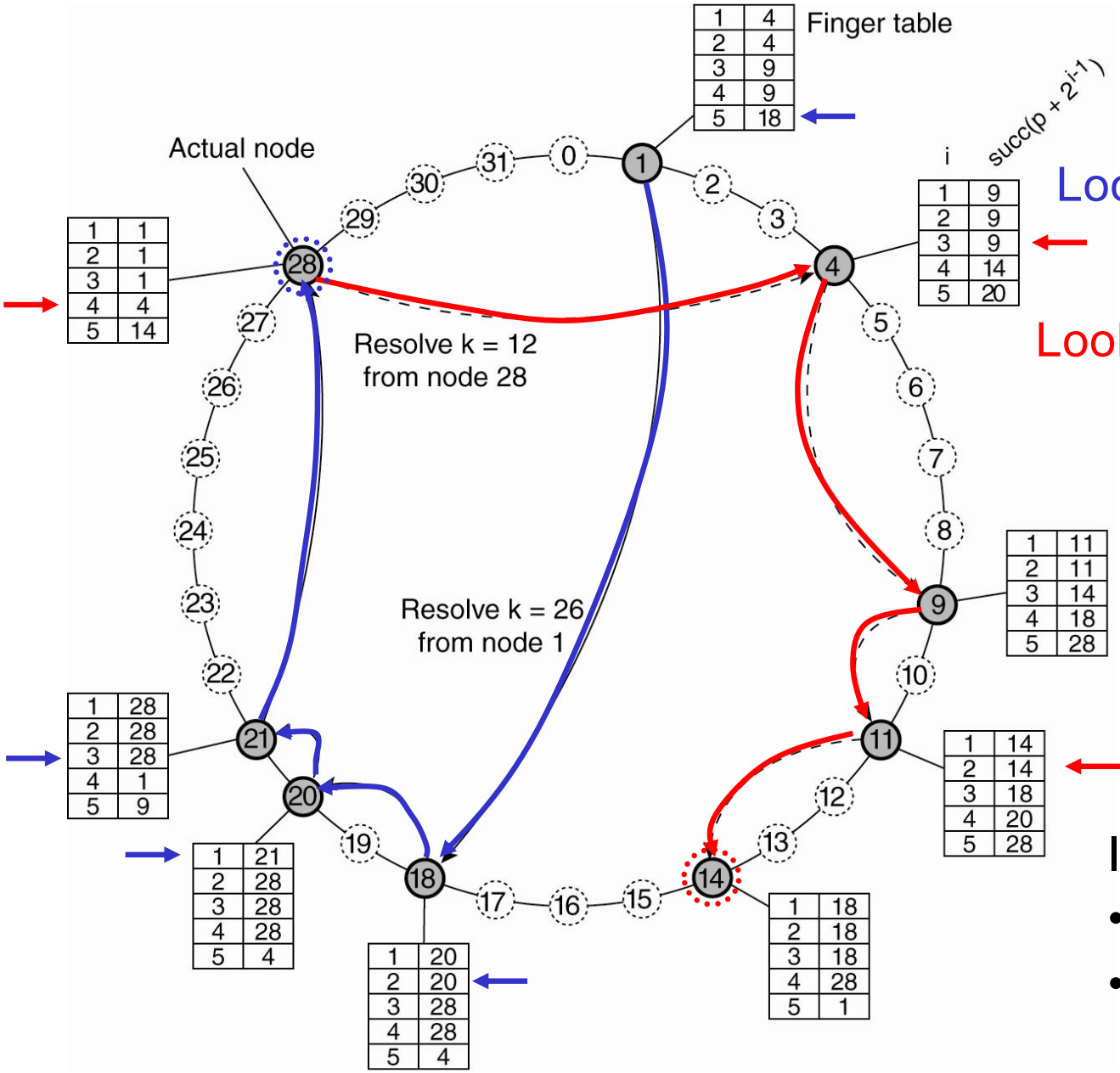
$$FT_p[j] \leq K < FT_p[j+1]$$

q is the farthest node from p whose key is less than or equal to K

Ogni nodo propaga una query per la chiave K al finger più distante che precede K, in senso orario. Si arriva fino al nodo N tale che N sia predecessore di K ed il successore di N sia successore di K. arrivato al successore di K ho trovato il nodo

- Features
  - It quickly reaches the vicinity of the searched point, then make gradually smaller jumps
  - Lookup cost: O(log N), where N is the number of nodes … not as cool as O(1) but way better than O(N)

Chord: routing algorithm example



Lookup for key 26 from node 1

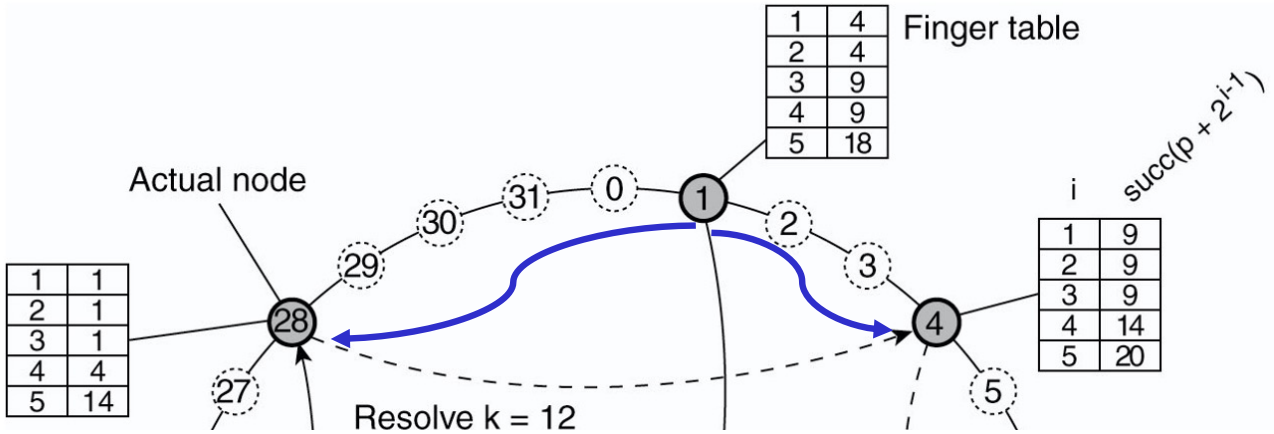
Lookup for key 12 from node 28

In the example:

- m=5
- Keys from 0 to 2^5-1

Chord: node join and leave

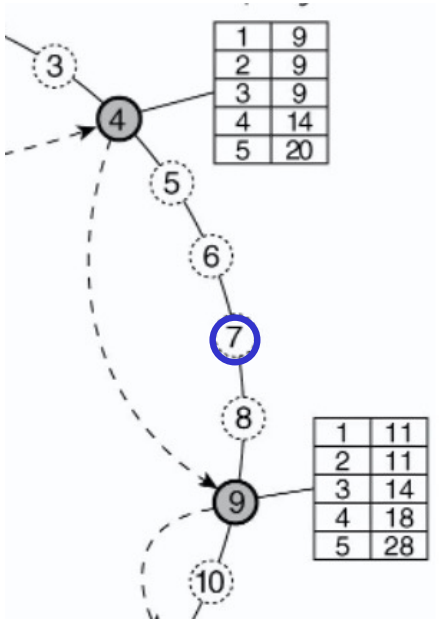
- In addition to maintaining a successor pointer, each node also keeps the pointer to its predecessor (i.e., linked list) to simplify ring maintenance operations
  - Predecessor of node *p* is the first node met in counter-clockwise direction starting at *p*-1
  - When a node joins or leaves, successor and predecessor pointers need to be updated



Chord: node join and leave

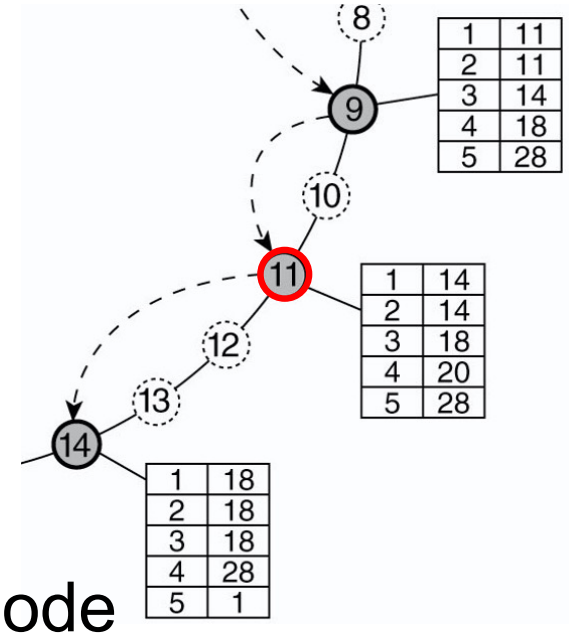
- When node *p* joins the Chord ring, it has to find its place:
  - Finds successor: asks another node to find its successor *succ*(*p*+1)
  - Joins the ring, linking to its successor and informing its successor of its presence
  - Initializes its FT by looking for $succ(p + 2^{i-1})$, $2 \leq i \leq m$
  - Informs its predecessor to update its FT
  - Transfers the keys from its successor to itself

- Example: node 7 joins
  - Node 7's successor is node 9
  - Node 9's predecessor changes to node 7
  - Node 4's successor changes to node 7
  - Keys 5, 6, and 7 are transferred to node 7



Chord: node join and leave

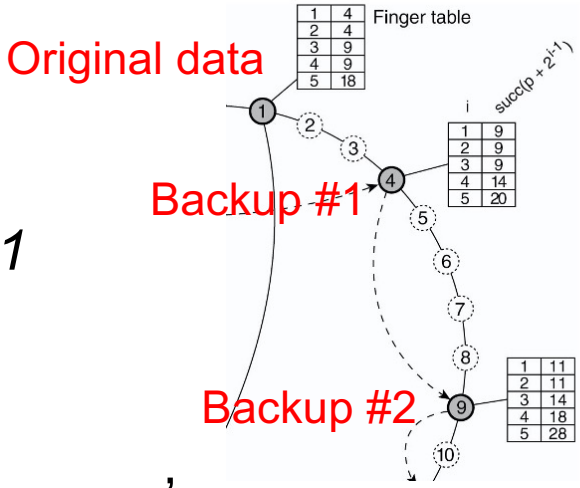
- When node *p* voluntary leaves the Chord ring:
  - Transfers the keys it is responsible for to its successor
  - Updates successor's predecessor pointer to the node that precedes *p*
  - Updates predecessor's successor pointer to *p* successor
- Example: node 11 leaves
  - Keys 10 and 11 are transferred to node 14
  - Node 14's predecessor changes to node 9
  - Node 9's successor changes to node 14
- Join/leave operations require $O(\log^2 N)$



- To keep the finger tables updated, each node periodically executes a ring stabilization procedure
  - Nodes can also leave the overlay abruptly because of failure

Chord: fault tolerance

- Nodes might crash
  - (*K, V*) should be replicated
  - Create *R* replicas, storing each one at *R-1* successor nodes in the ring
- Need to know multiple successors
  - A node needs to know how to find its successor's successor (or more)
    - Easy only if it knows all nodes!
- When a node is back up, it needs to:
  - Check with successors for updates of data it owns
  - Check with predecessors for updates of data it stores as backups



Chord: summing up

- Pros
  - ✓ Simple and elegant
  - ✓ Load balancing
    - Keys are evenly distributed among nodes
  - ✓ Scalability
  - ✓ Efficient lookup: O(log *N*)
  - ✓ Robustness
    - Nodes periodically update their finger tables to reflect changes in the overlay, keeping the system robust
- Cons
  - ✗ Physical network proximity not considered
  - ✗ Expensive support for non-exact matching
  - ✗ Incorrect Chord's original ring-maintenance protocol

    Reasoning about identifier spaces: How to make Chord correct, IEEE TSE, 2017 https://arxiv.org/pdf/1610.01140.pdf

se un nodo è lontano fisicamente, mentre vicino logicamente, la latenza c'è lo stesso
ricorda CHORD come la DHT va bene per EXACT MATCHING

Kademlia

- A popular structured P2P overlay network
- Based on DHT principles
- Designed for efficient, decentralized resource lookup and routing
- Every node and key is identified by a 160-bit GUID (typically SHA-1 hash)

Globally Unique IDentifier

- Each value is stored at the k nodes whose node IDs are closest to the key ID
- Supports parallel and asynchronous queries to improve speed and reliability
- Provides robustness to node churn

ricordando che churn è il tasso di abbandono o comunque l'abbandono da parte di un nodo

Maymounkov and Mazieres, Kademlia: A peer-to-peer information system based on the XOR metric, IPTPS 2002

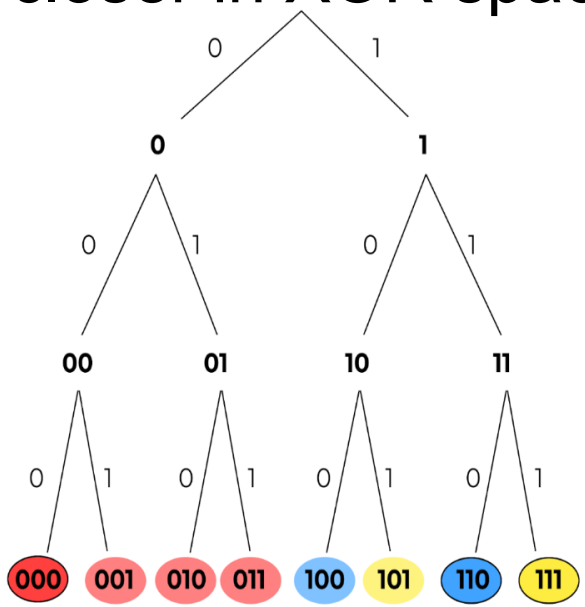
Kademlia: XOR distance and routing

- Uses XOR metric to measure distance between node IDs and keys e non la linear difference come CHORD
- Distance between nodes/keys *x* and *y* is:

$$d(x, y) = x \oplus y$$

  - Symmetric, fast computation
- XOR defines a binary tree structure over the ID space
- Prefix-based routing: nodes closer in XOR space have more similar prefixes
  - **Longest prefix matching**

Example: 3-bit identifier space

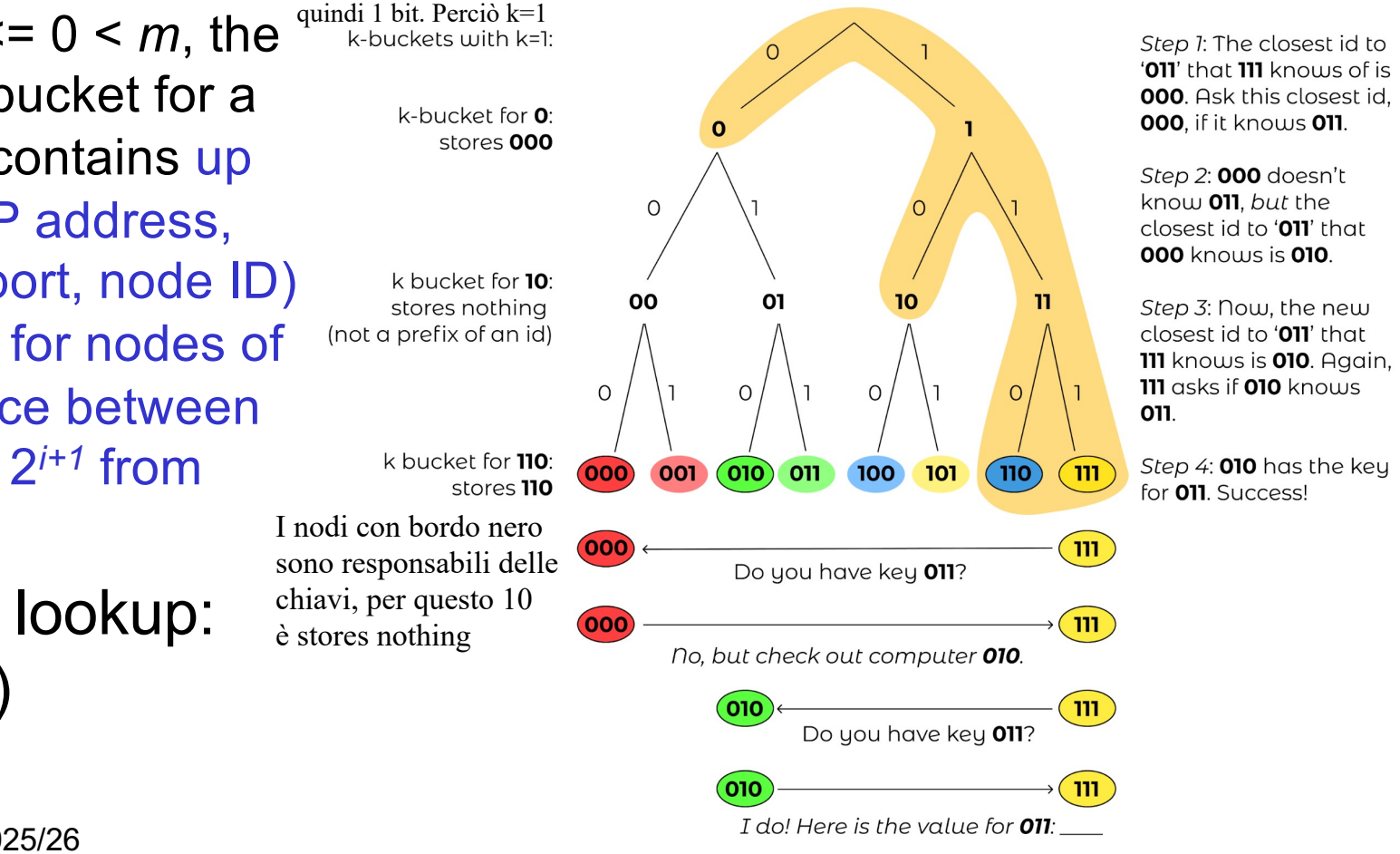


Kademlia: XOR distance and routing

- Each node maintains a routing table composed of *m* k-buckets, with *m* = # GUID bits
- k-buckets store contact info for nodes at exponentially increasing XOR distances
  - For *i* <= 0 < *m*, the *i*-th k-bucket for a node contains up to *k* (IP address, UDP port, node ID) triples for nodes of distance between 2^i and 2^{i+1} from itself
- Efficient lookup: O(log *N*)

Ogni nodo mantiene una tabella di routing che è divisa in bucket.
Un bucket contiene informazioni sui nodi che sono a distanza di k bit (dove k è un parametro) dalla chiave del nodo.

Calcolo per prima cosa 111 XOR 011 e trovo 100 quindi 1 bit. Perciò k=1



I nodi con bordo nero sono responsabili delle chiavi, per questo 10 è stores nothing

Kademlia: lookup and pros

- Lookup starts by querying nodes closest to the target key
- Queries sent in parallel to multiple nodes
- Results returned with nodes closer to the key until the responsible node is found
- Pros
  - ✓ Efficient lookups with O(log *N*) complexity
  - ✓ Handles churn gracefully with node replacement in k-buckets
  - ✓ Parallel queries improve latency and reliability
  - ✓ Widely used in real-world P2P systems (e.g., BitTorrent's DHT)
- Kademlia visualization: https://kelseyc18.github.io/kademlia_vis/

DHTs: summing up

- DHTs in retrospective
  - Promising for data lookup in large P2P systems
  - Decentralization enhance load balancing and fault tolerance
  - But: security remains difficult
  - But: churn is a problem, particularly if log(*N*) is big
  - DHTs haven't fully met initial high expectations

nel caso che un nodo sia malevolo e sfrutta il sistema a sua vantaggio senza subire grosse conseguenze. altro contro è il DDoS

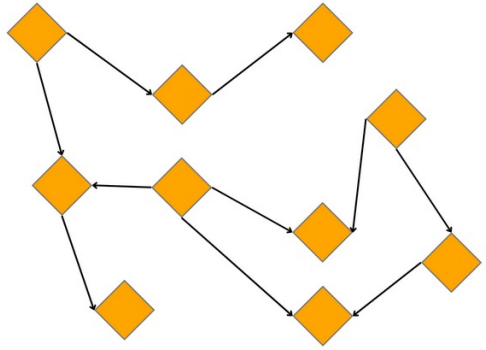
- What DHTs got right
  - Consistent hashing: effective tool to spread load across machines (e.g., used in Amazon Dynamo, Cassandra, etc.)
  - Incremental scalability: capacity grows with added nodes
  - Replication: ensures high availability and efficient recovery
  - Self-management: requires minimal configuration
  - No single point of failure or centralized control

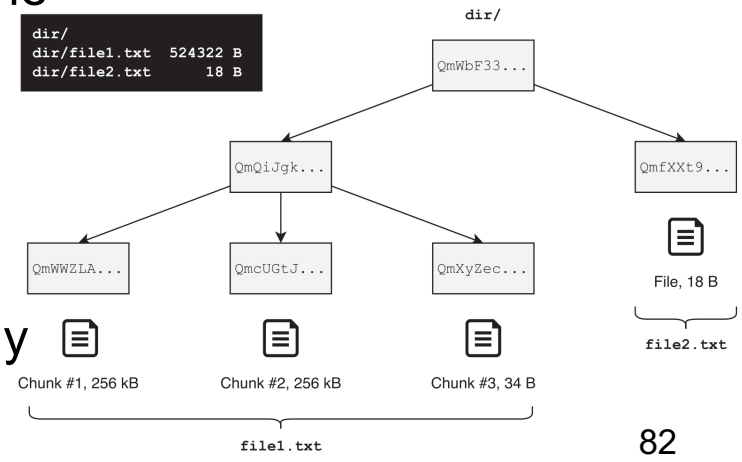
IPFS

- IPFS = InterPlanetary File System https://ipfs.tech/
- A P2P, distributed file system with global scale
- Aims to replace location-based web (URLs) with content-based addressing
- Think: Web + BitTorrent + Git, all in one
- Motivations
  - Current web: relies on central servers and location addressing (e.g., https://example.com/file.jpg)
  - Problems:
    - Censorship
    - Single points of failure
    - Redundancy and inefficiency
- IPFS: store, retrieve, and locate data based on the fingerprint of its actual content rather than its name or location

IPFS: content addressing

- Files
  - Split into small blocks
  - Each block has a unique cryptographic hash of content (**CID**)
    - Immutable: if content changes, CID changes
  - Blocks are linked via cryptographic hashes
  - Together, they form a **Merkle DAG**
- Merkle DAG
  - **DAG**: Directed Acyclic Graph Grafo orientato aciclico
  - **Merkle tree**: data structure where each node (or block) includes a hash of its contents
    - If a node has children, it also includes the hashes of those children
  - Benefits
    - Can verify integrity of any block
    - Can reuse blocks (data deduplication)
    - Can retrieve parts of a file independently
    - Versioning support (like Git)





IPFS: DHT

- DHT-based discovery
  - IPFS uses a DHT to locate content by CID
  - Variant of Kademlia DHT: modified and asynchronous version adapted for IPFS needs

Feature	IPFS Kademlia DHT
Node ID	256-bit (SHA-256 hash)
Key (CID)	Multihash-based content identifier
Routing able	Uses k-buckets as Kademlia
Lookup	Asynchronous + parallel lookup
Content providers	Nodes store "provider records" for CIDs (not content itself)
Data retrieval	DHT used to find who has the content (not content itself)

- For details, see https://docs.ipfs.tech/concepts/how-ipfs-works

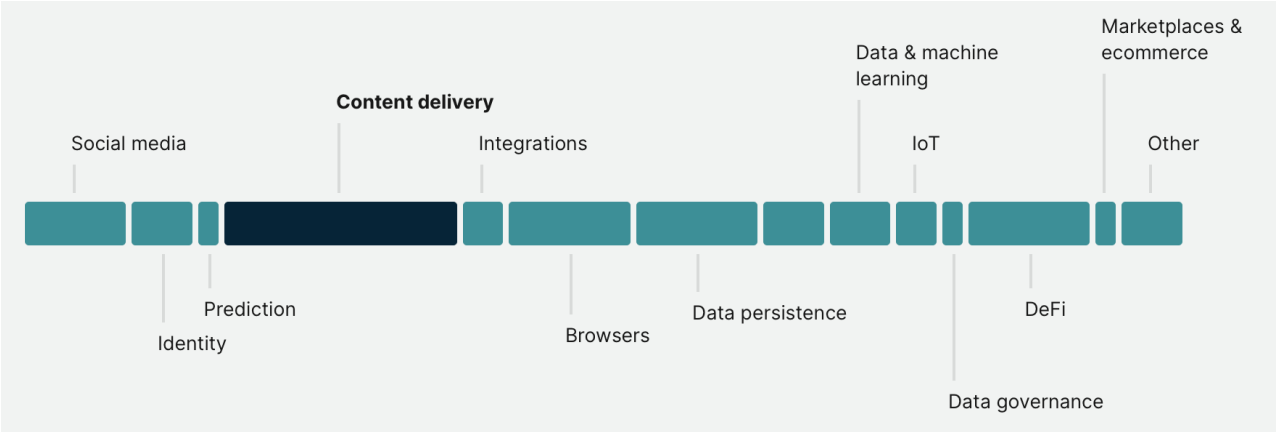
IPFS: DHT

- How IPFS uses the DHT
- Publishing content
  - When a node adds content, it advertises in DHT that it can provide the content identified by a given CID
- Retrieving content
  - Another node performs a DHT lookup using the CID to find which peers provide the content
- Fetching
  - Once providers are found, content is fetched directly via a P2P transfer (e.g., Bitswap protocol), possibly in parallel
- Two DHT modes in IPFS
  - Public DHT (default): nodes use a shared, global DHT to announce/provide content; access is through an IPFS public gateway
  - Private DHT: used in local or restricted networks (e.g., IPFS clusters)

IPFS: benefits and limitations

- Pros
  - ✓ Decentralized: no single point of failure
  - ✓ Versioned and immutable content
  - ✓ Efficient: caching (IPFS caches and stores data on local servers), deduplication, and content reuse
  - ✓ Censorship-resistant
  - ✓ Offline-first: content can be fetched locally or over LAN
- Limitations and challenges
  - ✗ Not ideal for dynamic or mutable data
  - ✗ Not a storage network by itself: depends on peers being online
  - ✗ Still evolving: performance and tooling improving but not perfect

IPFS: use cases



- Content delivery
  - E.g., Audius https://audius.org, Fleek https://fleek.xyz
- NFT and blockchain storage
  - Filecoin https://filecoin.io, OpenSea https://opensea.io
- Decentralized apps (dApps)
- Scientific data sharing and archiving

- IPFS and other distributed systems use libp2p
  - Open source networking library https://libp2p.io

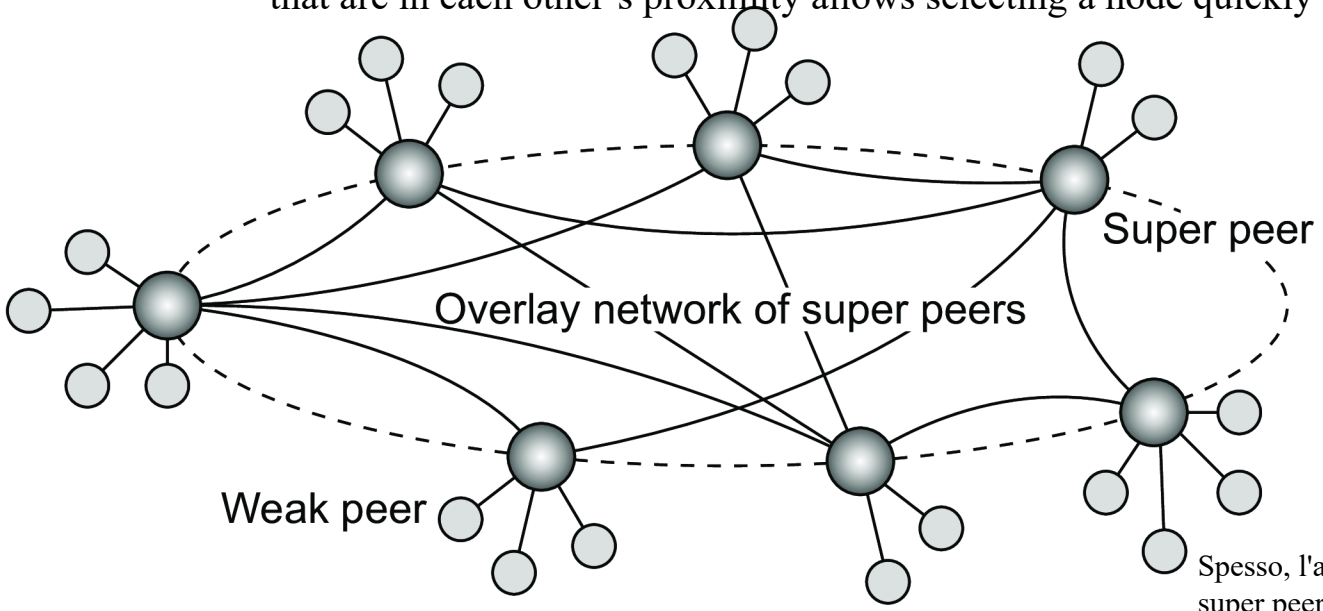
Hybrid architectures

- So far, we have examined centralized and decentralized architectures
- Hybrid architectures combine elements of centralized and decentralized organizations
- Goal: leverage the benefits of both models
- Examples of hybrid architectures (with different degree of decentralization)
  - Super-peer network
  - BitTorrent: tracker + peer swarm
  - Blockchain: decentralized by design, but in practice often hybrid
    - Many blockchain platforms combine decentralized infrastructure with centralized or semi-centralized control

O meglio hierarchically organized peer-to-peer network

Hybrid architectures: super-peer network

- It is sometimes sensible to break the symmetry in pure P2P networks: super peers (e.g., Gnutella 2)
- Super peers (index servers) improve lookup performance by acting as broker that collects data on resource usage and availability for several nodes that are in each other's proximity allows selecting a node quickly with sufficient resources



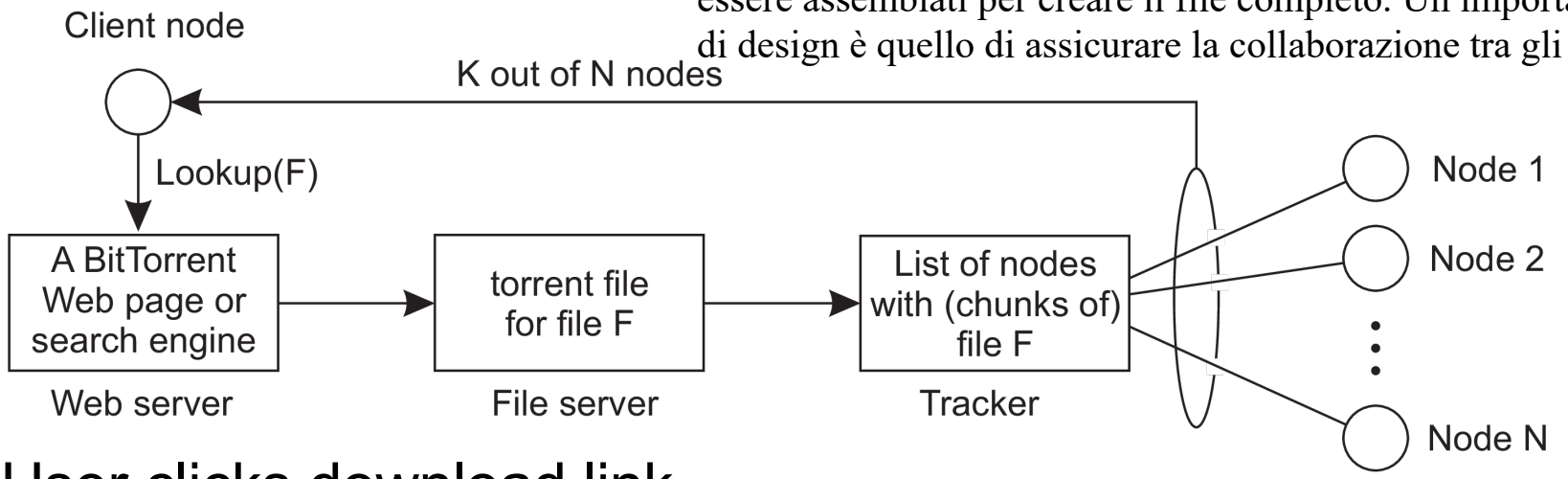
- Issues to address
  - Static or dynamic association of peer-super peer
  - How to select super peers nasce il leader election problem

Spesso, l'associazione tra un weak peer e il suo super peer è fissa: quando un weak peer si unisce alla rete, si attacca a un super peer finchè non lascia la rete. Tale associazione fissa non è sempre la migliore soluzione. In file-sharing networks, sarebbe meglio per il weak peer attaccarsi a un super peer che mantiene un indice di file di cui il weak peer è attualmente interessato.

Hybrid architectures: BitTorrent

- Unstructured P2P system for file sharing
  - Alternatively, can use a structured DHT based on Kademlia for trackerless operation (not examined)
- Steps to download file F

L'idea è che quando un utente sta cercando un file, lui scarica pezzi del file in parallelo da altri utenti finché i pezzi scaricati possono essere assemblati per creare il file completo. Un importante obiettivo di design è quello di assicurare la collaborazione tra gli utenti.



1. User clicks download link
   - BT client gets a torrent file, containing tracker reference
   - Tracker: server keeping track of active peers who have (chunks of) F; acts as bootstrapping node for the torrent
2. BT client contacts tracker
   - Tracker replies with a list of peers who have (chunks of) F

Hybrid architectures: BitTorrent

3. BT client downloads chunks of F from peers, joining a swarm of downloaders, who in parallel get file chunks but also distribute downloaded chunks to other peers

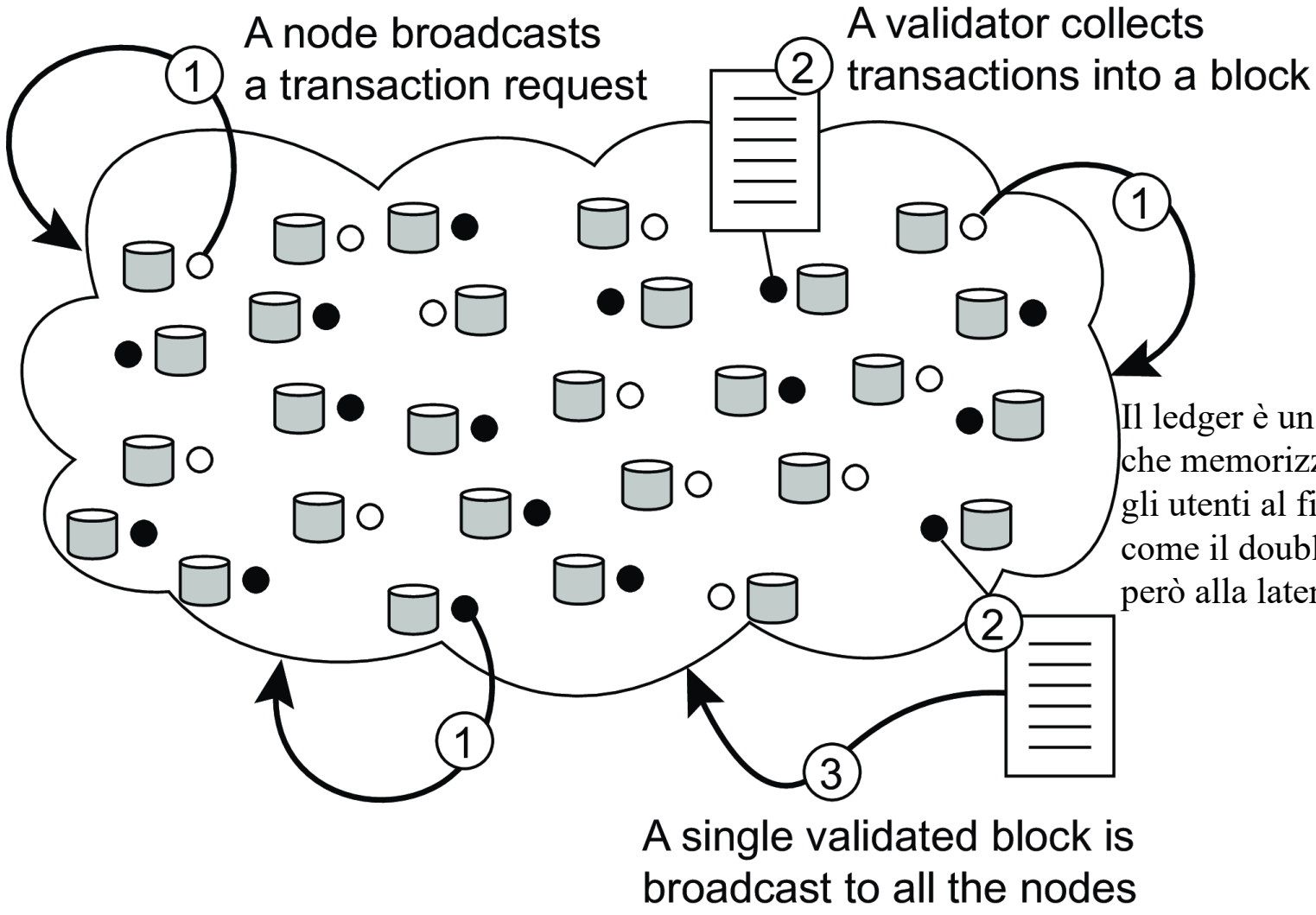
- BitTorrent incentivizes peers to exchange data
  - Chunk selection based on rarest piece first
  - Bandwidth allocation based on tit-for-tat
- Rarest piece first
  - Chunks that are most uncommon in the network are preferably selected for download
  - Goal: make file exchange more robust against node churn
- Tit-for-tat in base al ratio c'è insomma rate limiting
  - Peers decide to who to upload data based on how much data they have downloaded data from that peer
  - Goal: prevent peers from free-riding (only downloading without uploading) If a node P notices that a node Q is downloading more than it is uploading, P can decide to decrease the rate at which it sends data to Q.

Blockchain

- A transaction (e.g., Alice transfers €10 to Bob) needs to be:
  - Validated: verifying that transaction is legal (e.g., no malicious action, no double spending)
  - Stored: for auditing purposes
  - How to validate transactions and where to store transactions?
- What kind of transactions?
  - Not only cryptocurrency transfers (e.g., Bitcoin)
  - But also: identification documents, resource usage and allocation, electronic voting, health records, etc.
- A blockchain acts as a collaborative data store of transactions, replicated among untrusted peers, and guarantees a consistent and tamper-resistant view of all transactions by peers
  - A type of **distributed ledger technology (DHT)**

Blockchain: working principle

- Each peer stores a local replica of the ledger

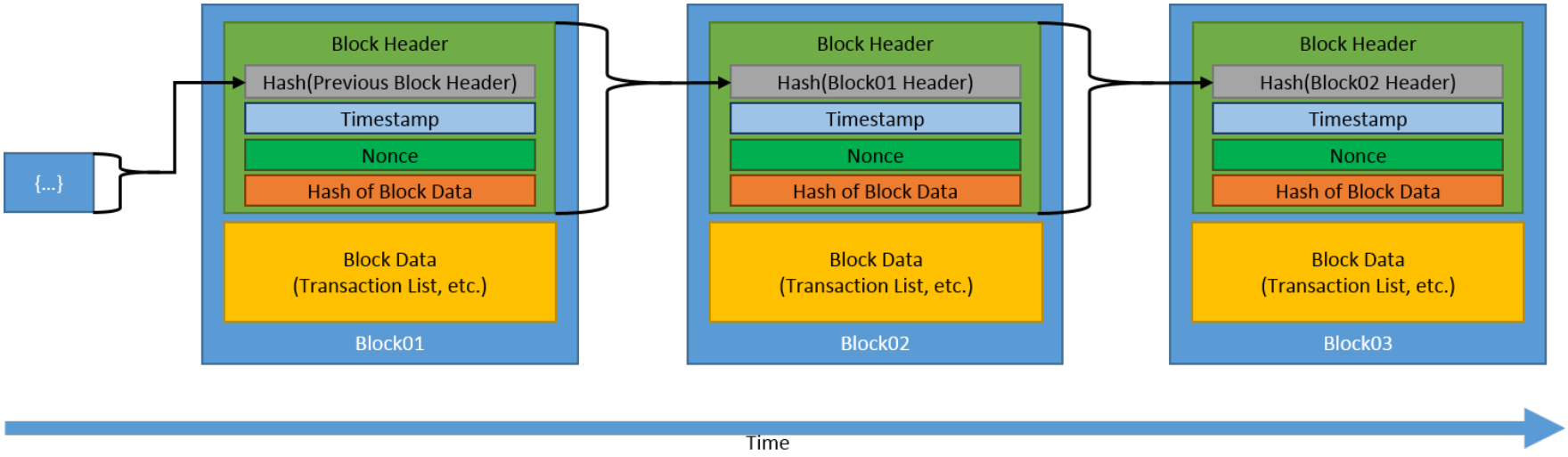


Il ledger è un database distribuito che memorizza le transazioni tra gli utenti al fine di evitare problemi come il double spending. Attenzione però alla latenza!

Blockchain: blocks

- Transactions are grouped into blocks
  - Block: header + body (set of transactions)
- Blocks are organized into an unforgeable append-only chain
  - A block is connected to the previous one by including a unique identifier (hash) based on previous block header
  - Changing a block invalidates all subsequent blocks
- Each block in the blockchain is immutable ⇒ massive replication

possibile ripasso con file di reti di calcolatori

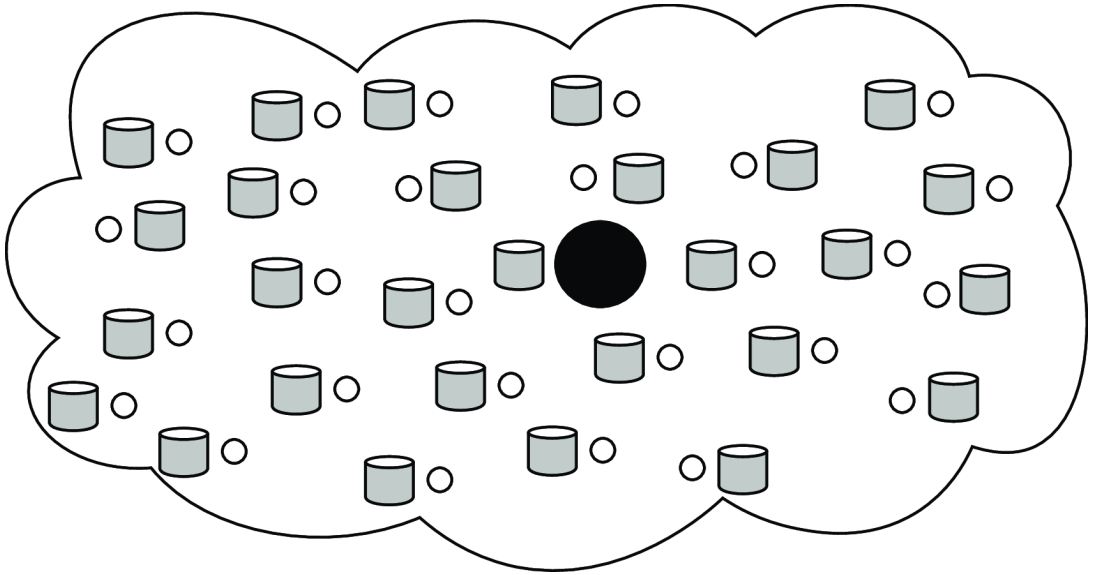


Blockchain: the key aspect

- Which validator is allowed to append a block of validated transactions to the chain?
- Deciding on which validator can move ahead requires (distributed) consensus

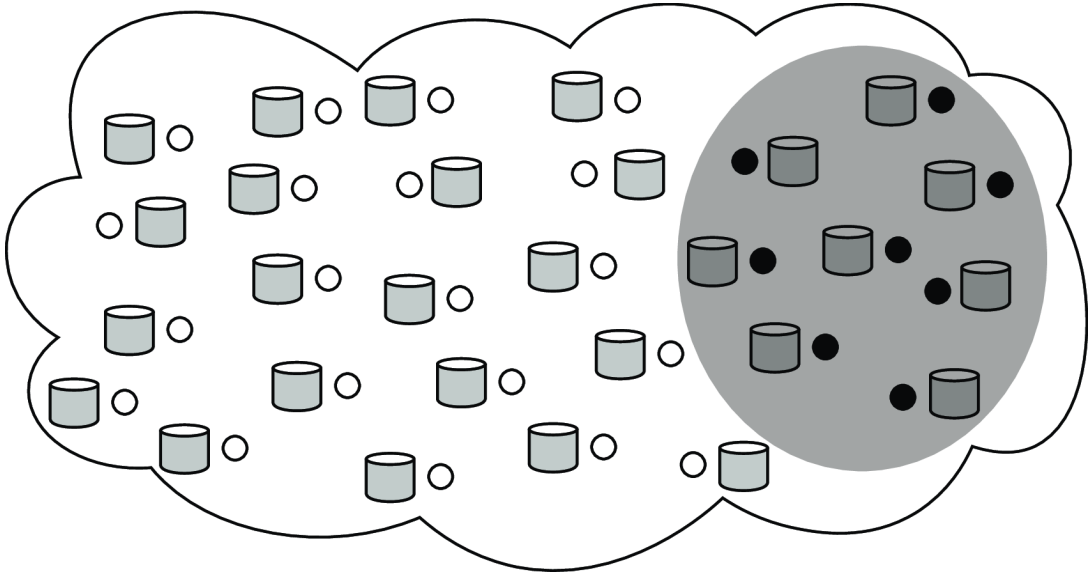
Appending a block: (distributed) consensus

- Centralized solution
  - A trusted single entity decides on which validator can go ahead and append a block
  - ✗ Does not fit the design goals of blockchain (no central authority)



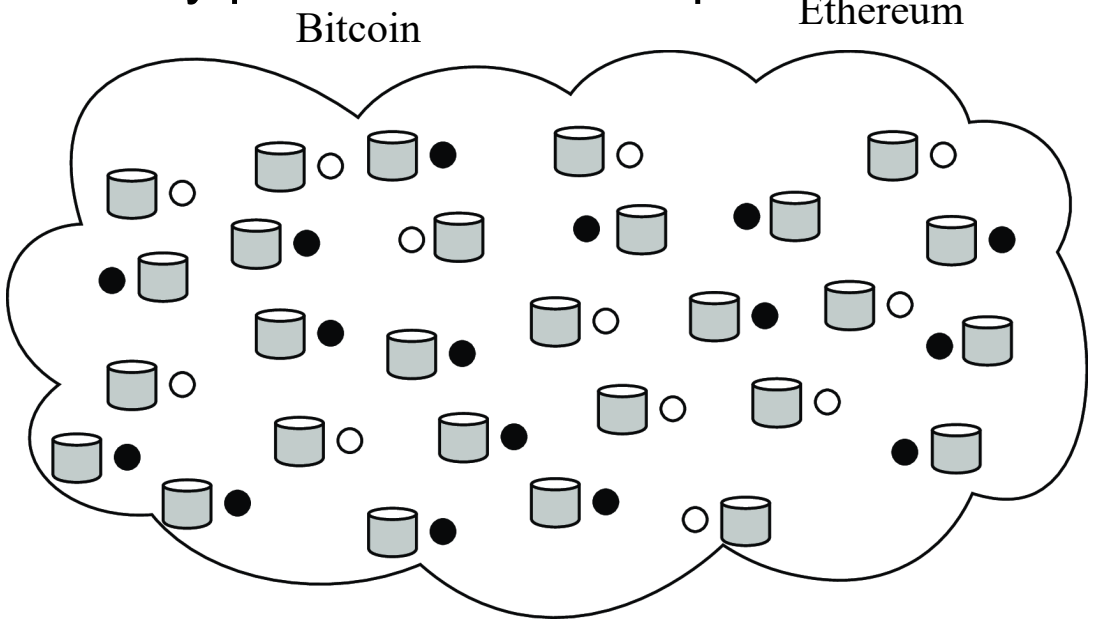
Appending a block: distributed consensus

- Distributed solution (permissioned blockchain)
  - A selected, relatively small group of servers jointly reach consensus on which validator can go ahead
  - None of these servers needs to be trusted, as long as roughly 2/3 behave according to their specifications
  - In practice, only a few tens of servers can be accommodated



Appending a block: distributed consensus

- Decentralized solution (permission-less blockchain)
  - All nodes collectively participate to validate transactions and engage in a leader election. Only the elected leader is allowed to append a block of validated transactions
  - E.g., Bitcoin, Ethereum
  - ✗ Large-scale, decentralized leader election that is fair, robust, secure, energy-efficient and so on, is far from trivial
    - We will study proof-of-work and proof-of-stake in futuro



Self-adaptive software systems

"Intelligence is the ability to adapt to changes"
S. Hawking

- We aim to a software system capable of adapting its operations at run-time with respect to itself and the environment: a self-adaptive (or autonomic) software system
  - Applications of self-adaptive systems in many computing environments
    - Cloud computing
    - Edge/fog computing
    - Compute continuum
    - HPC
    - Cyber-physical systems

I sistemi oggigiorno sono altamente automatizzati, questo perché altrimenti si perderebbero molte risorse umane e non per adattare il sistema al vario carico di lavoro per esempio.

Self-adaptive software systems

Definizione

- **Autonomic computing**: computing paradigm designed to the complexity and heterogeneity of IT systems through automatic adaptations
  - Inspired by human autonomic nervous system, which controls vital functions (heart rate, digestion, body temperature, ...) masking complexity from conscious human control

- Key capabilities of self-adaptive (or autonomic) software systems:
  - Autonomously manage their functionalities and goals, with little or no human intervention
  - Adapt to changes and uncertainty in both the environment and the system itself appunto self-adaptive

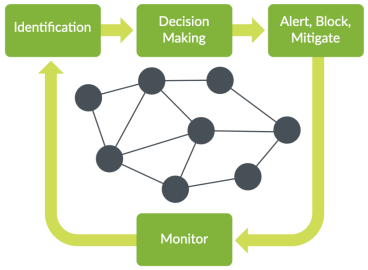
Kephart and Chess, The vision of Autonomic Computing, *IEEE Computer*, 2003

Self-adaptation is everywhere

kubernetes

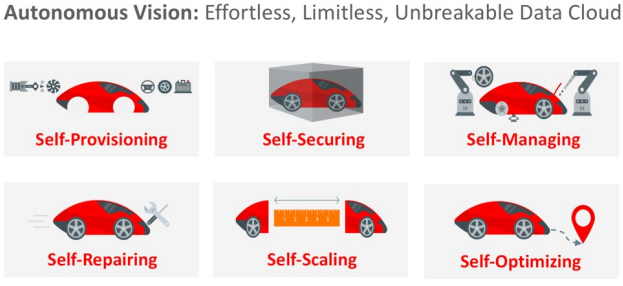
kubernetes.io

più gettonato tra i sistemi open-source



Self-healing networks, www.juniper.net

aws.amazon.com/ec2



Autonomous database, www.oracle.com/autonomous-database/what-is-autonomous-database/

Goals of self-adaptive systems

- A self-adaptive (***self-****) software system is able to self-manage, pursuing the following goals:
- Self-optimize
  - Capability of system to optimize its resource usage or performance while providing its required quality goals
  - E.g., change placement of application components onto system nodes to satisfy application response time

non ha senso avere due server che sono utilizzati al 40%, meglio aggregare tutto il lavoro su un singolo server per essere utilizzato al 80% che resta ancora libero.

- Self-heal
  - Capability of system to discover, diagnose and recover from faults to provide its required quality goals or degrade gracefully otherwise
  - E.g., detect crashed nodes and exclude them from serving requests

vedremo nelle lezioni future come il gossiping, è utilizzato da amazon proprio per gestire il self-heal e capire quali macchine non funzionano

Goals of self-adaptive systems

- Self-configure appunto come visto nella lezione precedente CHORD
  - Capability of system to automatically integrate new elements, without interrupting the system's normal operation, or tune some configuration parameters
  - E.g., discover new nodes and add them to serve requests

- Self-protect aspetto importante e delicato
  - Capability of system to detect anomalies (i.e., intrusion detection) and react to intrusion and attack actions and its consequences (i.e., intrusion response) so to protect from security threats
  - E.g., in a network of IoT devices detect a jamming attack that corrupts network traffic and adapt packets schedule

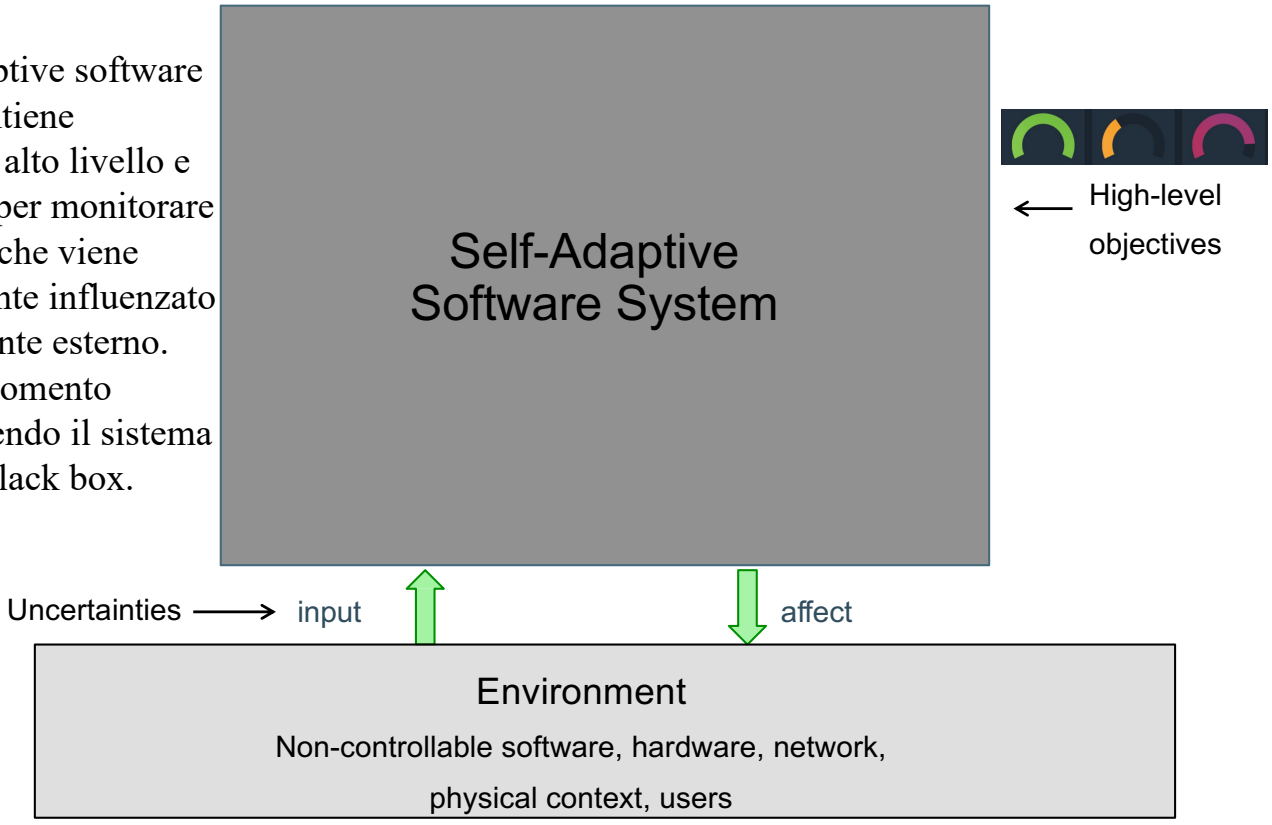
How to achieve the goals of a self-* system?

- The system should know its internal state (self-awareness) and the current external operating conditions (self-situation)
- Should identify changes regarding its state and the surrounding environment (self-monitoring)
- And should adapt consequently (self-adjustment)

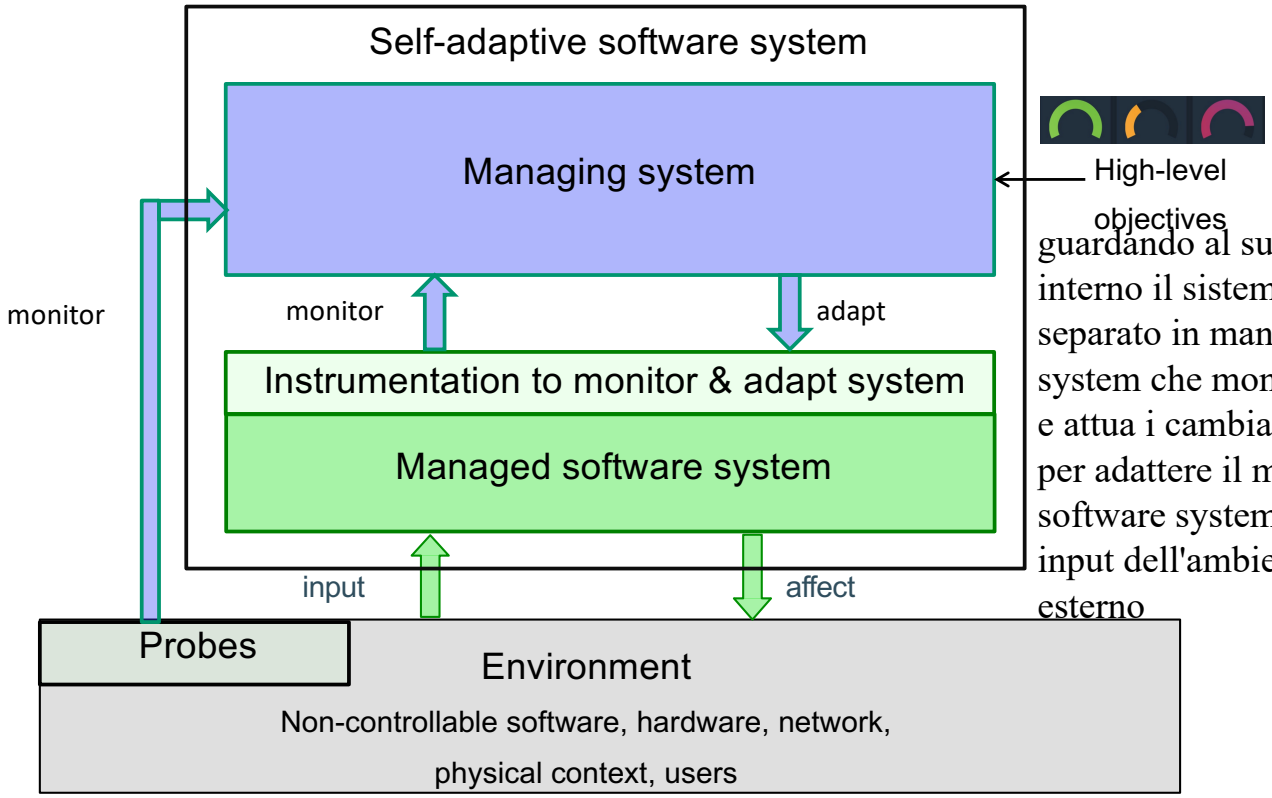
- These attributes are the implementation mechanisms

Conceptual model of self-adaptive system

un self-adaptive software system, contiene obiettivi ad alto livello e un sistema per monitorare il suo stato che viene costantemente influenzato con l'ambiente esterno. in questo momento stiamo vedendo il sistema come una black box.



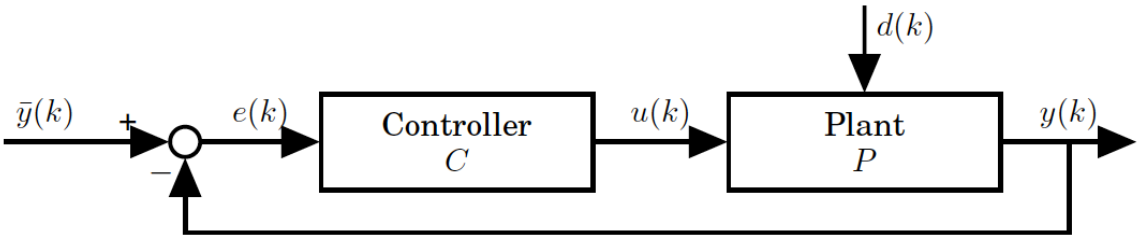
Conceptual model of self-adaptive system



guardando al suo interno il sistema è separato in managing system che monitora e attua i cambiamenti per adattere il managed software system agli input dell'ambiente esterno

You are already familiar with this model

- The control-theory perspective of a self-adaptive system

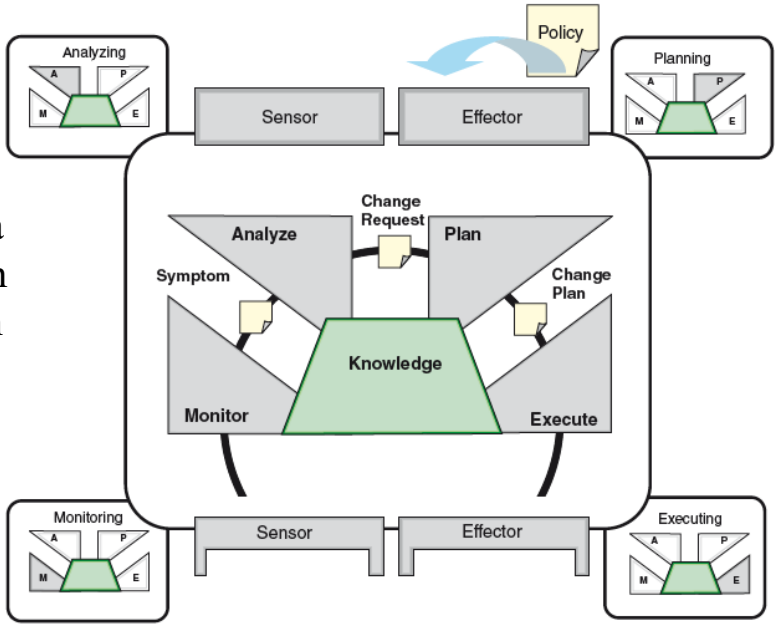


The capabilities of an autonomic computing system rely on a feedback loop, which maps to the conceptual feedback loop of the managing system in the conceptual model of a self-adaptive system. We direct our attention now to the internals of the managing system. In particular, we identify the primary functions of a managing system to define a reference model. The reference model enables us to explain in broad terms how the elements of a self-adaptive system work together to accomplish their collective purpose. A reference model is a commonly agreed decomposition of a known problem into functional elements that cooperatively solve the problem.

MAPE: reference architecture for self-adaptive system

- **MAPE** (Monitor, Analyze, Plan, Execute) loop

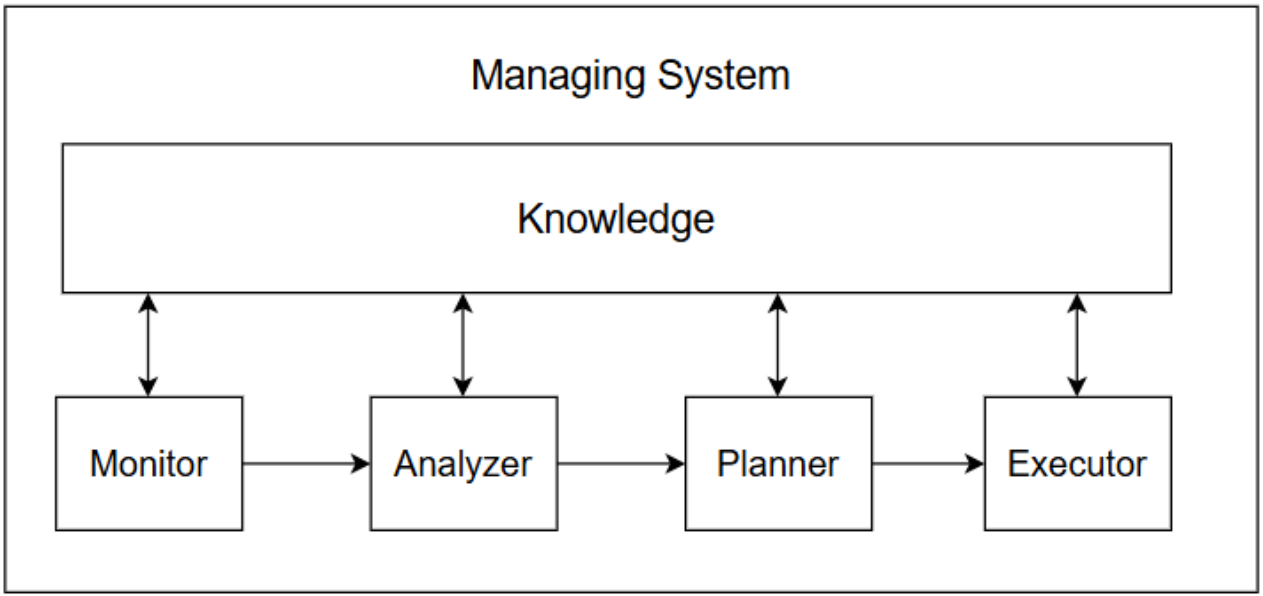
la nostra architettura di riferimento per un self-adaptive system



MAPE: building blocks (or phases)

- **M**onitor
  - Collects data from the managed system and execution environment through sensors; aggregates, filters and correlates these data into symptoms that can be analyzed
- **A**nalyze
  - Observes and analyzes situations to determine need for adaptation
  - If adaptation is required, it triggers Plan
- **P**lan
  - Determines which mitigation actions need to be performed so to enact a desired alteration in the managed system
- **E**xecute
  - Enacts the change plan by carrying out the actions determined by Plan through effectors so to adapt the managed system
- Plus **K**nowledge (MAPE-K)
  - Stores shared knowledge regarding relevant aspects of the managed system, environment, and the administrator's goals

La funzione di knowledge può essere equipaggiata con vari meccanismi a supporto dei client (elementi MAPE) per leggere, scrivere e aggiornare la knowledge. Un meccanismo di notifica può essere fornito per notificare i client interessati a certi cambiamente della knowledge. Nel caso in cui più client tentano di accedere alla knowledge simulataneamente, servono meccanismi per evitare race conditions assicurando che la knowledge rimanga consistente.

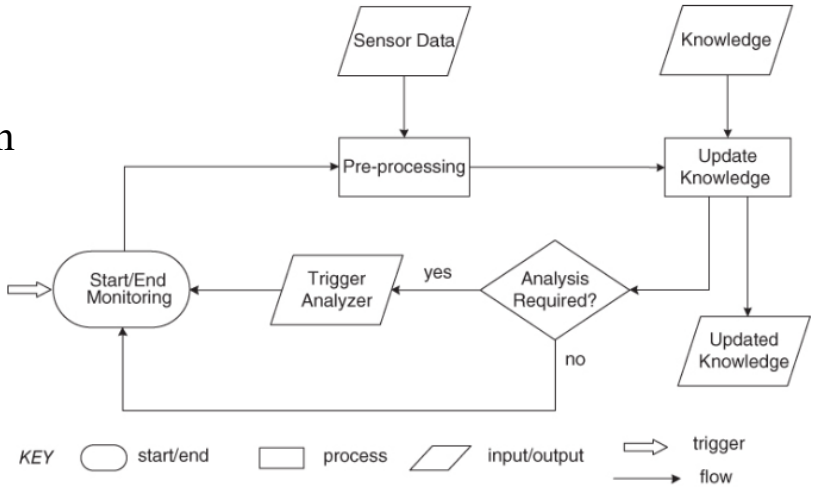


Reference model of a Managing System

MAPE: Monitor

- Main design options for Monitor:
  - *When* to monitor: continuously, on demand
  - *What* to monitor: resources, workload, performance, …
  - *How* to monitor: architecture (centralized vs. decentralized), methodology (active vs. passive)
  - *Where* to store monitored data (e.g., time-series database) and *how* (e.g., some pre-processing)

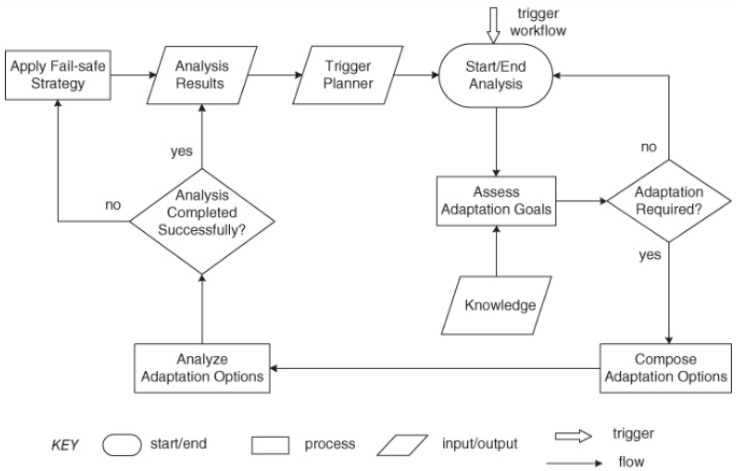
The monitor function has a dual role. On the one hand, it keeps track of the environment in which the managed system operates and reflects the relevant changes in the environment model. On the other hand, the monitor keeps track of changes of the managed system and updates the managed system model accordingly.



MAPE: Analyze

- Main design options for Analyze:
  - *When* to analyze: event- or time-triggered
  - Reactive vs. proactive adaptation
    - *Reactive*: *in reaction to* events that have already occurred (e.g., increase number of resources *after* workload increase)
    - *Proactive*: based on *prediction* so to plan adaptation actions in advance (e.g., increase number of resources *before* workload increase occurs)

The role of the analyzer function is to assess the up-to-date knowledge and determine whether the system satisfies the adaptation goals or not, and if not, to analyze possible configurations for adaptation.



MAPE: Plan

- The most challenging and studied MAPE phase
- A variety of methodologies and techniques can be used to plan adaptation, including
  - Optimization theory
    - ✓ Optimal adaptation actions
    - ✗ Con: formulation can be NP-hard, too expensive to solve at runtime
  - Heuristics
    - ✓ Faster
    - ✗ Sub-optimal adaptation actions
  - Machine learning, including reinforcement learning
  - Control theory
  - Queueing theory
- Example: optimal bin packing and heuristic policies to dynamically place virtual machines or containers on servers

usare problemi di ottimizzazione è valido solo se non sono NP-hard

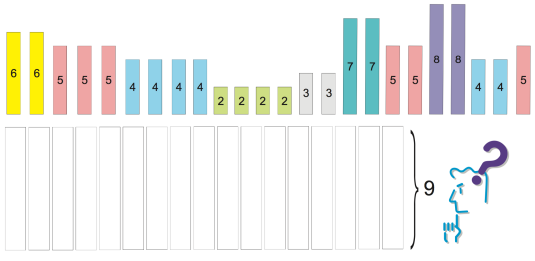
euristica, ovvero cercare di predirre i comportamenti

The role of the planner function is to select the best adaptation option and generate a plan for adapting the managed system from its current configuration to the new configuration defined by the best adaptation option. In the event that a fail-safe strategy needs to be applied, the planner only needs to generate an adaptation plan to bring the system to a safe state.

Example: VM/container placement

- A set of physical servers (bins) with limited capacity
- How to place a set of virtual machines (VMs) or containers, each with specific resource demands?
- We can use a bin packing optimization problem
- Bin packing: pack items of varying sizes (VM or containers) into the minimum number of bins (servers), each with limited capacity (resources), ensuring the total size in each bin does not exceed its capacity https://en.wikipedia.org/wiki/Bin_packing_problem
- Integer optimization problem ⇒ NP-hard ⇒ we need efficient heuristics to find new placement quickly when some change occurs

ricorda che NP è la complessità non polinomiale più alta!



euristico ovvero in maniera pragmatica

Example: VM/container placement

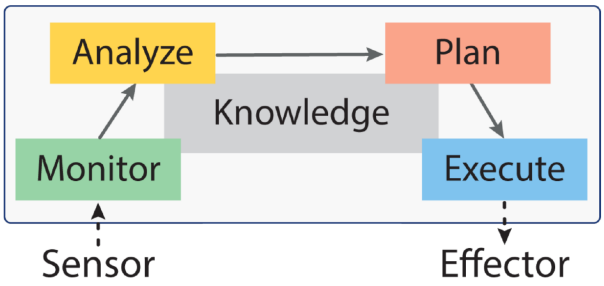
- Examples of baseline heuristic policies
- *Round robin:* organize bins in a *circular* list, saving the latest used bin; place each item in the next bin with enough capacity, starting from the current position
  - Does not minimize the number of used bins
- *First fit:* organize bins in a list and place each item into *the first bin* in which it fits, starting from the beginning of the list for each item
  - First fit decreasing: variant in which items are sorted in decreasing order
- *Best fit*: place the item in the bin with the *least remaining space* that still fits the item
- *Worst fit*: place the item in the bin with the *most remaining space* where it fits

se ho un bin che occupa 6
lo metto in uno che ha 6 di spazio

se ho un bin che occupa 6, lo metto in uno vuoto che può tenere 9

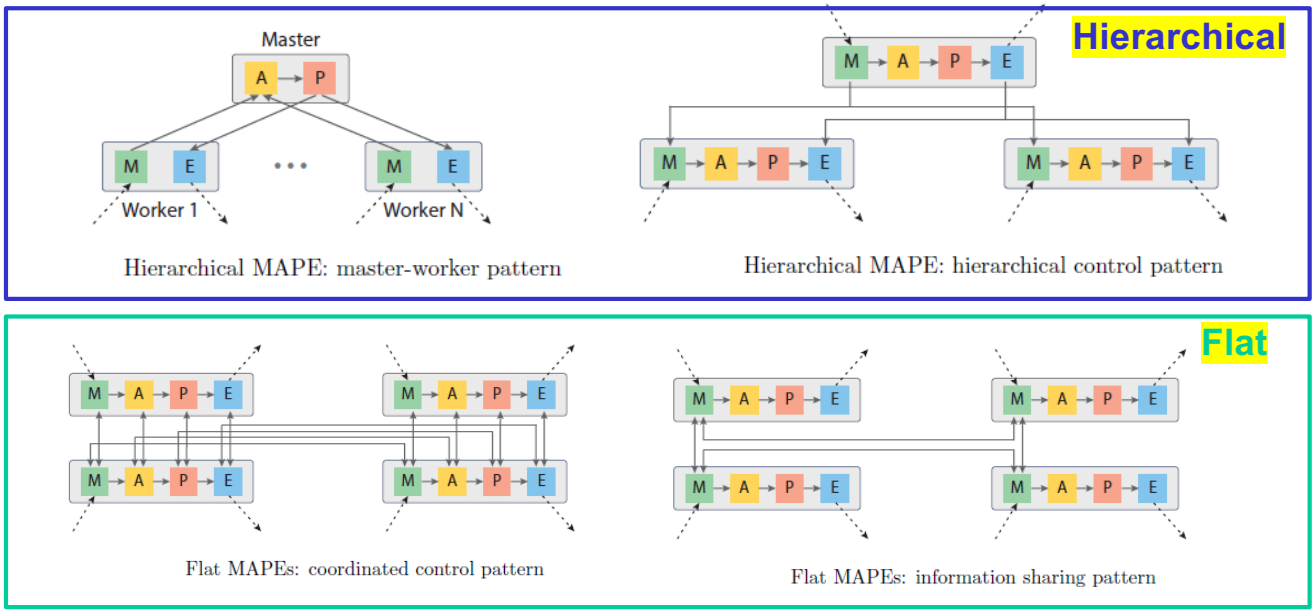
Alternative architectures for MAPE

- Designing the architecture of the managing system
- Centralized MAPE
  - All MAPE components are located on the same node
  - Simple, but lacks of scalability in geo-distributed environments
- **Decentralized MAPE**
  - MAPE components are distributed across multiple nodes
  - Various architectural patterns with its own trade-offs
    - There is no one-size-fits-all solution: the choice depends on system characteristics, application requirements, and operational context come nella gran parte dei casi



How to decentralize the adaptation control

- Main architectural patterns for decentralized MAPE



Weyns et al., On patterns for decentralized control in self-adaptive systems. In *SEfSAS II*, 2013

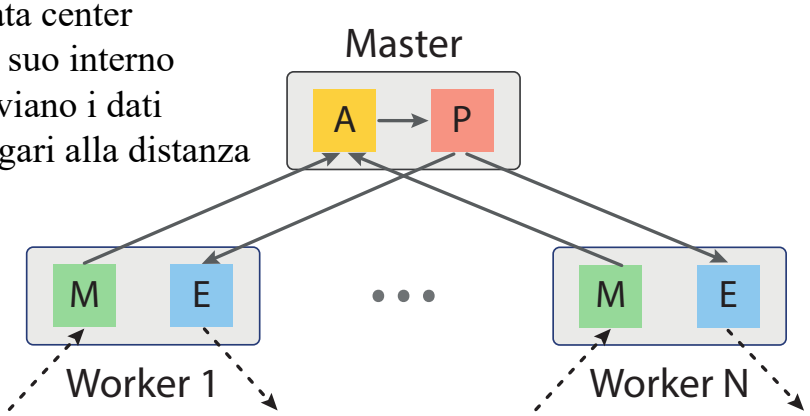
How to decentralize the adaptation control

- First design choice: ***hierarchical*** vs. ***flat***
  - Hierarchical: easier to design, but risk of bottleneck in top level of hierarchy
  - Flat: more difficult to coordinate, but can scale better
- Hierarchical MAPE patterns: multiple MAPE loops organized in a hierarchy, where a higher-level control loop manages subordinated control loops
  - Master-worker
  - Hierarchical control
- Flat MAPE patterns: multiple MAPE loops cooperate as peers
  - Coordinated control
  - Information sharing

Hierarchical MAPE: master-worker pattern

- Decentralize M and E on workers, keep A and P centralized on master
- ✓ Global view on master who can achieve global adaptation goals
- ✗ Communication overhead and risk of performance bottleneck and SPOF on master

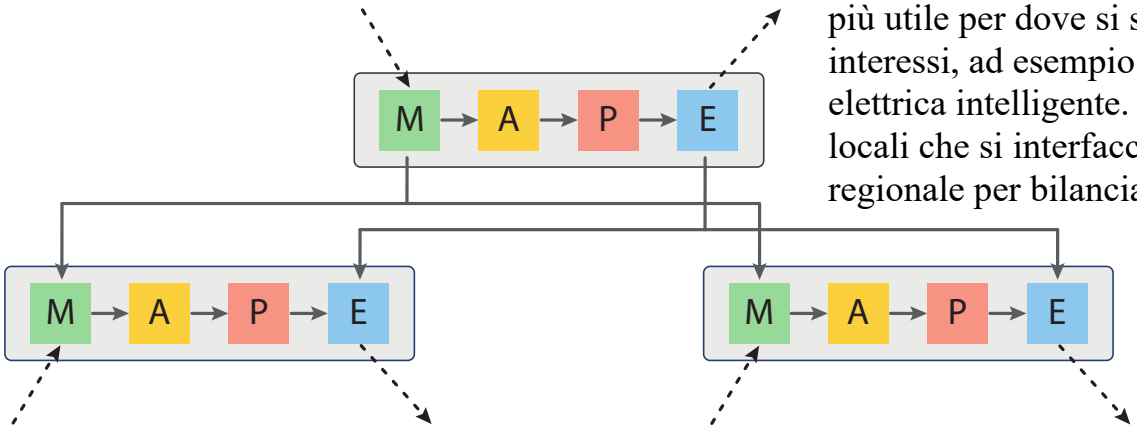
soluzione ideale in un data center per analizzare i server al suo interno che sono monitorati e inviano i dati senza latenza dovuta magari alla distanza fisica al master.



Hierarchical MAPE: hierarchical control pattern

- Multiple MAPE loops, which can operate at different time scales and with separation of concerns
- ✓ Top-level MAPE can achieve global goals, increased flexibility
- ✗ Can be non-trivial to identify different levels of control, depends on managed system characteristics

più utile per dove si separano gli interessi, ad esempio una rete elettrica intelligente. avrò MAPE locali che si interfacciano con uno regionale per bilanciare consumi

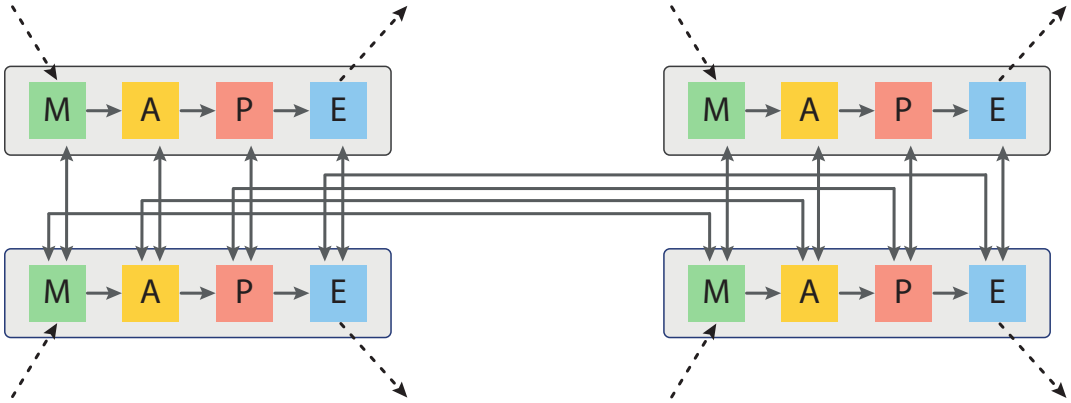


Flat MAPE: coordinated control pattern

- Multiple control loops, each one in charge of some part of the managed system but coordinated through interaction caso d'uso un gruppo di nodi edge

✓ Better scalability

✗ More difficult to take joint adaptation decisions

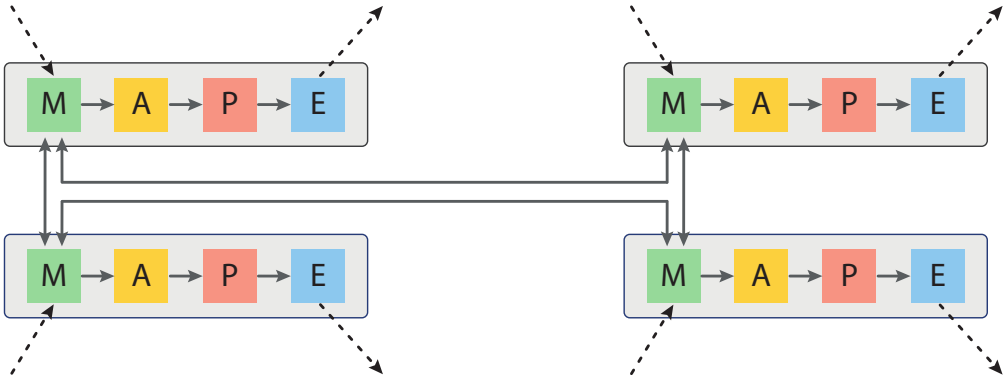


Flat MAPE: information sharing pattern

- Special case of coordinated control pattern: interaction only among M components nodi fog? super peer?

✓ Better scalability

✗ Lack of coordination on planning, conflicting or sub-optimal adaptation actions can be enacted

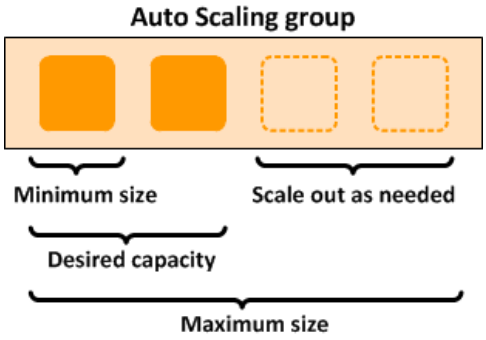


Examples of self-adaptive systems

- Let's analyze 3 examples of self-adaptive systems for resource management
  1. Auto-scaling EC2 instances
  2. Self-adaptive selection of application component services
  3. Auto-scaling microservice-based applications
- Common ground
  - Applications operate under uncertainty (e.g., workload surge and spikes, node crashes)
  - Adaptation goal: maintain target SLO (e.g., application response time, availability, cost)
- Examples vary in planning methodologies and control architectures

Example 1: Amazon EC2 Auto Scaling

- AWS service that automatically adds or removes EC2 instances based on user-defined conditions and health checks https://aws.amazon.com/ec2/autoscaling/



- MAPE Monitor: monitor scaling metrics on EC2 instances using CloudWatch https://aws.amazon.com/cloudwatch/
- Which scaling metrics?
  - CPU utilization, memory usage, network throughput, Application Load Balancer (ALB) request count, …

Example 1: Amazon EC2 Auto Scaling

- MAPE Plan: we study 2 policies implemented in AWS Auto Scaling for scale-out/scale-in decisions
  https://docs.aws.amazon.com/autoscaling/ec2/userguide/scale-your-group.html
  - Dynamic simple scaling: responds to real-time metrics (e.g., CPU utilization) with scaling actions triggered by thresholds
  - Predictive scaling: uses machine learning to forecast future traffic and proactively adjusts capacity

approccio di reazione

approccio appunto proattivo (predico lo scaling)

- Beyond scaling: self-healing capabilities
  - Auto Scaling is also a self-healing system
  - It detects unhealthy EC2 instances, automatically terminates them, and launches replacements to maintain availability

Example 1: Amazon EC2 Auto Scaling

- **Dynamic simple scaling**: a simple, rule-based policy based on predefined metric thresholds
- Threshold-based heuristic policy (*reactive*)
  - User sets upper and lower thresholds on some scaling metric(s)
  - Upper threshold → triggers scale-out
  - Lower threshold → triggers scale-in

```
if (metric > upper_thr) scale-out
else if (metric < lower_thr) scale-in
```

    - Example of *scale-out* rule: if average CPU utilization of all instances > 70% in last 1 minute, then add 1 new instance
    - Example of *scale-in* rule: if average CPU utilization of all instances is <35% in last 5 min, then remove 1 instance
  - CloudWatch monitors and sends alarms for scaling out (`upper_thr`) and scaling in (`lower_thr`)
  - Cooldown period: delay after a scaling action before any further scaling actions can be triggered

altrimenti non faccio assestare il sistema e corro il rischio di overprovvision o underprovvision

Threshold-based policy: pros & cons

✓ Simple and easy-to-understand
- Choose scaling metric(s), a monitoring period, and upper/lower thresholds for triggering alarms

- Basic idea, but in practice…

✗ Choosing the right metrics and thresholds is difficult
- Metrics are application-dependent: application components may be CPU-, memory-, I/O-intensive, or a mix
- Thresholds can be too aggressive or too conservative; examples of common issues:
  - Slow scale-out: upper threshold too high → not enough instances added → SLO violations
  - Rapid scale-out: upper threshold too low → too many instances added → underutilization and high cost
  - Slow scale-in: scaling period too long → not enough instances removed → wasted resources and cost
  - Rapid scale-in: lower threshold too high → too many instances removed → SLO violations

Threshold-based policy: pros & cons

- Basic idea, but in practice…

✗ No application-specific metrics (e.g., response time often missing)

✗ Not robust against varying load patterns

- Idea: we can exploit reinforcement learning to learn optimal threshold values

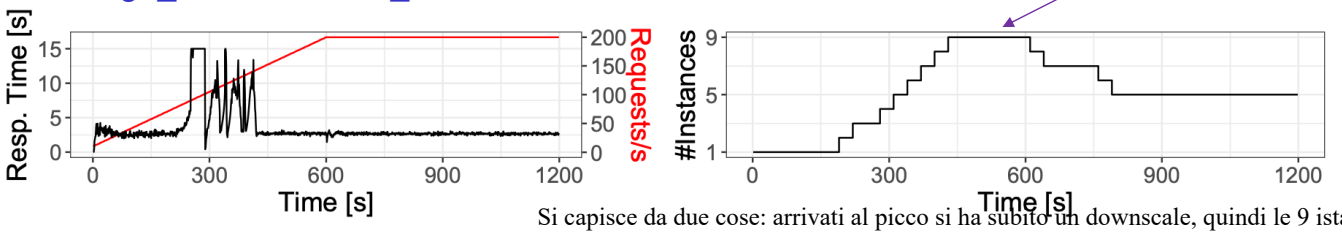
  Rossi et al., Dynamic multi-metric thresholds for scaling applications using reinforcement learning, *IEEE Trans. Cloud Comp.*, 2023 https://doi.org/10.1109/TCC.2022.3163357

Threshold-based policy: cons

- Examples of wrong policy configurations
  - CPU utilization as scaling metric

scaling period = 30 s.
high_thr = 80% low_thr = 25%

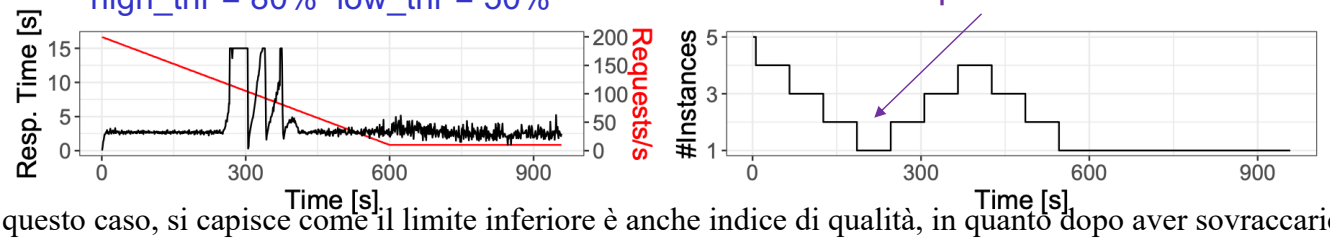
over-provisioned



Si capisce da due cose: arrivati al picco si ha subito un downscale, quindi le 9 istanze sono sotto il 25%. E che ogni incremento è dato dalla rapidità dello scaling period, che non permette di valutare il nuovo stato del sistema correttamente

scaling period = 60 s.
high_thr = 80% low_thr = 50%

under-provisioned

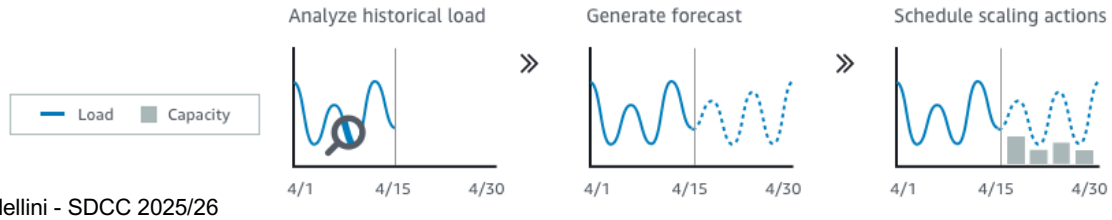


grazie a questo caso, si capisce come il limite inferiore è anche indice di qualità, in quanto dopo aver sovraccaricato il sistema si passa ad aumentare e dopo diminuire le istanze e questo appunto è sintomo di costi altalenanti ed inefficienza

Strasser et al., Autoscaler Evaluation and Configuration: A Practitioner's Guideline, *ICPE 2023*
https://research.spec.org/icpe_proceedings/2023/proceedings/p31.pdf

Example 1: Amazon EC2 Auto Scaling

- **Predictive scaling**
- ML-based policy (*proactive*)
  - Trained ML model predicts expected application traffic and EC2 usage, including daily and weekly usage patterns
    - Scaling metrics: CPU utilization, ALB request count
  - Requires historical data collected from CloudWatch
    - Minimum 1 day of data to start making predictions
    - Hourly forecasts for the next 48 hours based on the past 14 days
    - Forecast updated every 6 hours
  - Scaling action: scale out to reach a target utilization
  - Assumption: Auto Scaling group is homogeneous and all instances have equal capacity



Predictive scaling: pros and cons

✓ Proactive

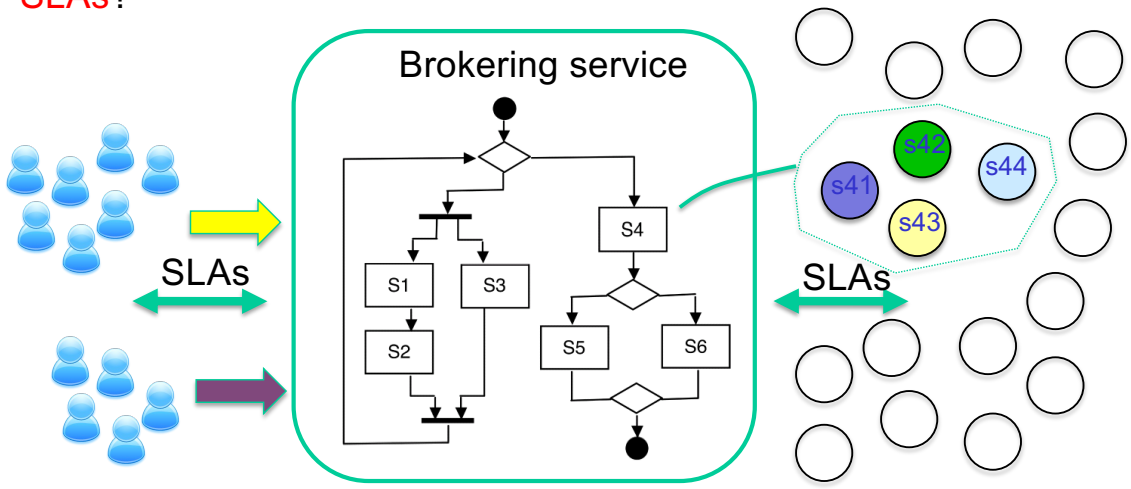
✗ Requires training
- More historical data → better forecast accuracy

✗ Choice of scaling metric (e.g., CPU utilization, request count) is critical

✗ Supports only scale-out decisions
- To scale-in a separate policy is needed
  Scale-in ricordiamo essere lo scaling verticale

Example 2: Self-adaptive service selection

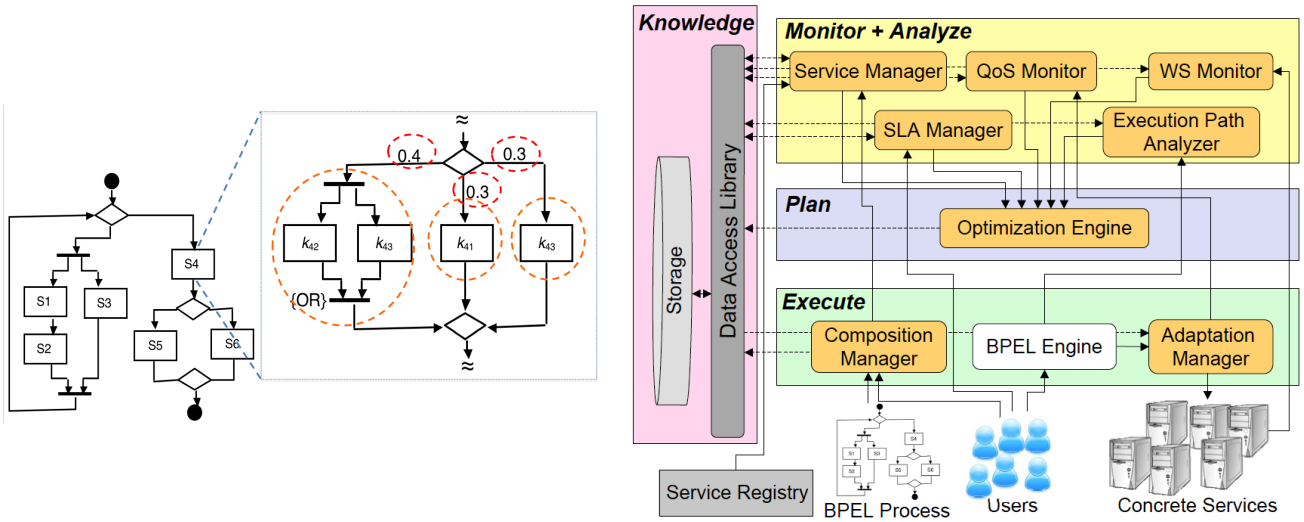
- QoS-driven self-adaptation of SLA applications
  - Each abstract service may have multiple concrete implementations with different non-functional properties
  - Which concrete services should be selected to meet application SLAs?



Cardellini et al., MOSES: a framework for QoS driven runtime adaptation of service-oriented systems, *IEEE Trans. Soft. Eng.*, 2012 http://www.ce.uniroma2.it/publications/tse2012.pdf

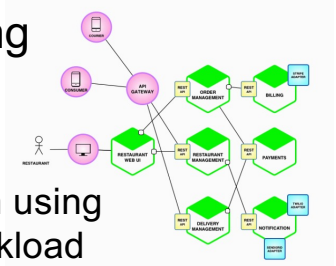
Example 2: Self-adaptive service selection

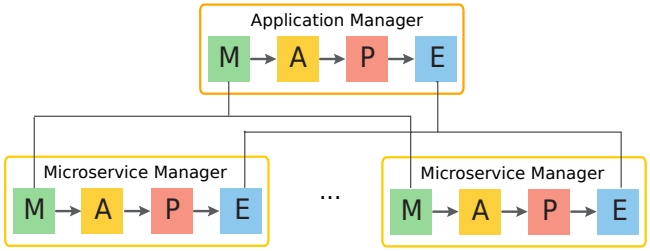
- MAPE Plan
  - Centralized policy: find the *optimal set of concrete services* (and their *coordination*) using linear programming optimization

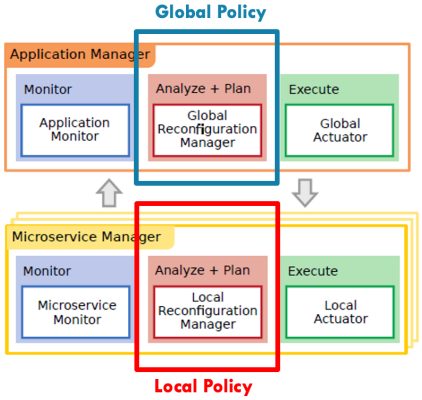


Example 3: Hierarchical auto-scaling of microservices

- Hierarchical control pattern for elastic scaling of microservices-based application
  - Goal: keep app response time below maximum
  - Each Microservice Mngr. determines scale-out/in using a local policy based on queueing theory and workload prediction and sends scaling proposal to Application Mngr.
  - Application Mngr. coordinates proposals by approving or rejecting them and sends decisions to local Execute components
  - Implemented in Kubernetes







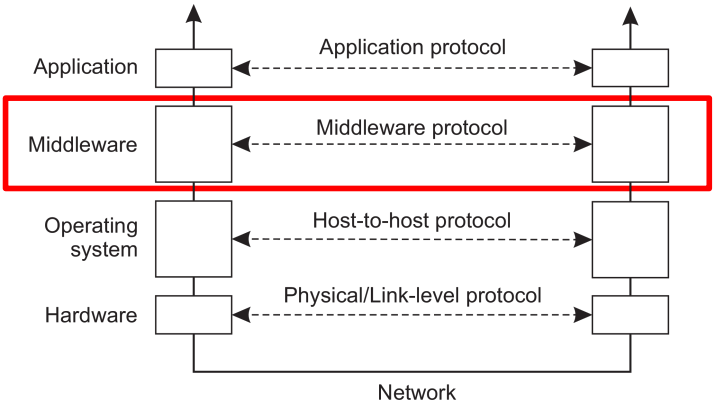
Rossi et al., Hierarchical scaling of microservices in Kubernetes, *Proc. IEEE ACSOS 2020*
http://www.ce.uniroma2.it/publications/acsos2020.pdf

Communication in distributed systems

- Message passing: the basics
- Communication is based on sending and receiving messages
- To make this work, parties must agree on many low-level communication details, such as:
  - How many volts represent a 0 or 1 bit?
  - How many bits represent an integer?
  - How does the receiver know when the message ends?
  - How can the receiver detect message corruption and what to do then?
- The good news: we don't care about these low-level details ☺

Basic networking model and its adaptation

- We already know the solution: divide network communication in layers
  - The well-known ISO/OSI reference model
  - For distributed systems we don't deal with low-level details: we focus on higher-level abstractions built over the network layer
- Adapted layering scheme in distributed systems



Middleware layer

- **Middleware** provides common services and protocols that support different distributed applications and systems
  - General-purpose
  - Application-independent
- Common middleware services
  - Communication: remote procedures calls, message queues, multicasting ci occuperemo nel dettaglio di questo servizio
  - Naming: identifies and enables sharing of resources
  - Security: ensures secure communication among distributed nodes
  - Distributed consensus: coordinates agreement across multiple distributed nodes
  - Distributed locking: prevents concurrent access to shared resources (e.g., multiple clients editing a file)
  - Data consistency: maintains data consistency across distributed nodes

Communication types in DS

- Let's distinguish communication types by 3 key aspects:

1. Persistency
   - Transient vs. persistent communication
2. Synchronization
   - Synchronous vs. asynchronous communication
3. Time dependence
   - Discrete vs. streaming communication

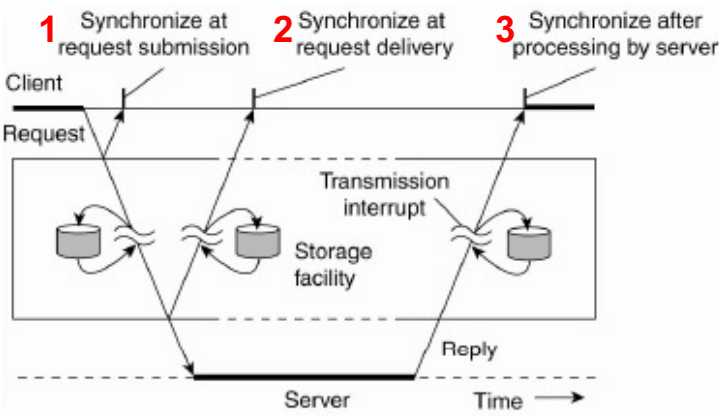
Persistent vs. transient communication

- Persistent communication Kafka è persistent
  - Message is stored by middleware until it can be delivered
  - Sender can continue execution after submitting the message
  - Receiver does not need to be active at the time of submission

- Transient communication
  - Message is kept only while sender and receiver are both active
  - If delivery is not possible, message is discarded
  - Requires both parties to be executing at the time of communication
  - Transport-layer example: routers store and forward messages, but discard them if forwarding fails

Synchronous communication…

- Once a message is submitted, the sender is blocked until the operation completes
- Send and receive are *blocking* operations
- Synchronization can occur:
  1. At request submission
  2. At request delivery
  3. After request processing



… vs asynchronous communication

- *Non-blocking send*
  - Once the message is submitted, the sender continues processing
  - The message is temporarily stored by middleware until it is transmitted
- *Receive* can be *blocking or non-blocking*
  - Non-blocking receive: immediately returns, either with data (if available) or with an indication that there is nothing to receive yet
- Benefits: decouples sender and receiver, enabling higher concurrency and responsiveness

Discrete vs. streaming communication

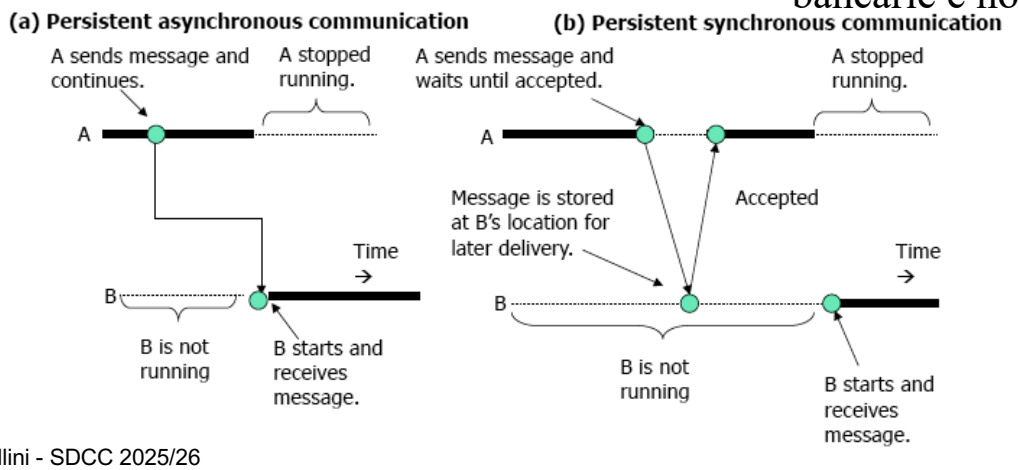
- Discrete communication
  - Each message forms a complete unit of information

- Streaming communication
  - Involves sending multiple messages, in temporal relationship or related to each other by sending order, which is needed to reconstruct complete information
  - E.g., audio, video

Combining persistence and synchronization

a) Persistent asynchronous communication
  - Message is stored until receiver is ready, sender does not wait for receiver MOM o comunque una chat
  - E.g., email, Teams chat, message-oriented middleware

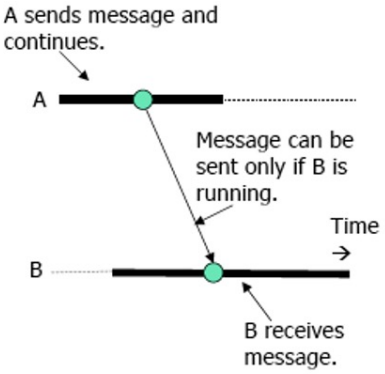
b) Persistent synchronous communication
  - Sender is blocked until message is delivered to receiver
  - Message is stored until delivery is confirmed tipico delle transazioni bancarie e non



Combining persistence and synchronization

c) Transient asynchronous communication
- Sender does not wait
- Message is not stored
- If the receiver is offline or unreachable, message is lost (e.g., UDP)

Una chiamata al telefono è transient, ovvero se A chiama e B non c'è non si ha alcuna telefonata.



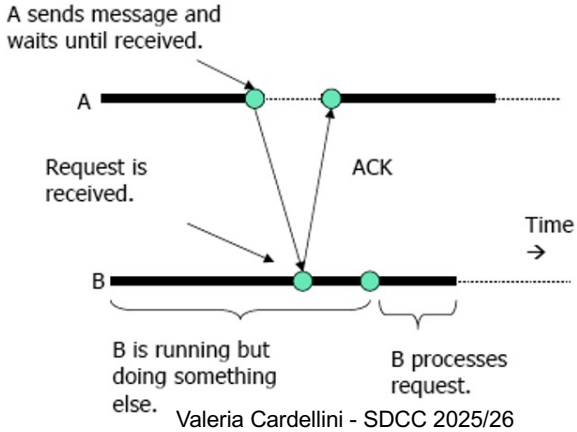
Invece una comunicazione transient con sincronizzazione dopo che la richiesta è stata processata corrisponde alle remote procedure call.

(c)Asynchronous communication

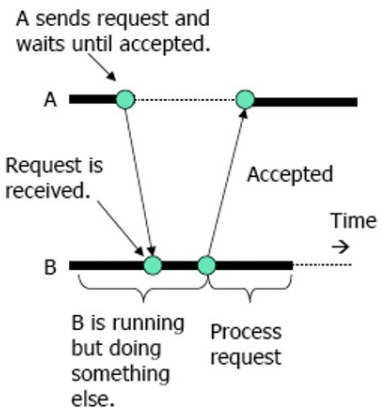
Combining persistence and synchronization

- Message is not stored caso Transient synchronous
- Sender is blocked until a specific condition is met, which alternatives?
  - d) Receipt-based synchronous: sender is blocked until message reaches the receiver buffer (e.g., asynchronous RPC)
  - e) Delivery-based synchronous: sender is blocked until message is actually delivered to receiver

(d) Receipt-based synchronous communication

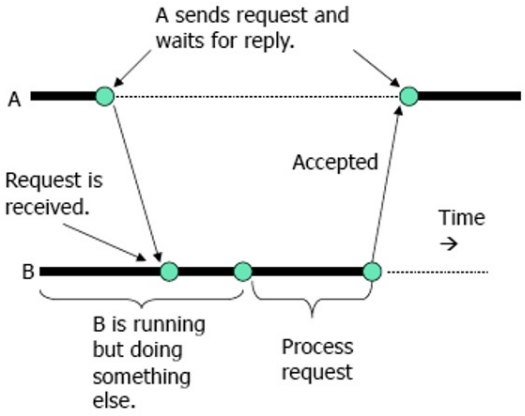


(e) Delivery-based synchronous communication



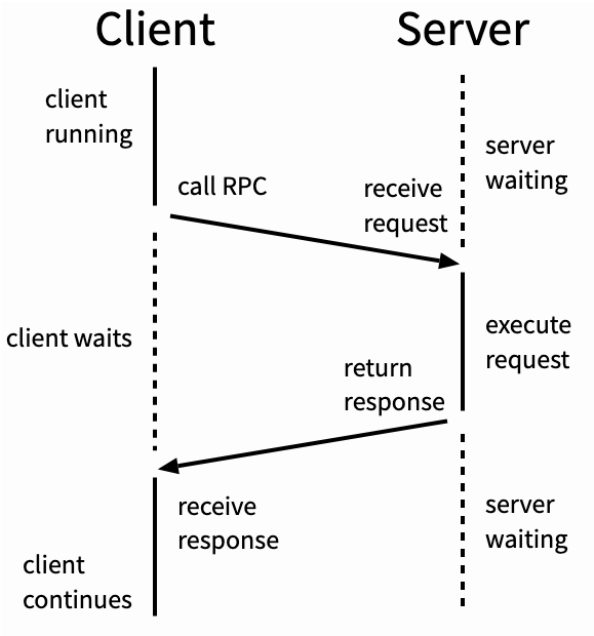
Combining persistence and synchronization

f) Response-based synchronous: sender is blocked until it receives a reply from receiver (e.g., synchronous RPC)



(f) Response-based synchronous communication

What happens when a client calls a RPC



- Which failures can happen?

L'obiettivo di RPC è l'access transparency camuffando la chiamata remota come locale. Tuttavia quando avviene un errore le differenze tra chiamate locali e remote non sono sempre facili da mascherare

Communication failures

Vedi sezione 8.3.2 Tanenbaum

- Different communication failures between sender (client) and receiver (server): what can go wrong?

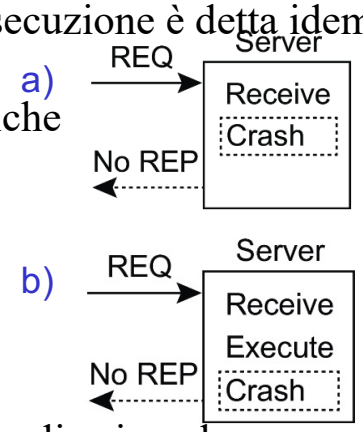
1. Message loss or delay
   - Request or reply is lost or delayed
   - Connection is reset
   - Pitfall: *Network is reliable*

se viene persa la request si usa un timer per la ritrasmissione. anche per la reply si può usare ma serve anche altro. una richiesta che non ha effetti collaterali dopo l'esecuzione è detta idempotent

2. Server crashes

si devono valutare le politiche di server e client

   a) Before performing the service
   b) After performing the service
   - Client can't distinguish between a) and b)



3. Client crashes after sending request

In questo caso si va a creare un orphan e va gestito come meglio si crede oppure si utilizza il checkpointing che si tratterà dopo (sezione 8.6)

4. Client cannot locate server: Unica soluzione richiedere una nuova connessione. N.B. si suppone che il server sia raggiungibile. Nelle prossime slide tutto si tratterà nel dettaglio

Failure semantics during communication

- In a DS, communication semantics define what guarantees the DS gives about message delivery in the presence of failures
  - **May-be** semantics
  - **At-least-once** semantics
  - **At-most-once** semantics
  - **Exactly-once** semantics

IMPORTANTE!

More guarantees, more complexity

- Failure semantics applies both to message delivery (e.g., MOM) and service processing (e.g., RPC)
- Let's focus on service processing
  - sender = client, receiver = server

ATTENZIONE QUESTE SONO SEMANTICHE

Failure semantics in a nutshell

- Maybe: no guarantees that service has been executed or not by server
- At-least-once: service is executed at least once (i.e., >= 1)
  - If client does not receive the reply: service *might* have been executed or it might not, or it might have been executed mutiple times
- At-most-once: service is executed at most once (i.e., 0 or 1)
  - If client receives the reply, it has been processed by server *no more than once*
  - The service may not be executed at all
- Exactly-once: service is executed precisely once
  - Even in the face of retries, crashes, or delays
  - Ideal for critical operations, but complex and costly

IMPORTANTI

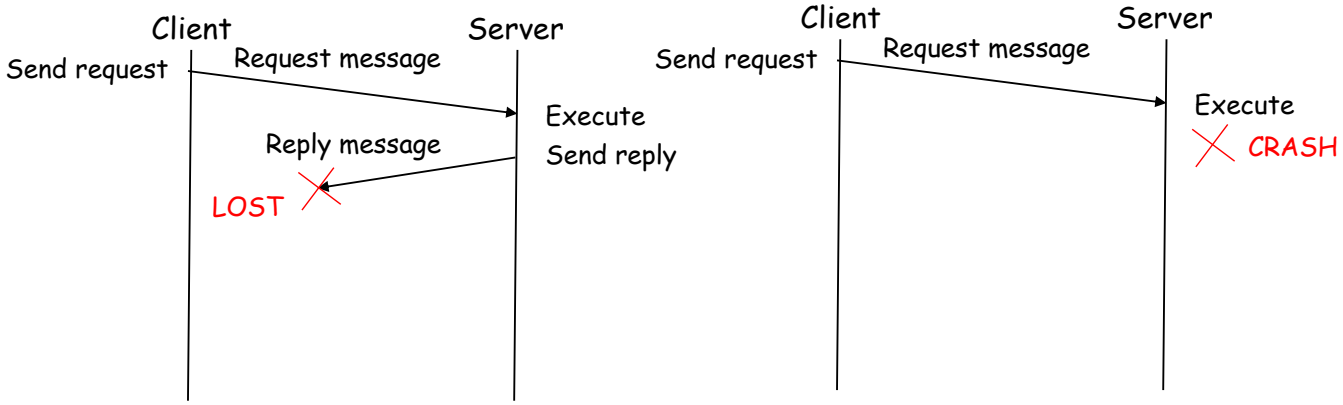
Basic mechanisms for implementation

- 3 basic mechanisms to implement failure semantics

1. Client side: Request Retry (RR1) con un timer magari
   - Client keeps trying until it gets a reply or is confident about server failure after a certain number of failed retries
2. Server side: Duplicate Filtering (DF) quindi tramite ID magari
   - Server discards any duplicate request from same client
3. Server side: Result Retransmit (RR2) se ricevo un duplicato ritrasmetto
   - Server keeps result (reply) so that it can be retransmitted without being computed again when server receives duplicate requests

May-be semantics

- No guarantee that service has been executed or not by server
- No action is taken to ensure reliable communication: no mechanism (among RR1, DF, RR2) is used
- E.g., best-effort in UDP
- Use cases: logging, sensor data (if occasional loss is tolerable)



At-least-once semantics

- Service, if executed, has been executed at least once
  - Perhaps more than once due to request duplicates caused by retransmissions

NON USA

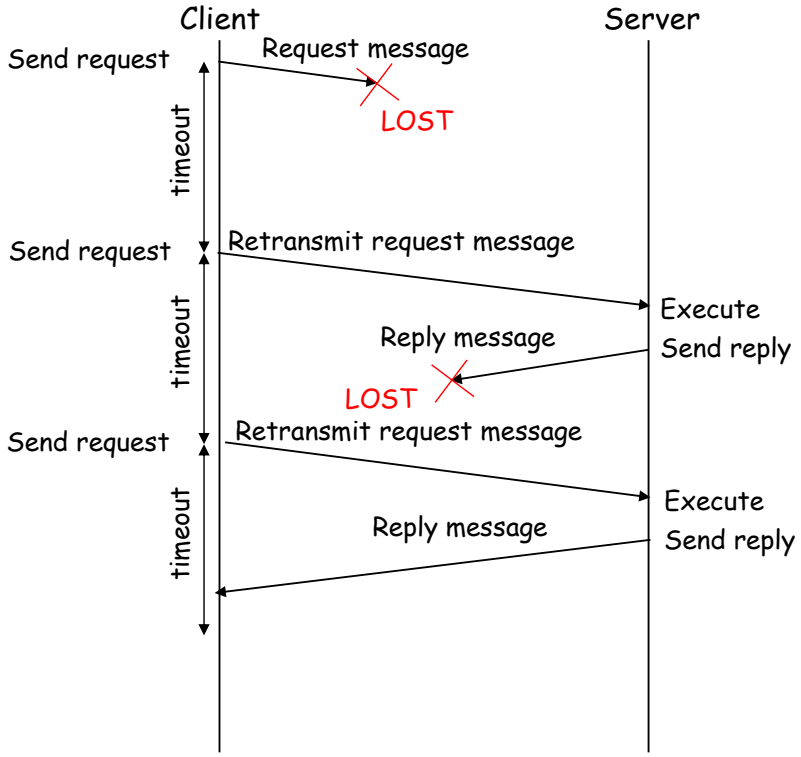
- Client uses RR1, server uses neither DF nor RR2
- Upon receiving the reply, client cannot know how many times its request has been processed by server
  - If server executes the service once or multiple times because the reply was lost or delayed
  - If server executed the service but crashed before replying and then recovered, and client retries the request after timeout; server processes the request again

non ci sono meccanismi di deduplicazione

- It works for idempotent services
  - **Idempotent** service: a service where applying it multiple times has the same effect as applying it once to the same input
    - E.g., `get(key)` or `put(key,value)` in KV data store

appunto una richiesta di pagamento NON È idempotent

At-least-once semantics

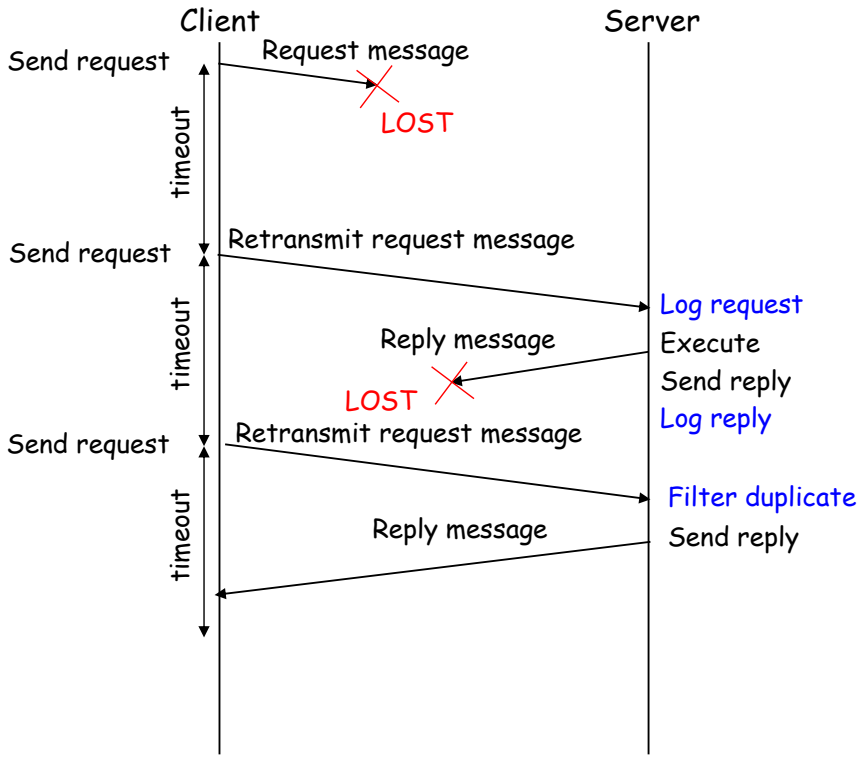


qui è stato eseguito 2 volte! quindi almeno 1

At-most-once semantics

- Service, if executed, has been executed at most once
  - *If* client receives the reply, it knows that the service was executed *once*
  - In case of failure, client cannot determine whether the request never reached the server, or server executed the service but the reply was lost (at-most-once: at most once, but possibly not all)
- Mechanisms used: RR1, DF, and RR2
  - Client retransmits request when timeout expires
  - Server maintains some state to identify duplicate requests and avoid processing the same request more than once
- It works also for non-idempotent services TRANSAZIONI
  - E.g., `increment(counter)` or `withdraw(amount)`
- No strict coordination between client and server
  - Client does not know if the service was executed (in case of no reply), server does not know if client received the reply

At-most-once semantics



At-most-once: implementation

- Server must:
  - Detect duplicate requests
  - Manage duplicate requests returning saved reply instead of re-running service `handler()`
  - Discard saved replies when it is safe

magari dopo un tot di tempo

- How to detect duplicates
  - Client includes a unique ID (`xid`) with each request and uses same `xid` when retransmitting the request due to timeout
  - Server uses `xid` to detect duplicates
  - If server knows the request with that `xid`
    - If reply is saved, resend it
    - If original request is still executing, don't re-run service !

Server:

```
if seen[xid]
 reply = old[xid]
else
  reply = handler()
  old[xid] = reply
  seen[xid] = true
```

At-most-once: implementation

- How to ensure unique `xid`?
  - Globally unique per client (and per server if needed)
  - Common pattern: `xid = client_id + sequence_number`
  - Sequence number: increments with every new request and wraps around only after large gaps
- How to handle duplicate requests during execution?
  - Server maintains also a status flag: `xid → {status, reply}`
  - Status is `pending` if service still executing
  - Status is `completed` if reply cached
  - If duplicate arrives while status is pending, server ignores it or queues it to wait for the original to finish
- Server must eventually discard cached replies: how?
  - Use sliding windows to track recent `xids`
  - Retain reply for a maximum lifetime

Idempotent Receiver pattern approfondimento utile
https://martinfowler.com/articles/patterns-of-distributed-systems/idempotent-receiver.html

Exactly-once semantics

- Strongest and hardest guarantee to implement, especially in large-scale DS
- Requires full agreement on interaction between client and server
  - Service is run only once or not run at all: all-or-nothing semantics
    - If everything goes well: service runs only once, duplicates are found
    - If something goes wrong: client or server knows if service has run (once - all) or not (never - nothing)
- Semantics with concordant knowledge of each other's state and without constraint on maximum duration of interaction protocol between client and server
  - No constraint on maximum duration: barely practical in real systems!

Exactly-once semantics: more mechanisms

- Basic mechanisms (RR1, DF, RR2) are not enough
  - Do not protect against server failures
- Additional mechanisms for fault tolerance
  - Transparent server replication presente in kafka
    - Multiple servers replicate the same service
    - E.g., leader-follower or primary-backup
  - Write-ahead logging (WAL) noi vedremo questo nel dettaglio
    - Before executing a service, the server logs the request to persistent storage
  - Recovery mechanisms
    - After failure, the server resumes processing from a safe state
    - *State checkpoint*: saves a local snapshot of server's state to persistent storage
      - Combined with WAL to reduce recovery time (only replay log entries after last checkpoint)
    - *Snapshot*: captures a consistent global state of replicated servers: more complex, requires coordination

Write-ahead logging pattern

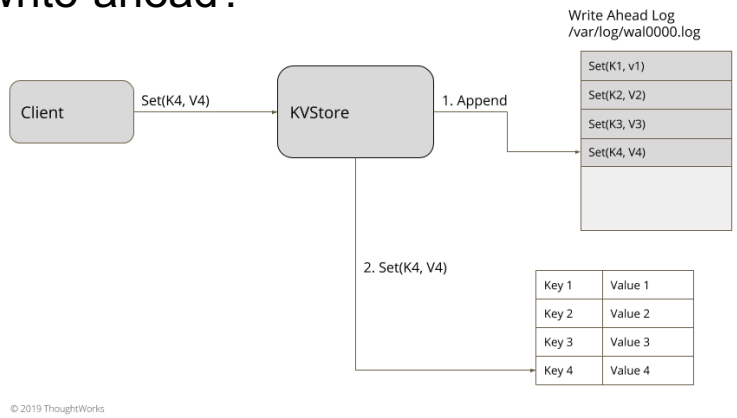
- aka Commit log
- Goal
  - Ensure durability by persisting every state change so server can survive crashes and restarts.
  - Log is stored
- How ogni volta che ricevo una richiesta aggiorno il log con un commit al file che è persistente
  - Each state change is stored as log entry in an append-only commit log file written on persistent storage (e.g., disk or SSD)
  - Entries are appended sequentially, ensuring fast write performance and crash-consistent ordering
  - On server restart, log is read and all entries are replayed in order to rebuild the in-memory state
  - No state is lost, as long as the log was flushed to disk before crash

WAL pattern

https://martinfowler.com/articles/patterns-of-distributed-systems/wal.html

Write-ahead logging pattern

- Why write-ahead?




```
public void put(String key, String value) {
    appendLog(key, value);
    kv.put(key, value);
}

private Long appendLog(String key, String value) {
    return wal.writeEntry(new SetValueCommand(key, value).serialize());
}
```


Summing up failure semantics

- In practice, at-least once and at-most once semantics are both feasible and widely used in DS
- Exactly-once is ideal, but often too complex, slow, or expensive to achieve in practice aggiunta di overhead maggiore rispetto a at-least once
- We often choose the lesser of two evils, i.e., at-least-once semantics
  - Easier to scale
    - Stateless retries
    - No need for coordination between sender and receiver importante
  - Works well with idempotent services
  - Can be combined with application-level deduplication if needed

Distributed systems are all about trade-offs!

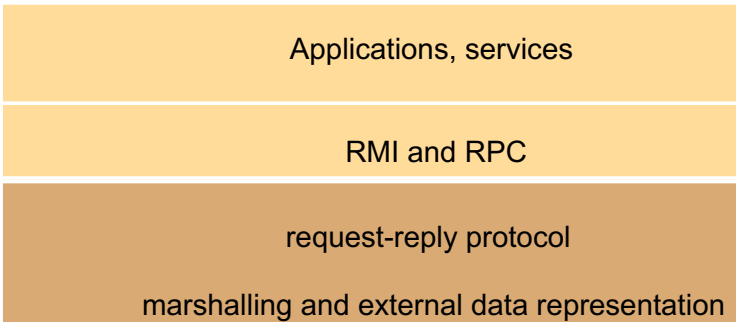
Distributed application programming

- Explicit network programming
  - Operating system constructs based on socket API and explicit management of message exchange
  - Foundational for network applications (e.g. web browser, web server, chat app)
  - ✗ Distribution is not transparent and requires developer effort

- How to increase the abstraction level?
- Communication middleware between OS and applications
  - Provides a higher-level API for communication
  - Hides complexity of underlying hw and sw layers
  - Frees developers from automatable tasks
  - Improves software quality by providing reusable, correct, and efficient solutions to common DS challenges

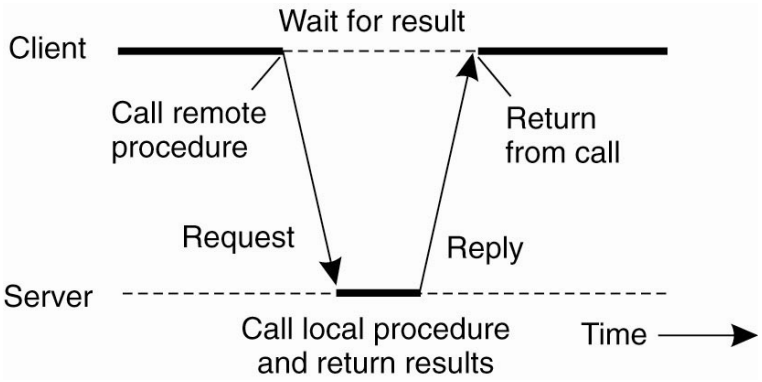
Distributed application programming

- Implicit network programming
  - Language-level constructs simplify distributed programming by hiding network details from developers
  - Developers work with **remote procedure calls (RPC)** or **remote method invocations (RMI)** as if they were local calls
  - In reality, the caller (client) and callee (server) are located on separate machines



Remote Procedure Call (RPC)

- Core idea (Birrel and Nelson, 1984): use client/server model to call procedures executed on remote machines
  - Client on machine A calls a procedure on machine B
  - Calling process on A is suspended
  - Called procedure is execute on B
  - Input/output parameters are transported as messages
  - No message passing is visible to developer



Why RPC

- Widely used in many distributed systems, including cloud computing, microservice architectures
- Many languages and frameworks have they own implementation of RPC:
  - **C (Sun RCP)**
  - **Java (Java RMI)**
  - **Python (RPyC)**
  - **Go**
  - **gRPC**

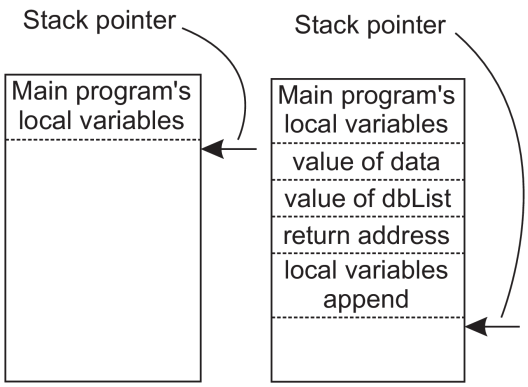
  Our case studies
  - Distributed Ruby https://github.com/ruby/drb
  - Ice https://zeroc.com/ice
  - Microsoft .NET https://dotnet.microsoft.com/
  - JSON-RPC https://www.jsonrpc.org

Local procedure call

- Example of local procedure call:

```
newlist = append(data, dbList)
```

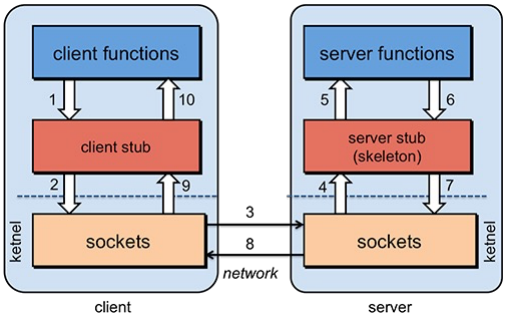
- Caller pushes to stack input parameters (`data`, `dbList`) and return address
- When callee returns, control is back to caller



How can RPC appear as close as possible to a local procedure call?

RPC: architecture

- Solution: proxies (aka **stubs**) to make RPC calls feel like local calls
- Client side: **client stub** exposes service's interface
  - Client calls client stub, which manages all communication details
- Server side: **server stub** receives request and dispatches it to the local procedure on the server
- Goal: distribution transparency
  - Automatic stub generation
  - Focus on application logic

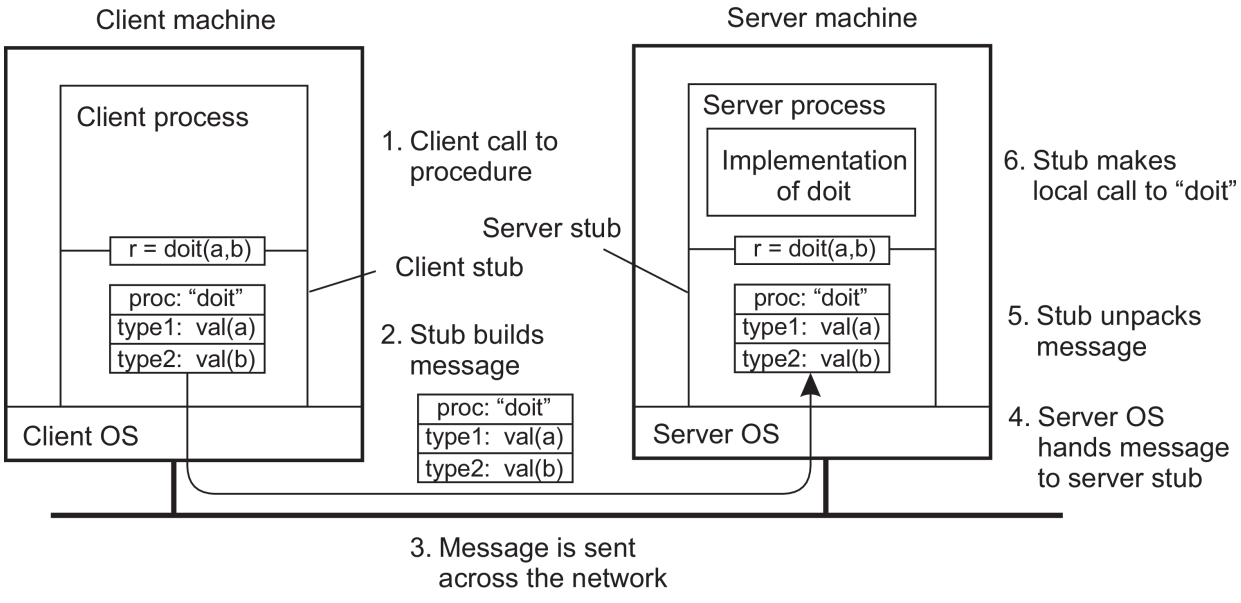


RPC: basic steps

1. On client side, client calls a local procedure, the client stub
2. Client stub marshals parameters (converts them to a common format), packages them into a request message, and call local OS
3. Client OS sends request message to remote OS
4. On server side, remote OS delivers request message to server stub
5. Server stub unmarshals parameters and calls server as it were a local procedure
6. Server executes local call and returns result to server stub
7. Server stubs marshals return value(s) and calls OS
8. Server OS sends reply message to client OS
9. On client side, Client OS delivers reply message to client stub
10. Client stub unmarshals return value(s) and returns result to client

RPC: example

- RPC `doit(a,b)`



RPC: key requirements

- To make an RPC appear as local procedure call, the middleware must:
- Exchange messages between client and server
  - Identify request and reply messages, and remote procedure
  - Pass parameters and return values
- Manage data heterogeneity
  - Parameters, return value(s)
  - Marshaling vs. serialization:
    - ***Marshaling***: bundle parameters into a transferable form that can be unmarshaled (reconstructed) by another process
    - ***Serialization***: convert objects into a sequence of bytes for transmission; serialization is a subset of marshaling
- Handle failures
  - During communication (e.g., message loss, delay)
  - User errors (e.g., invalid input)

RPC: implementation challenges

1. Manage heterogeneity in data representation
   - In addition, client and server must agree on transport protocol for message passing: TPC, UDP, both?
2. Perform parameter passing by reference
   - Client and server run on different machines with separate address space
3. Define failure semantics
   - Local procedure call: exactly-once come visto costo da implementare
   - Remote procedure call: typically at-least-once or at-most-once
4. Bind client to server
   - Locate server endpoint before performing the call

Data representation heterogeneity

- Client and server may use different data representations
  - E.g., byte ordering (little-endian vs big-endian), data size, padding, …
  - RPC must define how RPC messages are encoded on the wire

di solito è un formato indipendente sia dall'architetture dei processori sia dalla rete

- Alternatives to handle data representation heterogeneity (*general*, not only RPC):
  1. Specify encoding within the message itself vedasi http con un campo apposito
  2. Sender converts data into receiver encoding costosa come strategia
  3. Use a common encoding agreed by both parties
     - Sender converts from local to common format
     - Receiver converts from common to local format
  4. Use an intermediary to convert between different encodings

Data representation heterogeneity

- Comparing alternatives for data conversion, assuming *N* components
- #2: each component must know conversion functions for every other component
  - Number of conversion functions: *N**(*N*-1) N componenti devono conoscere N-1 altri formati
  - ✓ Faster conversion (direct)
  - ✗ Poor scalability
- #3: all components agree on common encoding, each component only needs to convert to and from local format
  - Number of conversion functions: 2*(*N*-1) N componenti hanno tutti due funzioni in comune una per passare da formato proprio a comune ed una inversa
  - ✗ Slower conversion (2 steps)
  - ✓ Better scalability
- #3: standard choice in most RPC systems

Data representation heterogeneity: patterns

- Patterns to implement alternatives #3 and #4
- Proxy (aka stub)
  - Goal: support access and location transparency
  - Manage access to a remote object via a proxy
    - Proxy resides in local address space and exposes the same interface of remote object
- Broker
  - Goal: separate and encapsulate communication details from its functionality
  - Enable components to interact without handling remote concerns themselves
  - Handles tasks such as server location, message routing, and communication hiding
- Proxy: standard choice in RPC systems
  - Who automatically generates stubs?

Parameter passing methods

- Call by value
  - Parameter value is copied into local storage (usually stack)
  - Callee acts on copy; changes do not affect caller data quindi server lavora su copia
- Call by reference
  - Reference (pointer) to parameter is passed (copied into stack) lo dice il nome stesso, si passa un riferimento, quindi si HANNO effetti
  - Callee acts directly on caller data
- Call by copy-restore
  - Special case of call by reference
  - Data is copied into callee stack; upon procedure return, updated data is copied back (restored) to caller
  - Supported in a few programming languages (e.g., Ada, Fortran)

nel copy-restore, i dati del client si trovano in un indirizzo e vengono copiati nella richiesta. il server lavora su questa copia e se modifica dati dopo il processamento, tale modifica sarà poi ricopiata nell'indirizzo originale del client

RPC parameter passing

- A reference is a memory address
  - Valid only in its context (local machine)
  - We need a pointer-less representation

- Solution: simulate call by reference in RPC by using call by copy-restore
  - Client stub copies the pointed data into the request message
  - Server stub works on copy in its own address space
  - If modified, the updated copy is restored by client stub overwriting original data
  - Size of data to be copied must be known per allocare spazio sufficiente e serializzare i dati. senza non si riesce
  - What happens if data contains pointers?

se ci sono puntatori serveno ulteriori meccanismi, ad esempio deep copy per gestire i dati, altrimenti i parametri sono inutilizzabili in quanto l'indirizzo copiato è sulla macchina del client ma sulla macchina del server quel indirizzo non punta a nulla.

RPC failure semantics

- Exactly-once semantics
  - Guarantees remote procedure is executed exactly once
  - Costly and complex to implement
- At-least-once semantics
  - If client receives a reply, the remote call has been executed at least once
  - Possible duplicate executions due to retransmissions
  - Suitable for idempotent operations
- At-most-once semantics
  - If client receives a reply, the remote call has been executed at most once
  - No duplicate executions, but call may not have been executed at all in failure scenarios
  - Requires duplicate request detection and state management on server

Binding: locating the server endpoint

- Binding: find the server endpoint, including address and specific process (e.g., port)
  - Types of binding: static or dynamic
- Static binding
  - Server endpoint known at design time (hard-coded)
  - ✓ Simple, no runtime overhead
  - ✗ Lacks transparency and flexibility
    - Changes require redeployment
- Dynamic binding
  - Server endpoint determined at run-time
  - ✓ Gains transparency and flexibility
    - Request redirection in case of server replication
  - ✗ Increased overhead: how to limit it?

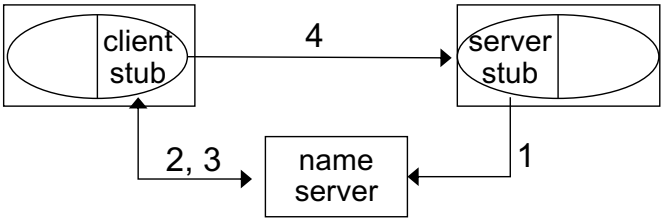
Dynamic binding

- Two phases in client/server relationship

1. Naming: static phase *before* execution
   - Client specifies whom it wants to connect to, using a unique service name
   - Unique names correspond to operations or abstract interfaces
   - Binding is made to the specific service interface, not a concrete address
2. Addressing: dynamic phase *during* execution
   - Actual server endpoint are resolved when client invokes the service
   - Middleware may dynamically select among multiple replica servers

Dynamic binding

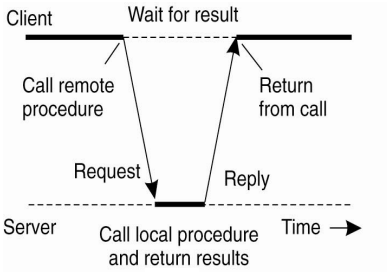
- Addressing can be explicit or implicit
  - Explicit addressing: client sends request using broadcast or multicast, waiting for the first reply
  - Implicit addressing: uses a name server (aka binder, directory service, registry service) that registers services and manages a binding table
    - Supports service lookup, registration, update, and deletion



- Addressing frequency
  - Each RPC requires addressing
  - To reduce overhead, binding results are cached and re-used

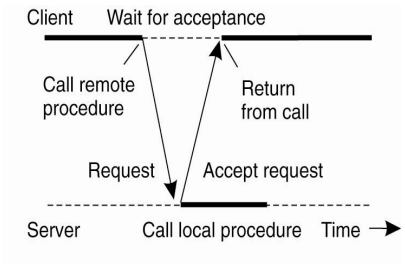
Other issues: Synchronous vs asynchronous RPC

- Synchronous RPC: strict request-reply behavior
  - RPC call blocks client that waits for server reply



Synchronous RPC

- Some RPC middleware supports asynchronous RPC
  - Client does not wait for server reply: continues execution
  - Server can send an ack upon receiving the request is received and execute procedure later



Asynchronous RPC

Other issues: transparency

- Is RPC truly transparent? Can RPCs be treated like local calls?
  - Performance, failures, concurrent requests, replication, migration, …

RPC NON È COMPLETAMENTE TRASPARENTE. Offre solo ACCESS, LOCATION e

- Performance
  - RPC is much slower than local calls: why?
  - Local procedure call: ~10 cycles (~3 ns)
  - RPC on LAN: 0.1-1 ms => ~100000 times slower
    - Major overheads: context switching, data copying, communication
  - RPC over WAN: can be millions of times slower

Other issues: transparency

Già viste nelle slide precedenti

- Common failures in RPC
  - Client cannot locate server
  - Lost request messages during transmission
  - Server crashes during execution
  - Lost reply messages
  - Client crashes before or after request

Other issues: security

- Authentication
  - Does client send messages to genuine server or to impostor?
  - Is the client who it claims to be?
- Authorization and access control
  - Does server accept messages only from legitimate clients?
  - Can server identify users at client side?
- Messages confidentiality and integrity
  - Messages may be visible and sniffed on network
  - Can messages be encrypted?
  - Are messages protected against accidental corruption or truncation while on network?
- Replay attacks
  - Can a malicious host capture and retransmit a message later?

Programming with RPC

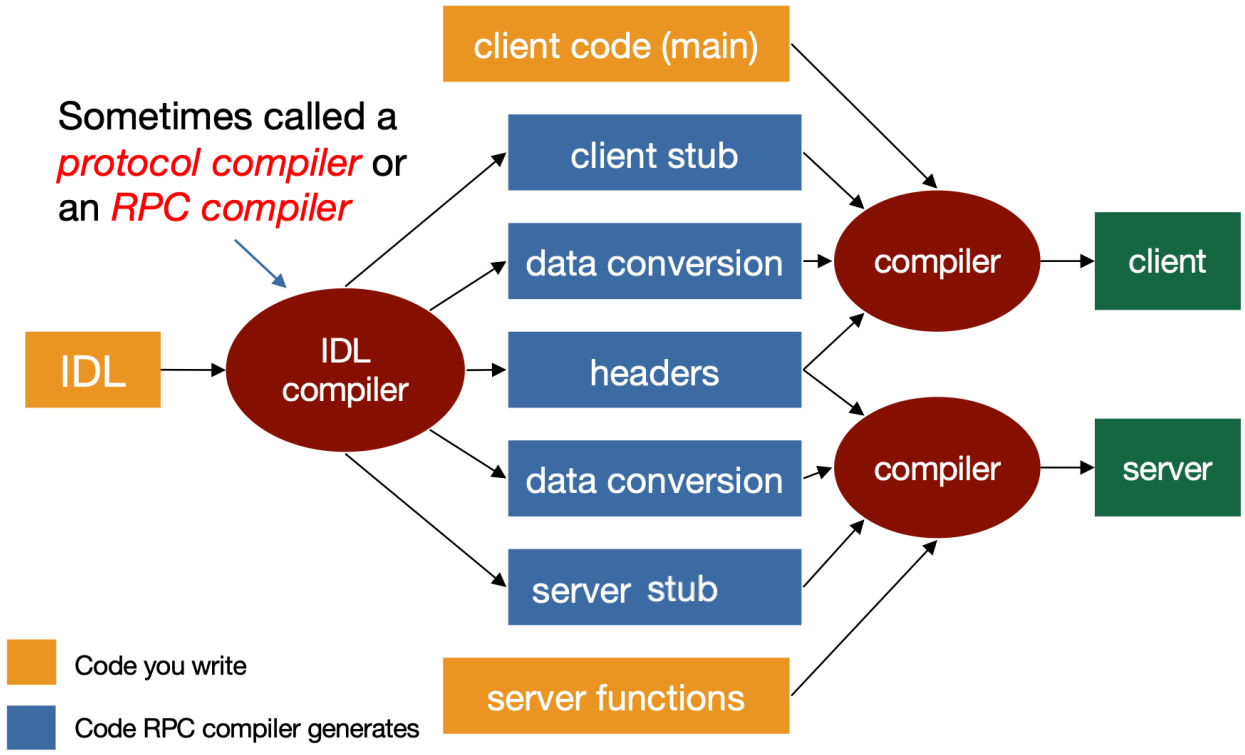
- Language-level support
  - Some languages (e.g., C, C++) lack built-in RPC support
  - Others (Java, Python, Go, Erlang, Haskell) provide direct support
    - Challenge: heterogeneous environments (e.g., Java service communicating with Python service)
- Common solution
  - Use an **Interface Definition Language (IDL)** to describe remote procedures
  - A dedicated IDL compiler automatically generates client and server stubs

Interface Definition Language (IDL)

- IDL allows developers to specify remote procedure interfaces (including *names, parameters, return values*) in a machine-independent way
- IDL compiler uses these interfaces to generate client and server stubs, containing:
  - Code to marshal
  - Code to unmarshal
  - Network transport code
- IDL definitions look similar to function prototypes
- Example: Thrift IDL

```
service KeyValueStore {
  i32 put(1: string key, 2: string value),
  string get(1: string key),
}
```


IDL and its compiler



RPC middleware: case studies

- Sun RPC
- Java RMI
- Python RPyC
- Go
- gRPC

RPC implementation: Sun RPC

- First-generation RPC
- Created by Sun Microsystems (now Oracle)
  - RFC 1831 (1995), RFC 5531 (2009)
  - Still used mainly for NFS (Network File System)
- Interfaces defined using XDR (eXternal Data Representation) as IDL
  - IDL compiler: `rpcgen`
  - Provides built-in conversion functions for:
    - Primitive types, e.g., `xdr_int()`
    - Structured types, e.g., `xdr_string()`
  - Binary format with *implicit typing*
    - Only values sent, no type or parameter metadata

Sun RPC: define RPC program using XDR

- Two main parts in a file `.x`
  1. Definition: procedure specifics to identify procedure names and parameters' data types
  2. XDR definitions: custom data definitions (if not built-in)

- Example: procedure to calculate the square of an integer

Sun RPC example: define remote procedure

`square.x`

```
struct square_in {       /* input (argument) */
  long arg1;
};
struct square_out {      /* output (result) */
  long res1;
};
program SQUARE_PROG {
  version SQUARE_VERS {
  square_out  SQUAREPROC(square_in) = 1; /* procedure number = 1 */
  } = 1;                 /* version number */
} = 0x31230000;          /* program number */
```

- Define remote procedure `SQUAREPROC`
  - Each procedure has only one input parameter and one output parameter
  - Identifiers are written in uppercase
  - Each procedure is associated with a unique procedure number within RPC program (e.g., 1)

Sun RPC: how to implement RPC program

- Developer responsibilities in RPC
- Client program: implements `main()` and logic needed to locate and bind remote procedure (`square_client.c`)
- Server program: implements remote procedures provided by RPC server (`square_server.c`)
  - Note: developer does not write server-side `main()`
  - Who calls the remote procedure on server side?

Example: local procedure

- Let's first consider the standard local procedure

`square_local.c`

```
#include <stdio.h>
#include <stdlib.h>

struct square_in {  /* input (argument) */
  long arg;
};
struct square_out {  /* output (result) */
  long res;
};
typedef struct square_in square_in;
typedef struct square_out square_out;

square_out *squareproc(square_in *inp) {
  static square_out out;

  out.res = inp->arg * inp->arg;
  return(&out);
}
```

Example: local procedure

- Local procedure *(continue)*

```
int main(int argc, char **argv) {
  square_in in;
  square_out *outp;

  if (argc != 2) {
    printf("usage: %s <integer-value>\n", argv[0]);
    exit(1);
  }
  in.arg = atol(argv[1]);

  outp = squareproc(&in);
  printf("result: %ld\n", outp->res);
  exit(0);
}
```

- Remote procedure: what changes?

Sun RPC example: remote procedure

- Remote vs. local: *similar*, not the same

server.c

```
#include <stdio.h>
#include <rpc/rpc.h>
#include "square.h"    /* generated by rpcgen */
square_out *squareproc_1_svc(square_in *inp, struct svc_req
   *rqstp) {
  static square_out out;

  out.res1 = inp->arg1 * inp->arg1;
  return(&out);
}
```

- Input and output parameters use pointers
- Output must point to a static variable so it persists after procedure returns
- Procedure naming convention: original RPC procedure name becomes `<procedure_name>_<version>_svc`, e.g., `squareproc_1_svc`, all lowercase)

Sun RPC example: client

- Run RPC client:

  `./square_client <server-hostname> <integer>`

client.c

```
#include <stdio.h>
#include <rpc/rpc.h>
#include "square.h"   /* generated by rpcgen */
int main(int argc, char **argv) {
  CLIENT *clnt;
  char *host;
  square_in in;
  square_out *result;
   if (argc != 3) {
    printf("usage: client <hostname> <integer-value>\n");
    exit(1);
  }
  host = argv[1];
  clnt = clnt_create(host, SQUARE_PROG, SQUARE_VERS, "tcp");
```

```
CLIENT *clnt_create(char *host, unsigned long prog,
unsigned number vers, char *proto)
```


Sun RPC example: client

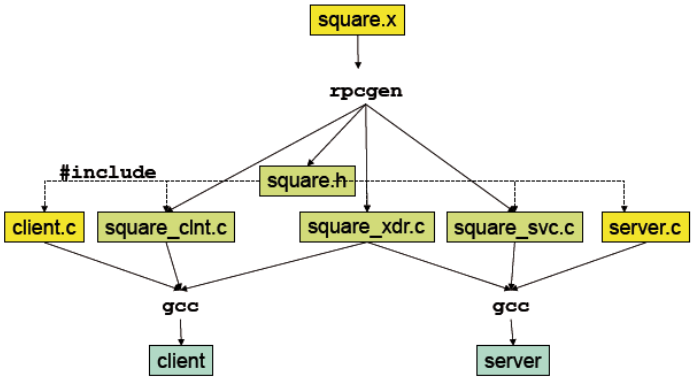
```
  if (clnt == NULL) {
    clnt_pcreateerror(host);
    exit(1);
  }
  in.arg1 = atol(argv[2]);
  if ((result = squareproc_1(&in, clnt)) == NULL) {
    printf("%s", clnt_sperror(clnt, argv[1]));
    exit(1);
  }
  printf("result: %ld\n", result->res1);
  exit(0);
}
```

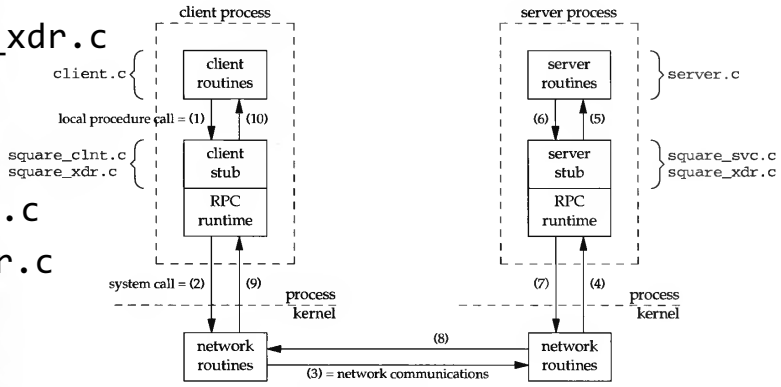
Sun RPC example: client

- Creation of client transport manager: `clnt_create()`
  - Handles communication (TPC or UDP)
  - Default request timeout and retransmission
- Client must know:
  - Server hostname
  - Remote procedure info: program (`SQUARE_PROG`), version (`1`), procedure (`squareproc`)
- To call the remote procedure:
  - Procedure name: lowercase + _ + version (`squareproc_1`)
  - Input: actual parameter + client transport manager
  - Output: pointer to result
  - RPC failure: function returns `NULL`
- Error handling:
  - `clnt_pcreateerror()` and `clnt_perror()`

SUN RPC example: development process

1. Define service specification
   - Using XDR: `square.x`
2. Use `rpcgen` to generate stubs
   - Header: `square.h`
   - Client stub: `square_clnt.c`
   - Server stub: `square_svc.c`
   - XDR routines: `square_xdr.c`
3. Implement client and server procedures
   - Client program: `client.c`
   - Server program: `server.c`





Sun RPC: server binding

- Client stub needs to know port number: how?
- Server stub registers RPC program
  - Each procedure is identified by program number, procedure number, version number
- Information stored in *port map*
  - Dynamic table of RPC services on server host
  - Managed by *port mapper* (`rpcbind`), listening on port 111
  - Client stub contacts port mapper to discover the port number, then sends the request to server stub

Sun RPC: port mapper (rpcbind)

- List RPC programs

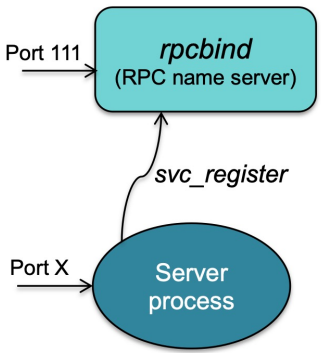
```
>$ rpcinfo -p
 program         vers     proto    port
   100000        4        tcp      111       rpcbind
   100000        4        udp      111       rpcbind
   824377344     1        udp      59528
   824377344     1        tcp      49311
```

Our RPC program `SQUAREPROC`

- 824377344 (= 0x31230000) is the program number in square.x
- Server supports both TCP and UDP: transport-protocol independent

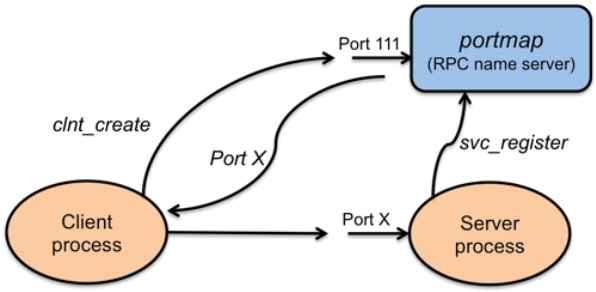
What goes on: server side

- Server stub code (`square_svc.c`)
- In `main()`:
  - Creates a socket and binds it to any available local port
  - Calls `svc_register` (RPC library function)
    - Registers procedures with port mapper (port 111)
    - Associates the specified program and version number pair with the dispatch routine
  - Calls `svc_run` (RPC library function)
    - `svc_run` enters the server's main loop
    - Listens for RPC requests
    - Dispatches to correct service procedure based on program and procedure numbers
    - Sends RPC replies



What goes on: client side

- When client program starts, `clnt_create` contacts server's port mapper to get port for that RPC interface
  - Early binding: done once, not per call



- Client stub (`square_clnt.c`) manages communication
  - Handles request timeout
  - Marshals/unmarshals between local and XDR format

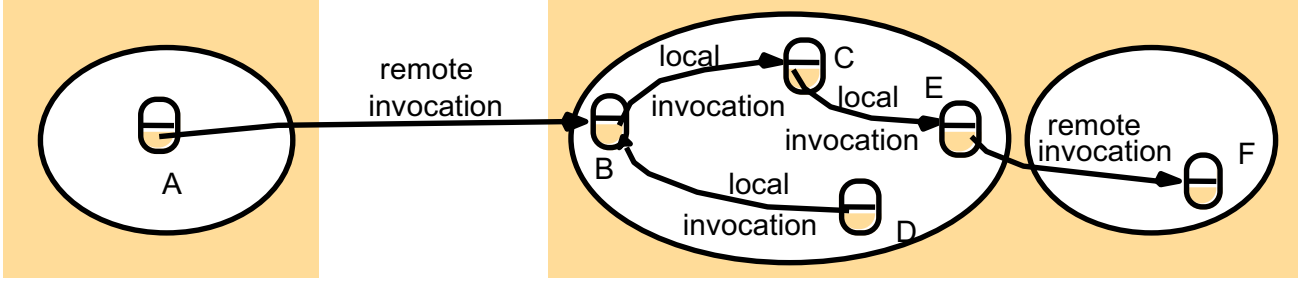
Sun RPC: summing up key features

- Remote procedures attraverso un IDL chiamato XDR
  - RPC program can expose multiple procedures
  - Versioning support
  - One input and one output per procedure
  - Call by copy-restore
- Transport independent: TCP or UDP at run-time
- Synchronous, blocking calls
- Concurrency: default is no concurrency, mutual exclusion guaranteed
- Stub code: visible, auto-generated by `rpcgen`
- Failure semantics: at-least-once
  - Retransmission on timeout
- Server binding: port mapper una volta sola, non per chiamata
- Security: authentication added with Secure RPC

Garantisce ACCESS tramite XDR e LOCATION transparency tramite port mapper parziali

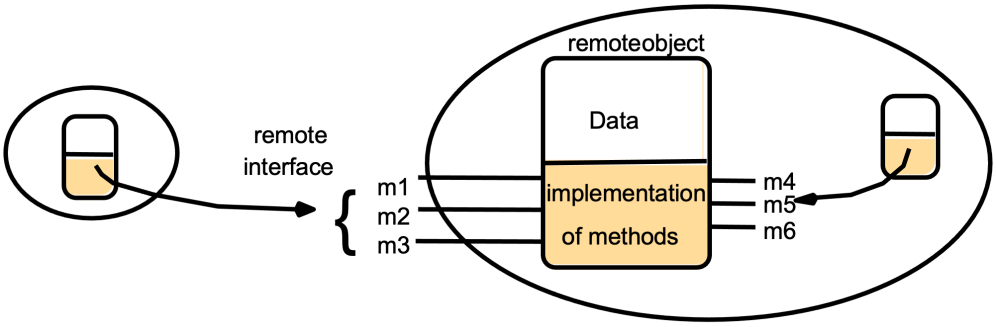
Java RMI

- Java RMI (Remote Method Invocation)
- Second-generation RPC
- RPC for Java, extended to distributed objects
  - Objects on one JVM can invoke methods on objects in another JVM
  - Goal: access transparency
  - Distribution transparency: not full



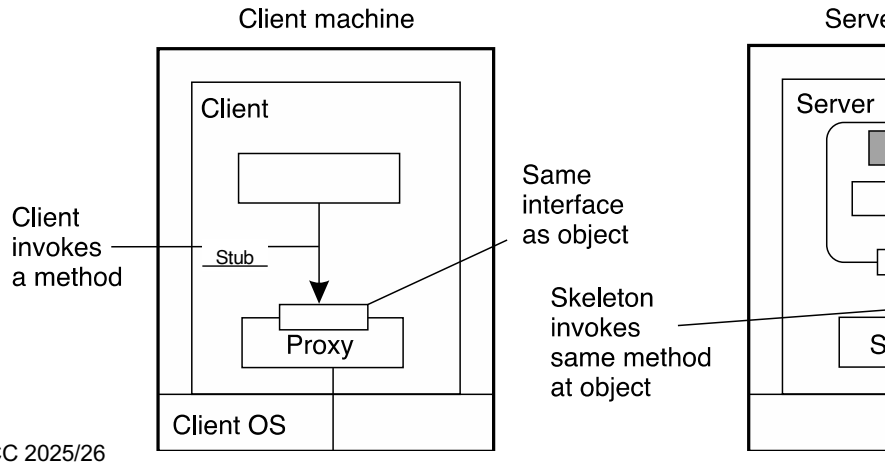
RMI: basics

- Physical separation possible due to logical separation
  - Java separation between definition (interface) and implementation (class)
- Remote interface: defines methods available for remote invocation
- Remote object: instance of class implementing remote interface
  - Important: internal state of remote object is not distributed



RMI: basics

- Remote method invocation: invoke methods of a remote interface on a remote object
  - Goal: keep same syntax as local method call
- Achieved via proxy pattern
  - Client-side **stub**: represents remote object locally
  - Server-side **skeleton**: dispatches calls to actual object

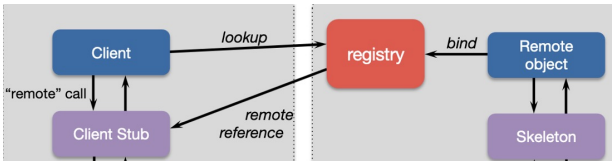


RMI: serialization/deserialization

- No external data representation needed
- Serialization/deserialization handled by Java
- Serialization: converts object passed as parameter to byte stream (`writeObject`)
- Deserialization: decodes byte stream to object (`readObject`)
- Stub and skeleton use this to exchange parameters and return values between JVMs
- Marshaling vs serialization
  - Marshaling: convert local data into network format for transmission
  - Serialization: convert object state into byte stream, reconstructs object at receiver
  - Note: Java distinguishes them (needs codebase), Python Pickle treats them as the same

RMI: call flow

- Client obtains object remote reference (stub) via **RMI registry**
  - Name server that relates objects with names
  - Server registers remote objects with registry
  - Client looks them up by name



- Client invokes remote method on stub
  - Syntax same as local call
- Stub serializes method ID and parameters and sends them to skeleton
- Skeleton receives message, deserializes data, invokes method, serializes return value, and sends it to stub
- Stub receives message, deserializes return value, and returns it to client

RMI example: echo remote interface

- Remote interface extends **`Remote`**
  - **`Remote`** identifies interfaces whose methods may be invoked from non-local JVMs
- Remote method
  - throws **`RemoteException`**
    - To handle failures (network, protocol, server)
    - Remote invocation is not fully transparent: failures are visible
  - passes parameters
    - by value: primitive types or objects implementing `Serializable`; stub serializes, skeleton deserializes
    - by reference: remote objects
      - Reference is a data structure containing {IP address, port, time, object ID, remote interface}, not a pointer!

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface EchoInterface
    extends Remote{
 String getEcho(String echo)
    throws RemoteException;
}
```


RMI example: echo server

- Class implements remote interface
  - Extend **`UnicastRemoteObject`**
  - Call class constructor **`UnicastRemoteObject`** which enables server to receive requests
  - Implement remote method, which throws remote exception

```
public class EchoRMIServer
 extends UnicastRemoteObject
 implements EchoInterface {

 // Constructor
 public EchoRMIServer()
  throws RemoteException {
    super();
 }

 // Implement remote method
 public String getEcho(String echo)
   throws RemoteException {
     return echo;
   }
```

RMI example: echo server

- Create instance of remote object
- Start RMI registry
- Register remote object with registry by name
  - Registry and server run on same host

```
public static void main(String[] args) {
  final int REGISTRYPORT = 1099;
  String registryHost = "localhost";
  String serviceName = "EchoService";
  try {
    // Create an instance of echo server
    EchoRMIServer serverRMI = new EchoRMIServer();
    // Bind remote object with RMI registry
    Registry registry =
      LocateRegistry.createRegistry(REGISTRYPORT);
    registry.bind(serviceName, serverRMI);
  }
  catch (Exception e) {
    System.err.println("EchoRMIServer exception: ");
    e.printStackTrace();
  }
 }
}
```


RMI example: echo client

- Lookup remote object in server's RMI registry by name
- Registry returns a remote reference (stub)
- Client invokes remote method
  - Synchronous, blocking

```
public class EchoRMIClient
{
  // Start RMI client
  public static void main(String[] args)
 {
   bufferedReader stdIn =
     new BufferedReader(
       new InputStreamReader(System.in));
  try
  {
    // Connect to remote RMI service
    EchoInterface serverRMI = (EchoInterface)
         Naming.lookup("EchoService");
    // Interact with user
    String message, echo;
    System.out.print("Message? ");
    message = stdIn.readLine();
    // Invoke remote service
    echo = serverRMI.getEcho(message);
    System.out.println("Echo: "+echo+"\n");
  }
  catch (Exception e)
  {e.printStackTrace(); System.exit(1); }
 }
}
```

RMI: key features

- Transparency: remote calls appear local
- Stub and skeleton auto-generated
- Synchronous, blocking calls
- Concurrency: multiple clients can invoke the same remote method
  - Thread-safety required (`synchronized`)
- Failure semantics: at-most-once
- Full support for object serialization/deserialization and dynamic class loading
- Server binding: RMI registry for object lookup
- Exception handling: `RemoteException`
- Security: optional security manager

registro degli oggetti in questo caso non delle procedure come per Sun

Garantito access transparency lo stub nasconde il tutto, parziale location grazieal RMI registry

Python RPC

- Various RPC implementations
  - Pyro https://pyro5.readthedocs.io/
  - RPyC
  - ZeroRPC https://www.zerorpc.io
- What helps Python achieve transparency
  - Runtime introspection: ability to examine objects during execution via `inspect` module https://docs.python.org/3/library/inspect.html
    - Inspect live objects, examine class contents, retrieve method source code, and extract function argument list
- General idea of RPC in Python:
  - Create connection using an RPC object
  - Invoke remote methods using that object
- Case study: RPyC (Remote Python Call) https://rpyc.readthedocs.io

RPyC: features

- Transparent RPC in Python
  - Transparent access to remote objects as if they were local
  - Proxy-based
  - Pass objects by value (immutable) or by reference (mutable, via `netref`)
  - Lightweight, pure Python: no interface definition file, IDL compiler, name server, transport service
- Symmetric operations
  - Client and server can call RPCs on each other (e.g., server can invoke callbacks on client)
- Synchronous and asynchronous calls
- Secure
  - Capability-based security model
  - Integrates with TLS/SSL, SSH

RPyC: server and client

- Server
  - Multiple implementations
    - `ThreadedServer`: thread for each connection
    - `OneShotServer`, `ThreadPoolServer`, `ForkingServer`: for different concurrency models
  - Binds to default port (18812, 18821 for SSL)
    - Developer can specify host IP and port
- Client
  - Connects to server
  - Uses `modules` property to access server module namespace
  - Transparent proxy forwards operations to remote objects

RPyC: passing data

- By value
  - Simple/immutable types (`int`, `string`, `tuple`)
  - Values are sent directly to remote side
  - Remote copy is independent
- By reference
  - Mutable or complex objects
  - Passed as reference via `netrefs`
  - Remote server contacts client to access attributes or call methods on the original object
    - Changes affect the original (local) object
  - Enables location-sensitive objects (files, OS resources, `sys.stdout`)
    - Remote process can print to local console by getting its `sys.stdout`

RPyC: stubs

- Transparent remote access
  - Client creates a local proxy object for remote modules/objects
  - Proxy acts like the actual object: all operations on proxy are automatically delivered to the remote side
  - Client is unaware that the object is remote
- Synchronous and asynchronous calls
  - Synchronous: client blocks and waits for return
  - Asynchronous: immediate return, notification when complete
  - Calls can be made asynchronous by wrapping the proxy with an *asynchronous wrapper* (`rpyc.async_`)

RPyC: services and security

- RPyC is built around services
  - Each connection exposes a service that defines the set of allowed (its policy)

- Service definition
  - Services are classes derived from `rpyc.core.service.Service`
  - Define methods that can be remotely accessed (exposed)
  - Methods can be called remotely when:
    - Their name begins with `exposed_` or are decorated with `@rpyc.exposed`
  - All exposed members of a service class are available to the remote side

RPyC example: calculator server

```
import rpyc

class CalculatorService(rpyc.Service):
    def exposed_add(self, a, b):
        return a + b
    def exposed_sub(self, a, b):
        return a - b
    def exposed_mul(self, a, b):
        return a * b
    def exposed_div(self, a, b):
        return a / b
    def exposed_fib(self,n):
        seq = []
        a, b = 0,1
        while a < n:
            seq.append(a)
            a, b = b, a+b
        return seq
    def foo(self):
        print("foo")

if __name__ == "__main__":
    from rpyc.utils.server import ThreadedServer
    t = ThreadedServer(CalculatorService, port=18861)
    print('Service started on port 18861')
    t.start()
```

RPyC example: calculator client

```
import rpyc

conn = rpyc.connect("localhost", 18861)
x = conn.root.add(4,7)
assert x == 11
print(conn.root.fib(1000))
# print(conn.root.div(1,0))
```

On the client, the remote service is exposed as the root object of the connection (`conn.root`).
This root object is a network reference (`netref`) to the service instance living in the server process
Any operation performed on `conn.root` is automatically forwarded to the server, making the remote service appear local.
This allows the client to transparently call exposed methods and access attributes of the server service.

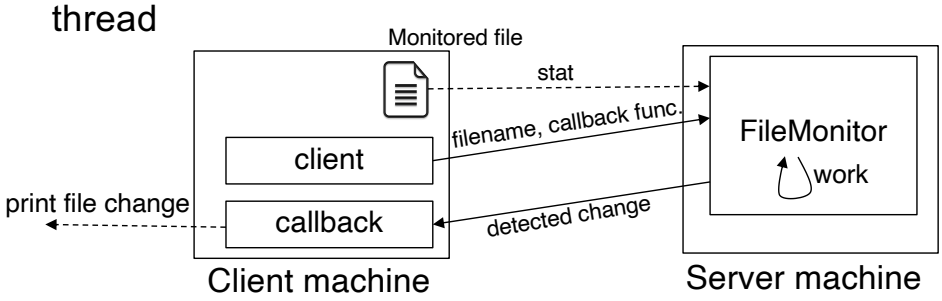
RPyC: async operation

- Key feature of RPyC
  - Client starts remote call and continues without blocking
  - Returns `AsyncResult` object that will *eventually* hold the result
- Asynchronous behavior must be *explicitly* enabled: to make a call asynchronous, wrap it with `async_()`
- Then, client can
  - test `AsyncResult` object for completion using `ready()`
  - wait until completion using `wait()`
  - retrieve result using `value`
  - set timeout for result using `set_expiry()`
  - register a callback function to execute when result arrives using `add_callback()`
- No guaranteed order of execution for async requests

See https://rpyc.readthedocs.io/en/latest/docs/async.html

RPyC: async operation and events

- Events can be implemented as asynchronous callbacks
  - Server produces events, client consumes them asynchronously
- Example: FileMonitor
  - Server periodically monitors file on the client machine using `os.stat()`
  - On file change, server sends an event to client (using an async callback) and provides old and new `stat` results
  - At client side, incoming events are handled by a background thread



RPyC example: FileMonitor server

```
import rpyc
import os
import time
from threading import Thread

class FileMonitorService(rpyc.Service):
    class exposed_FileMonitor(object): # expose class
        def __init__(self, filename, callback, interval = 1):
            self.filename = filename
            self.interval = interval
            self.last_stat = None
            self.callback = rpyc.async_(callback) # create async callback
            self.active = True
            self.thread = Thread(target = self.work)
            self.thread.start()
        def exposed_stop(self): # this method has to be exposed too
            self.active = False
            self.thread.join()
        def work(self):
            while self.active:
                stat = os.stat(self.filename) # get status of monitored file
                if self.last_stat is not None and self.last_stat != stat:
                    self.callback(self.last_stat, stat) # notify client of change
                self.last_stat = stat
                time.sleep(self.interval) # monitor file status periodically

if __name__ == "__main__":
    from rpyc.utils.server import ThreadedServer
    # spawn a thread for each connection
    ThreadedServer(FileMonitorService, port = 18871).start()
```

RPyC example: FileMonitor client

```python
import rpyc
import time
import os

# File to monitor
filename = "/tmp/floop.bloop"
if os.path.exists(filename):
    os.remove(filename)

# Open file to monitor
f = open(filename, "wb", buffering=0)

conn = rpyc.connect("localhost", 18871)

# Start background thread to process incoming events
bgsrv = rpyc.BgServingThread(conn)

# Callback function, obviusly written on client side
def on_file_changed(oldstat, newstat):
    print("\nfile changed")
    print(f"    old stat: {oldstat}")
    print(f"    new stat: {newstat}")

# Create file monitor and pass filename and callback function as arguments
filemon = conn.root.FileMonitor(filename, on_file_changed)

# Wait for file monitor to have a look at file
time.sleep(2)
```


RPyC example: FileMonitor client

```python
# Change file size and be notified by file monitor about change
print("change file size")
f.write(b"loop")

# Wait for file monitor to have a look at changed file
time.sleep(2)

# Change file size again
print("change size again")
f.write(b"groop")
time.sleep(2)

f.close()
filemon.stop()
bgsrv.stop()    # stop background thread
conn.close()
```

Seguito lezione Go

RPyC: summing up key features

- Transparency: transparent access to remote objects and methods via netrefs
- Only exposed classes, methods, or attributes are visible remotely
- Both synchronous and asynchronous calls
- Allows callbacks: server can call functions defined on client
- Concurrency: multiple clients can interact safely with the same server
- Pure Python, no stubs, no IDL, no service registry
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts or retries, no server deduplication

Garantisce access transparency grazie a netrefs, NO LOCATION bisogna conoscere hostname e porta, si concurrency transparency

RPC in Go

- Go provides built-in support for RPC
  - Package `net/rpc` https://pkg.go.dev/net/rpc
  - Type-safe RPC mechanism

- Main features
  - Provides remote access to exported methods of an object
  - TCP or HTTP as “transport” protocols
  - Supports concurrent client requests

RPC in Go: defining an RPC service

- Define a type (struct) whose methods will be exposed remotely
- Methods must satisfy the following rules:
  - Method type and method name are exported (start with capital letter)
  - Method has two arguments, both exported or built-in types
    - First argument: input (usually a struct)
    - Second argument: pointer to reply struct for response
  - Method returns an error

Defining an RPC service: example

```
package arith
import "errors"

type Arith struct{}  // Service struct
type Args struct {A, B int}  // Input arguments
type Result int  // Result type for Multiply method
type Quotient struct {Quo, Rem int}  //Result type for Divide method

func (t *Arith) Multiply(args *Args, res *Result) error {
        *res = args.A * args.B
        return nil
}
func (t *Arith) Divide(args *Args, quo *Quotient) error {
        if args.B == 0 {
                return errors.New("divide by zero")
        }
        quo.Quo = args.A / args.B
        quo.Rem = args.A % args.B
        return nil
}
```

RPC in Go: server

- On server side
  - Create a TPC (or HTTP) server to receive incoming requests
  - Use **`Register`** (or **`RegisterName`**) to make an object's exported methods visible as RPC procedures and publish them on server

```
func (server *Server) Register(rcvr any) error
func (server *Server) RegisterName(name string, rcvr
any) error
```

  - Takes interface as input: `any` is an alias for `interface{}`, allows any value (tipically a pointer to a struct)
  - `Register` uses the type name as service name and registers methods
  - `RegisterName` allows to specify a custom service name

RPC in Go: server

  - Use **`Listen`** to create a network listener and announce the server on a specific port

```
func Listen(network, address string) (Listener, error)
```

  - Use **`Accept`** to accept connections on the listener and serve requests

```
func (server *Server) Accept(lis net.Listener)
```

    - Concurrency out of the box: `Accept` spawns a separate goroutine for each request
    - `Accept` blocks; to handle other tasks concurrently in main, call `Accept` in a goroutine
  - Can also use an HTTP handler for RPC messages
    - Use `HandleHTTP` and `http.Serve`
    - See example on course site

RPC in Go: client

- On client side
  - Use **`Dial`** to connect to RPC server at the specified address and port using the given transport protocol (e.g., `"tcp"`)

```
func Dial(network, address string) (*Client, error)
```

    - Use `DialHTTP` for HTTP connection
  - Use **`Call`** to call synchronous RPC
    - `Call` waits for the remote procedure to complete

```
func (client *Client) Call(serviceMethod string, args
any, reply any) error
```

  - Use **`Go`** to call asynchronous RPC
    - `Go` invokes the call asynchronously and signals completion via `Done` channel in `Call` struct

```
func (client *Client) Go(serviceMethod string, args
any, reply any, done chan *Call) *Call
```


RPC in Go: client

- On client side
  - Struct **`Call`** represents an active RPC

```
type Call struct {
    ServiceMethod string  // The name of the service and method
                          // to call.
    Args any // The argument to the function (*struct)
    Reply any // The reply from the function (*struct)
    Error error // After completion, the error status.
    Done chan *Call // Receives *Call when Go is complete.
}
```

RPC in Go example: calculator

- Let's consider a simple RPC calculator with two functions: multiply and divide two integers
- Code available on course site

RPC in Go: synchronous call

- Need some setup in advance of this, see example
- `Call` makes a blocking RPC call: invokes the remote function, waits for it to complete, and returns its error status

```
// Synchronous call
args := arith.Args{n1, n2}
var mulReply arith.Result
err = client.Call("Arithmetic.Multiply", args, &mulReply)
if err != nil {
   log.Fatal("Error in Arithmetic.Multiply:", err)
}
fmt.Printf("Arithmetic.Multiply: %d*%d=%d", args.A, args.B,
   mulReply)
```

```
func (client *Client) Call(serviceMethod string, args any, reply any)
error
```

RPC in Go: asynchronous call

- Go uses a channel as parameter to retrieve the RPC reply when the call is complete
- The Done channel will signal when the call is complete by returning the same `Call` object
  - If no channel in input, a new one will be allocated by Go

```
// Asynchronous call
divReply := new(Quotient)
divCall := client.Go("Arithmetic.Divide", args, divReply, nil)
replyCall = <- divCall.Done
// check errors, print, etc.
```

channel

```
func (client *Client) Go(serviceMethod string, args any, reply any,
done chan *Call) *Call
```

- Go internal implementation: see https://go.dev/src/net/rpc/client.go?s=8029:8135 - L284

Go's RPC semantics

- Client-side: no automatic retries and built-in timeouts
- Client returns an error if no reply is received
  - TCP timeout
  - Server may not have received request
  - Server processed request but a failure occurred before sending reply or a network failure occurred
- Server-side: no automatic deduplication of requests
  - Client request has a unique sequence number but is not used for deduplication on server
  - Sequence number is used on client for request tracking
- Best-effort model
- Developer responsibility to implement retries, timeouts, and server-side deduplication

RPC in Go: marshaling and unmarshaling

- By default, Go uses `encoding/gob` package for marshaling (encoding) and unmarshaling (decoding) RPC parameters https://pkg.go.dev/encoding/gob
  - When you use `net/rpc`, gob is the default encoding/decoding mechanism used automatically
- gob manages binary streams (gobs) exchanged between Encoder (sender) and Decoder (receiver)
  - Each item in the stream is *self-describing*: item is preceded by a specification of its type; pointers are not transmitted, only the values they point to are
- Basic usage: create an encoder, transmit values, use a decoder to receive and decode values
  - Both RPC client and server must be written in Go
- Use gob explicitly when you need encoding/decoding of Go data outside the automatic RPC mechanism or for persistence

RPC in Go: marshaling and unmarshaling

- Alternatives to gob for Go RPC

1. `net/rpc/jsonrpc` provides JSON-RPC encoding https://pkg.go.dev/net/rpc/jsonrpc
   - Implements JSON-RPC 1.0 for `rpc` package
   - Provides ClientCodec and ServerCodec
   - JSON: human-readable but less efficient than gob
2. Protocol buffers and gRPC instead of `net/rpc`

Go RPC: summing up key features

- Transparency: transparent access to remote structs and methods
- Only exposed methods are visible remotely
- Both synchronous and asynchronous calls
- Concurrency: multiple clients can interact with the same server
- No IDL, no service registry
- Go automatically handles marshalling and unmarshalling
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts, retries, server deduplication

Garantisce ACCESS transparency, niente location in quanto server hostname e porta garantito anche concurrency transparency

Comparing RPC so far

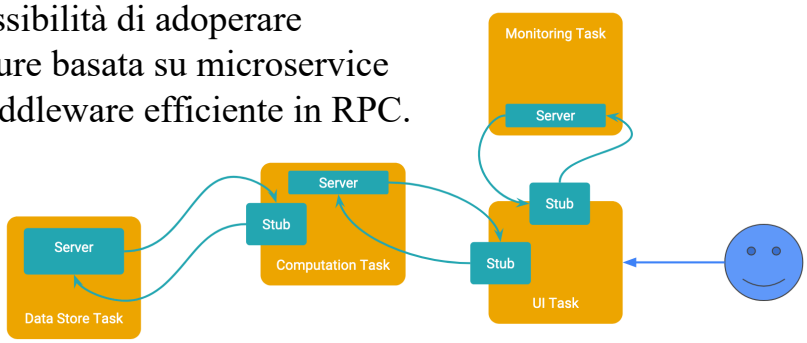
- Let's compare RPC implementations and how they differ in terms of distribution transparency
  - Access transparency
  - Location transparency
  - Concurrency transparency
  - Failure transparency
  - Replication transparency
    - Add server-side load balancer which acts as proxy between RPC clients and replicated RPC servers

tabella completa alla fine

Motivation for new RPC middleware

- Large-scale distributed applications composed of **microservices**
  - Microservices architecture: build application as a collection of *independent*, *autonomous* (developed, deployed, and scaled independently), *business capability–oriented*, and *loosely coupled* services
  - Multi-language (i.e., polyglot) development
  - Communication mainly structured as RPCs

come visto, data la possibilità di adoperare una software architecture basata su microservice viene introdotto un middleware efficiente in RPC. gRPC



gRPC

- High-performance, open-source universal RPC framework https://grpc.io/
- Can run in any environment
  - Multi-language, multi-platform support
- Background and adoption
  - Developed by Google and released in 2015, now CNCF project
  - Widely used by companies like Cisco, Discord, Netflix, Reddit, Spotify, Square
  - Etcd, Kubernetes, containerd, GPC use gRPC for communication between components
  - Has become a cornerstone for cloud-native environments

gRPC

- Main use cases
  - Connect polyglot microservices (request-reply communication style) across and within data centers
    - Supports load balancing, tracing, health checks, authentication
  - Connect devices, mobile apps and browsers to backend services
  - Generate efficient client libraries
- Also core part of many ML systems
  - Gemini, TensorFlow, NVIDIA Triton inference system use it
- And popular for serving models

  https://github.com/tensorflow/serving/blob/master/tensorflow_serving/apis/prediction_service.proto

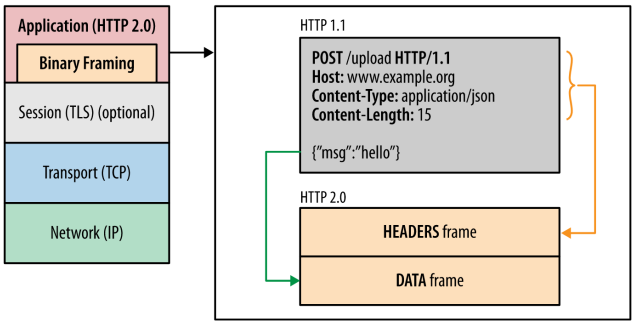
  https://bentoml.com/blog/3-reasons-for-grpc

gRPC: key features

- Transport and communication
  - Uses **HTTP/2** for efficient transport
  - Supports bidirectional streaming, multiplexing, and header compression
- Interface definition and code generation
  - **Protocol Buffers** as IDL
  - Simple service definition, automatic client/server code generation
  - Strict specification prevents errors
- Advanced features
  - Authentication and flow control
  - Blocking or non-blocking client bindings
  - Deadline, timeouts and cancellation
  - Interceptors/middleware: logging, monitoring, retires, metrics

gRPC transport: HTTP/2

- Basic idea: treat RPCs as references to HTTP objects, leveraging HTTP/2 as underlying transport
- Why HTTP/2?
  - Major revision of HTTP, providing significant performance improvements over HTTP 1.x
  - Enables low-latency, high-throughput communication
- HTTP/2 in a nutshell
  - Binary framing layer: requests and responses are divided into small messages framed in binary, making transmission efficient



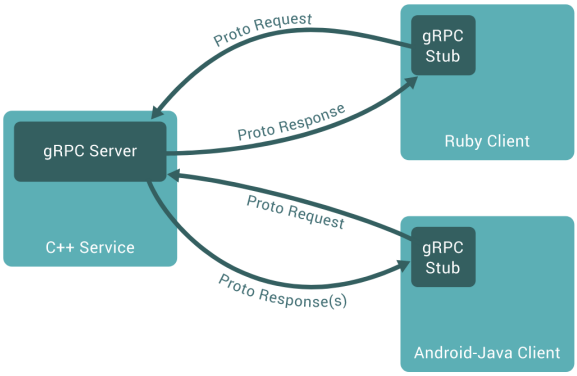
gRPC transport: HTTP/2

- HTTP/2 in a nutshell
  - From messages to streams
    - Stream: bidirectional flow of bytes within a single connection, may carry one or more messages
    - Message: complete sequence of frames, representing a logical request or response
    - Frame: smallest unit of communication, contains a frame header identifying its stream
  - Bidirectional streaming: supports client-server and server-client streaming simultaneously
  - Request/response multiplexing: multiple requests/responses per client share a single connection, allowing efficient use of TCP connections and avoiding head-of-line blocking
  - HTTP header compression: reduces protocol overhead

See https://hpbn.co/http2/

gRPC: Protocol buffers

- gRPC uses Protocol buffers (aka *protobuf*) as:
  - IDL for service interface definition and automatic generation of client stubs and abstract server classes
  - Message interchange format: gRPC messages are serialized using protobufs, thus resulting in compact, efficient message payloads
- Design pattern: proxy pattern
  - Client stub acts as a local proxy to the remote service
  - Server skeleton handles incoming RPC calls



Protocol buffers

- Google's mature open-source mechanism to serialize structured data
- Binary data representation: compact and efficient
- Strongly typed: enforces data safety and prevents erros

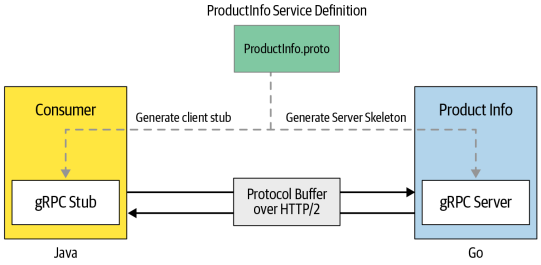
```
syntax = "proto3";

message Person {
  string name = 1;
  int32 id = 2;
  string email = 3;
}
```

- Data types are structured as messages
  - Message: logical record of information containing a series of name-value pairs called fields
  - Each field has a unique number (e.g., `string name = 1`)
    - Field numbers are used in the binary encoding to identify data

Protocol buffers: example

- ProductInfo service interface



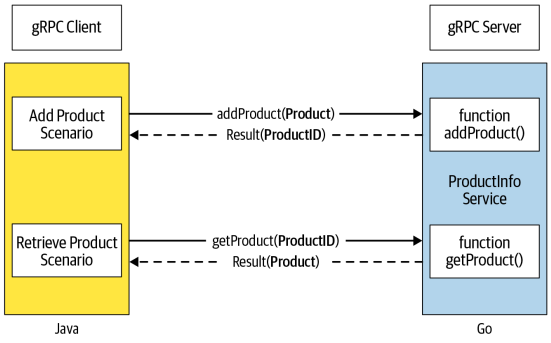

```
// ProductInfo.proto
syntax = "proto3";
package ecommerce;

service ProductInfo {
    rpc addProduct(Product) returns (ProductID);
    rpc getProduct(ProductID) returns (Product);
}

message Product {
    string id = 1;
    string name = 2;
    string description = 3;
}

message ProductID {
    string value = 1;
}
```

Client-server interaction



gRPC: basic steps

1. Define service and messages: write `.proto` file using protobufs as IDL and describing
   - Service: collection of remote methods
   - Message types exchanged between client and server
2. Generate server and client code
   - Use `protoc` (protocol buffer compiler) in your preferred language(s) from proto definition
   - Go: run `protoc` manually to generate stubs
   - Java: use build tools (e.g., Maven, Gradle)
3. Implement service and client
   - Use language-specific gRPC API (e.g., Go, Java, Python) to write client and server applications
   - Let's consider Go: gRPC-Go https://grpc.io/docs/languages/go

gRPC: helloworld example in Go

- See https://grpc.io/docs/languages/go/quickstart/ and https://github.com/grpc/grpc-go/tree/master/examples/helloworld/

1. Define service (`helloworld.proto` file)

```
package helloworld;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```


gRPC: helloworld example in Go

2. Compile gRPC service definition

```
$ protoc --go_out=. --go_opt=paths=source_relative \
    --go-grpc_out=. --go-grpc_opt=paths=source_relative \
    helloworld/helloworld.proto
```

- Automatically generated files:
  - `helloworld.pb.go`
    - Contains Go structs for protobuf message types: message types have become Go structs
    - Provides methods to populate, serialize, and retrieve request and response data
  - `helloworld_grpc.pb.go`
    - contains Go interfaces (*stub*) for RPC calls: RPC definitions have become Go interfaces
    - Client stub interface to call service methods
    - Server interface to implement service methods
    - Uses message structs from `helloworld.pb.go`

gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   a. Implement the interface generated from `.proto` service definition: the actual "work"

```
func (s *server) SayHello(ctx context.Context,
   in *pb.HelloRequest) (*pb.HelloReply, error) {
    // Implement service logic here
}
```

   - `ctx`: carries deadlines, cancellation signals and metadata
   - `in`: request message from the client

gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   b. Create and run gRPC server: listen on a network port, create a new gRPC server instance, register the service implementation, pass the listener to the gRPC server

```
lis, err := net.Listen("tcp", port)
if err != nil {
    log.Fatalf("failed to listen: %v", err)
}
s := grpc.NewServer()
pb.RegisterGreeterServer(s, &server{})
s.Serve(lis)
```

   - `Serve` blocks and starts handling requests

gRPC: helloworld example in Go

4. Create gRPC client
   - Establish a *gRPC channel* (connection) to communicate with server using `Dial`

```
conn, err := grpc.Dial(address, opts…)
```

     - `opts` can include options like `grpc.WithInsecure()`, TLS credentials, etc.
   - Create a client stub to perform RPCs: get it using `pb.NewGreeterClient` provided by `pb` package (generated from `.proto` file)

```
c := pb.NewGreeterClient(conn)
```

   - Call service methods on client stub: send request, receive response

```
r, err := c.SayHello(ctx, &pb.HelloRequest{Name: name})
```

     - Create and populate a request protobuf (`HelloRequest`)
     - Pass a `context` to support deadlines, cancellation or metadata
     - Receive a response protobuf (`HelloReply`)

gRPC: updating a service

- gRPC simplifies application updates
  - Incremental updates without breaking existing functionalities
- To add a new method
  1. Modify the service definition (`.proto` file): add new methods, update message types
  2. Regenerate client and server stubs
  3. Update server implementation to implement the new method
  4. Update client code to call the new method

gRPC: updating helloworld example

- Let's update helloworld adding `SayHelloAgain()`
  1. Update the `.proto` file
  2. Regenerate gRPC code using `protoc`
  3. Update server implementation with the new method

```
func (s *server) SayHelloAgain(ctx context.Context, in
   *pb.HelloRequest) (*pb.HelloReply, error) {
    log.Printf("Received: %v", in.GetName())
    return &pb.HelloReply{Message: "Hello again " +
   in.GetName()}, nil
}
```

  4. Update client code calling the new method

```
r, err = c.SayHelloAgain(ctx, &pb.HelloRequest{Name: name})
 if err != nil {
    log.Fatalf("could not greet: %v", err)
 }
 log.Printf("Greeting: %s", r.GetMessage())
```


gRPC: ProductInfo example

- ProductInfo microservice which manages products and their information
- Client operations:
  - Add new products, retrieve product details

1. Define service in `.proto` file (see slide 122)
2. Implement server and client in Go
3. Implement server and client in Java

Go code: Teams/course website

Java code: https://github.com/grpc-up-and-running/samples/tree/master/ch02

See chapter 2 of gRPC: Up and Running
https://www.oreilly.com/library/view/grpc-up-and/9781492058328/

gRPC: service method types

- gRPC supports 4 kinds of service methods
  - Defined in `.proto` file
  - Example: route guide
    https://github.com/grpc/grpc-go/tree/master/examples/route_guide

1. Simple (unary) RPC
   - Client sends a single request to server and waits for a single response
   - A normal function call

```
rpc GetFeature(Point) returns (Feature) {}
```

2. Server-streaming RPC
   - Client sends one request to server
   - Server responds with a stream of messages
   - Client reads until the stream ends
   - Message order is guaranteed within an RPC call

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```


gRPC: service method types

3. Client-streaming RPC
   - Client sends a stream of messages to the server
   - Server reads all messages and returns a single response
   - gRPC guarantees message ordering within each RPC call

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

4. Bidirectional streaming RPC
   - Both client and server send streams of messages simultaneously (i.e., full duplex)
   - Each side can read and write independently
   - Server can wait until it has received all client's messages before writing its messages, or server and client can play "ping-pong"
   - gRPC preserves message ordering in each stream

```
rpc RouteChat(stream RouteNote) returns (stream
RouteNote) {}
```

gRPC: route guide example in Go

- Goal: demonstrates all four types of gRPC communication in Go using a location-based route guide service
- protobufs: service and message definitions in `routeguide/route_guide.proto`
- Server: implements gRPC service methods (`GetFeature`, `ListFeatures`, `RecordRoute`, `RouteChat`)
- Client: calls RPCs using Go gRPC stubs
- Transport: supports both plaintext and TLS-secured connections

gRPC: distribution transparency

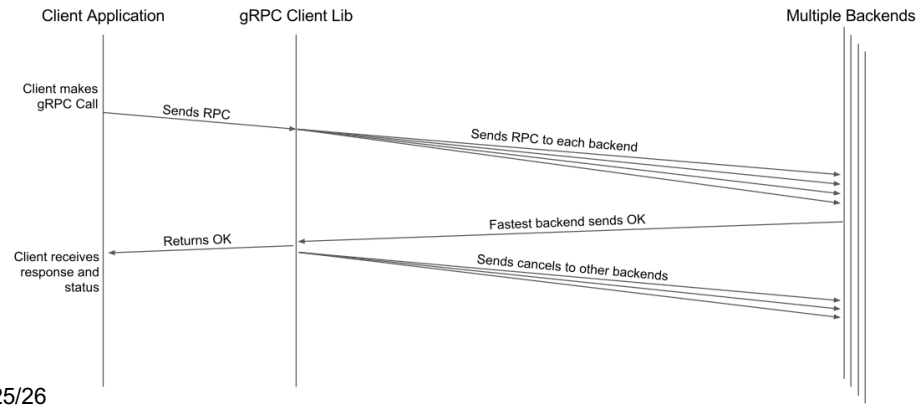
- Access transparency
  - Achieved through the usual proxy pattern
  - Cross-language
- Location transparency
  - Default: DNS name resolver
  - Custom name resolution
    https://grpc.io/docs/guides/custom-name-resolution
  - Service discovery supported via pluggable mechanisms (e.g., Consul, Eureka)
  - Integration with service meshes (e.g., Istio, Consul) that provide service discovery
    - Infrastructure layer for managing service-to-service communication in microservices

gRPC: distribution transparency

- Concurrency transparency
  - Provides concurrency-safe APIs, but concurrency management remains application-dependent (no full transparency)
  - gRPC-Go uses goroutines and synchronization mechanisms https://github.com/grpc/grpc-go/blob/master/Documentation/concurrency.md
- Replication transparency
  - Integrates with client-side or external load balancers (e.g., Envoy) to distribute client requests across replicated servers https://grpc.io/docs/guides/custom-load-balancing
  - Replication logic (e.g., keeping replica consistent) must be handled by the application
- Failure transparency
  - At-most-once semantics by default
  - Some failures (e.g., server crashes mid-call) may result in *partial execution*, not rolled back automatically: no exactly-once semantics

gRPC: distribution transparency

- Failure transparency
  - Retry strategies can be configured for transient failures, including exponential backoff https://grpc.io/docs/guides/retry → retry with backoff pattern
  - Request hedging: send multiple copies of the same request to reduce tail latency https://grpc.io/docs/guides/request-hedging
    - Inspired by: Dean & Barroso, The Tail at Scale, 2013 https://research.google/pubs/the-tail-at-scale/
  - Retries and hedging must be configured, not automatic



gRPC: additional features

- Security: SSL/TLS support
  - How to use
    - Generate certificates
      - Development: use self-signed certificates (via OpenSSL)
      - Production: use valid, trusted certificates from Certificate Authority
    - Server side: configure gRPC server with SSL/TLS certificates and private key
    - Client side: configure gRPC client to trust server's certificates
    - Mutual TLS (mTLS): for client-server authentication, ensuring both sides are verified
    - Example: https://github.com/grpc/grpc-go/tree/master/examples/features/encryption
- Benchmarking and load testing
  - ghz: modern load testing tool for gRPC https://ghz.sh
  - Test different gRPC call types (e.g., unary vs. streaming)
  - Benchmark with different payload sizes and concurrency levels to simulate real-world traffic

gRPC: weaknesses

- Limited browser support
  - Cannot directly call a gRPC service from a browser
  - Requires gRPC-Web for communication (but limited features) https://grpc.io/docs/platforms/web/basics/
- Binary protocol
  - protobufs is efficient but not human-readable
  - Requires additional tools like gRPC command-line tool or Wireshark https://wiki.wireshark.org/gRPC
- Load balancing and traffic management
  - No built-in server-side load balancing
  - Service mesh dependency
- Complexity in large-scale systems

Tabella riassuntiva di RPC

Tecnologia	Access	Location	Replication	Failure	Concurrency	Sincronia
Sun RPC	Parziale, serve sapere il nome della procedura e i parametri	Parziale, tramite il port mapper serve solo hostname	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	No, le procedure eseguono in mutua esclusione	No, supporta solo procedure sincrone
Java RMI	Si, attraverso pattern proxy, permette di manipolare oggetti remoti	Parziale, tramite RMI registry che mantiene gli oggetti remoti	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, gli oggetti devono essere invocati in contesto thread-safe	No, supporta solo procedure sincrone
RPyC	Si, con pattern proxy via netrefs, permette di manipolare oggetti remoti	Non è garantita, serve hostname e porta	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, ci sono diverse configurazioni ma vanno gestiti i lock	Si, supporta procedure asincrone
Go RPC (net/rpc)	Parziale, serve sapere il nome della procedura e i parametri	Non è garantita, serve hostname e porta anche se si usa il registrer nel server	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, si usano le goroutine ma vanno gestite	Si, supporta procedure asincrone
gRPC	Garantita, tramite Protobuf	Non è garantita, serve hostname e porta	Parziale, tramite un Load balancer esterno	Parziale, permette di fare retry e request hedging se configurato	Parziale, la gestione è application dependent	Si, supporta procedure asincrone

configurato

Message-oriented communication

- RPC provides distribution transparency over socket programming
- But still coupling between interacting entities
  - Temporal: caller waits for a reply
  - Spatial: shared data
  - Functionality and communication are tightly coupled
- How to improve decoupling and flexibility?
- Message-oriented communication
  - Transient
    - Berkeley sockets, Message Passing Interface (MPI)
  - Persistent
    - **Message Oriented Middleware (MOM)**

RPC non garantisce alcun tipo di decoupling
Spatial: deve conoscere minimo i metodi
Temporal: può essere sincrono
Synchronization: se sincrono si blocca il caller

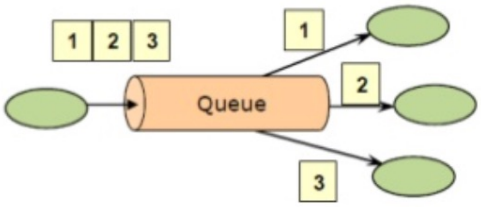
In particolare un message-queuing system o MOM fornisce un supporto esteso per comunicazioni persistenti asincrone, ovvero dove i messaggi rimangono sul disco e i due host non devono essere entrambi presenti al momento della comunicazione.

Message-oriented middleware

- Communication middleware that supports sending and receiving messages in a persistent way
  - Provides intermediate-term storage for messages
- Loose coupling among system/app components
  - Supports temporal and spatial decoupling
  - Can also support synchronization decoupling
- Goals: increase performance, scalability, and reliability
  - Commonly used in serverless and microservice architectures
- Communication patterns:
  - **Message queue** studieremo questi due pattern
  - **Publish-subscribe** (pub/sub)
- Related systems:
  - **Message queue system** (MQS) e questi due sistemi
  - **Pub/sub system**

Message queue pattern

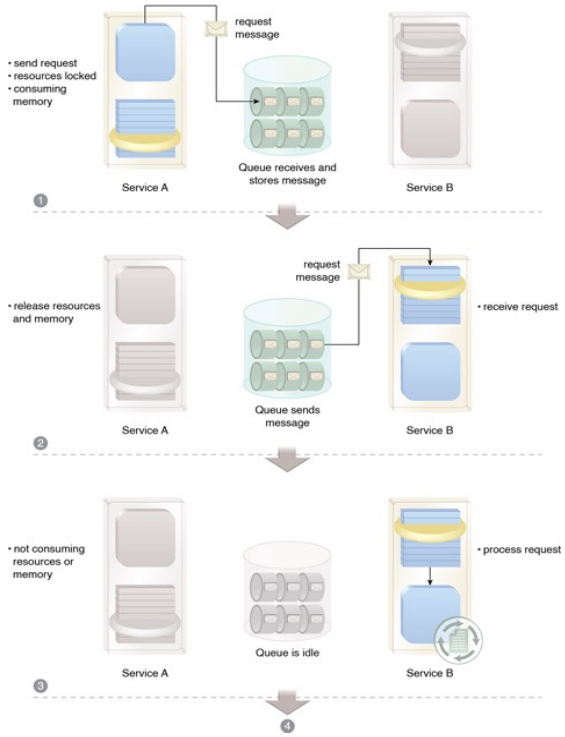
- Messages sent to a queue are stored until retrieved by a consumer
- Multiple producers can send messages to the queue
- Multiple consumers can receive messages from the queue
- Communication is one-to-one: each message is delivered to a single consumer



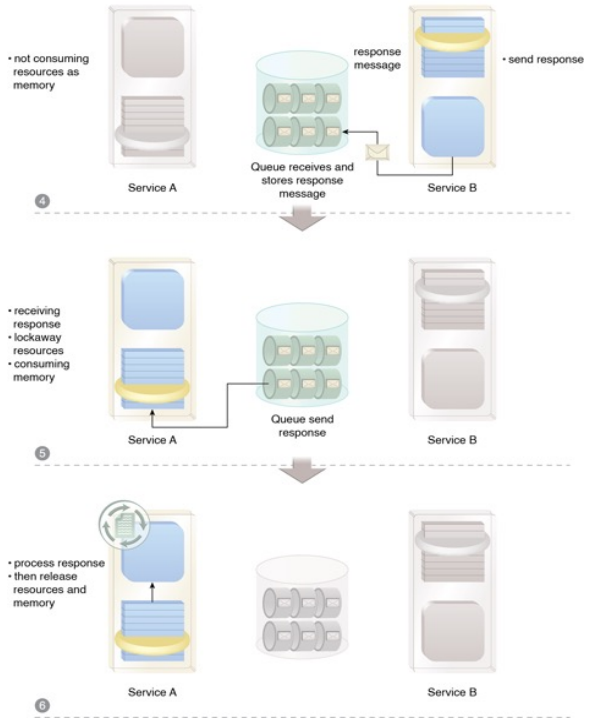
- Use cases:
  - Task scheduling, load balancing, logging or tracing

Message queue pattern

A sends a message to B



B processes the message and sends a response back to A



pattern abbastanza "rudimentale", da notare come il processamento sia operazione bloccante

Message queue API

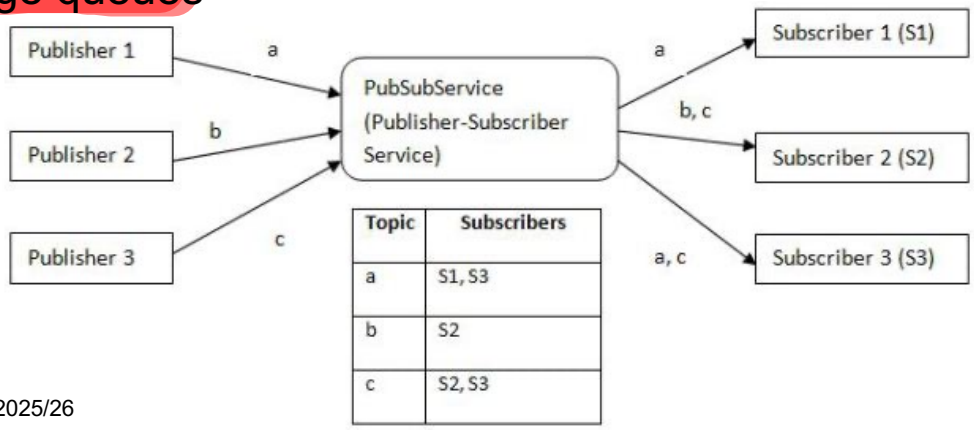
- Typical calls in MQS:
  - **put**: non-blocking send
    - Inserts a message into the queue immediately
  - **get**: blocking receive
    - Waits until the queue is non-empty and retrieve a message
    - Variant: can search for a specific message in the queue
  - **poll**: non-blocking receive
    - Checks the queue and retrieves a message if available
    - Never blocks
  - **notify**: non-blocking receive
    - Installs a callback handler that is automatically invoked when a message arrives in the queue

come elencati da Tanenbaum

il pattern odierno

Publish/subscribe pattern

- Producers (publishers) send asynchronous messages, such as event notifications
- Consumers (subscribers) express interest in specific topics by subscribing
- Decoupling in time and space: publishers and subscribers do not need to interact directly
- Supports one-to-many communication, unlike message queues



RECAP del pattern

Publish/subscribe pattern

- Multiple consumers can subscribe to a topic, with or without filters
- An *event dispatcher* collects subscriptions and route messages to all matching subscribers
  - For scalability, the dispatcher is typically distributed
- High degree of decoupling among components
  - Components can be easily added or removed: ideal for dynamic environments
- Use cases:
  - Event notification, multicasting
- Comparison with message queue

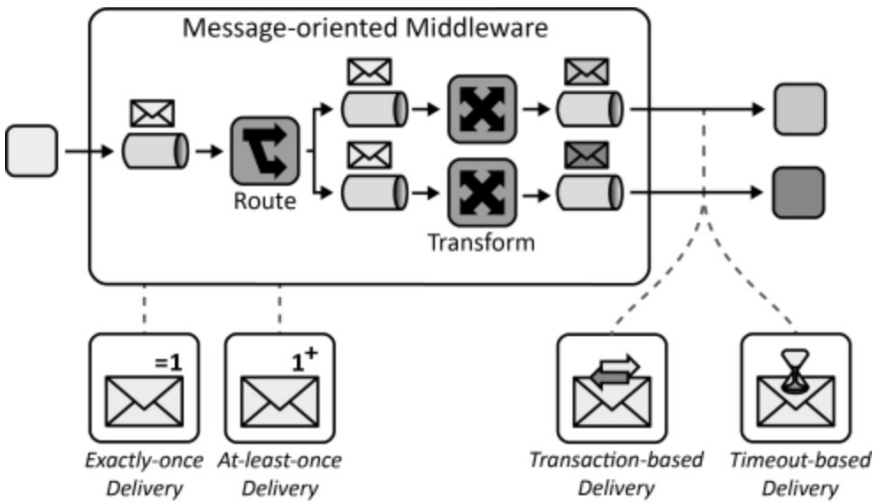
Feature	Message queue	Pub/sub
Delivery	One-to-one	One-to-many
Consumers	Single consumer per message	Multiple consumers per message

Publish/subscribe API

- Typical calls in a pub/sub system:
  - `publish(event)`: called by a publisher to publish an event
    - Events can be any data type and may include meta-data
  - `subscribe(filter_expr, notify_cb, expiry) → sub_handle`: called by a subscriber to subscribe to events
    - Inputs: filter expression, reference to a notification callback, and subscription expiry time
    - Returns a subscription handle
  - `notify_cb(sub_handle, event)`: called by pub/sub system to deliver to a matching event to subscribers
  - `unsubscribe(sub_handle)`: called by the subscriber to remove the subscription

MOM functionalities

- MOM simplifies distributed communication by handling:
  - Addressing: locating the recipient(s) for messages
  - Routing: delivering messages along appropriate paths
  - Availability of application components (or applications)
  - Message format transformations gestendo l'eterogeneità



https://www.cloudcomputingpatterns.org/message_oriented_middleware

MOM functionalities

- Let's analyze
  - Delivery semantics
  - Delivery model
  - Message routing
  - Message transformations

An important aspect of message-queuing systems is that a sender is generally given only the guarantees that its message will eventually be inserted in the recipient's queue. No guarantees are given about when, or even if the message will actually be read, which is completely determined by the behavior of the recipient. Dunque si utilizza il concetto di ack, per avvisare della lettura

ripassiamo perché non fa male

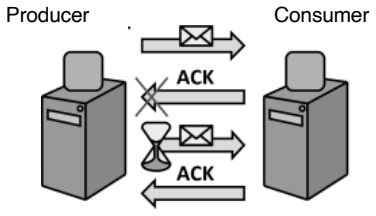
MOM delivery semantics

At-most-once delivery

- Each message is delivered no more than once
- Messages may be lost, but are never redelivered

At-least-once delivery

- Messages are never lost, but they may be delivered multiple times
  - Since messages can be duplicated, the consumer-side application must be able to handle duplicate processing (or be sure that re-processing the same message does not cause errors)
- Mechanism:
  - Consumer sends ack for each message; MOM resends message if ack is missing

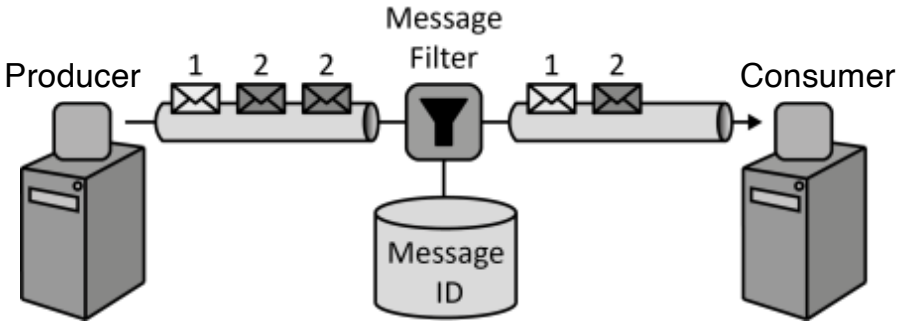


MOM delivery semantics

Exactly-once delivery

- Message is delivered exactly once to consumer
- Mechanisms
  - Deduplication logic: MOM filters message duplicates
    - Each message has a unique ID
  - Idempotent consumers: can safely process the same message multiple times
  - Messages survive MOM failures

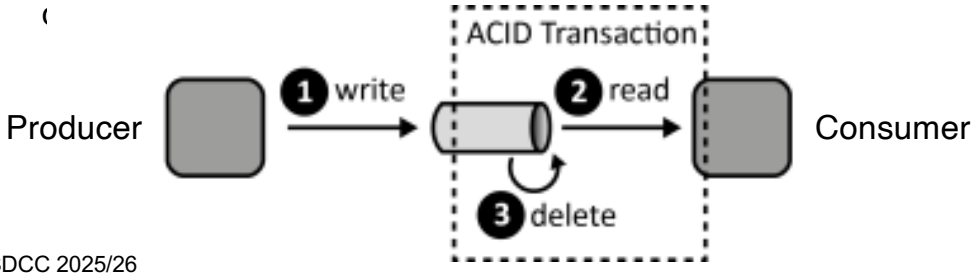
ricorda idempotent è una richiesta che può essere fatta più volte perché senza effetti sullo stato dell'oggetto



MOM delivery semantics

Transaction-based delivery

- Guarantees messages are deleted from the queue only after they have been successfully received and processed by the consumer
- Mechanism
  - MOM and consumer participate in a transaction: read and delete operations are part of the transaction, ensuring ACID properties
    - Commit: if successful, the message is permanently deleted
    - Rollback: if the transaction fails, the message remains in the (



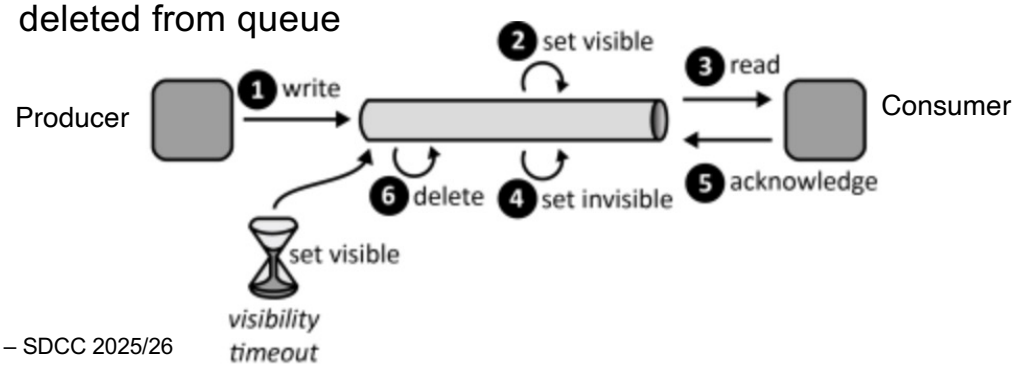
MOM delivery semantics

Timeout-based delivery

- Ensures messages are delivered at least once while preventing duplicate consumption
- Mechanism

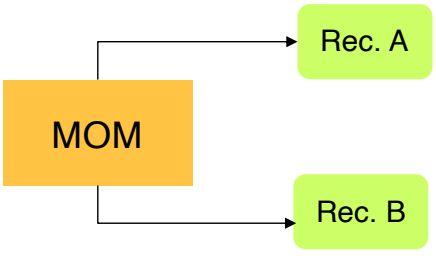
un messaggio appena letto diventa invisibile e se non cancellato torna visibile dopo che finisce il visibility timeout

  - Message is not deleted immediately from the queue, but marked as invisible until *visibility timeout* expires
  - Invisible message cannot be read by another consumer
  - After consumer's ack of message receipt, message is deleted from queue

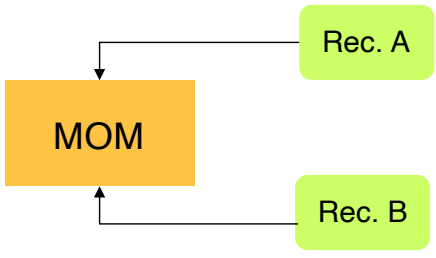


MOM delivery model

- How messages are retrieved by subscribers or consumers
- Options: push vs. pull delivery
- Push: MOM actively notifies the receiver(s) when a message is available
- Pull: receiver(s) periodically polls MOM for new messages



Push delivery

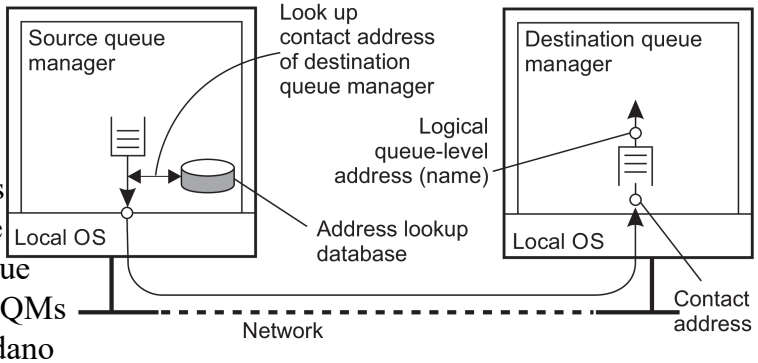


Pull delivery

Message routing in MOM

- Queues are managed by queue managers (QMs)
  - Applications can only put messages into a local queue
  - Applications can only retrieve a message from a local queue
- QMs are responsible for routing messages between local queues and remote queues
  - Act as message-queuing relays, communicating with distributed applications and other QMs
  - Form an overlay network
  - Some QMs may operate only as routers, without storing messages

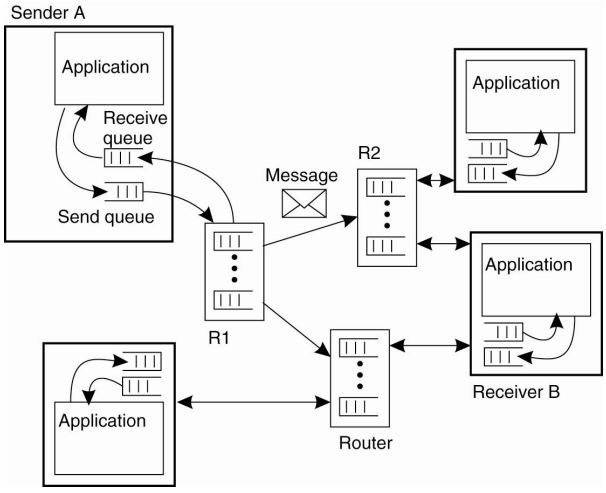
un po' ricorda RPC con gli stub



Un QM si deve occupare di instradare il messaggio, quindi possedere un contact address che indica il protocollo hostname e porta. Poi siccome due QMs possono collaborare un QM deve sapere i contact address gestiti dagli altri, dunque come soluzione per la scalabilità alcuni QMs possono opearre come router che instradano i messaggi, realizzando così una rete overlay

Message routing: overlay network

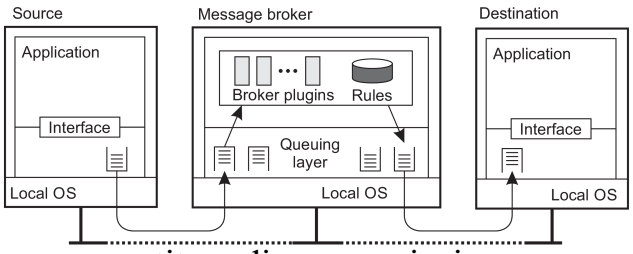
- Overlay network for message routing
  - Each QM maintains a routing table to map queue names to their location
- Overlay network maintenance
  - Static overlays: routing tables are set up and managed manually
    - Easier to configure but less flexible
  - Dynamic overlays: routing tables are updated at runtime
    - More complex but supports scalable and adaptive routing



ovviamente la soluzione migliore è quella dinamica

Application heterogeneity: message broker

- Problem: new and existing apps rarely share a common data format, how to integrate them into a single, coherent system?
- Solution: ***message broker***
  - Handles application heterogeneity in MOM
- Responsibilities
  - Message transformation: uses a repository of conversion rules to transform message types
  - Content-based routing: routes messages based on content or type
- Scalability and reliability: usually implemented in a distributed manner



18

Un message broker può comportarsi sia come un convertitore di messaggi, sia come un application-level gateway in a message-queuing system. Il suo scopo principale è convertire i messaggi in arrivo così che la destinazione li capisca.Come mostra la figura un message broker è software in più

MOM frameworks

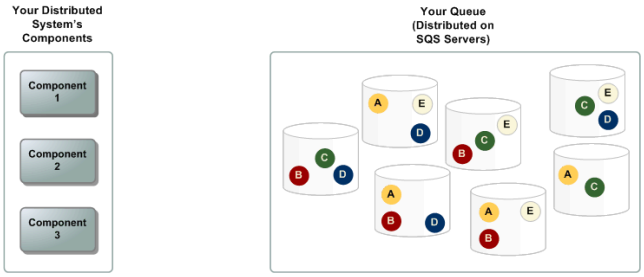
- Main MOM systems and libraries
  - Apache ActiveMQ https://activemq.apache.org
  - **Apache Kafka**
  - Apache Pulsar https://pulsar.apache.org
  - IBM MQ https://www.ibm.com/products/mq
  - NATS https://nats.io
  - **RabbitMQ**
  - Redis https://redis.io/
  - ZeroMQ https://zeromq.org
- Note: distinction between queue-based and pub/sub patterns is sometimes blurred
  - Some frameworks support both patterns, e.g., NATS, Pulsar and Kafka (with specific configurations)
  - Others only one, e.g., Redis is pub/sub only, ZeroMQ is queue only

MOM frameworks

- Cloud-based MOM services
  - **Amazon Simple Queue Service (SQS)**
  - Amazon Simple Notification Service (SNS)
  - CloudAMQP: RabbitMQ as a Service
  - Google Cloud Pub/Sub
  - Microsoft Azure Service Bus
- Note
  - Cloud MOMs abstract infrastructure management, allowing developers to focus on messaging patterns
  - Many support both queue and pub/sub patterns

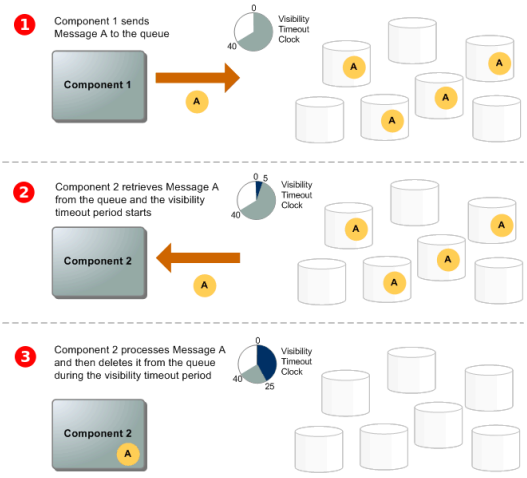
Amazon Simple Queue Service (SQS)

- Reliable, scalable cloud-based message queue service
  - Designed to decouple application components, allowing them to run independently and asynchronously
  - Supports integration across different languages and technologies
- Architecture
  - Message queues fully managed by AWS
  - Queues are distributed across multiple SQS servers
  - Replication within a region: messages stored on multiple servers for high availability
  - Pull delivery model: consumers poll the queue to retrieve messages



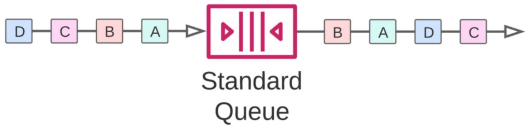
Amazon SQS: Message handling

- Consumer must delete message after processing
  - Queue = temporary holding location, not permanent storage
  - Message retention period: configurable (max 14 days)
- Implements timeout-based delivery
  - When a consumer retrieves a message, it remains in the queue but becomes invisible to others (visibility timeout)
  - If processing succeeds, the consumer deletes the message
  - If processing fails, the timeout expires and the message becomes visible again



Amazon SQS: Polling and queue types

- Consumers poll the queue to receive messages
  - *Short* polling: queries only a subset of SQS servers (may return empty)
  - *Long* polling: queries all servers, wait for messages (reduces empty responses)
- Queue types: standard or FIFO
- Standard queue (default)
  - Best-effort ordering: messages may arrive out-of-order
  - Duplicates possible



- FIFO queue
  - Guarantees in-order delivery (sent=processed order)
  - No duplicates
  - ✗ Lower throughput

Amazon SQS: API

- `CreateQueue`, `ListQueues`, `DeleteQueue`
  - Create a new queue, list all queues, delete an existing queue
- `SendMessage`
  - Add a message to queue (message size up to 256 KB)
  - For larger payloads: store on S3 and send an S3 reference instead
- `ReceiveMessage`
  - Retrieve one or more messages
  - Retrieve up to 10 messages: can specify the maximum number of messages to fetch (from 1 to 10)
  - No filtering before delivery
  - Post-delivery filtering
    - If a consumer wants to filter messages, it must inspect message body or attributes after receiving the messages and decide whether to process or discard them

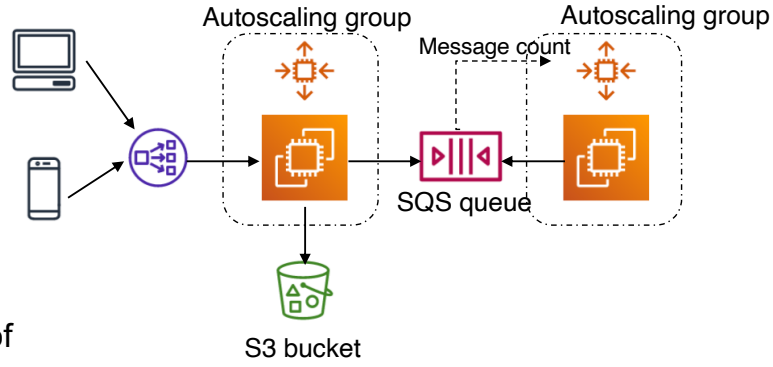
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/Welcome.html

Amazon SQS: API

- `DeleteMessage`
  - Remove a specified message from the queue after processing
- `ChangeMessageVisibility`
  - Change the visibility timeout of a specific message in the queue
  - A message remains in the queue until explicitly deleted by the consumer
  - Default visibility timeout: 30 sec.
- `SetQueueAttributes`, `GetQueueAttributes`
  - Modify queue attributes (e.g., message retention), retrieve current queue settings

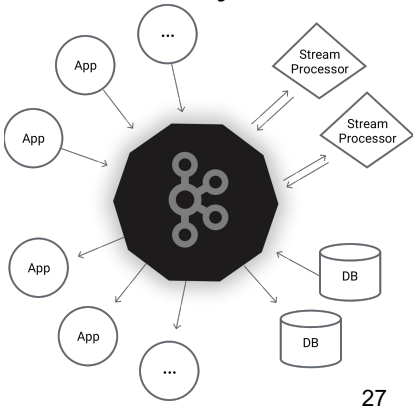
Amazon SQS: example

- Photo processing service using SQS
  - Goal: use SQS to decouple the front-end and back-end components, ensuring load balancing and fault tolerance
  - Front-end uploads a photo to S3 and sends to back-end an SQS message containing the S3 link
  - A pool of EC2 instances (back-end) retrieves messages from the queue
    - Each instances gets a photo, processes it (e.g., resize) and delete the message after processing
    - If processing fails, the message becomes visible again after the visibility timeout, allowing another instance to retry
    - The back-end pool can be scaled horizontally based on the number of messages in the queue

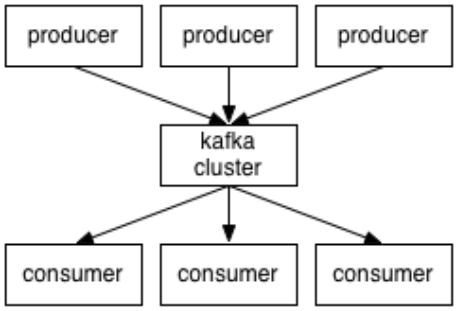


Apache Kafka

- Open-source, distributed event streaming platform https://kafka.apache.org/
- Designed to
  - Publish and subscribe to streams of events
  - Store event streams durably and reliably
  - Process event streams
- Started by LinkedIn in 2010, now Apache project
- Used at scale by LinkedIn, Netflix, Uber, and many others
- Written in Java and Scala
- Horizontally scalable and fault-tolerant
- High-throughput ingestion
  - Billions of events



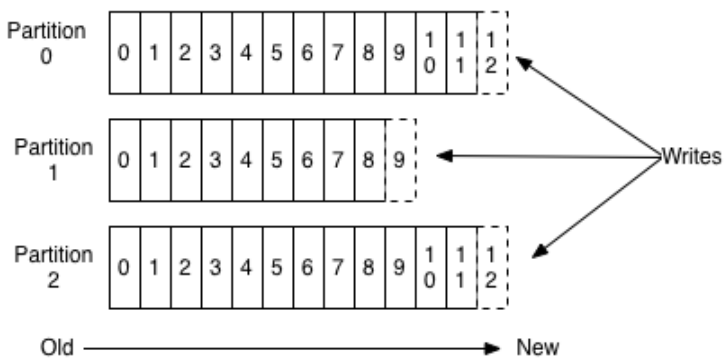
Kafka at a glance



- **Kafka topics**: Kafka stores streams of events (aka *messages* or *records*) in categories known as topics
- **Kafka cluster**: composed of servers called brokers, which can span multiple data centers or cloud regions
  - Brokers receive and store events
- **Producers**: publish (write) events to a Kafka topic
- **Consumers**: subscribe to Kafka topics, read published events and process them
  - A topic can have 0, 1, or many subscribing consumers

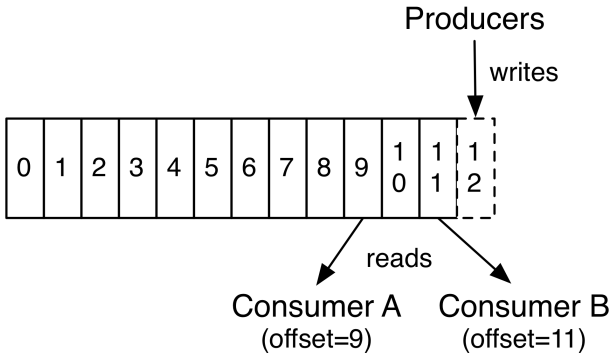
Kafka: topics and partitions

- Topic: category to which events are published
- For each topic, Kafka cluster maintains a **partitioned log**
- Log (data structure): *append-only*, *totally ordered* sequence of events
- Partitioned log: each topic is split into a pre-defined number of partitions
  - Partition: the unit of parallelism for a topic



Kafka: partitions and parallelism

- Parallelism and scalability come from having multiple partitions per topic IMPORTANTE
- Producers publish events to topic partitions
- Consumers read events from the topic
- Each partition is:
  - A numbered, ordered, immutable sequence of records
  - Appended to, never modified (immutable records)
  - Similar to a commit log
- Every record has a monotonically increasing **offset**
  - Uniquely identifies each record within a partition

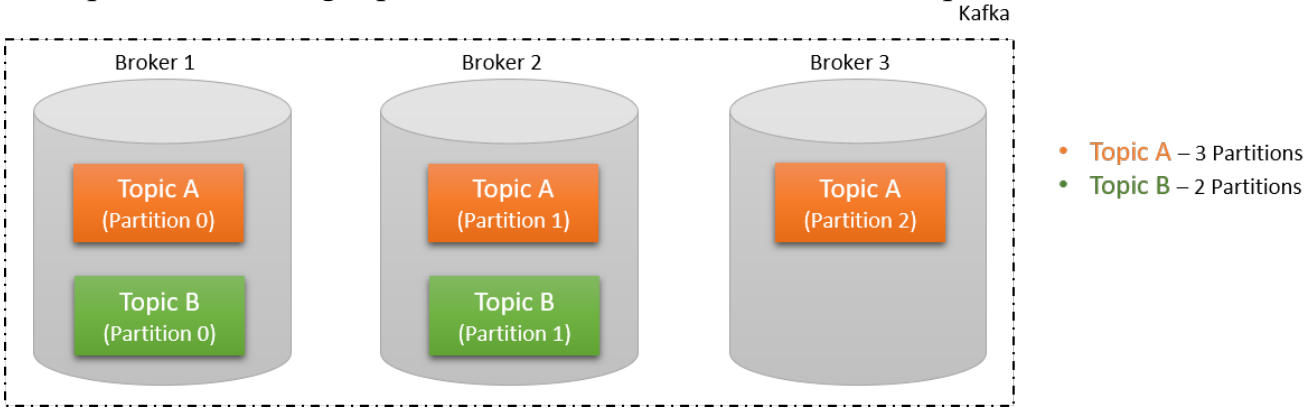


DA NOTARE COME UN CONSUMER LEGGE UN OFFSET DIVERSO

Kafka: partitions and design choices

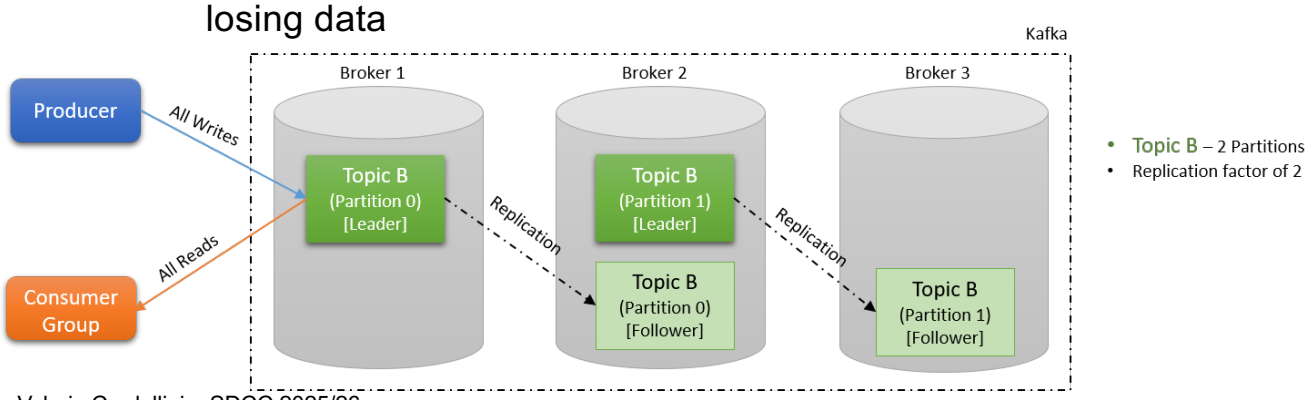
- Improve scalability: topic partitions are distributed across multiple brokers
  - ✓ I/O throughput increases through parallel reads and writes
    - Multiple producers can write concurrently to different partitions
    - Multiple consumers can read concurrently from different partitions

un topic viene partizionato e ogni partizione finisce sotto broker diversi per incrementare il throughput



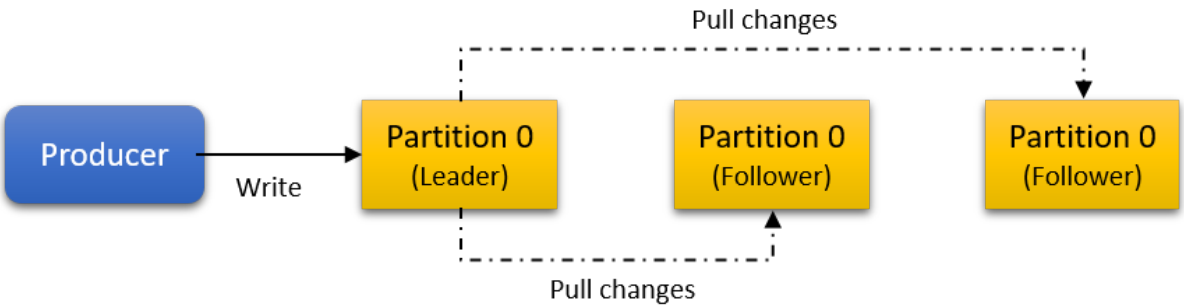
Kafka: partitions and design choices

- Improve fault tolerance: partitions can be *replicated* across brokers
  - Each partition has:
    - 1 leader: handles all writes and reads
    - 0 or more followers: replicate the leader data i follower fanno pull al leader
- *Replication-factor* = total number of replicas (including leader)
  - Replication-factor = *N* → up to *N*-1 brokers can fail without losing data



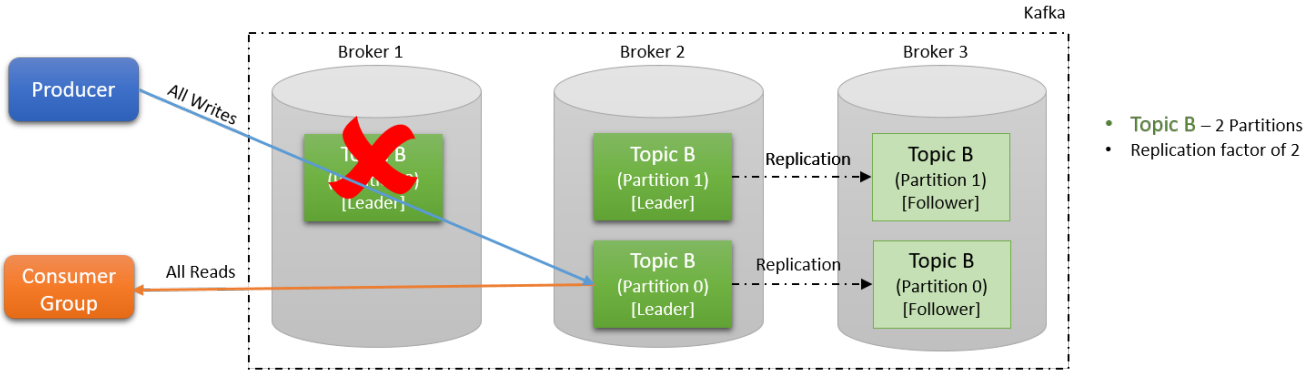
Kafka: partitions and design choices

- Simplify data consistency management: only leader handles all reads and writes
  - Producers write to the leader, consumers read from the leader
  - Followers replicate the leader and serve as backups
  - Followers can be:
    - in-sync: fully updated with the leader
    - out-of-sync: lagging behind the leader



Kafka: partitions and design choices

- Share responsibility and balance load: each broker acts as leader for some partitions and follower for others
- Coordination across brokers is managed by Apache Zookeeper or KRaft
  - Handles leader election
  - Maintains cluster metadata

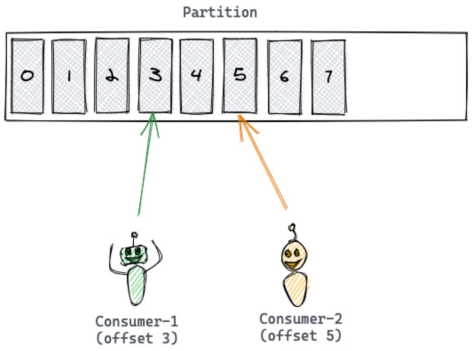


Kafka: producers

- Producers = data sources
- Publish events to topics of their choice
  - Send events directly to the broker that is leader for the target partition (no routing tier)
- Responsible for assigning events to partitions using:
  - Key-based partitioning
    - Event contains a partition key (e.g., order ID, user ID)
    - Partition is chosen by hashing the key
    - Example: if `key=user_id`, all events for the same user go to the same partition
  - Round-robin partitioning
    - Default if no key is specified
  - Custom partitioning
    - A partitioner can be provided to implement a custom partitioning logic
- Multiple producers can write concurrently to the same partition

Kafka: consumers

- Kafka uses a pull-based delivery model for consumers
  https://kafka.apache.org/documentation.html#design_pull
- A consumer track its progress using offset
  - Offset identifies which events have been consumed
- Multiple consumers can be read the same partition, each reading at different offsets



Kafka: consumers

- Why pull?
- Push model
  - Broker actively pushes events to consumers
  - ✗ Broker must manage
    - Different consumers with their needs and capabilities
    - Transmission rate control
    - Timing decisions: whether to send a message immediately or accumulate more data to send later
- Pull model
  - Consumers control event retrieval from broker
  - Consumers maintain an offset to identify the next event to read
  - ✓ More scalable and flexible
    - Less burden on brokers
    - Consumers can pull events when ready, adjusting the rate
  - ✗ If no events are available, consumers may end up busy waiting for events to arrive

Kafka: consumers

- How can consumers read in a fault-tolerant way?
  - After a consumer reads events, it stores its committed offset in a special Kafka topic called `__consumer_offsets`
  - In case of crash, the consumer can recover by reading events from the committed offset
  - By default, `auto-commit` is enabled

gli offset sono mantenuti in un topic speciale
e ciascun consumer accede al topic tramite
un file chiamato: __consumer_offset-xx
dove xx è il numero del offset

Kafka: message retention and storage

- Kafka brokers store messages reliably on disk
- Messages are stored in log segments (files on disk)
- Unlike other message queue and pub/sub systems, Kafka does not delete messages after delivery, but retains messages based on size or time

A DIFFERENZA
DI RABBITMQ

- Issue: how to free up disk space?
  - Topics are configured with retention time (how long events should be stored)
  - Upon expiry, events are marked for deletion importante
  - Alternatively, retention can be specified in bytes
- On each broker, messages are flushed from the broker's in-memory buffer to the disk
  - A log flush policy controls when messages are written to disk (time interval and number of messages)

Hands-on Kafka

- Preliminary steps
  - Download and install Kafka
    https://kafka.apache.org/downloads
    - Configure Kafka properties in `server.properties`
      - E.g., `listeners` which are network interfaces on Kafka broker: `listeners=PLAINTEXT://localhost:9092`
      - E.g., log flush policy
  - Start Kafka environment
    - Kafka can use KRaft or ZooKeeper (we will use KRaft)
      - Alternatively, start ZooKeeper first
    - Start Kafka broker (default port: 9092)

```
$ bin/kafka-server-start config/kafka.properties
```


Hands-on Kafka

- Let's use Kafka CLI to create a topic, publish and consume events to/from topic, and delete it
- Create a topic named test with 1 partition and no replication (`replication-factor=1`)

```
$ kafka-topics --create --bootstrap-server localhost:9092
  --replication-factor 1 --partitions 1 --topic test
```

  - Bootstrap server: an initial broker used by the producer to connect to the Kafka cluster
  - It provides the producer with metadata about the cluster, including a list of all the brokers
  - Multiple bootstrap servers for fault tolerance and load balancing
  - Once the producer discovers the partition leader, it can start sending messages

Hands-on Kafka

- Produce messages into the topic

```
$ kafka-console-producer --bootstrap-server localhost:9092 \
  --topic test
>msg 1
>msg 2
```

- Consume messages from the beginning of the topic

```
$ kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic test --from-beginning
```

- Consume messages from a given offset (e.g., 2) of a specific partition (e.g., partition 0) of the topic

```
$ kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic test --offset 2 --partition 0
```

- Produce messages with key

```
$ kafka-console-producer --bootstrap-server localhost:9092 \
  --topic test --property parse.key=true --property key.separator=:
>course:sdcc
>university:Tor Vergata
```

in /tmp trovo una cartella contenente i file legati a kafka
tra cui test-0, test-1, test-2 che sono le partizioni del topic test

Hands-on Kafka

- List available topics

```
$ kafka-topics --list --bootstrap-server localhost:9092
```

- Delete a topic

```
$ kafka-topics --delete --bootstrap-server localhost:9092 \
  --topic test
```

- Stop Kafka

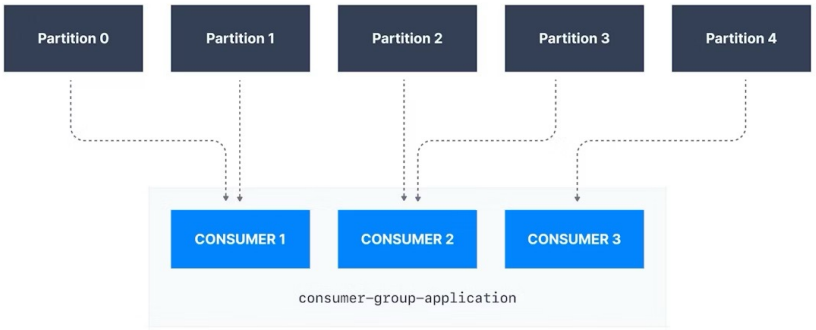
```
$ kafka-server-stop
```

  - If Zookeper: `$ zookeeper-server-stop`

kafka-topics --bootstrap-server localhost:porta --describe da più informazioni su i topic in uso come ad esempio Replicas, Elr che indica il numero di leader elegibili (vedi file)

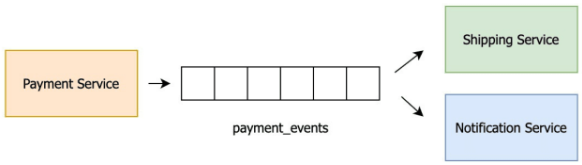
Kafka: consumer group

- Consumer Group: set of consumers that cooperate to consume data from a topic and share a group ID
  - A Consumer Group represents a *logical* subscriber
  - Topic partitions are divided among consumers in the group
    - Reassigned when consumers join/leave the group
  - Each event is delivered to *only one* consumer within the group
  - Each group maintains its own offset per topic partition

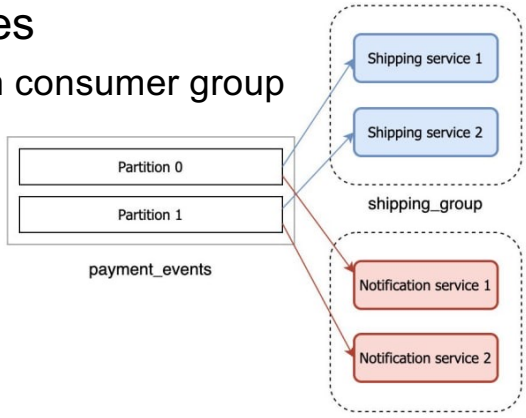


Kafka: consumer group

- How can many consumers read the same events?
  - By using different group IDs: consumers in different groups can independently read the same events from the same topic
- Example: microservices communicating through Kafka



- When we scale microservices
  - Each microservice has its own consumer group



Hands-on Kafka: consumer group

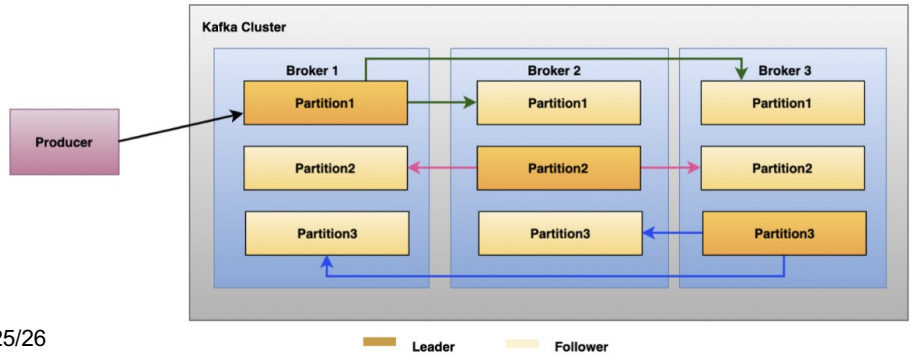
- Launch a consumer in a consumer group named `my-group`

```
$ kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic test \
  --group my-group
```

- Open two more terminal windows and run two additional consumers in `my-group`
  - Each consumer will be assigned to a different topic partition
- Produce messages in the topic and see how they are distributed among the consumers
  - Each consumer will read the messages from its assigned partition

Kafka: replication

- Topic partitions are replicates across brokers for fault tolerance
  - Leader-follower model: followers replicate the leader log
  - Replication is asynchronous: periodically, followers pull messages from the leader in order, using offsets
  - In-sync replicas (ISR): the leader and any followers that are updated with the leader's log
  - Durability (no message loss) is guaranteed as long as at least one in-sync replica is available
  - If the leader crashes, one in-sync replica is elected as new leader



Kafka: event ordering guarantees

- Events written by a producer to a topic partition are appended in the order they are sent

  message: m1 m2 m3 m4 m5
  offset: 10 11 12 13 15

- Consumer reads events in the same order they are stored in the partition

  m1 → m2 → m3 → m4 → m5

- Kafka provides ordering guarantees, but *only within a partition*
  - Total order of events within each partition (*per-partition ordering*)
- Kafka does not preserve order across partitions of the same topic
- In practice, per-partition ordering combined with key-based partitioning is sufficient for most applications

Kafka: write guarantees

- Controlled by `acks` and `min.insync.replicas`
- `acks` (producer setting): how many in-sync replicas must acknowledge a write before the producer considers it successful
  - `acks=0`: no ack (fire and forget)
  - `acks=1`: wait for leader only
  - `acks=all`: wait for all ISR replicas
- `min.insync.replicas` (topic/cluster setting): minimum number of ISR replicas that must acknowledge a write for it to succeed
- Strong durability: `acks=all + min.insync.replicas>=2`
  - Example: `replication-factor=3`
    - `min.insync.replicas=1`: only leader
    - `min.insync.replicas=2`: leader + 1 follower (recommended)
    - `min.insync.replicas=3`: leader + 2 followers (highest durability)

Kafka: read and visibility guarantees

- Message durability: depends on `acks` + ISR size/status
  - A write is kept only if enough in-sync replicas acknowledge it (as required by `acks` and `min.insync.replicas`)
- Message availability: consumers can read messages as soon as they are appended to the leader's log
  - Replication to followers is asynchronous
  - Exception: transactional consumers (`read_committed`)
- A write is successful only if:
  1. The leader appends the message, and
  2. The acknowledgment requirements (`acks` and `min.insync.replicas`) are satisfied
  - If reqs cannot be met, write is rejected (not visible to consumers)

è possibile che un consumer legga un dato che non ha raggiunto un numero di ack sufficiente per essere stabile. read_committed evita ciò

dunque se una delle repliche non è disponibile la write fallisce

Kafka: delivery semantics

- **At-most-once**: guarantees that events are never re-delivered but may be lost
  - Producer disables retries (i.e., `acks=0` on producer)

  - Consumer commits its offset before processing event
    - What happens if the consumer crashes after committing but before processing?

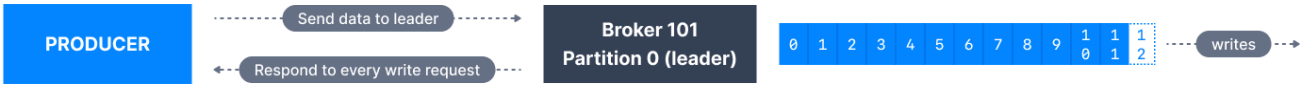
If the consumer process crashes after saving its position, but before saving the output of its message processing, the consumer that takes over processing would start at the saved position and the messages prior to that position would not be processed. Quindi si perde il messaggio

https://kafka.apache.org/documentation/#semantics

https://docs.confluent.io/kafka/design/delivery-semantics.html

Kafka: delivery semantics

- **At-least-once** (default): guarantees no event loss, but events may be duplicated
  - Producer waits for ack only from the partition leader; if no ack, it retries
  - How? Set `acks=1` on producer
  - Consumer commits its offset after processing an event
    - What happens if the consumer crashes before committing?

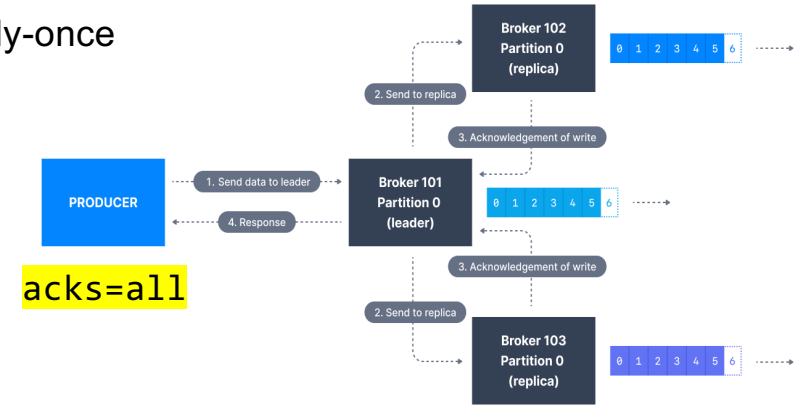


`acks=1`

In this case, if the consumer process crashes after processing messages, but before saving its position, the new process that takes over may process some messages a second time. Va quindi implementato lato consumer un meccanismo di duplication filtering

Kafka: delivery semantics

- **Exactly-once**: guarantees no event loss, no duplicates, but at the cost of higher latency and lower throughput
  - Producer: `acks=all`
    - Waits for ack from all in-sync partition replicas
  - Idempotent producer: detect duplicates caused by retries
    - Producer ID and sequence number are added to each event
  - Durability: `min.insync.replicas >= 2`
  - Transactions enabled: atomic writes across partitions + transactional offset commits + `read_committed` consumers
  - Not fully exactly-once



`acks=all`

Kafka: delivery semantics

- Take-away message: choose delivery semantics that fit your application
  - At-most-once: prioritize speed (e.g., telemetry, if some data loss is acceptable)
  - At-least-once: prioritize reliability (e.g., email notifications)
  - Exactly-once: prioritize correctness (e.g., financial transactions)

In informatica teorica, il **teorema CAP**, noto anche come **teorema di Brewer**, afferma che è impossibile per un sistema informatico distribuito fornire simultaneamente tutte e tre le seguenti garanzie:[1][2]

- Coerenza (*copy-consistency*): tutti i nodi vedano gli stessi dati nello stesso momento
- Disponibilità (*availability*): la garanzia che ogni richiesta riceva una risposta su ciò che è riuscito o fallito
- Tolleranza di partizione (*partitioning*): il sistema continua a funzionare nonostante arbitrarie perdite di messaggi o malfunzionamenti

Secondo il teorema, un sistema distribuito è in grado di soddisfare al massimo due di queste garanzie allo stesso tempo, ma non tutte e tre.[3]

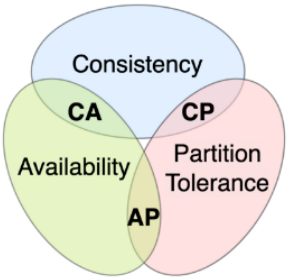


Diagramma di Venn del teorema CAP

Kafka è CA

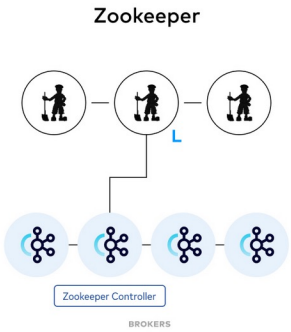
Kafka: trade-offs

- Consistency vs. availability trade-off
  - Kafka ensures strong consistency per partition: all in-sync replicas are up-to-date with the leader
  - If the leader fails and no ISR follower is available, the partition becomes unavailable

- Latency vs. durability trade-off
  - Producers control durability via `acks`
    - `acks=all`: highest durability, higher latency (waits for all ISRs)
    - `acks=1`: lower latency, but messages may be lost if leader crashes
    - `acks=0`: lowest latency, highest risk of loss
  - Lower latency increases the chance of data loss in case of failure

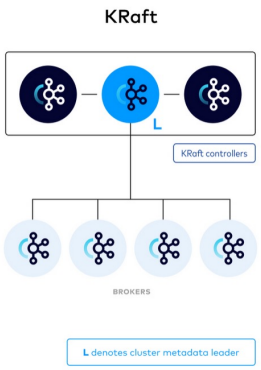
Kafka: from ZooKeeper to KRaft

- Zookeeper: distributed coordination service used for managing distributed systems https://zookeeper.apache.org/
  - Provides locking, leader election, monitoring
- Role of ZooKeeper in Kafka
  - Metadata management: list of brokers, configuration for topics and partitions
  - Leader election: to determine partition leader
  - Cluster health monitoring
- Cons
  - ✗ Operational complexity: separate Zookeeper cluster
  - ✗ Scalability: as Kafka cluster grows, Zookeeper becomes bottleneck
  - ✗ Single point of failure for coordination: if Zookeeper is unavailable, Kafka brokers may be unable to manage metadata, elect new leader, and know broker status



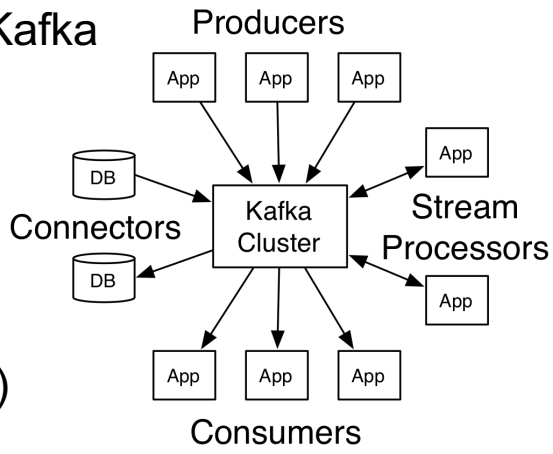
Kafka: from ZooKeeper to KRaft

- Apache Kafka Raft (KRaft)
  - Kafka-native consensus protocol that replaces Zookeeper for metadata management and leader election
  - Kafka itself stores metadata and partition leader information
  - Raft protocol coordinates metadata updates
  - KRaft is used for leader election
- Pros
  - ✓ Simplified architecture
  - ✓ Improved scalability
  - ✓ Better fault tolerance: metadata replicated to all brokers, making failover faster



Kafka: APIs

- A set of APIs for interacting with Kafka
- Producer API: allows clients to publish data to Kafka topics
- Consumer API: allows clients to consume data from Kafka topics
- Connect API: integrates external systems (databases, file systems) as data sources and sinks
  - Pre-built connectors: AWS S3, Lambda, MySQL, Postgres, …
- Admin API: manages clusters and resources (topics, partitions, brokers, etc.)
- Streams API: for real-time processing of streaming data



https://kafka.apache.org/documentation/#api

Kafka: client libraries

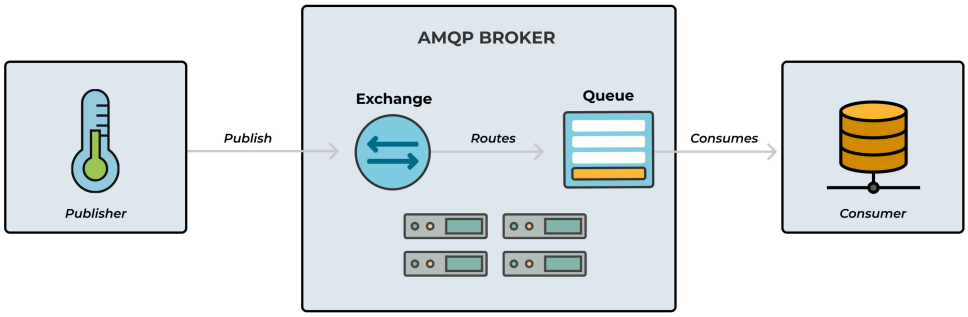
- Kafka officially provides an SDK for Java
  - Fully featured (supports all APIs except Admin), actively maintained
- Community-driven Kafka client libraries for other languages, including
  - Go
    - Confluent https://github.com/confluentinc/confluent-kafka-go
    - Kafka-go https://github.com/segmentio/kafka-go
  - Python
    - Confluent https://github.com/confluentinc/confluent-kafka-python

Interacting with MOM: messaging protocols

- Application-layer open standard protocols for MOM interaction
  - **AMQP** (Advanced Message Queueing Protocol) unico che trattiamo
    - Binary protocol
  - MQTT (Message Queue Telemetry Transport) https://mqtt.org
    - Binary, lightweight protocol, for low-bandwidth and low-power devices, commonly used in IoT applications
  - STOMP (Simple Text Oriented Messaging Protocol) https://stomp.github.io/
    - Simple, text-based protocol, commonly used in web applications and real-time communication
- Goals:
  - Platform- and vendor-agnostic
  - Interoperability: facilitates communication between different MOMs

Messaging protocols and IoT

- Widely used in Internet of Things (IoT)
  - Messaging protocols enable data exchange between sensors and services that process data



- Advantages on MOM in IoT
  - Decoupling: separate data producers from consumers
  - Resiliency: MOM provides temporary message storage
  - Traffic spikes handling: MOM can persist data and process it eventually

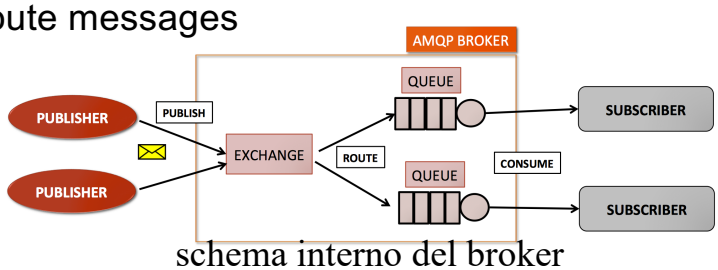
AMQP: features

- Open standard messaging protocol, supported by industry
  - Version 1.0, approved in 2014 by ISO and IEC

  https://docs.oasis-open.org/amqp/core/v1.0/amqp-core-complete-v1.0.pdf
- Application-level, binary protocol molto efficiente essendo binario
  - Based on TCP with additional reliability mechanisms (delivery semantics)
- Programmable protocol
  - Entities and routing schemes are primarily defined by apps
- Designed to provide reliable, scalable and secure messaging
- Supports queue-based and pub/sub patterns
- Implementations noi studieremo RabbitMQ
  - Apache ActiveMQ, RabbitMQ, Azure Event Hubs, Pika (Python client library), …

AMQP: architecture

- 3 key actors: publishers, subscribers, and brokers
  - Brokers manage and route messages
- AMQP entities (within broker): queues, exchanges and bindings



schema interno del broker

  - Queues: storage units that hold messages
  - Exchanges: routing entities that determined how messages are distributed to queues tramite politiche specifiche
    - Like post offices or mailboxes
  - Bindings: rules that define how messages are distributed to queues sarebbero le frecce nell'immagine
  - Message delivery
    - Push delivery: brokers push messages to consumers subscribed to queues
    - Pull delivery: consumers pull messages from queues on demand

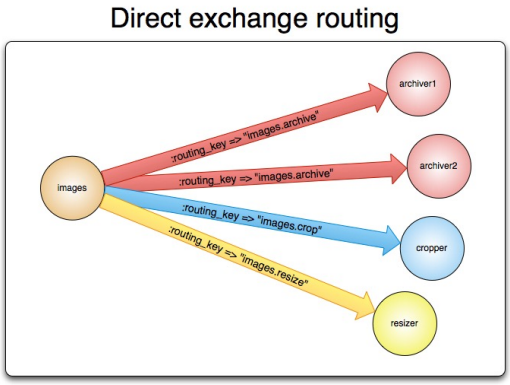
https://www.rabbitmq.com/tutorials/amqp-concepts

AMQP: routing

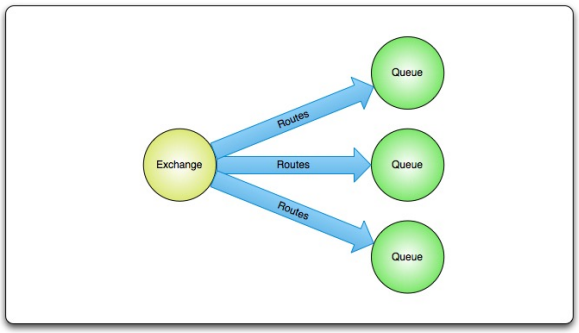
- Types of AMQP exchanges
  - Direct exchange: delivers messages to queues based on message routing key
    - Use case: routing to specific queues based on exact matching

la politica del exchange la sceglie il programmatore

poco flessibile



  - Fanout exchange: delivers messages to all queues that are bound to the exchange
    - Use case: broadcasting to multiple queues



AMQP: routing

- Types of AMQP exchanges
  - Topic exchange: delivers messages to one or more queues based on topic matching più flessibile rispetto al direct exchange
    - Routing key can include wildcards
    - Use case: commonly used to implement pub/sub patterns
    - Enables multicast routing, e.g., targeted notifications to users in different regions
    - Example: a message with routing key `"location.northAmerica.newYork"` will be delivered to queues bound with patterns like `"location.northAmerica.#"`, `"location.#"`, `"*.newYork"`
  - Headers exchange: delivers messages based on message headers rispetto ai primi 3 tipi che si basano sul body del messaggio, questo si basa sul header
    - To route on multiple attributes that are expressed as message headers instead of routing key
    - Use case: complex multi-attribute matching

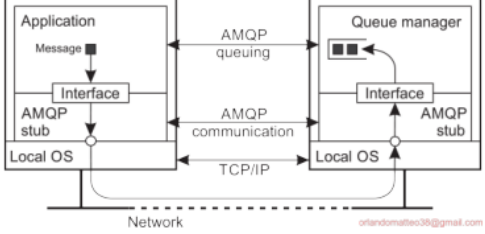
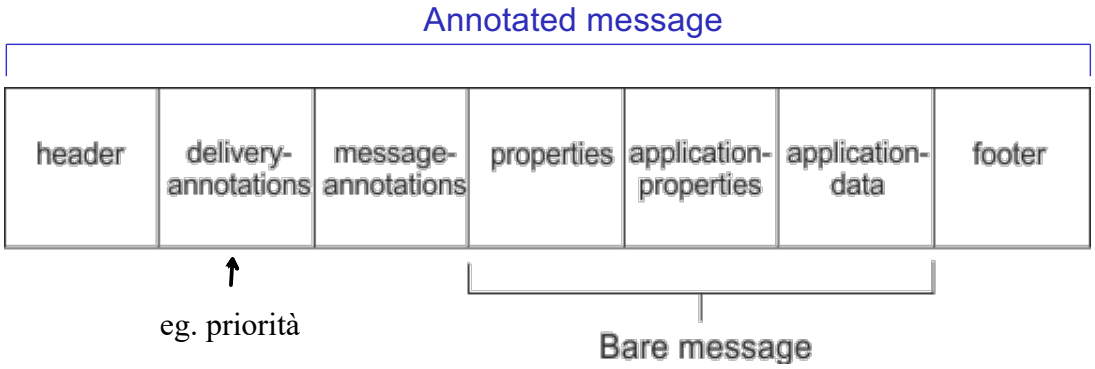


Figure 4.29: An overview of a single-server AMQP instance

figura di approfondimento di AMQP dal libro di Tanenbaum

AMQP: messages

- AMQP defines two message types:
  - Bare messages: raw messages supplied by sender immutabile!
  - Annotated messages: modified by intermediaries (e.g., brokers) during transit by adding headers and annotations
- Message header conveys delivery parameters
  - Including durability requirements, priority, time to live



RabbitMQ

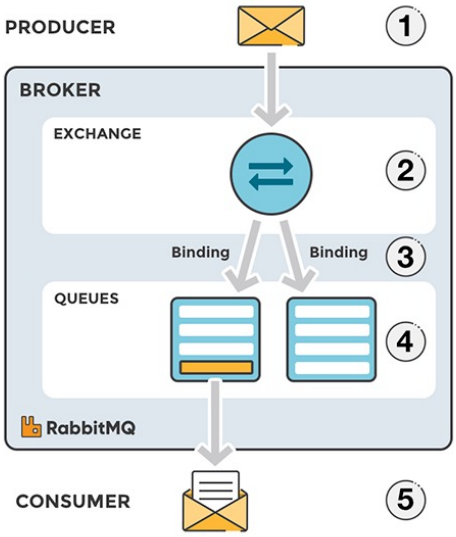
- Open-source message broker https://www.rabbitmq.com/

- Multiple messaging protocols supported
  - AMQP, MQTT, and STOMP
- Push delivery model (can also support pull) pull che non vedremo. ricorda kafka usa pull
- FIFO ordering guarantee at queue level detttaglio importante
- Cross-platform
  - Can run on multiple operating systems and cloud environments
- Wide range of development tools (Java, Go, Python, …)

RabbitMQ: architecture

- Producers do not publish messages directly to queues
- Producer sends messages to an exchange, which routes messages to one or more queues with the help of bindings and routing keys
  - Binding: link between an exchange and a queue

Message flow in RabbitMQ



qui si nota la somiglianza con il protocollo AMQP

- RabbitMQ broker can be distributed, e.g., forming a cluster https://www.rabbitmq.com/distributed.html
  - Supports quorum queues: durable, replicated FIFO queues based on Raft consensus algorithm come per KRaft

RabbitMQ: delivery semantics

- At-most-once delivery by default
  - Consumer does not explicitly send acks (`autoAck = true` in the consumer settings); if the consumer fails before processing the message, the message will be lost
- At-least-once can be configured
  - Explicit message ack must be configured (`autoAck = false` in the consumer settings) and the consumer is required to send an explicit ack after successfully processing the message (esempio di seguito)

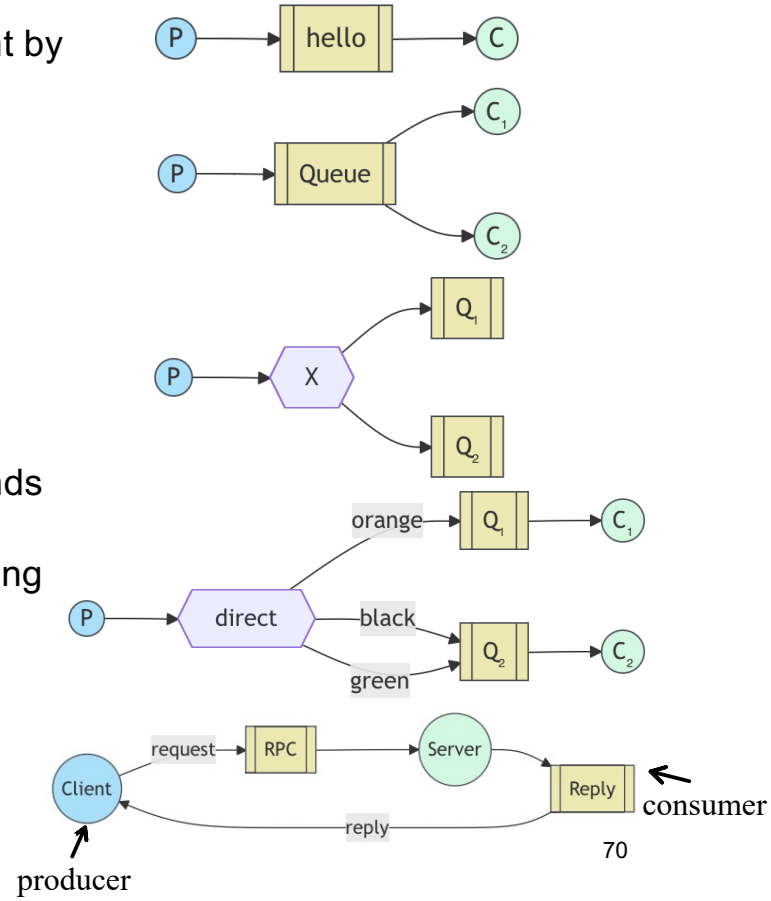
una volta mandato il messaggio, dopo la lettura viene cancellato. a differenza di Kafka che li mantiene persistenti

RabbitMQ: use cases

esempi in Go

1. Store and forward messages sent by a producer and received by a consumer (message queue)
2. Distribute tasks among multiple workers (work queue)
   quindi un load balancer
3. Deliver messages to many consumers (pub/sub) using a *message exchange*
4. Routing messages: producer sends messages to an *exchange,* that selects the queue based on binding rules and routing keys
5. Run a function on consumer and wait for result (RPC)

https://www.rabbitmq.com/tutorials



RabbitMQ and Go

- Let's use RabbitMQ, Go and AMQP (messaging protocol) for:

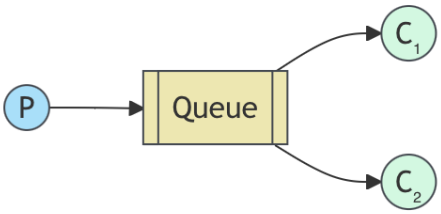
Ex. 1: Message queue

https://www.rabbitmq.com/tutorials/tutorial-one-go

i messaggi saranno distribuiti in maniera Round-robin

Ex. 2: Work queue

https://www.rabbitmq.com/tutorials/tutorial-two-go.html



Code on Teams/course website

RabbitMQ and Go

- Preliminary steps:

1. Install RabbitMQ and start RabbitMQ server on localhost (port 5672) https://www.rabbitmq.com/download.html

```
$ rabbitmq-server
```

   - RabbitMQ CLI tool: `rabbitmqctl`

```
$ rabbitmqctl stop
$ rabbitmqctl status
```

     Some useful commands for `rabbitmqctl`

```
list_channels
list_consumers
list_queues
reset
```

   - Also web UI for management and monitoring (port 15672)

2. Install Go AMQP client library

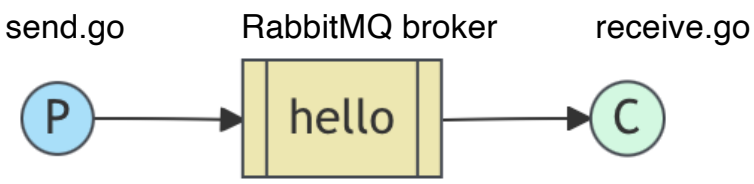
```
$ go get github.com/rabbitmq/amqp091-go
```

   Details on `ampq`: https://pkg.go.dev/github.com/rabbitmq/amqp091-go

RabbitMQ and Go: example 1

1. Message queue pattern

- Support single producer, single consumer or multiple producers, multiple consumers
- Note that:
  - Message is delivered to only one consumer
  - Delivery is push-based



RabbitMQ and Go: example 2

2. Work queue pattern

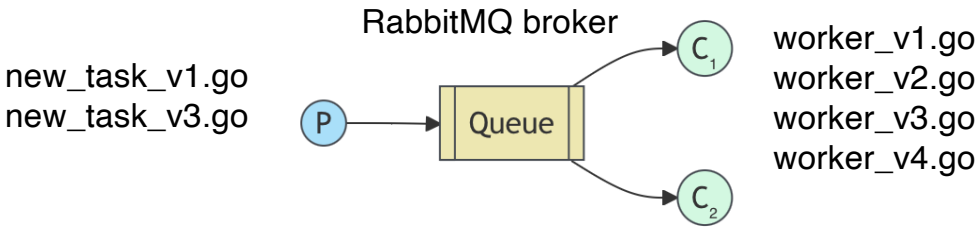
- Version 1 (new_task_v1.go and worker_v1.go):
  - Multiple consumers: tasks are distributed among consumers in a round-robin fashion
  - Message loss scenario: if consumer crashes after the message is delivered by RabbitMQ but before completing the task, the message is lost

    `auto-ack=true`: message is considered to be successfully delivered as soon as it is sent to consumer ("fire-and-forget")
- Version 2 (new_task_v1.go and worker_v2.go):
  - Set `auto-ack=false` in `Consume` and add explicit ack in consumer to tell RabbitMQ that message has been processed and RabbitMQ can safely discard it
  - What happens when RabbitMQ is restarted while there are pending messages?
  - Which delivery semantics with explicit acks? at-least-once

RabbitMQ and Go: example 2

2. Work queue pattern

- Version 3 (new_task_v3.go and worker_v3.go):
  - Use a durable queue so messages are persisted to disk and survives RabbitMQ crash and restart
  - Queue declared with `durable=true` in `QueueDeclare`
- Version 4 (new_task_v3.go and worker_v4.go):
  - Improve task distribution by preventing a worker from receiving a new message while it still has unacknowledged messages un load balancing più soffisticato
  - Achieved using channel prefetch (`Qos`) setting



Multicast communication

- **Multicast communication**: a group communication pattern where data is sent to *multiple* receivers (but not all) at once
  - Can be one-to-many or many-to-many
  - One-to-many multicast apps: video/audio resource distribution, file distribution
  - Many-to-many multicast apps: conferencing tools, multiplayer games, interactive distributed simulations
  - Broadcast: special case of multicast, where data is sent to *all* receivers
- Cannot be implemented via unicast replication (source sends indivudual copies to each receiver): lack of scalability
  - Solution: replicate data only when needed

Types of multicast

- How to realize multicast
  - **Network**-level multicast (IP-level)
    - Packet replication and routing managed by network routers
    - Uses IP Multicast
    - ✗ Limited in usage due to the need for specific router support and network infrastructure
  - **Application**-level multicast
    - Replication and routing managed by the hosts

Application-level multicast

- Basic idea:
  - Organize nodes into an overlay network
  - Use the overlay network to disseminate data
  - Can be structured or unstructured
- Structured application-level multicast
  - Explicit communication paths
  - How to build a structured overlay network?
    - Tree: one path between each pair of nodes (e.g., tree building based on Chord)
    - Mesh: multiple paths between each pair of nodes
- Unstructured application-level multicast che studieremo con il gossiping
  - No predefined communication paths, typically relying on random connections between nodes

Con un albero c'è solo un unico percorso tra ogni coppia di nodi. In una mesh ogni nodo possiede multipli vicini e quindi, multipli percorsi tra ogni coppia di nodi.La differenza principale tra le due reti è che la mesh è generalmente più robusta, in qunato se una connessione salta, c'è sempre un altro percorso per disseminare le informazioni senza dover riorganizzare l'intera rete.

Unstructured application-level multicast

- How to realize unstructured application-level multicast
  - ✓ Flooding
    - Node *P* sends the multicast message *m* to all its neighbors
    - Each neighbor forwards *m* to all its neighbors (except to *P)* if it has not seen *m* before
  - ✓ Random walk
    - Node *P* sends multicast message *m* to a randomly chosen neighbor
    - The chosen neighbor forwards *m* to another randomly chosen neighbor
  - 👉**Gossiping**

Gossip-based protocols

- Gossip-based protocols (or algorithms) are probabilistic (aka *epidemic* algorithms)
  - Gossiping effect: information spreads within a group just as it would be in real-life social interactions
  - Strongly related to epidemics, by which a disease is spread by infecting members of a group, which in turn can infect others
- Allow information dissemination in large-scale networks through random choice of successive receivers among those known to sender
  - Each node sends the message to a randomly chosen subset of nodes in the network
  - Each receiving node will forward the message to another randomly chosen subset, and so on

Origin of gossip-based protocols

- Proposed in 1987 by Demers et al. as a solution for data consistency in replicated databases with hundreds of servers
  - Assumption: no write conflicts (i.e., independent updates)
  - Updates are initially performed at one replica server
  - Each replica shares its updated state with only a few selected neighbors
  - Update propagation is *lazy*, i.e., not immediate
  - Eventually, each update should reach every replica

Demers et al., Epidemic Algorithms for Replicated Database Maintenance, PODC 1987 https://dl.acm.org/doi/pdf/10.1145/41840.41841

Why gossiping in large-scale DSs?

- Several attractive properties of gossip-based information dissemination for large-scale distributed systems
  - Simplicity of gossiping algorithms
  - No centralized control or management (and related bottleneck)
  - Scalability: each node sends only a limited number of messages, independently from system size
  - Reliability and robustness: thanks to message redundancy

Who uses gossiping? Examples

- AWS S3 uses a gossip protocol to rapidly disseminate information throughout its system. This allows S3 to quickly route around failed or unreachable servers, among other things
- Amazon's Dynamo uses gossiping for node failure detection
- BitTorrent uses a gossip-based information exchange
- Cassandra uses gossiping for group membership and node failure detection
- Gossip dissemination pattern
  https://martinfowler.com/articles/patterns-of-distributed-systems/gossip-dissemination.html

Strategies to spread updates

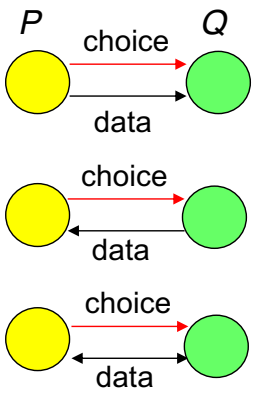
- Let's consider the two principal operations

1. Anti-entropy: each node periodically selects another node randomly and exchanges updates (i.e., state differences), with the goal of having identical states on both node afterwards

2. Rumor spreading: a node that has a new update (i.e., has been contaminated) periodically selects *F* (*F* >= 1) peers and sends them the update (contaminating them); a node that has received an update can stop further distributing it

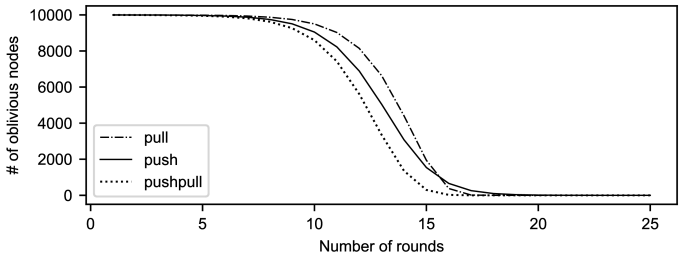
Anti-entropy

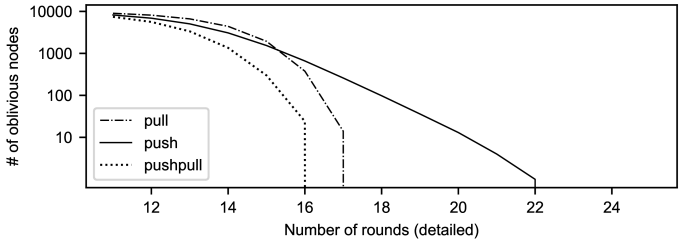
- Goal: increase node state similarity, thus decreasing "disorder" (the reason for the name!)
- Node *P* selects node *Q* randomly: how does P update *Q*?
- Different update strategies:
  1. Push: *P* only pushes its own updates to *Q*
  2. Pull: *P* only pulls new updates from *Q*
  3. Push-pull: *P* and *Q* send updates to each other, i.e., *P* and *Q* exchange updates



Anti-entropy: performance

- Push-pull
  - Fast and message-saving strategy: takes O(ln *N*) rounds to disseminate updates to *N* nodes, using O(*N* ln ln *N*) messages
  - *Round* (or *gossip cycle*): time interval in which every node initiates an exchange



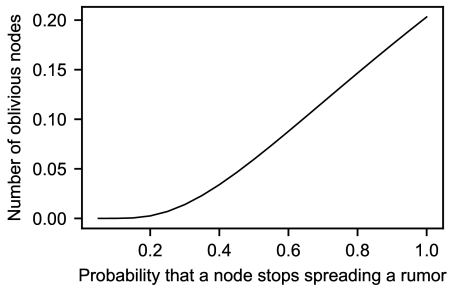


Rumor spreading

- Node *P*, having an update to report, contacts a randomly chosen node *Q* and forwards the update
- If *Q* has already been updated, *P* may lose interest in spreading the update further; with probability p_{stop} *P* stops contacting other nodes
- Fraction *s* of oblivious nodes (nodes that have not yet been updated) is equal to

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

p_{stop} probabilità che il nodo non è stato raggiunto dall'informazione



Consider 10,000 nodes		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

- To improve information dissemination (especially when p_{stop} is high), combine rumor spreading with anti-entropy

Sia anti-entropy che rumor spreading sono strategie periodiche ma il periodo non è lo stesso.

Framework for gossiping protocols

- Two nodes *P* and *Q*, where *P* selects *Q* to exchange information with
  - *P* runs at each round (Δ time units)

Active thread (node P):		Passive thread (node Q):
selectPeer(&Q)		
selectToSend(&buf)		
sendTo(Q, buf)	----->	receiveFromAny(&P, &req)
		selectToSend(&buf)
receiveFrom(Q, &resp)	<-----	sendTo(P, buf)
selectToKeep(view, resp)		selectToKeep(view, req)
processData(view)		processData(view)

selectPeer: randomly select a node to send the gossip message to
selectToSend: select some entries from node's local view to send
selectToKeep: select which received entries to store in node's local view; remove duplicate entries

Kermarrec and van Steen, Gossiping in distributed systems, *SIGOPS Oper. Syst. Rev.*, 2007 https://www.distributed-systems.net/my-data/papers/2007.osr.pdf

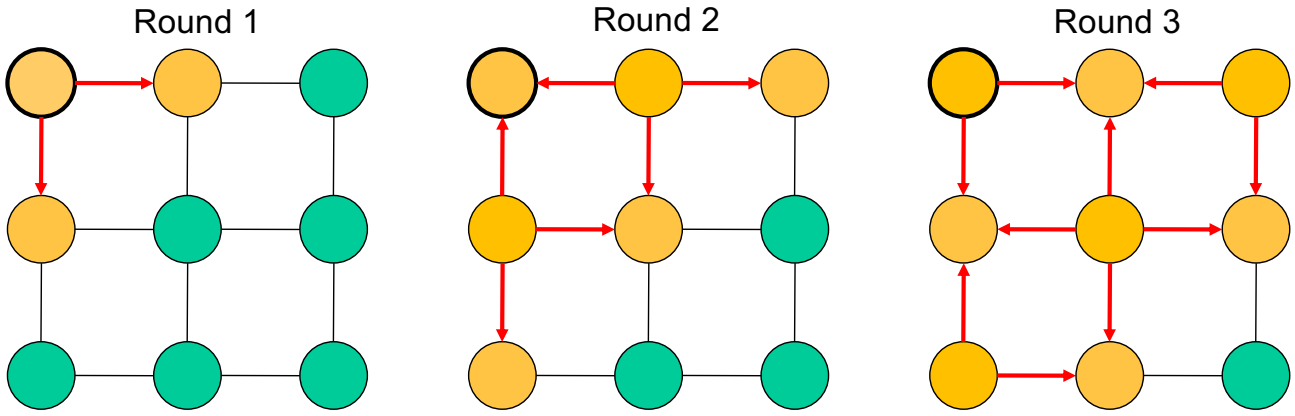
Framework for gossiping protocols

- Simple? Not quite
- Several crucial aspects
  - Node selection to gossip: many alternatives including
    - Uniformly selecting from the set of currently available (alive) nodes
    - Selecting the peer that has been least contacted (e.g., used by CoachroachDB)
  - Data exchanged
    - What is exchanged is highly application-dependent
    - Choice of update strategy
  - Data processing
    - Again, highly application-dependent

Insomma i dettagli implementativi, non saranno dimostrati nel corso ma diamo almeno un ragionamento superficiale

Gossiping vs flooding: example

- Information dissemination is the classic and most popular application of gossiping protocols in DSs
  - Gossiping is generally more efficient than flooding
- Flooding-based information dissemination
  - Each node that receives a message forwards it to all its neighbors (including the sender)
  - Message is eventually discarded when TTL reaches 0

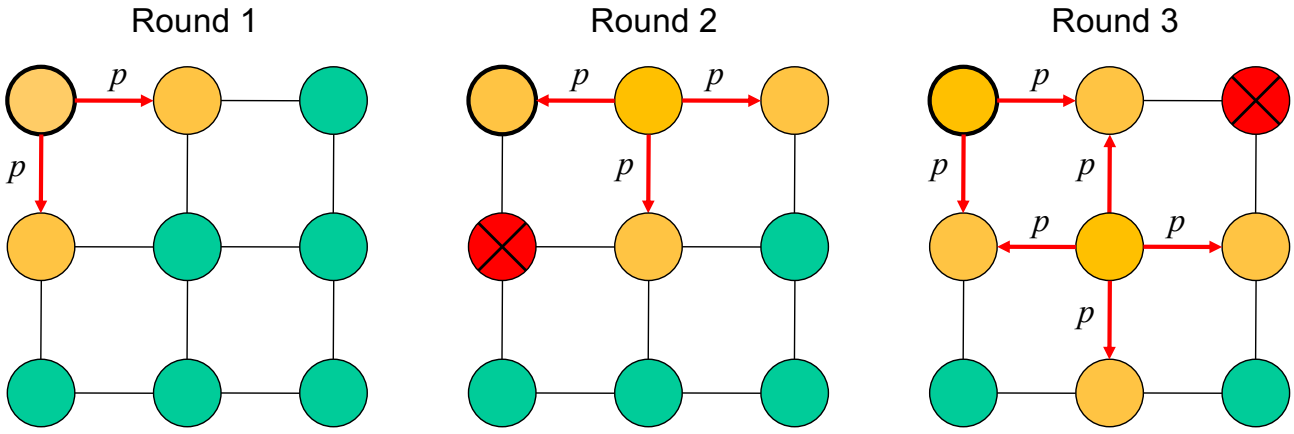


Sent messages: 18
Reached nodes: 8 out of 9_{90}

Gossiping vs flooding: example

- Use an example of rumor spreading algorithm
  - The node sends the message to each of its neighbors with probability *p*

```
for each msg m
    if random(0,1) < p then send m
```



Sent messages: 11
Reached nodes: 7 out 9

Dunque a parità di round il gossiping occupa meno la rete ma raggiunge meno nodi, mentre il carico con flooding è maggiore e si coprono più nodi.

Gossiping vs flooding

- Gossiping key features
  - Probabilistic
  - Localized decision, yet leads to a global state
  - Lightweight
  - Fault-tolerant
- Flooding pros and cons
  - ✓ Universal coverage and minimal state information needed
  - ✗ Can flood the network with redundant messages
- Gossiping goals
  - Reduce redundant transmissions compared to flooding while attempting to retain its advantages
  - However, due to its probabilistic nature, it cannot guarantee that all the peers are reached and it generally takes longer to complete than flooding

Other application domains of gossiping

- Besides information dissemination…
- Group membership
  - To know the list of nodes in DS (who is part of the system)
- Peer sampling
  - To select nodes from a larger set of available nodes to interact with
- Resource management in large-scale DS
  - Including monitoring and failure detection
- Distributed computations for data aggregation in large-scale DS (e.g., sensor network)
  - Computation of aggregates, e.g., sum, average, maximum, minimum
  - Example: computing average
    - Let $v_{t,i}$ and $v_{t,j}$ be the values at time t stored at nodes i and j
    - During a gossip exchange, i and j exchange their current local value v_i and v_j and adjust it to

$$v_{t+1,i},\ v_{t+1,j} \leftarrow (v_{t,i} + v_{t,j})/2$$

Gossiping case studies

1. Blind counter rumor mongering: example of gossip-based disseminatio protocol

2. Bimodal multicast: builds on gossiping to provide reliable multicast

Blind counter rumor mongering

- Why this name?
  - *Rumor mongering* (def.: "the act of spreading rumors", also known as gossip): a node with a "hot rumor" periodically infects other nodes
  - *Blind*: the node loses interest regardless of who the recipient is (*why*)
  - *Counter*: the node loses interest after a fixed number of contacts (*when*)
- Two parameters to control gossiping
  - *B*: max number of neighbors a message is forwarded to
  - *F*: number of times a node forwards the same message to its neighbors

un nodo che possiede hot rumor periodicamente diffonde il rumor. a tale nodo non importa a chi diffonde il rumor ma arrivato ad un counter smette di fare gossip

Portman and Seneviratne, The cost of application-level broadcast in a fully decentralized peer-to-peer network, ISCC 2002

Blind counter rumor mongering

- Gossiping protocol

```
A node n initiates a broadcast by sending message m to B
   of its neighbors, chosen at random
When node p receives a message m from node q
  If p has received m no more than F times
    p sends m to B neighbors, chosen uniformly at random,
    among those that p believes have not yet seen m
```

  - Note that *p* knows whether its neighbor *r* has already seen *m* only if *p* has previously sent *m* to *r*, or if *p* has received *m* from *r*

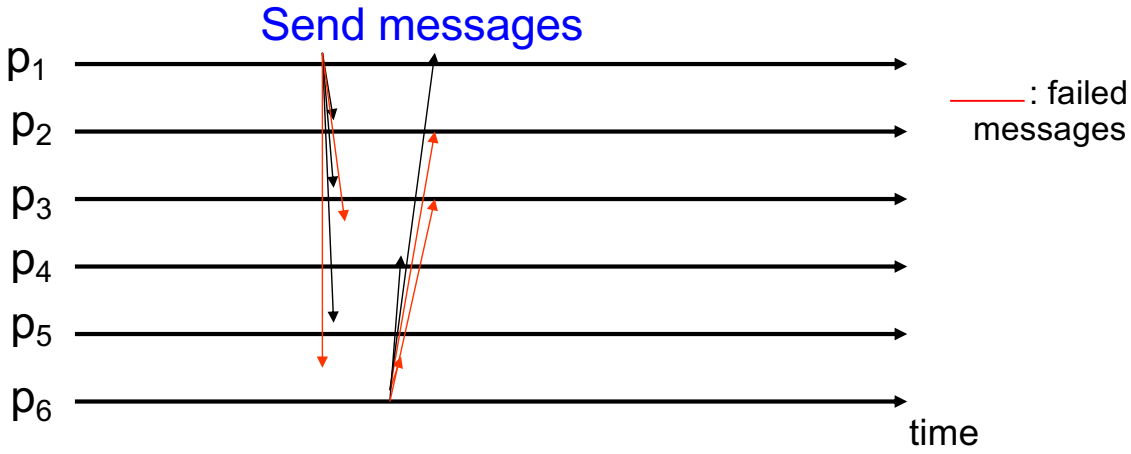
- Performance (*B*=*F*=2) compared to flooding
  - Lower number of messages: ~50%
  - Incomplete coverage: ~90%
  - Slower dissemination: ~2x

Bimodal multicast

- Aka pbcast (probabilistic broadcast)
- 2-phase protocol:
  1. **Message distribution**: a process sends a multicast message with no particular reliability guarantees
  2. **Gossip repair**: after receiving a message, a process begins to gossip about it to a set of peers
     - Gossip occurs at regular intervals, giving processes a chance to compare their states and fill gaps in their message sequence
- Used by Fastly CDN for cache invalidation
  https://www.fastly.com/blog/building-fast-and-reliable-purging-system

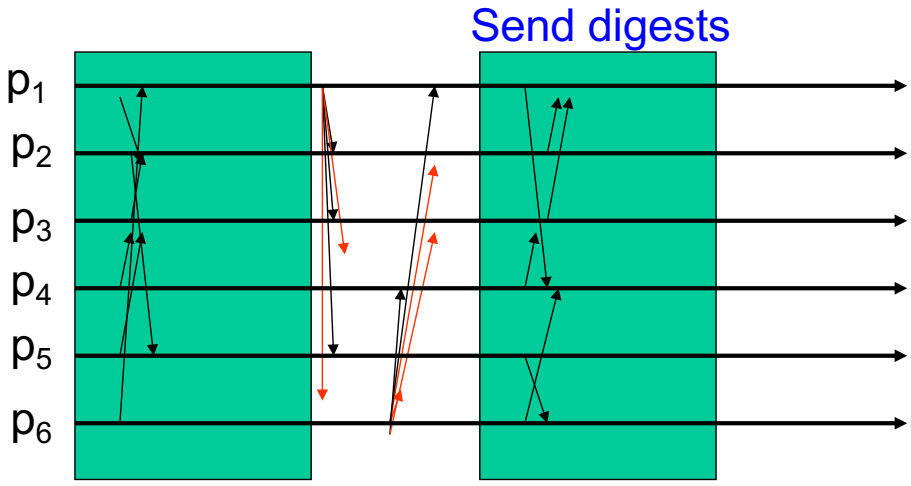
Birman et al., Bimodal multicast, *ACM Trans. Comput. Syst.,* 1999

Bimodal multicast: message distribution



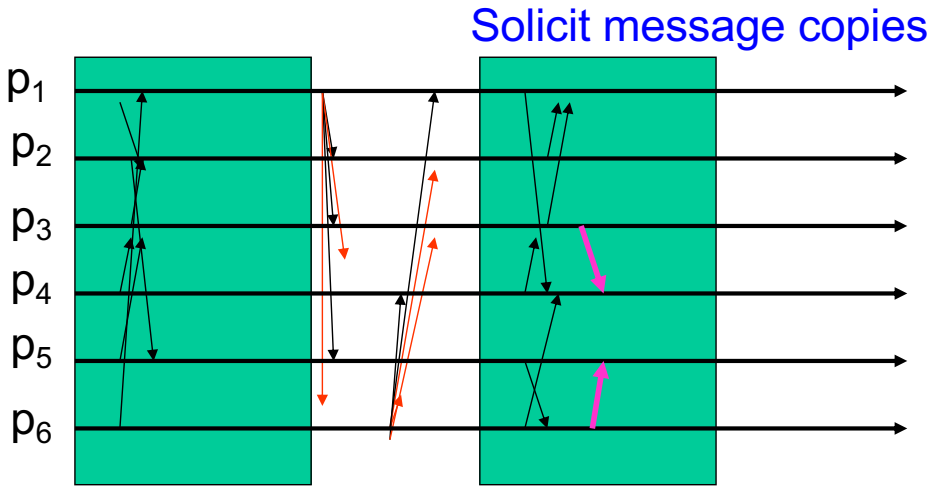
- Start with *unreliable* multicast to rapidly distribute messages
- Partial distribution may occur:
  - Some message may not reach all nodes
  - Some process may be faulty

Bimodal multicast: gossip repair



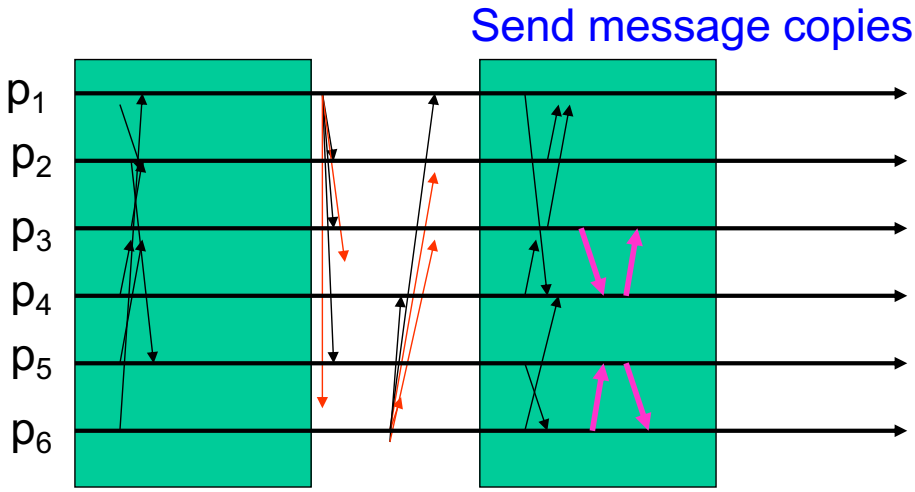
- Periodically (e.g., every 100 ms), each process sends a *digest* describing its state to a *randomly* selected process
- The digest only identifies messages, without including them

Bimodal multicast: gossip repair



- The recipient checks the gossip digest against its own message history and *requests* copies of any missing messages from the process that sent the gossip

Bimodal multicast: gossip repair



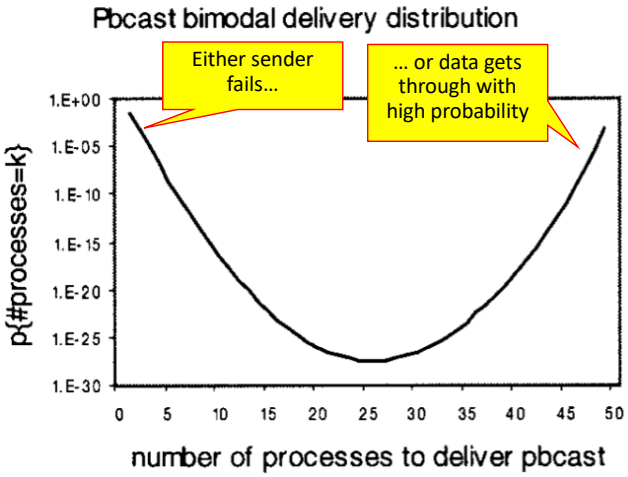
- Processes reply to solicitations received during a gossip round by retransmitting the requested messages
- Some optimizations exist (not examined)

Bimodal multicast: why “bimodal”?

- Are there two phases?
- No; dual “modes” of result

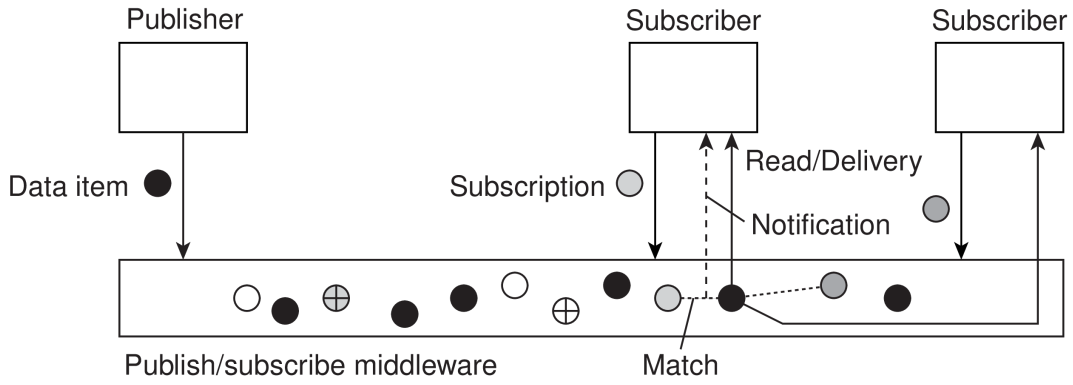
1. Bimodal delivery pattern: pbcast is almost always delivered to most or to few processes, and almost never to some processes

   Atomicity = almost all or almost none



2. Bimodal delivery latencies: one distribution of very low latencies (messages arriving without loss in the initial phase) and a second distribution with higher latencies (messages that had to be repaired afterward)

Pub-sub event matching



- Subscriber specifies which events it is interested in (subscription *S*)
- Publisher publishes event *N*: does *N* match *S*?
- Challenge: efficiently implement event matching

Event matching: centralized architecture

- Naive solution: centralized architecture
  - Single server handles all subscriptions and notifications
- Server responsibilities:
  - Handles subscriptions from subscribers
  - Receives events from publishers
  - Checks events against subscriptions
  - Notifies matching subscribers

✓ Simple to realize, feasible for small-scale deployments

✗ Scalability

✗ SPOF

Event matching: distributed architecture

- Achieve matching scalability
- Simple solution: partition subscriptions

1. Hierarchical architecture: master distributes matching across multiple workers
   - Each worker stores and handles a subset of subscriptions
   - Master receives events and distribute them to workers for matching
   - Partitioning strategy
     - Topic-based pub/sub: hash topic names to map subscriptions and events to workers

   ✗ Single master
2. Flat architecture: no single master, matching is spread across distributed servers
   - Partitioning strategy
     - Topic-based pub/sub: hash topic names to select server

Event matching: distributed architecture

- Other solutions: decentralized servers (overlay network)
- Challenge: how to route notifications to subscribers?

1. Unstructured overlay: use flooding or gossiping to disseminate event notifications
   - Store a subscription at only one server, but disseminate notifications to all servers: matching is distributed across servers
   - Selective routing: install filters to ignore paths toward nodes that are not interested, reducing unnecessary messages
2. Structured overlay: use a DHT to disseminate event notifications